ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ

ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΒΙΟΜΗΧΑΝΙΚΗΣ ΣΧΕΔΙΑΣΗΣ ΚΑΙ ΠΑΡΑΓΩΓΗΣ

ΣΧΕΔΙΑΣΗ ΚΑΙ ΚΑΤΑΣΚΕΥΗ ΑΥΤΟΜΑΤΟΠΟΙΗΜΕΝΗΣ ΡΟΜΠΟΤΙΚΗΣ ΣΚΑΚΙΕΡΑΣ ΜΕ ΕΝΣΩΜΑΤΩΜΕΝΗ ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ ΓΙΑ ΤΗΝ ΦΥΣΙΚΗ ΑΛΛΗΛΕΠΙΔΡΑΣΗ ΠΑΙΚΤΩΝ ΜΕ Η/Υ

Συγγραφέας

ΤΣΑΝΤΗΛΑΣ ΘΩΜΑΣ

Αριθμός Μητρώου: 21389261

Επιβλέπων

ΑΒΡΑΑΜ ΧΑΤΖΟΠΟΥΛΟΣ

Αθήνα, 2026



UNIVERSITY OF WEST ATTICA

SCHOOL OF ENGINEERING

DEPARTMENT OF INDUSTRIAL DESIGN AND PRODUCTION ENGINEERING

Design and implementation of an automated robotic chessboard with integrated artificial intelligence for natural player-computer interaction

Author

TSANTILAS THOMAS

Registration Number: 21389261

Supervisor

AVRAAM XATZOPOULOS

Athens, 2026

Δήλωση Συγγραφέα Διπλωματικής Εργασίας

Ο κάτωθι υπογεγραμμένος Τσαντήλας Θωμάς με αριθμό μητρώου 21389261, φοιτητής του Πανεπιστημίου Δυτικής Αττικής της Σχολής Μηχανικών του Τμήματος Βιομηχανικής Σχεδίασης και Παραγωγής, δηλώνω υπεύθυνα ότι:

«Είμαι συγγραφέας αυτής της πτυχιακής/διπλωματικής εργασίας και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, οι όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε ακριβώς είτε παραφρασμένες, αναφέρονται στο σύνολό τους, με πλήρη αναφορά στους συγγραφείς, τον εκδοτικό οίκο ή το περιοδικό, συμπεριλαμβανομένων και των πηγών που ενδεχομένως χρησιμοποιήθηκαν από το διαδίκτυο. Επίσης, βεβαιώνω ότι αυτή η εργασία έχει συγγραφεί από μένα αποκλειστικά και αποτελεί προϊόν πνευματικής ιδιοκτησίας τόσο δικής μου, όσο και του Ιδρύματος.

Παράβαση της ανωτέρω ακαδημαϊκής μου ευθύνης αποτελεί ουσιώδη λόγο για την ανάκληση του πτυχίου μου».

Ο Δηλών



Μέλη Εξεταστικής Επιτροπής

Η διπλωματική εργασία εξετάστηκε επιτυχώς από την κάτωθι Εξεταστική Επιτροπή:

α/α	Ονοματεπώνυμο	Ψηφιακή Υπογραφή
1	ΧΑΤΖΟΠΟΥΛΟΣ ΑΒΡΑΑΜ	Digitally signed by ΑΒΡΑΑΜ ΧΑΤΖΟΠΟΥΛΟΣ Date: 2026.07.13 09:46:29 +03'00'
2	ΖΑΧΑΡΙΑ ΠΑΡΑΣΚΕΥΗ	PARASKEVI ZACHARIA Digitally signed by PARASKEVI ZACHARIA Date: 2026.07.13 20:59:32 +03'00'
3	ΔΡΟΣΟΣ ΧΡΗΣΤΟΣ	Digitally signed by ΧΡΗΣΤΟΣ ΔΡΟΣΟΣ Date: 2026.07.13 19:50:17 +03'00'

Περίληψη

Μία αυτόματη σκακιέρα επιχειρεί να γεφυρώσει το ψηφιακό με το φυσικό σκάκι φέρνοντας πλεονεκτήματα του διαδικτύου στο φυσικό χώρο. Η διπλωματική εργασία πραγματεύεται τη μεθοδολογία σχεδίασης, υλοποίησης και προγραμματισμού μίας πλήρως αυτοματοποιημένης σκακιέρας. Σκοπός είναι η ανάπτυξη ενός συστήματος ικανού για αυτόνομη κίνηση των κομματιών του αντιπάλου και η προσομοίωση αυτού με ένα νευρωνικό δίκτυο. Η μεθοδολογία περιλαμβάνει το προσχέδιο, την επιλογή εξαρτημάτων με έρευνα αγοράς και αναλυτικά τα στάδια κατασκευής. Επιπρόσθετα επεξηγείται ο κώδικας που δίνει στο σύστημα τις δυνατότητες του και η διαδικασία εκπαίδευσης του νευρωνικού δικτύου. Συνδυάζοντας επιστημονικές γνώσεις από πολλούς κλάδους φέρεται εις πέρας ένα πρωτότυπο που αποδεικνύει πως αυτή η πολύπλοκη κατασκευή μπορεί να ολοκληρωθεί με κοινά υλικά και προσβάσιμες γνώσεις.

Συγκεκριμένα, οι κύριοι πυλώνες του συστήματος είναι η αντίληψη, η επεξεργασία και κίνηση. Η αντίληψη εκτελείται από ένα πλέγμα μαγνητικών αισθητήρων τοποθετημένων κάτω από τη σκακιέρα συνδεδεμένων σε πολυπλέκτες. Τα δεδομένα τους στέλνονται στο τμήμα της επεξεργασίας που εκτελείται από έναν μικροελεγκτή, ο οποίος τρέχει το κβαντοποιημένο νευρωνικό δίκτυο για τον υπολογισμό της απάντησης του υπολογιστή, όπως εκπαιδεύτηκε κατά τη διαδικασία της μηχανικής εκμάθησης. Η επιλεγμένη κίνηση του υπολογιστή εκτελείται από έναν μηχανικό σύστημα δύο αξόνων τύπου CoreXY, ελέγχοντας τη θέση ενός ηλεκτρομαγνήτη. Συνδυάζοντας αυτά τα υποσυστήματα, επιτυγχάνεται ο συγχρονισμός φυσικών και ψηφιακών αποφάσεων. Μεταφράζοντας τη κίνηση του παίκτη σε ψηφιακό δεδομένο, και μετά την επεξεργασία του, την επιστροφή της ως φυσική απάντηση, υλοποιώντας έτσι την αμφίδρομη αλληλεπίδραση ανθρώπου και υπολογιστή στο φυσικό χώρο.

Λέξεις κλειδιά:

Μηχατρονική, σκάκι, ESP32, AI, νευρωνικό δίκτυο, CoreXY, 3D printing, ηλεκτρονική, προγραμματισμός, κβαντοποίηση νευρωνικού δικτύου, αυτόματη σκακιέρα, HCI (Human-Computer Interaction), αλληλεπίδραση ανρθώπου-υπολογιστή, Edge AI, ενσωματωμένα συστήματα,

Abstract

An automated chessboard attempts to bridge digital and physical chess by bringing the advantages of the internet into the physical space. This thesis addresses the methodology of designing, implementing, and programming a fully automated chessboard. The aim is the development of a system capable of autonomously moving the opponent's pieces and the simulation of said opponent through a neural network. The methodology encompasses the preliminary design, the selection of components through market research, and the detailed stages of construction. In addition, the code that grants the system its capabilities and the training process of the neural network are explained. By combining scientific knowledge from multiple disciplines, a prototype is brought to completion that demonstrates how this complex construction can be realized using common materials and accessible knowledge.

Specifically, the main pillars of the system are perception, processing, and movement. Perception is carried out by a grid of reed switches placed beneath the chessboard and connected to multiplexers. Their data is sent for the processing part, which is executed by a microcontroller running a quantized neural network to compute the opponent's response, as trained during the machine learning process. The computer's selected move is executed by a two-axis CoreXY-type mechanical system, controlling the position of an electromagnet. By combining these subsystems, the synchronization of physical and digital decisions is achieved. Translating the player's move into digital data, and after processing it, returning it as a physical response thus realizing the bidirectional interaction between human and computer in the physical space otherwise known as HCI.

Keywords:

Mechatronics, chess, ESP32, AI, neural network, CoreXY, 3D printing, programming, neural network quantization, automated chessboard, HCI (Human-Computer Interaction), human-computer interaction, Edge AI, embedded systems,

Πίνακας Περιεχομένων

Περίληψη 5
Abstract 6
Κατάλογος Εικόνων – Σχημάτων – Διαγραμμάτων 9
Κατάλογος Πινάκων 12
Συντομογραφίες και απόδοση όρων 13
Εισαγωγή 19
Θεωρητικό Υπόβαθρο 20
Γλώσσα προγραμματισμού C++ 20
Κανόνες του Σκακιού 20
Αλγεβρική Σκακιστική Σημειογραφία 20
UCI (Universal Chess Interface) 21
Τρισδιάστατη μοντελοποίηση 21
Σχεδίαση και Κατασκευή Υλικού Μέρους (Hardware) 22
Προσχέδιο 22
Έρευνα Αγοράς 25
Επιλογή Ηλεκτρομαγνήτη 25
Επιλογή Βηματικού Κινητήρα 31
Επιλογή Driver Βηματικού Κινητήρα 41
Επιλογή μικροελεγκτή 45
Κατασκευή 48
Ηλεκτρονικό Κύκλωμα 66
Λίστα Υλικών 66
Οικονομικός Προϋπολογισμός 68
Ανάλυση και Συγγραφή Λογισμικού (Software) 70
Διαγράμματα Ροής 70
Κώδικας Προγράμματος 71
Βιβλιοθήκες και παγκόσμιες μεταβλητές 71
setup() 81
loop() 84
getLogicalPos() 89
muxEnable() και muxDisableAll() 90
muxSelectChannel() 91
muxScanAll() 92
muxToSquare(), muxToSquareLower() και muxBuildLcdString() 93
lcdWriteRow() 94
lcdUpdateRow1() 95
lcdUpdate() 97
printBoardState() 98
initPieceBoard() 99
selectPlayerColor() 100

startNewGame() 101
passOrConfirm() 103
Legal Move Generation 108
computeLegalDests() 110
nearestLegal() 115
collectSourceCandidates() 116
lmgIsAttackedBy() 118
lmgFindKing() 120
lmgInCheckAfterMove() 120
filterChecks() 121
resetBoardStateForLegalDests() 122
advanceCycling() 123
detectMove() 124
initNeuralNetwork() 129
encodeBoardForModel() 133
doComputerMove() 136
squareToMotorPos() 139
checkEStop() 140
moveToSquare() 141
squareToStepCoords() 144
distToSegSq() 145
squareInPath() 146
countNearSegment() 146
computeMagnetRoute() 147
findPathBlockersRoute() 149
findParkingSquare() 151
physicalPickAndPlace() 153
updatePendingCandidates() 154
physicalMoveViaWaypoint() 157
executeMove() 158
homeAndMeasure() 165
Εκπαίδευση Νευρωνικού Δικτύου 167
Βάση δεδομένων 167
Εκπαίδευση μοντέλου 184
Επαλήθευση αποτελεσμάτων 186
Κβαντοποίηση μοντέλου 186
Αποτελέσματα 191
Συμπεράσματα 193
Χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs) 194
Βιβλιογραφία 198
Προσθήκες 201

Κατάλογος Εικόνων – Σχημάτων – Διαγραμμάτων

Εικόνα 1 - Σύστημα Κίνησης Cross Slide ... 22
Εικόνα 2 - Σύστημα Κίνησης CoreXY ... 22
Εικόνα 3 - Κάμερες ... 23
Εικόνα 4 - Ultrasonic και Servo ... 23
Εικόνα 5 - Magnetic Sensors ... 23
Εικόνα 6 - RFID και NFC ... 23
Εικόνα 7 - Tree Search ... 24
Εικόνα 8 - Convolutional Neural Network ... 24
Εικόνα 9 - Ηλεκτρομαγνήτης KK-P25/20 ... 26
Εικόνα 10 - Ηλεκτρομαγνήτης WF-P30/22 ... 26
Εικόνα 11 - Ηλεκτρομαγνήτης WF-P25/20 ... 26
Εικόνα 12 - Ηλεκτρομαγνήτης ZYE1-P25/20 ... 26
Εικόνα 13 - Δίοδος 1N5404 ... 27
Εικόνα 14 - Διάγραμμα Επίδοσης Transistor ... 29
Εικόνα 15 - MOSFET Transistor ... 29
Εικόνα 16 - Βηματικός Κινητήρας 42BYGHW208 ... 32
Εικόνα 17 - Βηματικός Κινητήρας OK42STH22-0804AC ... 32
Εικόνα 18 - Βηματικός Κινητήρας 42BYGHM809 ... 32
Εικόνα 19 - Βηματικός Κινητήρας 42BYGHW804 ... 32
Εικόνα 20 - Βηματικός Κινητήρας Creality 42-34 ... 32
Εικόνα 21 - Τροχαλία 20T ... 33
Εικόνα 22 - Μηχανολογικό Σχέδιο Τροχαλίας και Άξονα Κινητήρα ... 40
Εικόνα 23 - Αποσβεστήρας από φελλό ... 40
Εικόνα 24 - Driver L9110S ... 42
Εικόνα 25 - Driver A4988 ... 42
Εικόνα 26 - Driver DRV8825 ... 42
Εικόνα 27 - Driver TMC2208 V3.0 ... 42
Εικόνα 28 - Schematic DRV8825 ... 44
Εικόνα 29 - Μικροελεγκτής Arduino UNO R3 ... 46
Εικόνα 30 - Μικροελεγκτής ESP32-S3 N8R8 ... 46
Εικόνα 31 - Μικροϋπολογιστής Raspberry Pi Zero 2 W ... 46
Εικόνα 32 - Βάση Ηλεκτρομαγνήτη ... 48
Εικόνα 33 - Πλαϊνή Δοκός ... 48
Εικόνα 34 - Κεντρική Δοκός ... 48
Εικόνα 35 - Εκτύπωση Δοκού ... 48
Εικόνα 36 - Slicer Settings ... 48
Εικόνα 37 - Πλαϊνές Βάσεις ... 49
Εικόνα 38 - V-wheel assembly ... 49
Εικόνα 39 - V-wheel kit ... 49
Εικόνα 40 - Διάταξη των ιμάντων στο coreXY ... 49

Εικόνα 41 - 3D μοντέλο coreXY 49
Εικόνα 42 - Πρώτο στάδιο κατασκευής του συστήματος κίνησης 50
Εικόνα 43 - Συνδεσμολογία DRV8825 50
Εικόνα 44 - Δεύτερο στάδιο κατασκευής του συστήματος κίνησης 51
Εικόνα 45 - Πρώτη έκδοση βάσης των κινητήρων 52
Εικόνα 46 - Δεύτερη έκδοση βάσης των κινητήρων 52
Εικόνα 47 - Κάτοψη κυκλώματος των Drivers 53
Εικόνα 48 - Συνδεσμολογία κυκλώματος ελέγχου ηλεκτρομαγνήτη 53
Εικόνα 49 - Μαγνήτες N52 53
Εικόνα 50 - Κάτοψη κυκλώματος ελέγχου ηλεκτρομαγνήτη 54
Εικόνα 51 - Χαλκοταινία 55
Εικόνα 52 - Τρεις πρώτες λωρίδες του πλέγματος αισθητήρων 55
Εικόνα 53 - Οι λωρίδες του πλέγματος αισθητήρων 56
Εικόνα 54 - Οργάνωση και κόλληση καλωδίων του πλέγματος αισθητήρων 57
Εικόνα 55 - Closeup οργάνωσης και κόλλησης καλωδίων του πλέγματος αισθητήρων 57
Εικόνα 56 - Κάτοψη πλέγματος αισθητήρων 58
Εικόνα 57 - Ένωση γειώσεων στο πλέγμα αισθητήρων 58
Εικόνα 58 - Κάτοψη οργάνωσης καλωδίων 59
Εικόνα 59 - Κλέμες για οργάνωση εξόδων 59
Εικόνα 60 - Συνδεσμολογία πολυπλεκτών 60
Εικόνα 61 - Κάτοψη κυκλώματος πολυπλεκτών 60
Εικόνα 62 - Βάσεις στήριξης 60
Εικόνα 63 - Σύνδεση καλωδίων στους πολυπλέκτες 2 61
Εικόνα 64 - Σύνδεση καλωδίων στους πολυπλέκτες 61
Εικόνα 65 - Πρόσοψη μετά τη σύνδεση του πλέγματος 62
Εικόνα 66 - Συνδεσμολογία πλέγματος αισθητήρων 62
Εικόνα 67 - Εκτύπωση βάσης οθόνης και κουμπιών 62
Εικόνα 68 - Βάση διεπαφής χρήστη συνδεδεμένη ανάποδα 63
Εικόνα 69 - 3D μοντέλο και wireframe του καλύμματος κουμπιού 63
Εικόνα 70 - Κάτοψη διεπαφής χρήστη 63
Εικόνα 71 - Σχέδιο εμφάνισης της σκακιέρας για εκτύπωση σε βινύλιο 64
Εικόνα 72 - Πιόνια διαφορετικών μεγεθών 65
Εικόνα 73 - Βάσεις προέκτασης για τους μαγνήτες 65
Εικόνα 74 - 3D μοντέλα των κομματιών 65
Εικόνα 75 - Πλήρες schematic του συστήματος 66
Εικόνα 76 - Παράδειγμα θέσης FEN 169
Εικόνα 77 - Parser running 182
Εικόνα 78 - Τελικά αποτελέσματα parser 183
Εικόνα 79 - Αρχεία από το parser 183
Εικόνα 80 - Αρχή εκπαίδευσης νευρωνικού δικτύου 184
Εικόνα 81 - Αποτελέσματα εκπαίδευσης 185

Εικόνα 82 - Αρχείο Pytorch..... 185
Εικόνα 83 - Εξαγωγή σε ONNX..... 185
Εικόνα 84 - Αρχείο ONNX..... 185
Εικόνα 85 - Διεπαφή χρήστη της ψηφιακής εφαρμογής για testing του μοντέλου..... 186
Εικόνα 86 - Κβαντοποίηση με ESP-PPQ..... 187
Εικόνα 87 - Κβαντοποιημένο μοντέλο ESP-DL..... 187
Εικόνα 88 - Κβαντοποιημένο μοντέλο tensor flow lite..... 188
Εικόνα 89 - Τέλος του header file του μοντέλου..... 190
Εικόνα 90 - Αρχή του header file του μοντέλου..... 190

Κατάλογος Πινάκων

Πίνακας 1 - Λίστα συντομογραφιών 18
Πίνακας 2 - Απόδοση Ηλεκτρομαγνητών 26
Πίνακας 3 - Σύγκριση Τροχαλίων 34
Πίνακας 4 - Ροπή ανά διαιρέτη βημάτων 35
Πίνακας 5 - Σύγκιση Βηματικών Κινητήρων προς Microstepping 36
Πίνακας 6 - Σύγκριση Απόδοσης Βηματικών Κινητήρων 36
Πίνακας 7 - Ρεύμα και Ισχύς Βηματικών Κινητήρων 39
Πίνακας 8 - Σύγκριση ρεύματος εισόδου στο driver των κινητήρων 39
Πίνακας 9 - Σύγκριση αντίστασης πηνίων των κινητήρων 40
Πίνακας 10 - Λίστα ηλεκτρονικών εξαρτημάτων 67
Πίνακας 11 – Λίστα μηχανικών εξαρτημάτων 68
Πίνακας 12 - Οικονομικός προϋπολογισμός 69
Πίνακας 13 – Memory partition 72
Πίνακας 14 - Memory partition σε CSV format 72
Πίνακας 15 – Παράδειγμα bit shifting 91
Πίνακας 16 - Κατανομή τετραγώνων στους πολυπλέκτες 93
Πίνακας 17 - Κατευθύνσεις κομματιών 109
Πίνακας 18 - Resolver additions 131
Πίνακας 19 - Ανάλυση επιπέδων του input tensor 170
Πίνακας 20 - Βιβλιοθήκες του parser της βάσης δεδομένων 172

Συντομογραφίες και απόδοση όρων

Συντομογραφία	*Πλήρης Ονομασία*	*Περιγραφή*
UCI	Universal Chess Interface	Πρωτόκολλο επικοινωνίας σκακιστικών προγραμμάτων· Αναπαριστά κινήσεις ως αρχικό και τελικό τετράγωνο (πχ e2e4). Χρησιμοποιείται στη βάση εκπαίδευσης.
FEN	Forsyth–Edwards Notation	Μορφή συμπίεσης θέσης σκακιέρας σε string. Περιλαμβάνει θέσεις κομματιών, σειρά, δικαιώματα ροκέ και en passant. Αριθμοί στις κενές συνεχόμενες θέσεις για συμπίεση τους.
ELO	Elo Rating System	Σύστημα αξιολόγησης επιπέδου. Αναφέρεται ως μέτρο δυσκολίας του νευρωνικού δικτύου (στόχος 1500 ELO).
AI	Artificial Intelligence	Τεχνητή Νοημοσύνη.
NN	Neural Network	Νευρωνικό Δίκτυο. Το μοντέλο που εκπαιδεύτηκε για να υπολογίζει τις απαντήσεις του υπολογιστή.
CNN	Convolutional Neural Network	Συνελικτικό Νευρωνικό Δίκτυο. Αρχιτεκτονική νευρωνικού δικτύου κατάλληλη για ανάλυση εικόνων/πινάκων.
DL	Deep Learning	Βαθιά Μάθηση. Κατηγορία μηχανικής μάθησης με πολλαπλά επίπεδα νευρώνων.
LLM	Large Language Model	Μεγάλο Γλωσσικό Μοντέλο. Αναφέρεται στη χρήση του Gemini και του Claude Sonnet.
TLFM	TensorFlow Lite for Microcontrollers	Βιβλιοθήκη για τη χρήση νευρωνικών δικτύων σε μικροελεγκτές. Χρησιμοποιείται για το inference στο ESP32-S3.
ONNX	Open Neural Network Exchange	Μορφή μοντέλων νευρωνικών δικτύων. Χρησιμοποιείται ως

		ενδιάμεσο βήμα εξαγωγής πριν την κβαντοποίηση.
NHWC	Number of samples, Height, Width, Channels	Μορφή διαστάσεων input tensor νευρωνικού δικτύου. Το μοντέλο δέχεται [1,8,8,18] NHWC.
PPQ	Post-Processing Quantization	Εργαλείο κβαντοποίησης μοντέλων της Espressif (ESP-PPQ). Χρησιμοποιείται για μετατροπή σε .espdl.
ESP-DL	Espressif Deep Learning	Βιβλιοθήκη βαθιάς μάθησης για ESP32 από την Espressif. Χρησιμοποιείται για την χρήση του κβαντοποιημένου μοντέλου.
Edge AI	Edge Artificial Intelligence	Εκτέλεση AI απευθείας σε ενσωματωμένο hardware όπως μικροελεγκτής χωρίς cloud.
ESP32	Espressif Systems MCU 32-bit	Μικροελεγκτής της Espressif. Χρησιμοποιείται το μοντέλο ESP32-S3 N8R8 (8MB Flash, 8MB PSRAM).
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor	Τύπος τρανζίστορ ελεγχόμενος από τάση. Χρησιμοποιείται ως διακόπτης για τον ηλεκτρομαγνήτη (μοντέλο IRLB8721PBF).
BJT	Bipolar Junction Transistor	Τύπος τρανζίστορ ελεγχόμενος από ρεύμα. Συγκρίθηκε με MOSFET, δεν επιλέχθηκε.
PNP	Positive-Negative-Positive	Τύπος BJT τρανζίστορ για high-side switching.
NPN	Negative-Positive-Negative	Τύπος BJT τρανζίστορ για low-side switching. Ο τύπος που χρησιμοποιείται στο κύκλωμα.
TVS	Transient Voltage Suppressors	Δίοδοι προστασίας από παροδικές υπερτάσεις. Αναφέρονται ως ακατάλληλο είδος διόδου για το κύκλωμα.
EMF	Electromotive Force (Back EMF)	Αντίθετη ηλεκτρική δύναμη που παράγεται από τον κινητήρα. Επηρεάζει τη λειτουργία του driver.

NEMA	National Electrical Manufacturers Association	Πρότυπο διαστάσεων βηματικών κινητήρων. Χρησιμοποιείται NEMA-17 (42×42mm).
PCB	Printed Circuit Board	Τυπωμένη πλακέτα κυκλώματος. Εξετάστηκε αλλά δεν επιλέχθηκε στην τελική κατασκευή λόγω κόστους.
PLA	Polylactic Acid	Υλικό τρισδιάστατης εκτύπωσης. Χρησιμοποιείται για όλα τα εκτυπωμένα εξαρτήματα.
GT2	Gates Tooth Profile 2mm	Ιμάντας χρονισμού με βήμα δοντιών 2mm. Χρησιμοποιείται στο σύστημα CoreXY.
RFID	Radio Frequency Identification	Εξετάστηκε ως εναλλακτική ανίχνευσης κομματιών, απορρίφθηκε.
NFC	Near Field Communication	Εξετάστηκε μαζί με RFID, απορρίφθηκε λόγω μαγνητικών παρεμβολών.
DIY	Do It Yourself	Ατομική κατασκευή. Χαρακτηρίζει τη φύση της κατασκευής.
CoreXY	Core XY Motion System	Σύστημα κίνησης δύο αξόνων με συνεργατικούς ιμάντες. Επιλέχθηκε για κίνηση ηλεκτρομαγνήτη σκακιέρας.
PWM	Pulse Width Modulation	Διαμόρφωση πλάτους παλμού. Χρησιμοποιείται για έλεγχο τάσης/ρεύματος στους βηματικούς κινητήρες.
MCPWM	Motor Control Pulse Width Modulation	Ενσωματωμένο hardware παλμών του ESP32 για έλεγχο κινητήρων. Αποδεσμεύει τον επεξεργαστή από δημιουργία παλμών.
LEDC	LED Control (ESP32 peripheral)	Δεύτερο hardware peripheral παλμών του ESP32. Αναφέρεται παράλληλα με MCPWM.
GPIO	General Purpose Input/Output	Γενικής χρήσης pins εισόδου/εξόδου μικροελεγκτή.

		Χρησιμοποιούνται για αισθητήρες, κινητήρες, οθόνη.
UART	Universal Asynchronous Receiver-Transmitter	Πρωτόκολλο σειριακής επικοινωνίας.
I2C	Inter-Integrated Circuit	Πρωτόκολλο επικοινωνίας δύο καλωδίων. Χρησιμοποιείται για σύνδεση LCD οθόνης.
LCD	Liquid Crystal Display	Οθόνη υγρών κρυστάλλων. Εμφανίζει κατάσταση συστήματος κατά τη διάρκεια του παιχνιδιού.
LED	Light Emitting Diode	Δίοδος εκπομπής φωτός. Onboard LED του ESP32 για οπτική ένδειξη κατάστασης.
GRB	Green-Red-Blue	Σειρά χρωματικών καναλιών του onboard LED του ESP32-S3 (αντί RGB).
USB	Universal Serial Bus	Σύνδεση για προγραμματισμό και αποσφαλμάτωση του ESP32.
USB-CDC	USB Communication Device Class	USB για σειριακή επικοινωνία. Ενεργοποιείται στο ESP32 για καλύτερο flashing αποδεσμεύοντας το UART.
MUX	Multiplexer	Πολυπλέκτης. Χρησιμοποιείται για ανάγνωση 64 αισθητήρων reed switch από λίγα pins (CD74HC4067).
RAM	Random Access Memory	Μνήμη τυχαίας πρόσβασης. Χρησιμοποιείται για runtime δεδομένα.
ROM	Read Only Memory	Μνήμη μόνο ανάγνωσης / Flash. Αποθηκεύεται εκεί το πρόγραμμα.
SRAM	Static Random Access Memory	Εσωτερική στατική μνήμη RAM του ESP32.
PSRAM	Pseudo-Static RAM	Εξωτερική ψευδο-στατική μνήμη. Στο ESP32-S3 N8R8 είναι 8MB· εκεί φορτώνεται το μοντέλο (6.89MB).

SPIFFS	SPI Flash File System	Σύστημα αρχείων για SPI Flash. Αναφέρεται στη διαχείριση partition μνήμης ESP32.
NVS	Non-Volatile Storage	Μνήμη μόνιμης αποθήκευσης. Partition μνήμης ESP32 για μόνιμη αποθήκευση ρυθμίσεων.
OTA	Over-The-Air	Ασύρματη ενημέρωση firmware. Αναφέρεται στο partition table του ESP32 ως πιθανή λειτουργία.
CSV	Comma-Separated Values	Μορφή αρχείου τιμών διαχωρισμένων με κόμμα. Χρησιμοποιείται για τον ορισμό partition table του ESP32.
JSON	JavaScript Object Notation	Μορφή δεδομένων. Χρησιμοποιείται στη βάση εκπαίδευσης (Lichess dataset) για αποθήκευση θέσεων/evals.
ASCII	American Standard Code for Information Interchange	Πρότυπο κωδικοποίησης χαρακτήρων. Χρησιμοποιείται για μετατροπή indexes σε γράμματα σκακιέρας (a-h).
IDE	Integrated Development Environment	Ολοκληρωμένο περιβάλλον ανάπτυξης. Επιλέχθηκε το Arduino IDE για απλότητα.
CLI	Command Line Interface	Διεπαφή γραμμής εντολών. Αναφέρεται ως Arduino CLI για διαχείριση βιβλιοθηκών.
ESP-IDF	Espressif IoT Development Framework	Επίσημο framework ανάπτυξης ESP32. Υποστηρίζεται μέσω wrappers στο Arduino IDE.
FreeRTOS	Free Real Time Operating System	Ενσωματωμένο λειτουργικό σύστημα πραγματικού χρόνου. Διαχειρίζεται tasks και συγχρονισμό πυρήνων ESP32.

PROGMEM	Program Memory	Macro για αποθήκευση δεδομένων στη flash αντί RAM. Χρησιμοποιείται για τη βελτιστοποίηση μνήμης.
HCI	Human-Computer Interaction	Αλληλεπίδραση Ανθρώπου-Υπολογιστή. Κεντρική έννοια της κατασκευής, φυσική αλληλεπίδραση παίκτη με AI.
XY	Two-Axis Coordinate System	Καρτεσιανό σύστημα συντεταγμένων δύο αξόνων. Αναφέρεται στο σύστημα κίνησης (CrossSlide / CoreXY).
DC	Direct Current	Συνεχές ρεύμα. Τροφοδοσία κυκλώματος (12V DC).
SD	Secure Digital	Κάρτα μνήμης. Αναφέρεται ως εναλλακτική αποθήκευση για Arduino UNO που δεν επιλέχθηκε.

Πίνακας 1 - Λίστα συντομογραφιών

Εισαγωγή

Στη σύγχρονη εποχή το σκάκι είναι πιο δημοφιλές από ποτέ (Chess Stats, 2026), με την ραγδαία αύξηση της προσβασιμότητας του χάρη σε διαδικτυακές πλατφόρμες όπως το Lichess και το Chess.com (Blind Mode, 2026). Ωστόσο, με την ανάπτυξη του ψηφιακού κόσμου το παιχνίδι μετακινήθηκε από το φυσικό χώρο στην οθόνη, κάτι που υποβαθμίζει την ιστορία του ως επιτραπέζιο και δυσχεραίνει την εμπειρία των παικτών με εκτεταμένο χρόνο στην οθόνη κατά τη παρτίδα. Οι αυτόματες σκακιέρες επιχειρούν να γεφυρώσουν αυτό το κενό φέρνοντας τα πλεονεκτήματα από το ψηφιακό σκάκι στον φυσικό χώρο (Square Off by Miko, 2026) (Millennium Chess Shop, 2026) (Tournament e-Boards, 2026).

Ο σκοπός της συγκεκριμένης κατασκευής είναι η δημιουργία μιας σκακιέρας η οποία είναι ικανή να αναγνωρίζει τις κινήσεις του παίκτη και να μετακινεί αυτόνομα τα κομμάτια του αντιπάλου εφαρμόζοντας πρακτικές γνώσεις μηχατρονικής. Ο αντίπαλος προσομοιώνεται με ένα νευρωνικό δίκτυο έχοντας τη μελλοντική δυνατότητα για συνδεσιμότητα με άλλους παίκτες για πραγματικό απομακρυσμένο σκάκι. Για την υλοποίηση αυτού του σκοπού η διαδικασία κατασκευής χωρίζεται σε τρία κύρια μέρη, την αντίληψη, την επεξεργασία και την κίνηση (Greg06, 2026).

Η αντίληψη υλοποιείται μέσω ενός δικτύου αισθητήρων τοποθετημένων κάτω από κάθε τετράγωνο της σκακιέρας, οι οποίοι ανιχνεύουν την θέση των μαγνητών στις βάσεις των κομματιών. Για τη μετακίνηση των μετρήσεων στο κομμάτι της επεξεργασίας χρησιμοποιείται η τεχνική πολυπλεξίας ώστε να συμπιεστούν τα δεδομένα και να διαβαστούν από έναν μικροελεγκτή.

Η επεξεργασία αφορά τον μικροελεγκτή και το λογισμικό όπου τα δεδομένα επικυρώνονται με βάση τους κανόνες του παιχνιδιού, μετατρέπονται σε μορφή ενός tensor που μπορεί να διαβαστεί από το νευρωνικό δίκτυο, και υπολογίζεται η απάντηση του υπολογιστή στη κίνηση του παίκτη. Ύστερα, η απάντηση μετατρέπεται σε κινήσεις των βηματικών κινητήρων για το σύστημα κίνησης.

Η κίνηση η ίδια εκτελείται από δύο βηματικούς κινητήρες, συνδεδεμένους με ιμάντες και τροχαλίες σε ένα κεντρικό ηλεκτρομαγνήτη. Ο ηλεκτρομαγνήτης μετακινείται στο υπολογισμένο τετράγωνο και εκτελεί τη κίνηση μετακινώντας το μαγνητικό πιόνι στο φυσικό χώρο προτού δοθεί η σειρά πίσω στον παίκτη. Ο παίκτης καθόλη τη διάρκεια της κίνησης και της αντίληψης μπορεί να δει τη κατάσταση του συστήματος σε οθόνη.

Συνδυάζοντας τη σύγχρονη τεχνητή νοημοσύνη και τον αυτοματισμό, η συγκεκριμένη κατασκευή αποτελεί μία ολοκληρωμένη σπουδή στον συνδυασμό διαφορετικών επιστημονικών κλάδων. Η επιτυχής μετάβαση από το φυσικό ηλεκτρικό σήμα των αισθητήρων στην υπολογιστική επεξεργασία του νευρωνικού δικτύου και, τελικά, στην ακριβή μηχανική κίνηση του ηλεκτρομαγνήτη αποτελεί μία απόδειξη πώς οι σύγχρονες τεχνολογίες μπορούν να επαναφέρουν την ανθρώπινη αλληλεπίδραση στο φυσικό χώρο, διατηρώντας αναλλοίωτη την παράδοση του παιχνιδιού.

Θεωρητικό Υπόβαθρο

Γλώσσα προγραμματισμού C++

Χρησιμοποιούνται μεταβλητές, συναρτήσεις, αντικείμενα (objects), references και pointers των οποίων ο σκοπός αναλύεται στην επεξήγηση κώδικα. Ωστόσο η κατανόηση της σύνταξης και βασικών γνώσεων της γλώσσας είναι σημαντική για την ανάγνωση του σχετικού κεφαλαίου. Δεν χρησιμοποιήθηκαν templates και unions στη συγγραφή του προγράμματος και δεν απαιτήθηκαν compiler intrinsics (Chess Programming Wiki, 2024) (C++ Reference, 2026) (Language Reference, 2026).

Κανόνες του Σκακιού

Απαιτείται κατανόηση των κανόνων και του σκοπού του παιχνιδιού, συμπεριλαμβανομένων των λιγότερο γνωστών κανόνων όπως το en passant ή το threefold repetition και stalemate. En passant είναι μια ειδική περίπτωση που ένα εχθρικό πιόνι μετακινήθηκε δύο τετράγωνα μπροστά (δηλαδή μόνο κατά την αρχική του κίνηση) και υπάρχει φιλικό πιόνι σε διπλανή στήλη, τότε μόνο για αυτή τη κίνηση επιτρέπεται η διαγώνια αιχμαλωσία του εχθρικού πιονιού. Threefold repetition είναι μία συνθήκη ισοπαλίας κατά την οποία αν μία θέση επαναληφθεί τρεις φορές κατά τη διάρκεια της παρτίδας τότε η παρτίδα είναι ισοπαλία. Stalemate είναι μία άλλη συνθήκη ισοπαλίας, συναντάται όταν ένας παίκτης δεν έχει καμία επιτρεπόμενη κίνηση και ο βασιλιάς του δεν βρίσκεται υπό απειλή, τότε η παρτίδα είναι ισοπαλία (Rules of Chess, 2026) (FIDE LAWS of CHESS, 2026).

Αλγεβρική Σκακιστική Σημειογραφία

Αυτού του είδους γραφής χρησιμοποιείται από ανθρώπους λόγω της συντομίας και της ευαναγνωσιμότητας της. Ο λόγος που προτιμάται είναι γιατί εμπεριέχει εξίσου πληροφορίες που βοηθούν τους παίκτες να αναγνωρίσουν τη θέση, κάτι που δεν είναι απαραίτητο για ένα πρόγραμμα. Τα προγράμματα χρησιμοποιούν UCI αντί της κανονικής αλγεβρικής γραφής. Σε κάθε κίνηση αναγράφεται το κομμάτι (εκτός από πιόνι) και το τετράγωνο στο οποίο μετακινήθηκε. Για παράδειγμα, βασίλισσα στο g2 θα γραφόταν Qg2 χωρίς να παρέχεται η πληροφορία τετραγώνου προέλευσης, άρα απαιτείται η γνώση της αρχικής θέσης και των διαδοχικών κινήσεων. Τα ροκέ συμβολίζονται με O-O για μικρό ροκέ και O-O-O για μεγάλο. Η αιχμαλωσίες με ένα x μεταξύ του κομματιού και του τετραγώνου, όπως Qxg2 δηλώνει πως η βασίλισσα αιχμαλώτισε κάποιο εχθρικό κομμάτι στο g2. Το ρουά συμβολίζεται με σταυρό + μετά το τετράγωνο, όπως Bxe7+ που σημαίνει αξιωματικός αιχμαλώτισε κάτι στο e7 και είναι ρουά. Για το en passant μπορεί προαιρετικά να γραφτεί e.p. μετά τη κίνηση αλλιώς γράφεται ως διαγώνια αιχμαλώτιση πιονιού όπως exd6 (το κομμάτι δεν αναφέρεται γιατί είναι κίνηση πιονιού). Το ματ γράφεται με # στο τέλος της κίνησης. Για την αναγνώριση της σειράς οι κινήσεις γράφονται σε ζευγάρια όπως 2.Nc3 Nf6 με τα λευκά αριστερά. Υπάρχουν περισσότεροι κανόνες αλλά δεν αναφέρονται στη παρούσα κατασκευή διότι είναι αρκετά σπάνιοι και συναντώνται μόνο σε συγκεκριμένες θέσεις (Algebraic Notation (chess), 2026).

UCI (Universal Chess Interface)

Το UCI είναι ένα σύστημα σκακιστικής γραφής που περιγράφει τις κινήσεις ως τετράγωνο αρχικής θέσης και τελική κολλητά. Για παράδειγμα η κίνηση e2->e4 σε UCI γράφεται e2e4. Οι προαγωγές γράφονται αναφέροντας το κομμάτι μετά τη κίνηση, η προαγωγή σε βασίλισσα θα γραφόταν e7e8q όπου q συμβολίζει τη βασίλισσα. Τα ροκέ συμβολίζονται ως διπλή κίνηση του βασιλιά με τα ροκέ των λευκών να αναγράφονται ως e1g1 για το μικρό ροκέ και e1c1 για το μεγάλο. Το en passant δεν έχει συγκεκριμένο σύμβολο, γράφεται μόνο με τη διαγώνια κίνηση του πιονιού και συμπεραίνεται γνωρίζοντας τη θέση. Το UCI χρησιμοποιείται από προγράμματα για συμπίεση δεδομένων καθώς μειώνει τις συνολικές κινήσεις, αλλά είναι πιο δυσανάγνωστο για ανθρώπους. Στη παρούσα κατασκευή συναντάται στη βάση δεδομένων εκπαίδευσης του νευρωνικού δικτύου (Universal Chess Interface, 2026).

Τρισδιάστατη μοντελοποίηση

Τα τρισδιάστατα μοντέλα που χρησιμοποιήθηκαν παρουσιάζονται στην ενότητα κατασκευής. Ωστόσο δεν αναλύεται η μεθοδολογία σχεδίασης αυτών των μοντέλων ούτε η διαδικασία της εκτύπωσης τους γιατί είναι ενδεικτικά και διαφέρουν μεταξύ κάθε κατασκευής αυτόματης σκακιέρας ανάλογα με τις επιθυμητές διαστάσεις. (Greg06, 2026) (Peiling, 2025)

Σχεδίαση και Κατασκευή Υλικού Μέρους (Hardware)

Προσχέδιο

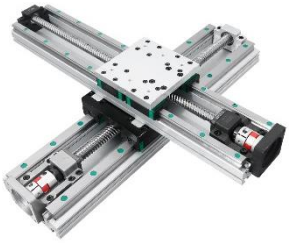
Αρχικά πρέπει να καθοριστούν τα επιμέρους μέρη που απαιτούνται για να λειτουργήσει η κατασκευή. Μελετώντας υπάρχουσες αυτόματες σκακιέρες, που μετακινούν αυτόματα τα κομμάτια και δεν απλώς ανιχνεύουν κινήσεις, διαπιστώθηκε πως χωρίζονται σε τρεις ομάδες ανάλογα με το τρόπο κίνησης των κομματιών:

- Μαγνητικό Σύστημα Κίνησης XY: Τοποθετείται ένας ηλεκτρομαγνήτης σε ένα μηχανικό σύστημα συντεταγμένων κάτω από τη σκακιέρα, για να μετακινούνται τα πιόνια πρέπει η βάση τους να είναι επίσης μαγνητική. Είναι το πιο κοινό είδος λόγω τις ευκολίας κατασκευής της συγκριτικά με τις υπόλοιπες μεθόδους κίνησης. Το κύριο μειονέκτημα είναι πως μπορεί να εκτελεστεί μόνο μία κίνηση τη φορά.

- Αυτόνομα Ρομπότ: Το κάθε πιόνι έχει ένα ολόκληρο σύστημα κίνησης με κινητήρες και τροχούς ώστε να μπορεί αυτόνομα να ελέγξει τη θέση του στη σκακιέρα. Αυτό σημαίνει μεγάλη δυσκολία κατασκευής λόγω των μικροσκοπικών εξαρτημάτων και του πλήθους τους, για όλα τα κομμάτια συνολικά απαιτούνται 32 ρομπότ. Το πλεονέκτημα που προσφέρει υπέρ του μαγνητικού συστήματος είναι πως μπορούν να εκτελεστούν ταυτόχρονα πολλές κινήσεις όπως για να επιστρέψουν όλα αυτόματα στις αρχικές τους θέσεις.

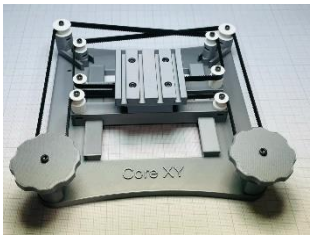
- Ρομποτικός Βραχίονας: Χρησιμοποιείται ένας εξωτερικός ρομποτικός βραχίονας για την εκτέλεση των κινήσεων που δεν καθίσταται έμφυτο μέρος της σκακιέρας. Χρειάζονται αρκετοί βαθμοί ελευθερίας κίνησης για τη σωστή μετακίνηση χωρίς να χαλάσει τη παρούσα θέση των κομματιών.

Η πιο κοινή λύση του μαγνητικού συστήματος κίνησης είναι η πιο προσβάσιμη χωρίς να απαιτείται ειδικός εξοπλισμός, με σχετικά χαμηλό κόστος υλικών και εκτεταμένες πηγές εκμάθησης. Μία πηγή που αναλύει τη διαδικασία καθόρισε το σύστημα κίνησης σε αυτό το στάδιο της κατασκευής (Greg06, 2026). Όσων αφορά το συγκεκριμένο σύστημα κίνησης που θα χρησιμοποιηθεί εξετάστηκαν δύο επιλογές:



Εικόνα 1 - Σύστημα Κίνησης Cross Slide

- Cross Slide: Το σύστημα κίνησης δύο αξόνων χρησιμοποιεί δύο δοκούς στοιβαγμένες η μία πάνω στη άλλη, αυτό σημαίνει πως το χαμηλότερο μοτέρ πρέπει να μετακινεί τη πάνω δοκό μαζί με τον κινητήρα της. Χρησιμοποιείται ως μία διαδομένη γερανογέφυρα με δυνατότητα ευελιξίας.



Εικόνα 2 - Σύστημα Κίνησης CoreXY

- CoreXY: Το σύστημα κίνησης που αξιοποιούν πολλοί τρισδιάστατοι εκτυπωτές, θεωρείται αξιόπιστο και ακριβής αλλά παρουσιάζει μεγαλύτερη δυσκολία κατασκευής λόγω της διαδρομής των ιμάντων (CoreXY, 2020).

Μεταξύ των δύο συστημάτων κίνησης λόγω του κλειστού χώρου της σκακιέρας και των παρόμοιων προδιαγραφών με 3D εκτυπωτές που παρουσιάζεται να ανταπεξέλθει το σύστημα κίνησης επιλέχτηκε το σύστημα CoreXY.

Με το σύστημα κίνησης επιλεγμένο πρέπει να καθοριστεί πώς θα γίνεται η ανίχνευση των κομματιών του παίκτη. Εξετάστηκαν τρεις γενικοί τρόποι που βρέθηκαν από υπάρχουσες κατασκευές.



Εικόνα 3 - Κάμερες

❖ Κάμερα: μία κάμερα τοποθετείται σε πρόβολο πάνω από τη σκακιέρα ώστε να βλέπει όλα τα τετράγωνα, αυτό επιτρέπει με έναν αισθητήρα και image recognition software να διαπιστωθεί η θέση όλων των 32 κομματιών.



Εικόνα 4 - Ultrasonic και Servo

❖ Περιστροφικοί αισθητήρες απόστασης: τοποθετούνται τρεις ή τέσσερις αισθητήρες που μπορούν να μετράνε απόσταση, όπως ultrasonic, laser ή lidar, στις άκρες τις σκακιέρας πάνω σε περιστρεφόμενες βάσεις. Με τις μετρήσεις μπορούν με τριπλευρισμό να χαρτογραφηθούν οι θέσεις των κομματιών στη σκακιέρα



Εικόνα 5 - Magnetic Sensors

❖ Μαγνητικοί αισθητήρες: τα πιόνια λόγω του επιλεγμένου μαγνητικού συστήματος κίνησης θα έχουν μαγνητικές βάσεις άρα μπορούν αν χρησιμοποιηθούν μαγνητικοί αισθητήρες όπως hall-effect sensors ή reed switches κάτω από τη σκακιέρα για τον εντοπισμό τους. Θα χρειαζόταν ένας μεγάλος αριθμός αισθητήρων.



Εικόνα 6 - RFID και NFC

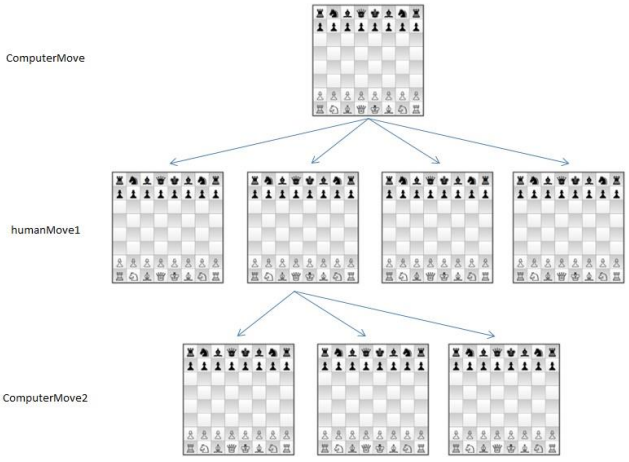
❖ Ετικέτες RFID/NFC: τοποθετούνται στις βάσεις των πιονιών παθητικές ετικέτες που μπορούν να αναγνωριστούν από 64 κεραίες κάτω από τη σκακιέρα. Επειδή η κάθε ετικέτα εκπέμπει έναν μοναδικό κωδικό αυτό θα επέτρεπε την αναγνώριση του τύπου του κομματιού κάτι που οι υπόλοιποι τρόποι στερούνται.

Η κάμερα θα ήταν η πιο πρακτική λύση, αλλά ο πρόβολος πάνω από τη σκακιέρα θα ήταν αντιαισθητικός προς την υπόλοιπη κατασκευή και θα εμπόδιζε ενεργά τον παίκτη από τις κινήσεις του. Οι περιστροφικοί αισθητήρες απόστασης αντιμετωπίζουν το πρόβλημα μεγάλου κόστους, καθώς η απαιτούμενη ανάλυση είναι πολύ υψηλή και θα χρειαζόντουσαν τέσσερις επιπλέον σερβοκινητήρες να δουλεύουν ασταμάτητα κατά τη διάρκεια της παρτίδας. Παρουσιάζουν επίσης το ρίσκο να είναι «τυφλοί» σε κάποια κομμάτια κατά τη διάρκεια της παρτίδας αν κανείς τους δεν έχει καθαρή προβολή προς αυτό αν εμποδίζουν ενδιάμεσα τα υπόλοιπα πιόνια. Οι ετικέτες και τα ραδιοκύματα παρουσιάζουν μεγάλο ρίσκο αποτυχίας λόγω των μαγνητικών παρεμβολών από τους μαγνήτες στη βάση των κομματιών και τον ηλεκτρομαγνήτη. Απαιτούνται μαζί με τις ετικέτες και ένα πλέγμα από κεραίες που θα έδιναν την ακριβή θέση του κομματιού αλλά η αναγνώριση του είδους του πιονιού μπορεί να γίνει στο software καθώς δεν υπάρχει ασάφεια στις κινήσεις και η αρχική θέση είναι πάντα η ίδια. Η καλύτερη λύση είναι οι μαγνητικοί αισθητήρες, ένα πλέγμα αυτών θα είχε την

απαιτούμενη ανάλυση για τα 64 τετράγωνα χωρίς να απαιτούνται ετικέτες στα κομμάτια. Το ρίσκο που παρουσιάζουν είναι πως το πλέγμα θα παρεμβάλλεται ανάμεσα στους μαγνήτες των πιονιών και του ηλεκτρομαγνήτη μειώνοντας τη μαγνητική έλξη. Η δυσκολία συνδεσμολογίας 64 αισθητήρων στις σωστές θέσεις είναι επίσης αρκετή. Ωστόσο ως μεθοδολογία είναι η πιο συμβατή με το μαγνητικό σύστημα κίνησης και για αυτό επιλέχτηκε σε αυτό το στάδιο έναντι των υπολοίπων υποψηφίων συστημάτων.

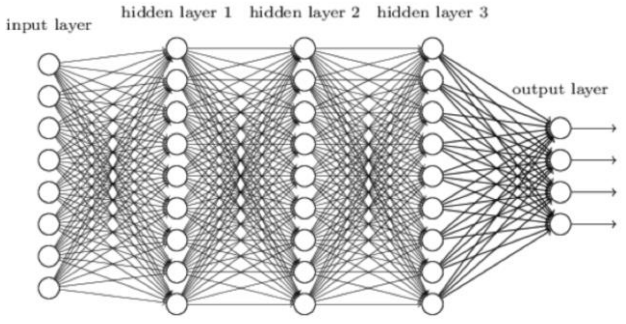
Τελευταία άμεση επιλογή για το προσχέδιο είναι πώς το σύστημα θα αποφασίζει τις κινήσεις που θα παίξει όταν είναι η σειρά του ως απάντηση στον παίκτη. Αυτό το βήμα είναι ιδιαίτερα δύσκολο διότι πρέπει να βρεθεί μια ισορροπία μεταξύ ακρίβειας και διασκέδασης, έτσι ώστε οι κινήσεις που να επιλέγονται να μην είναι πάντα τέλειες. Τέλειες κινήσεις θα καθιστούσαν το σύστημα άδικο προς τους περισσότερους παίκτες και δεν θα ήταν διασκεδαστικό να νικάει πάντα ο υπολογιστής. Αξίζει να σημειωθεί πως το σκάκι ως παιχνίδι δεν έχει «λυθεί» ώστε να γνωρίζεται ποια είναι η τέλεια κίνηση σε μία θέση, αλλά η φύση του ως παιχνίδι αναγνώρισης μοτίβων και υπολογισμού σειρών κινήσεων έχουν οδηγήσει στην διαφορά επιπέδου μεταξύ των καλύτερων υπολογιστών και ανθρώπων να είναι εξαιρετικά μεγάλο. Επιπρόσθετα η κατασκευή δεν αναφέρεται σε παγκόσμιους πρωταθλητές άρα ένας μέσος παίκτης με ELO 1500 (Elo Rating System, 2026) θα έπρεπε να μπορεί να νικήσει ή να φτάσει σε ισοπαλία με το πρόγραμμα (Chess Stats, 2026).

Η δύο κύριες προσεγγίσεις σκακιστικών αλγορίθμων που αξιολογούν μία θέση χωρίζονται στη κλασική και τη σύγχρονη.



Εικόνα 7 - Tree Search

Η κλασική αφορά αλγορίθμους που ψάχνουν κάθε δυνατή θέση και της δίνουν ένα βαθμό μέχρι ένα συγκεκριμένο «βάθος» που καθορίζει πόσο μπροστά μπορεί να δει ο αλγόριθμος. Υπάρχουν διαφορετικοί αλγόριθμοι που επιτυγχάνουν αυτή τη διαδικασία με διαφορετικούς τρόπους αλλά όλοι βασίζονται σε αυτή τη θεμελιώδη λογική. Αυτή η μέθοδος είναι γνωστή και ως Tree Search (Alpha-Beta, 2026) (Shannon, 1950).



Εικόνα 8 - Convolutional Neural Network

Η σύγχρονη πειραματική προσέγγιση είναι η χρήση ενός νευρωνικού δικτύου για την αξιολόγηση θέσης (Silver, et al., 2017). Το δίκτυο καλείται να αναθέσει μία πιθανότητα σε κάθε πιθανή κίνηση και τις οργανώνει σε μία λίστα. Πρέπει να γίνει έλεγχος νομιμότητας της κίνησης και τα αποτελέσματα εξαρτώνται από την εκπαίδευση (Neural Networks, 2026).

Και οι δύο προσεγγίσεις είναι έγκυρες αλλά λόγω προσωπικού ενδιαφέροντος επιλέχτηκε το νευρωνικό δίκτυο για το σύστημα. Ακολουθεί η έρευνα αγοράς για τα εξαρτήματα.

Έρευνα Αγοράς

Επιλογή Ηλεκτρομαγνήτη

Ο ηλεκτρομαγνήτης είναι το κύριο εξάρτημα πάνω στο οποίο βασίζεται η κίνηση των πιονιών άρα η επιλογή του είναι σημαντική. Τα χαρακτηριστικά που εξετάστηκαν ήταν:

- Η τάση λειτουργείας του ηλεκτρομαγνήτη θα καθορίσει την υψηλή τάση του συστήματος. Σε αυτό το στάδιο δεν έχουν επιλεχτεί τα υπόλοιπα εξαρτήματα άρα αυτή η τάση είναι ακόμη ευέλικτη. Για να περιέχει το κύκλωμα δύο επίπεδα τάσης, τη τάση λογικής του μικροελεγκτή, και τη υψηλή τάση λειτουργίας. Μετά την επιλογή αυτής της τάσης θα πρέπει οι κινητήρες και τα υπόλοιπα εξαρτήματα να δουλεύουν με αυτήν, για να αποφευχθεί επιπλέον κύκλωμα μετατροπής τάσης για την αντιμετώπιση τριών διαφορετικών τάσεων.

- Το ρεύμα που απαιτείται από τον ηλεκτρομαγνήτη δεν πρέπει να ξεπερνά το ασφαλές όριο μίας μεμονωμένης σύνδεσης σε breadboard, το οποίο κυμαίνεται στο 1A (APDahlen, 2024). Ανάλογα με το ρεύμα πρέπει να επιλεχτεί και ένα transistor ικανό να το διοχετεύσει, που θα επιτρέψει στον μικροελεγκτή τον έλεγχο του ηλεκτρομαγνήτη.

- Η διαθεσιμότητα είναι σημαντική καθώς από τον ηλεκτρομαγνήτη θα κριθούν οι περιοριστικοί παράγοντες του συστήματος, οι οποίοι θα βοηθήσουν στην επιλογή των υπόλοιπων εξαρτημάτων. Εφόσον τα υπόλοιπα εξαρτήματα χρειάζονται σε πιο προχωρημένα στάδια κατασκευής η γρήγορη παραλαβή τους δεν καθίσταται όσο σημαντική. Η κατασκευή ξεκίνησε από το κύκλωμα του ηλεκτρομαγνήτη.

- Η διάμετρος και το ύψος του ηλεκτρομαγνήτη καθορίζουν το μέγεθος του τετραγώνου της σκακιέρας, όπως και μερικώς το ύψος της κατασκευής.

- Η τιμή καθώς σε αυτό το στάδιο η κατασκευή βρίσκεται στα πρώτα στάδια η αγορά κάποιου ακριβού ηλεκτρομαγνήτη δεν είναι απαραίτητη.

- Το βάρος θα καθορίσει την απαιτούμενη ροπή από τα μοτέρ, καθώς είναι το βαρύτερο φορτίο που πρέπει να μετακινηθεί.

Οι ηλεκτρομαγνήτες είναι αρκετά απλά εξαρτήματα φτιαγμένα από ένα χάλκινο σύρμα που δρα ως πηνίο γύρω από ένα μεταλλικό πυρήνα. Το μαγνητικό πεδίο από το πηνίο είναι πολύ ασθενές και ο πυρήνας λειτουργεί ως πολλαπλασιαστής του ανάλογα με τη διαπερατότητα (permeability) του. (Electromagnet, 2026)

Όπου υπάρχει πηνίο χρειάζεται μία δίοδος προστασίας, για να προστατεύσει τα υπόλοιπα συνδεδεμένα εξαρτήματα από το πηνίο όταν αποφορτίζεται παγιδεύοντας το ρεύμα σε ένα κλειστό βρόγχο μέχρι να μετατραπεί σε θερμότητα. Αυτή η χρήση διόδων είναι τόσο κοινή που η δίοδος που χρησιμοποιείται για αυτό το σκοπό ονομάζεται flyback ή freewheeling δίοδος. Αυτή η ονομασία αναφέρεται στη χρήση της διόδου στο κύκλωμα και όχι στην δίοδο την ίδια, μπορεί να χρησιμοποιηθεί οποιαδήποτε δίοδος για αυτή τη λειτουργία αρκεί να μπορεί να ανταπεξέλθει στη τάση του πηνίου.

Οι υπολογισμοί σε αυτό το στάδιο είναι ανώφελοι λόγω της πληθώρας άγνωστων παραγόντων, με τον ηλεκτρομαγνήτη να είναι το πρώτο εξάρτημα που επιλέγεται, και για αυτό το λόγο εξετάστηκαν προτού υπολογισμών τα διαθέσιμα μοντέλα στην αγορά.

Οι διαθεσιμότητα διαφορετικών μοντέλων από τοπικούς πωλητές είναι πολύ περιορισμένη, με τα παρακάτω τα άμεσα διαθέσιμα μοντέλα:



Εικόνα 9 – Ηλεκτρομαγνήτης KK-P25/20

Μοντέλο: KK-P25/20
Τάση: 12V
Δύναμη συγκράτησης: 8kg
Κατανάλωση: ≈2W
Τιμή: 9.99€



Εικόνα 10 - Ηλεκτρομαγνήτης WF-P30/22

Μοντέλο: WF-P30/22
Τάση: 12V
Δύναμη συγκράτησης: 10kg
Κατανάλωση: ≈5W
Τιμή: 7.90€



Εικόνα 11 - Ηλεκτρομαγνήτης WF-P25/20

Μοντέλο: WF-P25/20
Τάση: 24V
Δύναμη συγκράτησης: 5kg
Κατανάλωση: ≈3.5W
Τιμή: 7.90€



Εικόνα 12 - Ηλεκτρομαγνήτης ZYE1-P25/20

Μοντέλο: ZYE1-P25/20
Τάση: 12V
Δύναμη συγκράτησης: 5kg
Κατανάλωση: ≈4W
Τιμή: 6.20€

Επιλέχθηκαν τα χαρακτηριστικά από τα datasheets των παραπάνω εξαρτημάτων, όπου διαθέσιμα, αντί από τις ιστοσελίδες των μεταπωλητών.

Όλοι οι υποψήφιοι ηλεκτρομαγνήτες είναι χαμηλής κατανάλωσης. Υπολογίστηκε το ρεύμα του καθενός από το νόμο του Ohm $I = \frac{P}{V}$ στο παρακάτω πίνακα:

	KK-P25/20	*WF-P30/22*	*WF-P25/20*	*ZYE1-P25/20*
I (A)	0.17	0.42	0.15	0.33

Όπως και η απόδοση ανά κιλό δύναμης συγκράτησης του καθενός διαιρώντας τη δύναμη με τη κατανάλωση:

Απόδοση (kg/W)	4	2	1.43	1.25

Πίνακας 2 - Απόδοση Ηλεκτρομαγνητών

Το μοντέλο KK-P25/20 προσφέρει σημαντικά μεγαλύτερη απόδοση, αλλά είναι το πιο ακριβό λαμβάνοντας υπόψη τις πολύ αυξημένες τιμές τοπικών πωλητών συγκριτικά με της παγκόσμιας αγοράς. Η μεγαλύτερη απόδοση και τη πανομοιότυπη φύση όλων των επιλογών οδήγησαν στην επιλογή του μοντέλου KK-P25/20 για την κατασκευή.

Η παρουσία πηνίου στο κύκλωμα απαιτεί μία δίοδο flyback για την προστασία των υπόλοιπων εξαρτημάτων κατά την αποφόρτιση του ηλεκτρομαγνήτη (Flyback diode, 2026). Τα χαρακτηριστικά που απαιτούνται να πληρεί η δίοδος είναι:

- Η αντίστροφη τάση (reverse voltage) πρέπει να υπερβαίνει την τάση λειτουργίας του κυκλώματος. Η αιχμή τάσης που παράγεται από την αποφόρτιση ενός πηνίου μπορεί να υπερβεί σημαντικά την τάση τροφοδοσίας, άρα απαιτείται ένα άνετο περιθώριο ασφαλείας πάνω από τα 12V του συστήματος.

- Το μέγιστο ρεύμα που μπορεί να ανακατευθύνει η δίοδος πρέπει να υπερβαίνει το ρεύμα λειτουργίας του ηλεκτρομαγνήτη.

- Η τιμή, καθώς πρόκειται για απλό εξάρτημα προστασίας που δεν δικαιολογείται υψηλό κόστος και οι δίοδοι είναι φθηνές και πωλούνται σε δεκάδες (bulk).

Τα είδη διόδων στην αγορά χωρίζονται σε rectifiers και schottky, εξαιρώντας τις φωτοδίοδους, zeners και TVS (transient voltage suppressors), που θα ήταν αναποτελεσματικές για αυτή τη χρήση.

Η πιο κοινή δίοδος rectifier είναι η 1N5404, η οποία προσφέρει μέγιστη αντίστροφη τάση 400V και ανταπεξέρχεται ρεύμα ως 3A. Συγκρίνοντας με τις απαιτήσεις του κυκλώματος, το ρεύμα λειτουργίας του *KK-P25/20* είναι 0.17A και η τάση τροφοδοσίας 12V, χαρακτηριστικά που μπορεί να ανταπεξέλθει η 1N5404. Αυτή η διαφορά μεταξύ πραγματικών συνθηκών και ανώτατων ορίων δρα ως συντελεστής ασφαλείας, εξασφαλίζοντας την σίγουρη προστασία χωρίς κίνδυνο της διόδου και κατ' επέκταση του υπόλοιπου κυκλώματος.



Εικόνα 13 – Δίοδος 1N5404

Να σημειωθεί πως μία δίοδος Schottky θα ήταν θεωρητικά βέλτιστη επιλογή λόγω της ταχύτερης ανταπόκρισης της και της χαμηλότερης πτώσης τάσης σε ορθή πόλωση. Ωστόσο, ο ηλεκτρομαγνήτης διακόπτεται με χαμηλή συχνότητα, εξαιτίας των καθυστερήσεων μεταξύ κινήσεων, οπότε ο χρόνος απόκρισης της 1N5404 είναι επαρκής. Λόγω της χαμηλής τιμής και της άμεσης διαθεσιμότητας της επιλέχθηκε η 1N5404 για αυτή τη χρήση.

Για τον έλεγχο του ηλεκτρομαγνήτη από τον μικροελεγκτή απαιτείται επίσης ένα transistor που θα λειτουργεί ως ηλεκτρικός αυτόματος διακόπτης. Ο μικροελεγκτής δεν μπορεί να τροφοδοτήσει απευθείας τον ηλεκτρομαγνήτη, καθώς τα pins εξόδου του περιορίζονται συνήθως σε λίγα mA, συνήθως 20-40mA, ενώ ο ηλεκτρομαγνήτης απαιτεί 0.17A στα 12V.

Με κάθε επιλογή transistor πρέπει να αποφασιστεί αν θα τοποθετηθεί να λειτουργεί ως high-side ή low-side switching, high-side μεταξύ της πηγής και του ηλεκτρομαγνήτη ενώ low-side μεταξύ του ηλεκτρομαγνήτη και της γείωσης. Αυτό είναι σημαντική επιλογή γιατί αλλάζει το είδος transistor που καθίσταται βέλτιστο για τη κάθε χρήση. Για high-side switching χρησιμοποιούνται PNP transistors ενώ για low-side NPN.

Θεωρητικά το high-side switching είναι πάντα καλύτερο από τη προοπτική πως το συνδεδεμένο κύκλωμα, με τη πτώση του διακόπτη, αποσυνδέεται πλήρως ηλεκτρικά αντί να είναι floating, δηλαδή δεν έχει ούτε τάση ούτε ρεύμα δίνοντας απόλυτο έλεγχο και αξιοπιστία στο transistor. Ο λόγος που δεν χρησιμοποιείται σε κάθε κύκλωμα είναι γιατί έρχεται με συγκεκριμένες προκλήσεις για να αξιοποιηθεί σωστά.

Η μεγαλύτερη πρόκληση είναι πως η τάση εισόδου στο gate, σε ένα MOSFET, πρέπει να είναι μεγαλύτερη από τη τάση του ερχόμενου ρεύματος της πηγής στο Drain, η οποία είναι 12V. Αυτό είναι δύσκολο για έναν μικροελεγκτή, που η τάση του κυμαίνεται μεταξύ 3.3V και 5V, και θα απαιτούσε κάποιο κύκλωμα που ανεβάζει τη τάση όπως ένα step-up converter ή/και ένα δεύτερο transistor. Επίσης τα transistors τύπου PNP είναι πιο δυσεύρετα και ακριβά, κάτι που οδήγησε στην επιλογή του low-side switching.

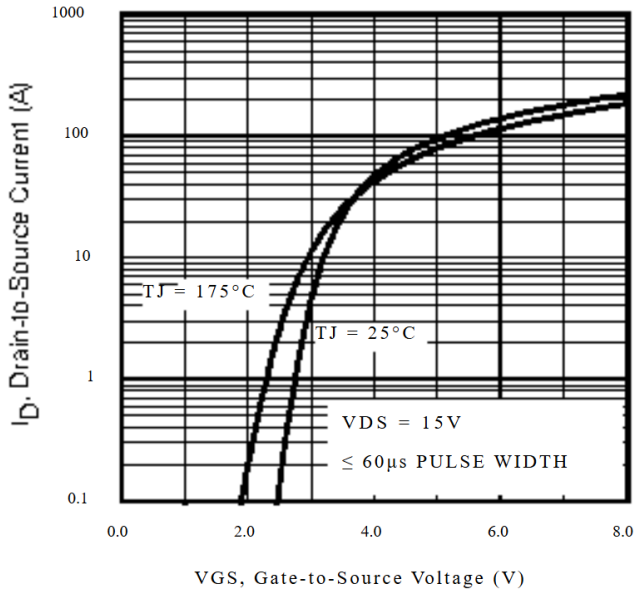
Στο low-side switching το κύριο πλεονέκτημα είναι η ευκολία σύνδεσης, το κύριο μειονέκτημα πως το κύκλωμα δεν αποσυνδέεται πλήρως, θα έχει ακόμη τάση αλλά όχι ρεύμα. Αυτό θα ήταν προβληματικό για κάποιο ευαίσθητο αισθητήριο ή υψηλής συχνότητας εξάρτημα, αλλά για έναν ηλεκτρομαγνήτη που θα ελέγχεται με χαμηλή συχνότητα, μία φορά ανά κίνηση, είναι διαχειρίσιμοι και επαρκές. Πρέπει να διαπιστωθεί από αυτό το μειονέκτημα πως απαιτείται ένα pull-down resistor, υψηλής αντίστασης αρκετών kΩ, το οποίο κλείνει τη πύλη του transistor όταν το pin του μικροελεγκτή δεν το ελέγχει ενεργά, διότι είναι floating κατά την εκκίνηση ή reset.

Η διαφορά μεταξύ BJT και MOSFET είναι πως τα BJT ελέγχονται με ρεύμα στο gate τους ενώ τα MOSFETS με τάση. Τα MOSFETS είναι πιο σύγχρονα και παρέχουν καλύτερη απόδοση και χαρακτηριστικά από όλες τις απόψεις, και είναι πιο ασφαλή για το pin του μικροελεγκτή. Τα μειονεκτήματα τους είναι το μεγάλο φυσικό μέγεθος τους και η τιμή τους, επειδή η σύνδεση θα γίνει σε breadboard και απαιτείται μόνο ένας για το κύκλωμα του ηλεκτρομαγνήτη, το φυσικό μέγεθος του δεν καθιστά περιοριστικό παράγοντα. Η τιμή παραμένει μικρή συγκριτικά των υπολοίπων εξαρτημάτων και τα πλεονεκτήματα είναι σημαντικά.

Τα transistors δεν είναι δυαδικοί διακόπτες με μόνο δύο καταστάσεις, πλήρως ενεργοποιημένο ή πλήρως απενεργοποιημένο, αλλά αναλογικά συστήματα που η τάση και το ρεύμα εξόδου εξαρτάται από το πόσο ανοιχτή είναι η πύλη τους. Όπως αναφέρθηκε προηγουμένως, πόσο ανοιχτή είναι η πύλη εξαρτάται από τη τάση της σε ένα MOSFET και το ρεύμα σε ένα BJT. Εφόσον για το παρόν σύστημα ο σκοπός είναι το transistor τύπου MOSFET να χρησιμοποιηθεί ως δυαδικός διακόπτης, πρέπει να ελεγχθεί η τάση που απαιτείται για να ανοίξει η πύλη του, η οποία αναγράφεται ως $V_{gs(th)}$, για gate to source threshold. Σε αυτή τη τάση το transistor άγει ελάχιστα και πρέπει να βρεθεί στο σχετικό γράφημα του datasheet ο λόγος $V_{gs(th)}$ προς I_{gs}, ώστε να μπορεί να περάσει αρκετό ρεύμα για την εφαρμογή, στη προκειμένη περίπτωση τουλάχιστον 0.17A. Γράφημα που συσχετίζει το $V_{gs(th)}$ με το I_{gs} δεν παρέχεται σε αυτή τη μορφή αλλά σε γράφημα $V_{gs} - I_{gs}$ που παρέχει τις ίδιες πληροφορίες.

Για την απλοποίηση των υπολογισμών, επειδή ανάλογα με τη θερμοκρασία και την εσωτερική αντίσταση αλλάζουν, πωλούνται transistors συγκεκριμένα με logic-level gates τα οποία είναι φτιαγμένα για τάσεις λειτουργίας μεταξύ 3.3V και 5V.

Ένα logic-level MOSFET είναι το μοντέλο *IRLB8721PBF* που έχει ανώτατο όριο τάσης 30V, ρεύμα 44A έως 62A ανάλογα με τη θερμοκρασία και εξαιρετικά μικρή αντίσταση $R_{ds(on)}$ στα 16mΩ για V_{gs} 4.5V και I_d 25A (IRLB8721PbF Datasheet, 2026). Πριν την αγορά του εξετάστηκε το γράφημα $V_{gs} - I_{gs}$ ως η τελευταία εγγύηση πως άγει το απαιτούμενο ρεύμα στη τάση των 3.3V, που είναι η μικρότερη που χρησιμοποιούν οι κοινοί μικροελεγκτές:



Εικόνα 14 - Διάγραμμα Επίδοσης Transistor

Από το γράφημα διαπιστώνεται πως σε τάση των 3.3V μπορεί να διοχετεύσει αρκετά περισσότερο ρεύμα από το απαιτούμενο των 0.14A ανεξαρτήτως θερμοκρασίας και επικυρώνεται η λειτουργία του ως logic-level transistor με το $V_{gs(th)}$ στα 1.8V και ρεύμα μεταξύ 10-80A σε τάσεις 3.3-5V στη πύλη που υπερέχει των συνηθισμένων εφαρμογών.

Με αυτό επιλέχτηκε το μοντέλο *IRLB8721PBF* ως το transistor για το κύκλωμα του ηλεκτρομαγνήτη.



Εικόνα 15 - MOSFET Transistor

Με το μοντέλο επιλεγμένο πρέπει να αποφασιστεί η αντίσταση στο gate, διότι παρόλο που τα MOSFETS λειτουργούν με τάση όταν ανοίγει η πύλη συμπεριφέρεται ως πυκνωτής, ο οποίος απαιτεί αρκετό ρεύμα μέχρι να είναι πλήρη φορτισμένος. Αυτή η αιχμή ρεύματος συμβαίνει κάθε φορά που το MOSFET ανοίγει και αντιμετωπίζεται με μία αντίσταση που μειώνει το ρεύμα που δέχεται προστατεύοντας το pin του μικροελεγκτή αλλά κάνοντας το transistor να ανοίγει πλήρως πιο αργά.

Επειδή το transistor είναι τύπου MOSFET και η πύλη είναι πλήρως απομονωμένη από το drain αρκεί ο υπολογισμός με τον τύπο του ohm για την προσέγγιση μίας ασφαλής τιμής αντίστασης:

$$R = \frac{V}{I} = \frac{5V}{0.02A} = 250\Omega$$

Επιλέχτηκε η τάση στα 5V για τη «χειρότερη περίπτωση» στην οποία απαιτείται η μεγαλύτερη αντίσταση, τουλάχιστον 250Ω. Οποιαδήποτε τιμή αντίστασης πάνω από την υπολογισμένη είναι κατάλληλη, αφού το ρεύμα στο drain-source εξαρτάται από τη τάση. Με το ρεύμα στη πύλη καθορίζεται μόνο η ταχύτητα φόρτισης. Λόγω διαθεσιμότητας επιλέχτηκε η ασφαλής τιμή αντίστασης των 440Ω, η οποία μειώνει τη ταχύτητα φόρτισης, αλλά παραμένει εξαιρετικά γρήγορη συγκριτικά με το ρυθμό φόρτισης και αποφόρτισης του πηνίου του ηλεκτρομαγνήτη.

Σε αργότερο στάδιο της κατασκευής, λόγω μικρότερης έλξης από αναμενόμενης μελετήθηκε περαιτέρω πως προκύπτει η αναγραφόμενη δύναμη των 8kg. Η ελκτική δύναμη ενός ηλεκτρομαγνήτη πάνω σε ένα μαγνητικό αντικείμενο σε απόσταση g καθορίζεται από τον ακόλουθο τύπο της μαγνητικής έλξης, να σημειωθεί πως ο τύπος λήφθηκε από το LLM Gemini της Google καθώς καμία πηγή δεν περιέχει ακριβώς αυτή τη μορφή που συνδυάζει το νόμο του Ampère με τις εξισώσεις του Maxwell, δύο νόμους που δεν κατανοήθηκαν άρα η εξίσωση είναι ενδεικτική χωρίς εγγυημένη επιστημονική ακρίβεια:

$$F = \frac{(N * I)^2 * \mu_0 * A}{2 * (L_g + \frac{L_{core}}{\mu_r})^2}$$

Όπου *F* η ελκτική δύναμη σε Newtons, *I* το ρεύμα σε Αμπέρ, N ο αριθμός στροφών του σύρματος γύρω από τον πυρήνα το οποίο είναι αδιάστατο νούμερο, μ_0 η μαγνητική διαπερατότητα του κενού που είναι η σταθερά 1.25663706127(20)*10^{-6} σε N*A^{-2}, A το εμβαδόν διατομής πόλου του ηλεκτρομαγνήτη σε μέτρα, L_g το μήκος διακένου σε m. L_{core} το μήκος του πυρήνα σε m, μ_r η σχετική διαπερατότητα του πυρήνα που είναι αδιάστατο νούμερο .

Όσο μεγαλύτερος ο αριθμητής τόσο μεγαλύτερη η μαγνητική έλξη και αντίθετα όσο μεγαλύτερος ο παρονομαστής τόσο μικρότερη. Το L_{core} του επιλεγμένου μοντέλου *KK-P25/20* είναι 0.02m και το μ_r εξαρτάται από το υλικό του πυρήνα. Συνήθως χρησιμοποιείται μαλακός σίδηρος ή χάλυβας χωρίς νικέλιο, υλικά που το μ_rτείνει μεταξύ 500-4,000 κάνοντας το κλάσμα $\frac{L_{core}}{\mu_r}$ στον παρονομαστή πάρα πολύ μικρό που σημαίνει πως δεν ευθύνεται σημαντικά για τη μείωση της δύναμης.

Η πιο σημαντική παράμετρος είναι το διάκενο L_g που δείχνει πως η δύναμη μειώνεται στο τετράγωνο της απόστασης. Επειδή στη τελική κατασκευή μεταξύ του ηλεκτρομαγνήτη και του μαγνήτη νεοδυμίου τοποθετήθηκε το πλέγμα μαγνητικών διακοπτών, η δύναμη μειώθηκε εκθετικά. Επίσης οι τρύπες μέσα στα πιόνια πρέπει να ελαχιστοποιούν το ύψος του μαγνήτη, δίνοντας προτεραιότητα στην ελαχιστοποίηση αυτής της απόστασης συγκριτικά με το βάρος ολόκληρου του πιονιού.

Στην πρώτη προσπάθεια η τρύπες στα πιόνια ήταν 5.17mm βαθιές για να αφεθεί χώρος στο μαγνήτη ύψους 4.8mm ώστε η επιφάνεια του πιονιού να έρχεται σε επαφή με τη σκακιέρα. Αυτό είχε ως αποτέλεσμα ο μαγνήτης να μην πατάει στη σκακιέρα αλλά να ασκείται η ελκτική δύναμη μέσω του πιονιού, κάτι που σημαίνει αύξηση της τριβής και μειωμένη απόδοση. Το δεύτερο αποτέλεσμα αυτού, ήταν πως η διαφορά 5.17-4.8=0.37mm επέφερε μείωση μεγαλύτερης της αναμενόμενης. Όπως προκύπτει από τον παραπάνω τύπο, αφού η δύναμη είναι αντιστρόφως ανάλογης του τετραγώνου της απόστασης, τότε ο λόγος που δίνει τη διαφορά των δύο είναι:

$$\frac{F_1}{F_2} = \frac{L_{g2}^2}{L_{g1}^2} = \frac{(5.17)^2}{(4.8)^2} \approx 1.16$$

Αυτός ο λόγος σημαίνει πως η διαφορά των δύο θέσεων, μία διαφορά 0.37 χιλιοστών, μειώνει τη μαγνητική δύναμη κατά 16%, ένα ποσοστό μεγαλύτερο από το αναμενόμενο σε μία τόσο μικρή αλλαγή. Αποτέλεσμα της διαπίστωσης ήταν η ελαχιστοποίηση του διάκενου L_g έναντι της αγοράς δυνατότερου ηλεκτρομαγνήτη.

Επιλογή Βηματικού Κινητήρα

Η επιλογή των κατάλληλων κινητήρων για το σύστημα κίνησης του ηλεκτρομαγνήτη είναι σημαντική καθώς από αυτούς εξαρτάται η ακρίβεια και η ταχύτητα των φυσικών κινήσεων. Τα χαρακτηριστικά που πρέπει να εξεταστούν είναι τα εξής:

- Η ροπή πρέπει να είναι αρκετή για να υπερνικήσει της τριβής των ιμάντων και των τροχών στις δοκούς. Οι μαγνήτες επίσης παρουσιάζουν μία μικρή τριβή όταν σέρνονται από τον ηλεκτρομαγνήτη πάνω στην επιφάνεια.

- Το φυσικό μέγεθος και το ύψος θα καθορίσει πόσο ψηλή είναι η κατασκευή, καθώς οι ιμάντες πρέπει να ευθυγραμμιστούν με τον άξονα περιστροφής του βηματικού κινητήρα. Το πιο συνηθισμένο μέγεθος σε 3D εκτυπωτές με συστήματα κίνησης CoreXY είναι το NEMA-17 (42x42mm) (Stepper Motor Selection Guide for 3D Printers: NEMA Size, Torque & Precision Explained, 2026).

- Το ρεύμα που καταναλώνεται πρέπει να είναι όσο το δυνατόν λιγότερο, για να μπορεί να το διοχετεύσει ένα breadboard χωρίς να λιώσει. Το προτεινόμενο όριο σε μία σύνδεση στο breadboard με εξαίρεση τις ράγες είναι ≈1A, μετά από αυτή την ένταση αρχίσει πτώση της τάσης (APDahlen, 2024).

- Η ονομαστική τάση, αν είναι υψηλότερη της τάσης 12V του ηλεκτρομαγνήτη, θα πρέπει να συμπεριληφθεί ένα επιπλέον κύκλωμα που να χαμηλώνει τη τάση. Για παράδειγμα, αν ο βηματικός κινητήρας για να λειτουργήσει απαιτούταν τροφοδοσία 24V, θα έπρεπε η τάση να μετατραπεί σε 12V για τον ηλεκτρομαγνήτη και σε 3.3V ή 5V για τον μικροελεγκτή. Για την αποφυγή τριών διαφορετικών τάσεων στο σύστημα η ιδεατή τάση τροφοδοσίας είναι τα 12V. Ονομαστική τάση (nominal voltage) είναι η τάση που απαιτεί ο ίδιος ο βηματικός κινητήρας και τη διαχειρίζεται το driver το οποίο τροφοδοτείται με 12V ή 24V. Προκύπτει από το νόμο του Ohm πολλαπλασιάζοντας το ρεύμα ανά φάση με την αντίσταση των πηνίων ανά φάση.

- Η ακρίβεια της γωνίας του κάθε βήματος καθορίζει τη μικρότερη απόσταση που μπορεί να διανύσει ο ηλεκτρομαγνήτης. Δεν απαιτείται η ίδια ακρίβεια με έναν 3D εκτυπωτή ή plotter, καθώς η μικρότερη κίνηση είναι ίση με την απόσταση από το κέντρο ενός τετραγώνου ως το κέντρο ενός άλλου, ή 3.7cm για μια σκακιέρα με διαστάσεις 30x30cm. Ανάλογα με το driver που επιλεχθεί, η ακρίβεια θα γίνει μεγαλύτερη χάρη στο microstepping που παρέχει. Οι περισσότεροι βηματικοί κινητήρες έχουν ακρίβεια 200 steps ανά πλήρη περιστροφή ή 1.8° ανά βήμα. Για τους παραπάνω λόγους η ακρίβεια είναι χαμηλής προτεραιότητας σε αυτό το σύστημα.

- Η τιμή, διότι το σύστημα κίνησης απαιτεί δύο βηματικούς κινητήρες που τείνουν να είναι ακριβοί συγκριτικά με τα υπόλοιπα εξαρτήματα της συνολικής κατασκευής, ενδέχεται να το καθιστά το πιο ακριβό μεμονωμένο εξάρτημα της κατασκευής.

Γνωρίζοντας τα χαρακτηριστικά που καθίστανται σημαντικά για το σύστημα, ακολουθεί η έρευνα αγοράς, αρχίζοντας από τοπικούς πωλητές. Οι υποψήφιοι βηματικοί κινητήρες που εξετάστηκαν ήταν οι εξής:



Εικόνα 16 – Βηματικός Κινητήρας 42BYGHW208

Stepper Motor 42BYGHW208
Ονομαστική τάση: 12V
Ρεύμα: 0.4A/φάση
Ροπή: 31N*cm (3100g*cm)
Μέγεθος: NEMA-17
Ακρίβεια: 1.8°/βήμα (200steps/περιστροφή)
Τιμή: 12.60€



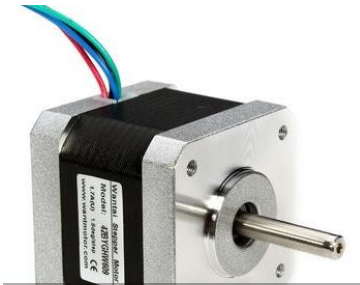
Εικόνα 20 - Βηματικός Κινητήρας Creality 42-34

Creality Stepper Motor 42-34 L20 Round Shaft
Ονομαστική Τάση: 4.83V
Ρεύμα: 0.84A/φάση
Ροπή: 40N*cm (4000g*cm)
Μέγεθος: NEMA-17
Ακρίβεια: 1.8°/βήμα (200steps/περιστροφή)
Τιμή: 14.80€



Εικόνα 19 - Βηματικός Κινητήρας 42BYGHW804

Stepper Motor 42BYGHW804
Ονομαστική Τάση: 3.6V
Ρεύμα: 1.2A/φάση
Ροπή: 48N*cm (4800g*cm)
Μέγεθος: NEMA-17
Ακρίβεια: 1.8°/βήμα (200steps/περιστροφή)
Τιμή: 15.90€



Εικόνα 18 - Βηματικός Κινητήρας 42BYGHM809

Stepper Motor 42BYGHM809
Ονομαστική Τάση: 2.77V
Ρεύμα: 1.68A/φάση
Ροπή: 42N*cm (4200g*cm)
Μέγεθος: NEMA-17
Ακρίβεια: 0.9°/βήμα (400steps/περιστροφή)
Τιμή: 16.90€



Εικόνα 17 - Βηματικός Κινητήρας OK42STH22-0804AC

Stepper short fuselage motor OK42STH22-0804AC
Ονομαστική Τάση: 3V
Ρεύμα: 0.8A/φάση
Ροπή: 9N*cm (900g*cm)
Μέγεθος: NEMA-17
Ακρίβεια: 1.8°/βήμα (200steps/περιστροφή)
Τιμή: 11.16€

Επιλέχθηκαν τα χαρακτηριστικά από τα datasheets των παραπάνω εξαρτημάτων, όπου διαθέσιμα, αντί από τις ιστοσελίδες των μεταπωλητών.

Έχοντας μερικά διαθέσιμα μοντέλα, ακολουθεί η προσέγγιση της απαιτούμενης ροπής και η σύγκριση των χαρακτηριστικών τους. Η ροπή υπολογίζεται από τον ακόλουθο μαθηματικό τύπο θεωρώντας πως η δύναμη είναι κάθετη στην απόσταση:

$$\tau = F * r \iff \tau = m * a * r$$

Όπου τ το μέτρο της ροπής σε N*m στο S.I., *F* το μέτρο της δύναμης σε N, r το μήκος του διανύσματος μεταξύ του σημείο ως προς το οποίο μετριέται η ροπή ως το σημείο εφαρμογής της δύναμης σε μέτρα. Η δύναμη *F* ισούται με μάζα επί επιτάχυνση από το δεύτερο νόμο του Νεύτωνα, όπου η μάζα *m* μετριέται σε kg και η επιτάχυνση σε m/s^2. (Torque, 2026)

Ο επιλεγμένος ηλεκτρομαγνήτης, μοντέλο KK-P25/20, ζυγίζει 56g και η βάση του θα τυπωθεί με PLA το οποίο είναι εξαιρετικά ελαφρύ. Για να γίνει ο υπολογισμός επιλέχτηκε η θεωρητική τιμή της «χειρότερης περίπτωσης» των 25g. Άρα η μάζα που πρέπει να κινηθεί είναι $56\text{g} + 25\text{g} = 81\text{g} = 0.081kg$ αγνοώντας το βάρος από τον ιμάντα και της δοκού του άξονα X λόγω δυσκολίας προσέγγισης τους. Στο τελικό υπολογισμό θα προστεθεί ένας μεγάλος συντελεστής ασφαλείας εξαιτίας αυτής της απλοποίησης.

Ένας 3D εκτυπωτής με σύστημα κίνησης CoreXY έχει τη δυνατότητα να επιταχύνει μεταξύ 10,000-20,000mm/s² (MadisonJames, 2025). Επιλέχθηκε η ενδιάμεση τιμή των 15,000mm/s² για μία μέση ζητούμενη επιτάχυνση στη προσέγγιση. Στο S.I. για τον υπολογισμό της ροπής μετατρέπεται σε 15m/s². Αυτή η τιμή είναι σημαντικά υψηλότερη της ταχύτητας μιας αυτόματης σκακιέρας, η οποία περιορίζεται λόγω της μαγνητικής δύναμης μεταξύ του ηλεκτρομαγνήτη και του πιονιού. Επιλέχτηκε ως ένας δεύτερος συντελεστής ασφαλείας.

Η ακτίνα, στη περίπτωση του συστήματος κίνησης CoreXY, αναφέρεται στον «κύκλο» που σχηματίζει ο ιμάντας γύρω από τον άξονα περιστροφής του κινητήρα. Οπότε πρέπει να υπολογιστεί πόση απόσταση διανύει ο ιμάντας σε μία πλήρη περιστροφή του μοτέρ.

Σε αυτό το μέρος της έρευνας πρέπει να αποφασιστεί το είδος του ιμάντα για αυτόν τον υπολογισμό. Ο πιο διαδεδομένος ιμάντας στην αγορά και ο πιο κοινός σε υπάρχοντα συστήματα CoreXY είναι ο ιμάντας χρονισμού GT2. Αυτός ο ιμάντας έχει απόσταση μεταξύ των δοντιών του (Pitch) 2mm (GT2 Timing Belt Datasheet, 2026).



Εικόνα 21 - Τροχαλία 20T

Πόσα δόντια του ιμάντα θα μετατοπιστούν ανά περιστροφή του κινητήρα εξαρτάται από τον αριθμό των δοντιών της τροχαλίας (Pulley) που θα τοποθετηθεί στον άξονα του ιμάντα. Οι δύο πιο συνηθισμένες επιλογές είναι τροχαλίες 16T, 16-δοντιών, και 20T, 20-δοντιών. Περισσότερα δόντια στη τροχαλία αναλογούν σε μεγαλύτερη ταχύτητα περιστροφής, επειδή περισσότερα δόντια του ιμάντα θα μετατοπιστούν έναντι μιας τροχαλίας με λιγότερα δόντια. Ωστόσο αυτό έχει ως αποτέλεσμα να αυξηθεί η απαιτούμενη ροπή λόγω αύξησης της ακτίνας.

Για τον υπολογισμό της απόστασης που θα μετατοπιστεί ο ιμάντας σε μία πλήρη περιστροφή του μοτέρ υπολογίζεται η περίμετρος του «κύκλου». Με άλλα λόγια, αν η τροχαλία 20-δοντιών κάνει μία πλήρη περιστροφή πόσα δόντια του ιμάντα θα μετατοπιστούν.

Γνωρίζοντας πως τα δόντια του ιμάντα απέχουν 2mm μεταξύ τους και υποθέτοντας επιλεγμένη τροχαλία με 20 δόντια η απόσταση που θα μετατοπιστεί δίνεται από τον τύπο:

$$C = N * P \Rightarrow C = 20\ \delta\acute{o}ντια * 2\frac{mm}{\delta\acute{o}ντι} = 40mm\ \alpha\nu\acute{\alpha}\ \pi\lambda\acute{\eta}ρη\ περιστροφ\acute{\eta}$$

Όπου *N* είναι ο αριθμός των δοντιών στη τροχαλία και *P* η απόσταση μεταξύ των δοντιών του ιμάντα.

Η περίμετρος του κύκλου δίνεται επίσης από τον τύπο $C = 2\pi * r$, όπου C η περίμετρος του κύκλου (Circumference) σε mm, π ο λόγος της περιμέτρου προς τη διάμετρο που είναι σταθερά με άπειρα δεκαδικά ψηφία 3.14159… και r η ακτίνα του κύκλου σε mm.

Φέρνοντας τον τύπο σε διαφορετική μορφή μπορεί να βρεθεί η ακτίνα για τον υπολογισμό της ροπής, εφόσον η περίμετρος είναι γνωστή από τον προηγούμενο υπολογισμό:

$$C = 2\pi * r \Leftrightarrow r = \frac{C}{2\pi} \Longrightarrow r = \frac{40mm}{2 * 3.14} \simeq 6.37mm = 0.00637m$$

Στον τύπο της ροπής:

$$\tau = m * a * r \Longrightarrow \tau = 0.081kg * 15\ m \cdot s^2 * 0.00637m = 0.00773955\ N \cdot m$$

Σε N*cm είναι $0.00773955\ N \cdot m * 100 = 0.773955\ N \cdot cm$, για τροχαλία με 20 δόντια.

Με την ίδια διαδικασία μπορεί να υπολογιστεί η ροπή για τροχαλία με 16 δόντια και να συγκριθούν.

$$C = N * P \Rightarrow C = 16\ \delta\acute{o}\nu\tau\iota\alpha * 2\frac{mm}{\delta\acute{o}\nu\tau\iota} = 32mm\ \alpha\nu\acute{\alpha}\ \pi\lambda\acute{\eta}\rho\eta\ \pi\varepsilon\rho\iota\sigma\tau\rho o\phi\acute{\eta}$$

$$r = \frac{C}{2\pi} \Longrightarrow r = \frac{32mm}{2 * 3.14} \simeq 5.1mm = 0.0051m$$

$$\tau = m * a * r \Longrightarrow \tau = 0.081kg * 15\ ms^2 * 0.0051m = 0.0061965N \cdot m = 0.61965\ N \cdot cm$$

Συγκρίνοντας τις δύο τροχαλίες βγαίνουν τα ακόλουθα ποσοστά:

	Τροχαλία 16T	Τροχαλία 20T	Απόδοση 20T/16T
Απόσταση/περιστροφή	32mm	40mm	+25% ταχύτητα
Ακτίνα βήματος	≈5.1mm	≈6.37mm	+25% ακτίνα
Απαιτούμενη ροπή	0.62 N*cm	0.77 N*cm	+24% απαιτούμενη ροπή

Διαιρώντας την απόσταση ανά περιστροφή με την συνηθισμένη ανάλυση των κινητήρων 1.8°/βήμα (200steps/περιστροφή), συγκρίνεται και η ακρίβεια

Ακρίβεια	0.16mm/βήμα	0.20mm/βήμα	-25% ακρίβεια

Πίνακας 3 - Σύγκριση Τροχαλίων

Εξαιτίας της απλουστευμένης προσέγγισης της ροπής προστίθεται ένας μεγάλος συντελεστής ασφαλείας ο οποίος επιλέχτηκε +50% στην απαιτούμενη ροπή. Άρα για τροχαλία 16T η απαιτούμενη ροπή είναι 0.93 N*cm, ενώ για τροχαλία 20T είναι ≃1.16 N*cm.

Πολλά εξαρτήματα, όπως οι βηματικοί κινητήρες και τα drivers τους είναι αλληλένδετα, που σημαίνει πως η επιλογή του ενός καθορίζει τη βέλτιστη επιλογή του άλλου. Δεν μπορούν να γίνουν ταυτόχρονα δύο έρευνες, όμως μία πρόχειρη μελέτη των drivers μπορεί να βοηθήσει στην επιλογή και του κινητήρα. Έτσι μελετήθηκε το microstepping προτού επιλεχτεί driver.

Ο φθηνότερος βηματικός κινητήρας, το μοντέλο *OK42STH22-0804AC,* φαίνεται καλή επιλογή σε χαρακτηριστικά, αλλά η χαμηλή ροπή του των 9N*cm συγκριτικά με τα υπόλοιπα μοτέρ απαιτεί έναν υπολογισμό για τη μείωση της ροπής από το microstepping, το οποίο μειώνει τον θόρυβο και τις δονήσεις του κινητήρα, αλλά με κόστος τη μείωση της ροπής. Η μείωση του θορύβου σε ένα αυτοματοποιημένο επιτραπέζιο παιχνίδι είναι ιδιαίτερα σημαντική για τους παίκτες. Η ποσοστιαία εναπομένουσα ροπή από το microstepping δίνεται από τον παρακάτω πίνακα (Chang & Tran, 2025):

Διαιρέτης βημάτων	*Ροπή*
1	100.00%
2	70.709%
4	38.267%
8	17.508%
16	9.801%
32	4.907%
64	2.454%
128	1.227%
256	0.614%

Πίνακας 4 - Ροπή ανά διαιρέτη βημάτων

Από τον πίνακα διαπιστώνεται πως το microstepping μειώνει απότομα την ροπή. Οι περισσότεροι drivers στην αγορά προσφέρουν ως 1/32 ή 1/256 microstepping *(Stepper Motor Drivers, 2026)*. Με απαιτούμενη ροπή ≃1.16 N*cm, το μέγιστο microstepping που μπορεί να χρησιμοποιηθεί, για τροχαλία 20T, είναι 1/8 με ≃1.58 N*cm, το 17.508% της αρχικής των 9 N*cm. Με 1/16 microstepping η ροπή μειώνεται σε 0.88 N*cm, λιγότερο της απαιτούμενης υπολογισμένης. Με τροχαλία 16T δεν μπορεί να χρησιμοποιηθεί 1/32 microstepping, καθώς μειώνει τη ροπή σε 0.44 N*cm έναντι της απαιτούμενης 0.93 N*cm. Αυτό σημαίνει πως δεν μπορεί να αξιοποιηθεί η μέγιστη μείωση θορύβου από τους περισσότερους drivers, για το συγκεκριμένο μοντέλο, κάτι που οδήγησε στην απόρριψη του.

Οι υπόλοιποι κινητήρες έχουν πολύ περισσότερη ροπή από την απαιτούμενη και η ακρίβεια τους είναι μεγαλύτερη της ζητούμενης για μία αυτόματη σκακιέρα. Για αυτό είναι δυνατή η χρήση της τροχαλίας 20T στο σύστημα, για μεγαλύτερη ταχύτητα ανά περιστροφή εφόσον πληρούνται οι παραπάνω προϋποθέσεις.

Για να επιλεχτεί ένας από τους υπόλοιπους κινητήρες εξετάζονται επίσης ως προς το μέγιστο microstepping που μπορεί να αξιοποιηθεί, με πορτοκαλί οι τιμές μικρότερες των 1.16 N*cm και κόκκινο οι τιμές μικρότερες των 0.93 N*cm:

Διαιρέτης βημάτων	*Ροπή*	*42BYGHW208 (31 N*cm)*	*Creality 42-34 (40 N*cm)*	*42BYGHW804 (48 N*cm)*	*42BYGHM809 (42 N*cm)*
16	9.801%	3.03	3.92	4.70	4.11
32	4.907%	1.52	1.96	2.35	2.06
64	2.454%	0.76	0.98	1.18	1.03
128	1.227%	0.38	0.49	0.59	0.51
256	0.614%	0.19	0.24	0.29	0.25

Πίνακας 5 - Σύγκιση Βηματικών Κινητήρων προς Microstepping

Το συμπέρασμα από αυτές τις τιμές είναι πως δεν μπορεί να αξιοποιηθεί ένα driver με 1/128 ή 1/256 microstepping, και μερικώς με 1/64 microstepping. Η ασφαλέστερη τιμή είναι αυτή των 1/32 microstepping, που συμπίπτει με τη μέγιστη δυνατότητα διαίρεσης βημάτων πολλών drivers στην αγορά. Επίσης αυτή η τιμή επιτρέπει τη χρήση της τροχαλίας 20-δοντιών, με τα πλεονεκτήματα που υπολογίστηκαν προηγουμένως.

Για να επιλεχτεί ένας από τους υποψήφιους κινητήρες εξετάστηκε η ροπή που προσφέρουν συγκριτικά με την κατανάλωση τους. Η κατανάλωση σε βατ (Watts) δίνεται από τον τύπο:

$$P = V_{nominal} * I_{phase}$$

Όπου P η ηλεκτρική ισχύς σε βατ, $V_{nominal}$ ή V_{rated} η ονομαστική τάση του κινητήρα και I_{phase} η ένταση ανά φάση.

Είναι σημαντικό πως η ονομαστική τάση αναφέρεται στην τάση του ίδιου του κινητήρα εσωτερικά και όχι στη τροφοδοσία του ανάλογου driver. Τα drivers τροφοδοτούνται με υψηλότερη τάση από τους βηματικούς κινητήρες, συνήθως μεταξύ 12V και 48V, ενώ η κινητήρες μπορεί να έχουν πολύ μικρότερη ονομαστική τάση, όπως 3V στη περίπτωση του OK42STH22-0804AC.

Διαιρώντας την ροπή με την ισχύ, υπολογίζεται η απόδοση του κάθε κινητήρα ως προς τη ροπή. Με άλλα λόγια, πόση ροπή προσφέρει ανά βατ.

Τα παρακάτω είναι τα αποτελέσματα των πράξεων:

	42BYGHW208	*Creality 42-34*	*42BYGHW804*	*42BYGHM809*
Ροπή (N*cm)	31	40	48	42
Ισχύς (Watts/Φάση)	4.8	4.06	4.32	4.65
Απόδοση	6.45	9.85	11.11	9.03

Πίνακας 6 - Σύγκριση Απόδοσης Βηματικών Κινητήρων

Το μοντέλο 42BYGHW804 δίνει τη μεγαλύτερη απόδοση για τη ροπή του. Το μοντέλο 42BYGHW208 καταναλώνει τη περισσότερη ενέργεια λόγω της υψηλής ονομαστικής τάσης του. Εφόσον το ρεύμα/φάση που καταναλώνει είναι τόσο μικρό στα 0.4A/φάση, αυτό σημαίνει πως η εσωτερική αντίσταση του είναι υψηλή.

Αυτοί οι υπολογισμοί αναφέρονται στην απόδοση όταν ο κινητήρας είναι σταματημένος, αφού οι βηματικοί κινητήρες καταναλώνουν τη πλήρη ενέργεια τους ακόμη και όταν δεν κινούνται για να κρατήσουν σταθερή τη περιστροφή τους. Εξαιτίας αυτής της συμπεριφοράς, η περισσότερη θερμότητα παράγεται όταν οι κινητήρες δεν περιστρέφονται, αλλά είναι απαραίτητη ώστε το σύστημα κίνησης να μην χάσει τη θέση του.

Σε αυτό το σημείο της έρευνας διαπιστώθηκε πως είτε μεταξύ του κινητήρα και του driver, είτε μεταξύ του driver και της πηγής, θα μεσολαβεί κάποια σύνδεση σε breadboard. Σε τέτοιες περιπτώσεις η χρήση των γραμμών τροφοδοσίας του breadboard είναι καλύτερη των μεμονωμένων συνδέσεων, επειδή η μεγαλύτερη επιφάνειά τους μπορεί να διαχύσει περισσότερη θερμότητα άρα και ρεύμα. Παχύτερα καλώδια ή πολλαπλά καλώδια συνδεδεμένα παράλληλα μπορούν να βοηθήσουν με αυτό το όριο των ≈1A. Η συνδέσεις θα μπορούσαν να γίνουν απευθείας στα pins του driver και η τροφοδοσία του driver στη πηγή, αλλά χρειάζεται ένας πυκνωτής στη τροφοδοσία του driver καθιστώντας τη λύση δυσκολότερη. Ένα perfboard λόγω του πλήθους συνδέσεων και την κοντινή απόσταση μεταξύ τους, παρουσιάζει μία δυσκολία κατασκευής. Τέλος, ένα PCB πρέπει να σχεδιαστεί προσεκτικά και είναι χρηματικά ακριβό, άρα δεν είναι η πρώτη λύση προτού εξέτασης των διαφορετικών λύσεων.

Εξαιτίας αυτής της διαπίστωσης αναθεωρήθηκε το καταλληλότερο μοντέλο με βάση την ασφάλεια του για ένα breadboard. Οι κινητήρες είναι διφασικοί, που σημαίνει πως τα πηνία τους θεωρητικά μπορούν να καταναλώσουν διπλάσιο του ρεύματος φάσης τους. Αυτό στη πράξη δεν συμβαίνει, διότι τα πηνία φορτίζονται διαδοχικά για τη κίνηση του εσωτερικού μαγνήτη. Το πόσο φορτίζεται η κάθε ομάδα πηνίων εξαρτάται από το επιλεγμένο microstepping με την πιο απλή περίπτωση το full stepping, κατά το οποίο μόνο μία ομάδα πηνίων είναι φορτισμένη σε μία χρονική στιγμή. Το microstepping, ανάλογα με τη διαίρεση βημάτων, ο διαχωρισμός του ρεύματος στις δύο ομάδες πηνίων ακολουθεί μία ημιτονοειδή συνάρτηση (Chang & Tran, 2025). Χρησιμοποιώντας τη θεωρητική περίπτωση, κατά την οποία όλα τα πηνία είναι φορτισμένα, δημιουργείται ένας συντελεστής ασφαλείας στους υπολογισμούς.

Οι συνδέσεις που απαιτούν προσοχή είναι μεταξύ του driver και των δύο ομάδων πηνίων, όπως και μεταξύ της πηγής και της τροφοδοσίας του driver. Λόγω του microstepping που θα χρησιμοποιηθεί το ρεύμα χωρίζεται μεταξύ των δύο ομάδων πηνίων κάνοντας τον υπολογισμό του δυσκολότερο. Για αυτό εξετάζεται αρχικά το ρεύμα στη τροφοδοσία του driver το οποίο χωρίζεται εσωτερικά στις δύο ομάδες πηνίων. Λόγω αυτής της μετατροπής τα drivers τροφοδοτούνται με υψηλότερη τάση αυτής της ονομαστικής τάσης του κινητήρα τους. Έτσι μετριάζεται το ρεύμα καθώς είναι σε μεγαλύτερη τάση και το driver λειτουργεί παρόμοια με έναν μετατροπέα υποβιβασμού τάσης (buck converter). Για παράδειγμα σε ένα κινητήρα με ονομαστική τάση 3V το ανάλογο driver του θα τροφοδοτηθεί με 12V, με αποτέλεσμα στη τροφοδοσία του να διοχετεύεται 4 φορές λιγότερο ρεύμα συγκριτικά με αν τροφοδοτούταν με 3V, αγνοώντας απώλειες μετατροπής.

Αυτή η συμπεριφορά προστατεύει το breadboard και τα καλώδια, διότι το συναθροισμένο ρεύμα από τις ομάδες των πηνίων δεν επιβαρύνει καμία μεμονωμένη σύνδεση. Ωστόσο, η συνολική θερμότητα που παράγεται στην είσοδο του driver είναι ελαφρώς μεγαλύτερη από τη συναθροισμένη των ομάδων πηνίων, λόγω απωλειών στην αποδοτικότητα μετατροπής τάσης. Ανάλογα με τη διαφορά μεταξύ ονομαστικής τάσης του μοτέρ και τροφοδοσίας του driver υπάρχει καταπόνηση λόγω της χρήσης PWM για τη πτώση της τάσης. Το μεγαλύτερο

ρεύμα που παράγεται από αυτή τη διεργασία παράγει θερμότητα εσωτερικά στα transistors του driver, που είναι ο λόγος γιατί αγοράζονται μαζί με ψύκτρα (heat sink) για να ανταλλάσσουν ταχύτερα θερμότητα με τον αέρα. Το πλεονέκτημα ενός κινητήρα που λειτουργεί με αρκετό ρεύμα και χαμηλή ονομαστική τάση, είναι πως η εσωτερική του αντίσταση είναι μικρή, κάτι που επιτρέπει μεγαλύτερη απόδοση ροπής και ταχύτητας για λιγότερες θερμικές απώλειες εσωτερικά στα πηνία.

Στην σύνδεση τροφοδοσίας του driver, από την οποία περνά το συνδυασμένο ρεύμα όλων των φάσεων του κινητήρα σε υψηλότερη τάση, η συνολική ισχύς παραμένει σταθερή από την αρχή διατήρησης της ενέργειας. Για την εσωτερική λειτουργεία του driver χρησιμοποιείται διαφορετική είσοδος μικρότερης τάσης, έτσι ώστε η είσοδος με τη μεγάλη τάση να αξιοποιείται αποκλειστικά από το κινητήρα. Οι εσωτερικές αντιστάσεις των εξαρτημάτων θεωρούνται αμελητέες. Από την αρχή διατήρησης της ενέργειας, για την είσοδο τροφοδοσίας υψηλής τάσης του driver και τις δύο εξόδους των ομάδων πηνίων ισχύει το εξής:

$$V_{input} * I_{input} = V_{output} * I_{output}$$

Όπου V_{input}, V_{output} η τάσεις εισόδου/εξόδων σε V και I_{input}, I_{output} οι εντάσεις εξόδων σε A. Ο τύπος ισχύει μόνο όταν ο κινητήρας είναι σταματημένος, καθώς με τη κίνηση τα πηνία θα δημιουργήσουν αντίθετη ηλεκτρομαγνητική δύναμη (Back EMF), η οποία αναγκάζει το driver να αυξήσει τη τάση εξόδου του για να συνεχίσει να «σπρώχνει» αρκετό ρεύμα προς τον κινητήρα. Ο λόγος που εξετάζεται σε ακινησία είναι διότι στο ολοκληρωμένο σύστημα θα περνάει το μεγαλύτερο χρόνο σε αυτή τη κατάσταση όσο θα αναμένεται η κίνηση του παίκτη και η απόφαση του υπολογιστή, οι κινητήρες θα περιστρέφονται μόνο στην εκτέλεση αυτής της κίνησης. Δεδομένου πως η παραγόμενη θερμότητα είναι αλληλένδετη με την ισχύ και το χρόνο, η περισσότερη θερμότητα που θέτει σε κίνδυνο τα εξαρτήματα θα παραχθεί στην ακινησία του κινητήρα.

Η συνολική θερμότητα που παράγεται από τον κινητήρα ανά φάση δίνεται από:

$$Q_{phase} = P_{phase} * t = V_{nominal} * I_{phase} * t = {I_{phase}}^2 * R_{coil} * t$$

Όπου Q_{phase} θερμότητα ανά φάση σε Joules, P_{phase} η ηλεκτρική ισχύς σε βατ ανά φάση, t ο χρόνος σε δευτερόλεπτα, $V_{nominal}$ η ονομαστική τάση σε V, I_{phase} το ρεύμα ανά φάση σε αμπέρ και R_{coil} η αντίσταση μίας ομάδας πηνίων σε ohms.

Εφόσον η θερμότητα οφείλεται στην ισχύ, τότε ο κινητήρας με τη λιγότερη ισχύ παράγει τη λιγότερη συνολική θερμότητα για το ίδιο χρονικό διάστημα λειτουργείας. Το μοντέλο Creality 42-34 με 4.06W ανά φάση είναι από τα υποψήφια το μοτέρ με τη μικρότερη ισχύς. Επιπρόσθετα πρέπει να εξεταστεί η κατανομή της θερμότητας στο κύκλωμα, δηλαδή πού παράγεται η ζέστη εκτός από το ποσοτικό μέγεθος της.

Αρχικά η συναθροισμένη απαίτηση ρεύματος και των δύο φάσεων για κάθε μοτέρ είναι $I_{phase} * 2$ για διφασικούς κινητήρες, θεωρώντας πως όλα τα πηνία είναι φορτισμένα, κάτι θεωρητικά αδύνατο αλλά χρήσιμο ως συντελεστής ασφαλείας, για κάθε κινητήρα:

	42BYGHW208	*Creality 42-34*	*42BYGHW804*	*42BYGHM809*
I (A)	0.8	1.68	2.4	3.36
Η ισχύς διπλασιάζεται λόγω του διπλασιασμού του ρεύματος:				
P (W)	9.6	8.12	8.64	9.3

Πίνακας 7 - Ρεύμα και Ισχύς Βηματικών Κινητήρων

Το ρεύμα στη τροφοδοσία του driver, υποθέτοντας ότι το driver τροφοδοτείται με 12V για τάση ίδιας με του ηλεκτρομαγνήτη, βρίσκεται αναδιατυπώνοντας τον τύπο:

$$V_{input} * I_{input} = V_{output} * I_{output} \Leftrightarrow I_{input} = \frac{V_{nominal} * I}{V_{output}} = \frac{P}{V_{output}}$$

Το $V_{nominal}$ παραμένει σταθερό παρά την αλλαγή από μία σε δύο φάσεις στους υπολογισμούς, επειδή είναι ορισμένο ως $V_{nominal} = I_{phase} * R_{coil}$ τότε φαινομενικά διπλασιάζοντας το I_{phase} θα διπλασιαζόταν και το $V_{nominal}$. Αυτό δεν συμβαίνει εξαιτίας της παράλληλης σύνδεσης των δύο ομάδων πηνίων, όταν εξετάζεται το συνολικό κύκλωμα, η αντίσταση R_{coil} διαιρείται με το 2 αντισταθμίζοντας την αλλαγή του I_{phase} και διατηρώντας το $V_{nominal}$ σταθερό.

Για κάθε κινητήρα το I_{input} είναι:

	42BYGHW208	*Creality 42-34*	*42BYGHW804*	*42BYGHM809*
$\mathbf{I_{input}}$ (A)	0.8	0.68	0.72	0.78

Πίνακας 8 - Σύγκριση ρεύματος εισόδου στο driver των κινητήρων

Όλες οι τιμές βρίσκονται χαμηλότερα των ≈1A άρα δεν απορρίπτεται κάποιος κινητήρας λόγω ασφάλειας του breadboard.

Το ρεύμα παράγει θερμότητα ανάλογα με την αντίσταση που συναντά στη πορεία του, και υπολογίζοντας μόνο με το R_{coil}, υπολογίζεται μόνο η θερμότητα που θα παραχθεί από το ρεύμα εσωτερικά του μοτέρ λόγω της αντίστασης των πηνίων. Στη πραγματικότητα κοιτώντας το κύκλωμα καθολικά, διαπιστώνεται ότι το ρεύμα συναντά διαφορετικές αντιστάσεις και ευθύνεται για τη περισσότερη παραγομένη θερμότητα, συγκριτικά με το ρόλο του R_{coil} στον υπολογισμό θερμότητας. Η συνολική θερμότητα για το κύκλωμα μεταξύ της πηγής και των πηνίων του βηματικού κινητήρα δίνεται από:

$$Q_{total} = P_{total} * t = I^2 * (R_{coil} + R_{breadboard} + R_{driver} + \cdots) * t$$

Από αυτόν το τύπο προέκυψε η σημαντικότητα του ρεύματος στη συνολική θερμότητα, κάτι που συνέβαλε στην επιλογή του τελικού κινητήρα.

Με βάση αυτό ο κινητήρας με το χαμηλότερο ρεύμα και τη μεγαλύτερη αντίσταση πηνίων θα είναι η ασφαλέστερη επιλογή για το breadboard και το driver, διότι η περισσότερη θερμότητα από τη λειτουργία του παράγεται εσωτερικά, λόγω της μεγάλης αντίστασης πηνίων αντί κατανεμημένα στα πιο ευαίσθητα εξαρτήματα.

Υπολογίζοντας την αντίσταση μίας ομάδας πηνίων, δηλαδή το R_{coil} κάθε κινητήρα, αναδιατυπώνοντας τον τύπο της ονομαστικής τάσης $V_{nominal} = I_{phase} * R_{coil}$ καταλήγουν οι τιμές:

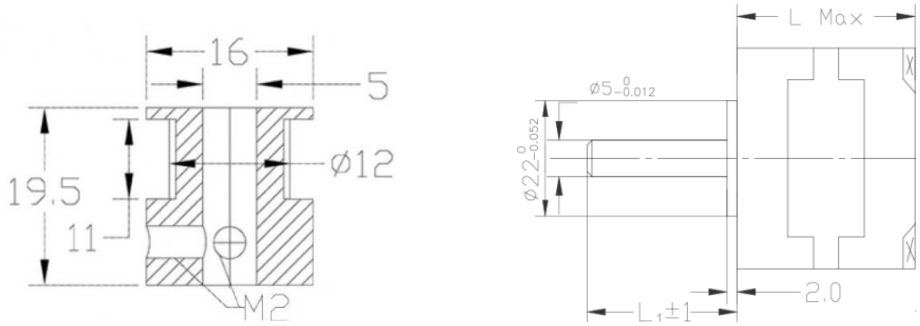
	42BYGHW208	*Creality 42-34*	*42BYGHW804*	*42BYGHM809*
$\boldsymbol{R_{coil}(\Omega)}$	30	5.75	3	1.64

Πίνακας 9 - Σύγκριση αντίστασης πηνίων των κινητήρων

Ο κινητήρας με την μικρότερη απόδοση ροπής ανά βατ έχει τη μεγαλύτερη εσωτερική αντίσταση, που σημαίνει ότι η περισσότερη ζέστη θα παραχθεί εσωτερικά του αντί για κατανεμημένα στη πορεία του ρεύματος ανάλογα με τη σημειακή αντίσταση.

Ακόμη και αν δεν είναι αποδοτικός, είναι η πιο ασφαλής επιλογή για τη προστασία του breadboard και του driver. Επιπρόσθετα, το γεγονός ότι πληρεί την απαιτούμενη ροπή με 1/32 microstepping και είναι ο φθηνότερος χρηματικά των επιλογών οδήγησαν στην επιλογή του μοντέλου *42BYGHW208* βηματικού κινητήρα για την κατασκευή.

Πριν την αγορά ελέγχθηκε η διάμετρος του άξονα περιστροφής του κινητήρα, από το μηχανολογικό του σχέδιο, να είναι συμβατή με την εσωτερική διάμετρο της τροχαλίας 20T:



Εικόνα 22 - Μηχανολογικό Σχέδιο Τροχαλίας και Άξονα Κινητήρα

Για περεταίρω μείωση θορύβου σε συνδυασμό με το microstepping αγοράστηκε και μία λεπτή επιφάνεια φελλού, η οποία δρα ως μηχανικός αποσβεστήρας μειώνοντας τις δονήσεις που μεταδίδονται από το υλικό στήριξης.



Εικόνα 23 - Αποσβεστήρας από φελλό

Σε μεταγενέστερο στάδιο αναγνωρίστηκε το πρόβλημα της αντίθετης ηλεκτρομαγνητικής δύναμης που μείωνε τη ροπή του κινητήρα περαιτέρω. Στο συγκεκριμένο κινητήρα, λόγω της υψηλής ονομαστικής τάσης του που ισούται με την τροφοδοσία του driver στα 12V, το driver δεν μπορεί να αυξήσει τη τάση για να «σπρώξει» αρκετό ρεύμα για να διατηρήσει το I_{phase}. Αυτό συμβαίνει μόνο όταν ο κινητήρας κινείται και έχει ως αποτέλεσμα να υπολειτουργεί ο κινητήρας όσο δεν δέχεται το πλήρες ρεύμα φάσεως του. Ωστόσο, λόγω χρήσεων συντελεστών ασφαλείας κατά την προσέγγιση των υπολογισμών σε αυτό το στάδιο, η ροπή αρκεί για τη λειτουργεία του συστήματος ακόμη και όταν υπολειτουργεί, άρα δεν χρειάστηκε αλλαγή των κινητήρων εξαιτίας αυτής της παράλειψης.

Επιλογή Driver Βηματικού Κινητήρα

Μετά την επιλογή του βηματικού κινητήρα είναι εξίσου σημαντική η επιλογή του κατάλληλου driver, μέσω του οποίου ελέγχονται οι παλμοί που επιτρέπουν την περιστροφή του κινητήρα με ακρίβεια. Κατά την προηγούμενη έρευνα διαπιστώθηκαν χαρακτηριστικά (Stepper Motor Drivers, 2026) που πρέπει να πληρεί το driver καθιστώντας αυτή την έρευνα αγοράς ευκολότερη.

Τα χαρακτηριστικά με τα οποία γίνεται η έρευνα είναι:

- Ικανότητα για 1/32 microstepping, το οποίο είναι το μέγιστο αξιοποιήσιμο όπως διαπιστώθηκε κατά την επιλογή του βηματικού κινητήρα λόγω μείωσης της ροπής. Μεγαλύτεροι διαιρέτες βημάτων δεν μπορούν να αξιοποιηθούν.

- Ευέλικτη τάση λογικής 3.3V και 5V, σε αυτό το στάδιο δεν έχει γίνει επιλογή μικροελεγκτή, άρα αν υποστηρίζονται και οι δυο κοινές τάσεις λογικής δεν εισάγεται ένας περιοριστικός παράγοντας στην μελλοντική επιλογή μικροελεγκτή. Διαφορετική τάση θα απαιτούσε χρήση περαιτέρω κυκλώματος, όπως ένα level shifter για την επικοινωνία μεταξύ driver και μικροελεγκτή.

- Οι απώλειες μετατροπής τάσης, λόγω του επιλεγμένου βηματικού κινητήρα να έχει ίδια ονομαστική τάση με τη τάση τροφοδοσίας του driver δεν είναι όσο σημαντικό χαρακτηριστικό, αλλά στη πράξη πάντα θα γίνεται μία μικρή μετατροπή τάσης. Για αυτό η χρήση σύγχρονων transistor MOSFET από το driver είναι προτιμότερη των BJT, για τη καλύτερη απόδοση τους και τις μικρότερες θερμικές απώλειες.

- Η ικανότητα περιορισμού ρεύματος και η προστασία υπερέντασης (overcurrent protection) είναι ιδιαίτερα σημαντική και το κύριο πλεονέκτημα ενός driver φτιαγμένο ειδικά για βηματικούς κινητήρες συγκριτικά με κάποιο driver γενικής χρήσης. Ο περιορισμός ρεύματος επιτρέπει την ασφαλή λειτουργία του μοτέρ και το αποτρέπει από το να υπερθερμανθεί.

- Η προστασία υπέρτασης (overvoltage protection) είναι σημαντική, γιατί τα πηνία του κινητήρα κατά τη κίνηση παράγουν αντίθετη ηλεκτρομαγνητική δύναμη η οποία ευθύνεται για αιχμές υψηλής τάσης. Δίοδοι ή άλλα κυκλώματα προστασίας πρέπει να συμπεριλαμβάνονται.

- Η μέγιστη τάση τροφοδοσίας, αν είναι υψηλότερη της τάσης 12V του ηλεκτρομαγνήτη θα πρέπει να συμπεριληφθεί ένα επιπλέον κύκλωμα που να χαμηλώνει τη τάση.

- Η προσθήκη ψύκτρας ή η προστασία υπερθέρμανσης (over-temperature thermal shutdown) προσφέρει παραπάνω ασφάλεια για να μην υπερθερμανθεί το ίδιο το driver. Ακόμη και με τον επιλεγμένο κινητήρα χαμηλού ρεύματος μία ψύκτρα δεν έχει μειονεκτήματα πέρα από το φυσικό μέγεθος της.

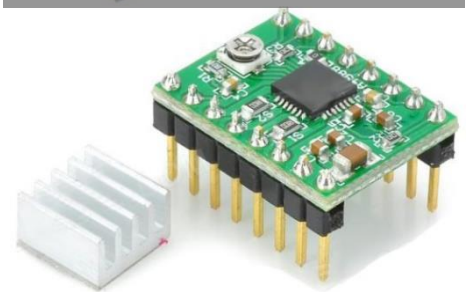
- Η τιμή δεν είναι όσο σημαντική όσο με τους κινητήρες, καθώς τείνουν τα drivers να είναι φθηνότερα, αλλά πρέπει να δικαιολογείται από τα υπόλοιπα χαρακτηριστικά του driver που αξιοποιούνται στο σύστημα.

Γνωρίζοντας τα χαρακτηριστικά που καθίστανται σημαντικά για το σύστημα, ακολουθεί η έρευνα αγοράς, αρχίζοντας από τοπικούς πωλητές. Τα υποψήφια drivers που εξετάστηκαν ήταν τα εξής:



Εικόνα 24 - Driver L9110S

Dual Motor Driver Module L9110S
Μέγιστος διαιρέτης βημάτων: N/A
Τάση λογικής: N/A
Τάση τροφοδοσίας: 2.5-12V
Transistor: MOSFET
Περιορισμός ρεύματος: ΟΧΙ
Μέγιστο ρεύμα ανά ομάδα πηνίων: 1.5A
Προστασία: διόδους για επιστροφή ρεύματος
Τιμή: 2.50€



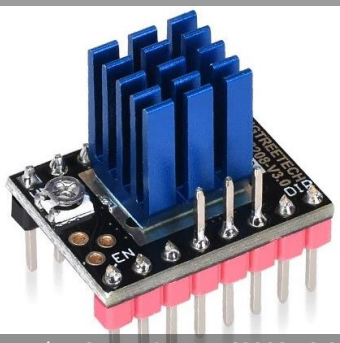
Εικόνα 25 - Driver A4988

A4988 Stepper Motor Driver
Μέγιστος διαιρέτης βημάτων: 16
Τάση λογικής: 3-5.5V
Τάση τροφοδοσίας: 8-35V
Transistor: MOSFET
Περιορισμός ρεύματος: ΝΑΙ
Μέγιστο ρεύμα ανά ομάδα πηνίων: 2A
Προστασία: υπερθέρμανσης, βραχυκυκλώματος, χαμηλής τάσης
Τιμή: 3.20€



Εικόνα 26 - Driver DRV8825

DRV8825 Stepper Motor Driver
Μέγιστος διαιρέτης βημάτων: 32
Τάση λογικής: 2.5-5.25V
Τάση τροφοδοσίας: 8.2-45V
Transistor: MOSFET
Περιορισμός ρεύματος: ΝΑΙ
Μέγιστο ρεύμα ανά ομάδα πηνίων: 2.2A
Προστασία: υπερθέρμανσης, overcurrent, βραχυκυκλώματος, χαμηλής τάσης,
Τιμή: 3.20€



Εικόνα 27 – Driver TMC2208 V3.0

TMC2208 V3.0 Stepper Motor Driver
Μέγιστος διαιρέτης βημάτων: 256
Τάση λογικής: 3-5.25V
Τάση τροφοδοσίας: 4.75-36V
Transistor: MOSFET
Περιορισμός ρεύματος: ΝΑΙ
Μέγιστο ρεύμα ανά ομάδα πηνίων: 2A
Προστασία: υπερθέρμανσης, βραχυκυκλώματος, χαμηλής τάσης, αντίθετης πόλωσης, διαγνωστικά
Τιμή: 4.80€

Επιλέχθηκαν τα χαρακτηριστικά από τα datasheets των παραπάνω εξαρτημάτων, όπου διαθέσιμα, αντί από τις ιστοσελίδες των μεταπωλητών.

Όλα τα drivers μπορούν να διοχετεύσουν 0.8A στις εισόδους τροφοδοσίας των κινητήρων, το υπολογισμένου του επιλεγμένου μοντέλου *42BYGHW208*, χρησιμοποιούν MOSFET transistors και υποστηρίζουν ένα εύρος τάσεων λογικής. Έχουν διαφορετικά είδη προστασίας και είναι φθηνοί συγκριτικά με τα μοτέρ.

Το μοντέλο *Dual Motor Driver Module L9110S* είναι κατάλληλο μόνο σε ηλεκτρικά χαρακτηριστικά και υποδεικνύει την έντονη διαφορά μεταξύ ένα driver γενικής χρήσης με ένα εξειδικευμένο driver βηματικών κινητήρων. Δεν έχει την ικανότητα να αυξήσει την ακρίβεια με microstepping και κυρίως δεν έχει περιορισμό ρεύματος για τη προστασία του κινητήρα. Μπορεί να κινήσει ένα βηματικό κινητήρα αλλά σε μία βασική μορφή που δεν είναι κατάλληλη για το σύστημα κίνησης που απαιτεί ακρίβεια. Για τους παραπάνω λόγους το μοντέλο *Dual Motor Driver Module L9110S* απορρίφθηκε από τα υποψήφια drivers.

Το μοντέλο *A4988 Stepper Motor Driver* είναι το πιο κοινό driver στην αγορά και παρέχει πολλά από τα ζητούμενα χαρακτηριστικά. Το μέγιστο microstepping είναι 1/16 έναντι του μέγιστου 1/32 που μπορεί να αξιοποιηθεί, όπως υπολογίστηκε στην έρευνα αγοράς των κινητήρων, αλλά παραμένει μία καλή επιλογή.

Το μοντέλο *DRV8825 Stepper Motor Driver* στην ίδια τιμή με το *A4988* παρέχει καλύτερα ηλεκτρικά χαρακτηριστικά, μεγαλύτερο διαιρέτη βημάτων μέχρι 1/32 microstepping που είναι το μέγιστο αξιοποιήσιμο, και προστασία υπερέντασης (overcurrent) (DRV8825 Datasheet, 2014) που το *A4988* στερείται. Λόγω των καλύτερων χαρακτηριστικών του, που χωρίς φαινομενικά κάποιο μειονέκτημα που απαιτεί σύγκριση μεταξύ των δύο, απορρίφθηκε το μοντέλο *A4988.*

Το μοντέλο *TMC2208 V3.0 Stepper Motor Driver* είναι το πιο σύγχρονο από τα υποψήφια με το μέγιστο διαιρέτη βημάτων μέχρι 1/256 microstepping και διαγνωστικά σε πραγματικό χρόνο μέσο επικοινωνίας UART. Στερείται προστασία υπερέντασης (overcurrent) συγκριτικά με το *DRV8825* κάτι αρκετά σημαντικό, αλλά εφόσον ο κινητήρας απέχει από το μέγιστο όριο των 2A δεν καθίσταται κρίσιμο. Χρησιμοποιεί τεχνολογία StealthChop2™ για να κάνει τον κινητήρα αθόρυβο. Μπορεί να ρυθμιστεί να χρησιμοποιήσει και τεχνολογία SpreadCycle™, η οποία βελτιώνει τη σταθερότητα και τη ροπή του κινητήρα (TMC2208/2 & TMC2224 family Datasheet, 2023). Τα ηλεκτρικά χαρακτηριστικά του είναι παρόμοια με του *DRV8825* άρα αυτό το driver προσφέρει τα περισσότερα και πρέπει να εξεταστεί ποιες από τις τεχνολογίες του μπορούν να αξιοποιηθούν για να δικαιολογηθεί το ελαφρώς υψηλότερο κόστος του.

Το StealthChop2™ είναι το κύριο χαρακτηριστικό του *TMC2208* που το *DRV8825* στερείται καθώς κάνει τη περιστροφή των κινητήρων αθόρυβη. Αυτό όμως μειώνει τη ροπή, από 10% σε ακινησία έως και 50% σε πολύ υψηλές ταχύτητες. Με την απαιτούμενη ροπή να είναι 1.16 N*cm και τη ροπή του επιλεγμένου κινητήρα με 1/32 microstepping 1.52 N*cm, βρισκόμενη 24% πάνω από το όριο, αυτή η μείωση ροπής επιφέρει ένα ρίσκο σε υψηλές ταχύτητες. Για αυτό αντί για StealthChop2™ μπορεί να ενεργοποιηθεί η λειτουργία SpreadCycle™ η οποία προσφέρει μεγαλύτερη ροπή αλλά χάνεται το πλεονέκτημα της αθόρυβης περιστροφής.

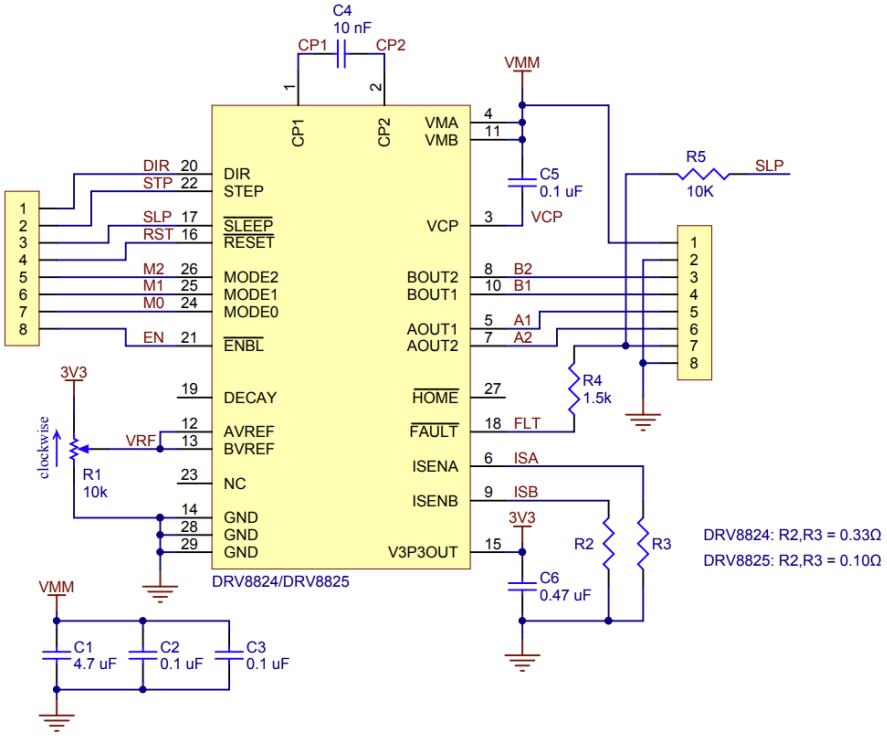
Αυτός ο περιορισμός, μαζί με την αδυναμία αξιοποίησης του προσφερόμενου 1/256 microstepping, σημαίνει πως το *TMC2208* θα λειτουργεί παρόμοια του *DRV8825* χωρίς να χρησιμοποιούνται τα καλύτερα χαρακτηριστικά του. Αναγνωρίζοντας πως το *TMC2208* είναι πιο σύγχρονο, επιλέχθηκε για το παρόν σύστημα η φθηνότερη επιλογή των δύο, το *DRV8825* λόγω αυτής της παρόμοιας λειτουργικότητας στο τελικό κύκλωμα.

Σε μεταγενέστερο στάδιο της κατασκευής αναγνωρίστηκε ότι το *DRV8825* χάνει βήματα (DRV8825 missing microsteps, 2015) στο 1/32 microstepping, γεγονός που το καθιστά επικίνδυνο για το σύστημα κίνησης που βασίζεται στην ακρίβεια των βημάτων για τον εντοπισμό της θέσης του. Δεν αναφέρεται ως πιθανότητα στο datasheet του κατασκευαστή. Αυτό οδήγησε στη χρήση 1/16 microstepping αντί του βέλτιστου 1/32 για την αντιμετώπιση αυτού το προβλήματος στη τελική κατασκευή, που έχει ως κόστος αυξημένο θόρυβο και λιγότερη ομαλότητα για περισσότερη ροπή.

Αυτό το πρόβλημα συναντάται συγκεκριμένα σε χαμηλά ρεύματα λειτουργίας, όπως το 0.4A/φάση του κινητήρα, και σε κινητήρες με χαμηλή ονομαστική τάση που δεν επηρεάζει το συγκεκριμένο σύστημα.

Οφείλεται σε ένα συνδυασμό προβλημάτων: Η συχότητα λειτουργίας του PWM από το datasheet είναι 30,000Hz ή 33.3μs, τόσο είναι ένας παλμός που στέλνεται στον κινητήρα. Ο ελάχιστος χρόνος απόσβεσης (Blanking Time) του *DRV8825* είναι 3.75μs που περιορίζει το «βήμα» τάσης που μπορεί να δώσει το driver σε ≈12% της πηγής, για 12V είναι 1.4V. Για το συγκεκριμένο μοτέρ λόγω της υψηλής εσωτερικής αντίστασης το ελάχιστο ρεύμα είναι $I = \frac{V}{R} = \frac{1.4}{30} = 0.046A = 46mA$, δεν μπορεί να σταλθεί ρεύμα λιγότερου αυτού για την ημιτονοειδή καμπύλη που ακολουθεί το microstepping με αποτέλεσμα τα χαμένα βήματα. Παρατηρείται σε μεγάλους διαιρέτες βημάτων γιατί εκεί απαιτούνται τόσο μικρά βήματα.

Η θεωρητική λύση σε αυτό το πρόβλημα είναι η αλλαγή της λειτουργίας του εσωτερικού chip από Mixed Decay σε Fast Decay που λύνει αυτόν τον περιορισμό με κόστος πιο ηλεκτρικά θορυβώδη κύματα. Αυτό γίνεται αλλάζοντας το DECAY pin από FLOATING σε HIGH , αλλά αυτό το pin δεν είναι προσβάσιμο στο breakout board που πωλείται και χρειάζεται μία πολύ ακριβής συγκόλληση για να βραχυκυκλωθεί και να αλλάξει σε Fast Decay mode. Για αυτό το λόγο κρίθηκε ανέφικτη για τη τελική κατασκευή.



Εικόνα 28 - Schematic DRV8825

Επιλογή μικροελεγκτή

Ο μικροελεγκτής ευθύνεται για το κομμάτι της επεξεργασίας της κατασκευής και για τον έλεγχο των επιμέρους κυκλωμάτων άρα είναι ο κεντρικός πυλώνας πάνω στον οποίων εξαρτάται η λειτουργία του συστήματος. Επειδή υπάρχουν πολλές επιλογές στην αγορά η έρευνα αγοράς του αφέθηκε για το τέλος, έτσι ώστε να έχουν ήδη διαπιστωθεί οι περιοριστικοί παράγοντες και οι προκλήσεις στις οποίες πρέπει να ανταπεξέλθει από προηγούμενες επιλογές εξαρτημάτων. Τα χαρακτηριστικά με τα οποία εξετάστηκαν οι μικροελεγκτές ήταν:

- Τάση: εξαιτίας των προηγούμενων επιλογών εξαρτημάτων, η τάση μπορεί να είναι ευέλικτη μεταξύ των 3.3V και 5V χωρίς να έχει περιοριστεί σε μία από τις δύο τιμές.

- Επεξεργαστική ισχύς: το νευρωνικό δίκτυο και οι παλμοί των βηματικών κινητήρων απαιτούν αρκετή επεξεργαστική ισχύ για να τρέχουν σε «πραγματικό χρόνο», δηλαδή αρκετά γρήγορα ώστε να είναι διασκεδαστική η παρτίδα. Να σημειωθεί πως το ενσωματωμένο hardware μπορεί να αντικαταστήσει την απαίτηση στην επεξεργαστική ισχύ και δεν απαιτείται να έχει ο ίδιος ο μικροεπεξεργαστής μεγάλη ισχύ. Για παράδειγμα, το περιφερειακό hardware του MCPWM παράγει τους παλμούς για τους κινητήρες απελευθερώνοντας ισχύ από τον επεξεργαστή, ή ένας επιταχυντής μπορεί να κάνει το ίδιο για το νευρωνικό δίκτυο. Ο αριθμός πυρήνων ή threads ανά πυρήνα σχεδόν πολλαπλασιάζει την διαθέσιμη επεξεργαστική ισχύς.

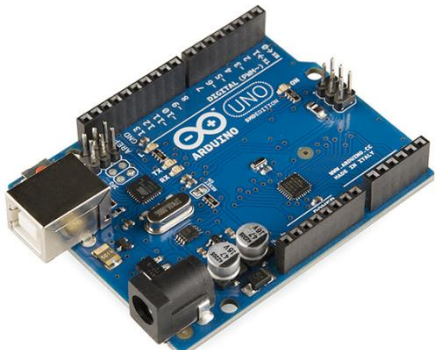
- Χωρητικότητα: το νευρωνικό δίκτυο και οι σχετικές βιβλιοθήκες του τείνουν να είναι απαιτητικά στη μνήμη αποθήκευσης. Παρ ’όλα αυτά δεν είναι μεγάλη προτεραιότητα λόγω της πληθώρας λύσεων και βελτιστοποιήσεων όσον αφορά την μνήμη, όπως η προσθήκη εξωτερικής μνήμης, η κβαντοποίηση του νευρωνικού δικτύου, in-place reading από τη flash χωρίς να χρειαστεί να αντιγραφτεί κάτι σε RAM και custom partition tables για διαχωρισμό της μνήμης ανάλογα με τις απαιτήσεις δίνουν μεγάλη ευελιξία στην αντιμετώπιση προβλημάτων χωρητικότητας.

- Τιμή: η επιπλέον επεξεργαστική ισχύς και χωρητικότητα δεν αξιοποιούνται με αποτέλεσμα να μην μπορεί να δικαιολογηθεί το κόστος σημαντικής υπερδιαστασιολόγησης.

- Αριθμός ακίδων (pins): δεν είναι μεγάλης προτεραιότητας καθώς μπορούν να χρησιμοποιηθούν shift registers και port expanders για την επέκταση του αριθμού ανάλογα με τις ανάγκες του συστήματος.

- Περιβάλλον προγραμματισμού: η οικειότητα με τις γλώσσες που υποστηρίζονται είναι σημαντική, με προτεραιότητα να δίνεται στη C++ έναντι της Python ή της Rust. Οι βιβλιοθήκες που μπορεί να χρησιμοποιηθούν παίζουν ρόλο με τη προτίμηση της Arduino.h αντί της ESP-IDF. Τέλος για το περιβάλλον ανάπτυξης (IDE) προτιμήθηκε το Arduino IDE για την απλότητα του και την αυτόματη διαχείριση παρόμοιας με της CMake με το Arduino CLI (Command Line Interface).

Με βάση τα παραπάνω χαρακτηριστικά εξετάστηκαν οι ακόλουθοι μικροελεγκτές:



Εικόνα 29 – Μικροελεγκτής Arduino UNO R3

Arduino UNO R3
Τάση: 5V
Επεξεργαστικής ισχύς: 16 MHz
Πυρήνες: 1
ROM: 33 KB
RAM: 2 KB
GPIO Pins: 20
Τιμή: 2 - 30€



Εικόνα 30 - Μικροελεγκτής ESP32-S3 N8R8

ESP32-S3 N8R8
Τάση: 3.3V
Επεξεργαστικής ισχύς: 240 MHz
Πυρήνες: 2
ROM: 8.384 MB
RAM: 8.512 MB
GPIO Pins: 31 - 36
Τιμή: 11 - 15€



Εικόνα 31 - Μικροϋπολογιστής Raspberry Pi Zero 2 W

Raspberry Pi Zero 2 W
Τάση: 3.3V
Επεξεργαστικής ισχύς: 1 GHz
Πυρήνες: 4
ROM: εξωτερική microSD
RAM: 512 MB
GPIO Pins: 26
Τιμή: 20€

Επιλέχθηκαν τα χαρακτηριστικά από τα datasheets των παραπάνω εξαρτημάτων, όπου διαθέσιμα, αντί από τις ιστοσελίδες των μεταπωλητών.

Από τους υποψήφιους μικροελεγκτές το *Arduino UNO R3* είναι το πιο φθηνό, όμως τα χαρακτηριστικά του δεν είναι αρκετά για να ανταπεξέλθει στις απαιτήσεις της κατασκευής. Με μόνο 32KB flash θα χρειαζόταν εξωτερική κάρτα SD η οποία θα κατανάλωνε επιπλέον GPIO pins, τα οποία είναι μόνο 20 και θα απαιτούταν επέκταση τους. Η επεξεργαστική ισχύς είναι πάρα πολύ περιοριστική με μόνο ένα πυρήνα, χωρίς ενσωματωμένο λειτουργικό σύστημα και RAM στα 2KB, χαρακτηριστικά που οδήγησαν στην απόρριψη του μικροελεγκτή από τους υποψήφιους.

Το *ESP32-S3 N8R8* παρουσιάζει βελτιωμένα χαρακτηριστικά, με σημαντική αύξηση της χωριτικότητας τόσο στη flash όσο και στη RAM με 8MB δεν προβλέπεται να χρειαστεί επέκταση. Η επεξεργαστική του ισχύς είναι μεγαλύτερη της απαιτούμενης με 240 MHz και δύο πυρήνες, οι οποίοι συγχρονίζονται από το ενσωματωμένο λειτουργικό σύστημα FreeRTOS (Real Time Operating System). Το FreeRTOS κάνει το συγχρονισμό ασύγχρονων πόρων και πυρήνων εύκολο με τη χρήση tasks και semaphores (FreeRTOS Reference Manual, 2026). Επίσης προσφέρει το μεγαλύτερο αριθμό pins από τους υποψήφιους μικροελεγκτές. Προγραμματίζεται με το framework ESP-IDF αλλά υποστηρίζεται από το Arduino IDE μέσω wrappers που μετατρέπουν τις συναρτήσεις της Arduino.h στις ανάλογες εντολές της ESP-IDF με κόστος αυξημένο compile time. Το μοντέλο S3 παρέχει AI acceleration προσφέροντας συγκεκριμένα special registers με χωριτικότητα 128-bit για πράξεις πινάκων 4 φορές πιο γρήγορα από κανονικά 32-bit registers. Έχει δύο peripherals για παλμούς ώστε να μην επιβαρύνεται ο επεξεργαστής με τον υπολογισμό τους, το LEDC και το MCPWM (ESP32-S3 Technical Reference Manual, 2026) (ESP32-S3 Datasheet, 2026).

Το *Raspberry Pi Zero 2 W* δείχνει εξαιρετικά μεγάλη επεξεργαστική ισχύς με 1 GHz και τέσσερις πυρήνες, ενσωματωμένο λειτουργικό σύστημα Linux (default το Debian distro) που το καθιστά μικροϋπολογιστή αντί για απλό μικροελεγκτή. Έχει εξαιρετικά πολύ RAM αλλά απαιτείται αγορά εξωτερικής κάρτας microSD για ROM, η οποία ενδεικτικά έχει χωρητικότητα μερικά GB. Επειδή τρέχει Linux δεν συνδέεται σε υπολογιστή για το προγραμματισμό του, αλλά μπορεί να εγκατασταθεί τοπικά το Arduino IDE και να προγραμματιστεί απευθείας από το desktop του.

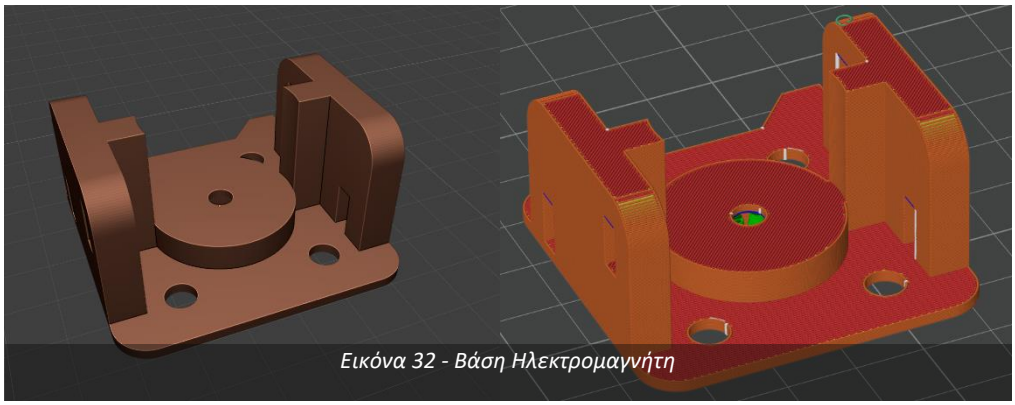
Με βάση τα παραπάνω και γνωρίζοντας τις απαιτήσεις του συστήματος η επιλογή είναι μεταξύ του *ESP32-S3 N8R8* και του *Raspberry Pi Zero 2 W.* Το Raspberry έχει καλύτερα χαρακτηριστικά αλλά δεν μπορούν να αξιοποιηθούν όλα και θα μείνει σημαντική επεξεργαστική ισχύς και χωρητικότητα ανεκμετάλλευτες. Αν προηγουμένως είχε επιλεχτεί η χρήση κάμερας για τον εντοπισμό των πιονιών, το Rasberry θα μπορούσε να τρέξει δύο νευρωνικά δίκτυα ταυτόχρονα και θα είχε εξεταστεί περισσότερο. Ωστόσο με τις προδιαγραφές του παρόν συστήματος, το *ESP32-S3 N8R8* παρέχει πόρους κοντά στους απαιτούμενους με ευελιξία για αλλαγές κατά τη κατασκευή όπως και χαμηλότερη τιμή. Το AI acceleration και το MCPWM που συμπεριλαμβάνωνται στο devkit μπορούν να αξιοποιηθούν για επιπλέον επεξεργαστική ισχύς, δηλαδή αυξημένη ταχύτητα. Για αυτούς τους λόγους επιλέχτηκε ως ο μικροελεγκτής της κατασκευής.

Κατασκευή

Τα λειτουργικά μέρη όσον αφορά το hardware χωρίζονται στο σύστημα κίνησης, το σύστημα ανίχνευσης, το κύκλωμα ελέγχου του ηλεκτρομαγνήτη, το σύστημα διεπαφής χρήστη (user interface), και το κύκλωμα τροφοδοσίας μαζί με τον μικροελεγκτή που παρουσιάζονται ως μέρος των υπολοίπων.

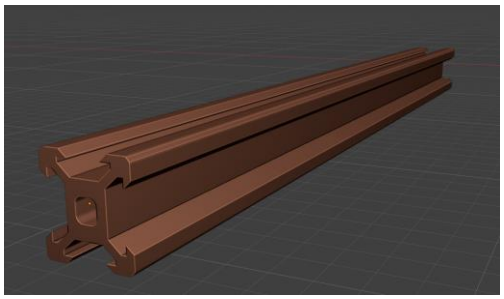
Το πρώτο που κατασκευάστηκε ήταν το σύστημα κίνησης, το οποίο αποτελείται από τις δοκούς, τους ιμάντες, τις τροχαλίες, τον φορέα και τις ράβδες του, τους βηματικούς κινητήρες και το κύκλωμα ελέγχου τους.

Σχεδιάστηκε η βάση του ηλεκτρομαγνήτη και τυπώθηκε τροποποιώντας το τρισδιάστατο μοντέλο από υπάρχουσα κατασκευή (Greg06, 2026):

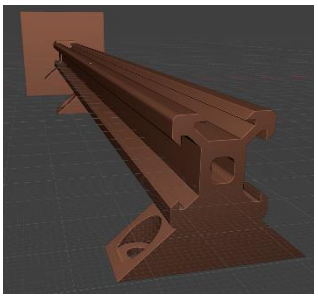


Εικόνα 32 - Βάση Ηλεκτρομαγνήτη

Η κεντρική δοκός και οι δοκοί στήριξης:

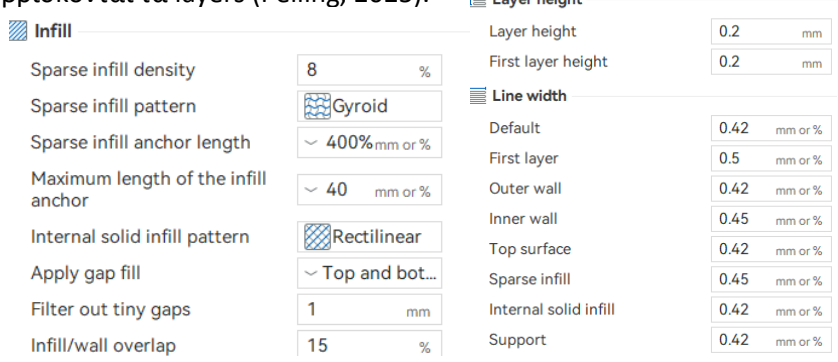


Εικόνα 34 - Κεντρική Δοκός

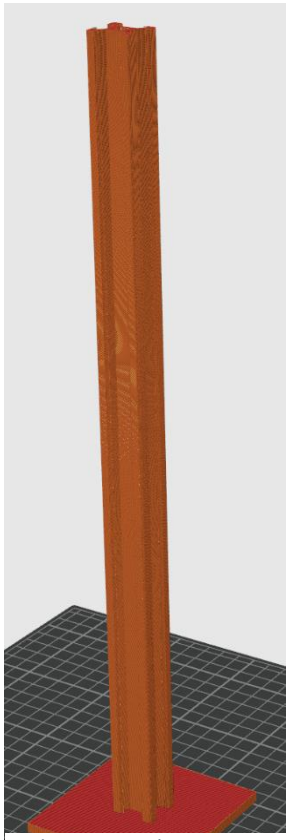


Εικόνα 33 - Πλαϊνή Δοκός

Για την εκτύπωση των δοκών χρειάστηκε βάση για να αυξήσει την επιφάνεια προσκόλλησης στο bed του 3D εκτυπωτή. Λόγω των απαιτούμενων διαστάσεων για μία σκακιέρα 30cm*30cm έπρεπε να τυπωθούν κατακόρυφα, κάτι που εισάγει αδυναμία στον άξονα που βρίσκονται τα layers (Peiling, 2025).

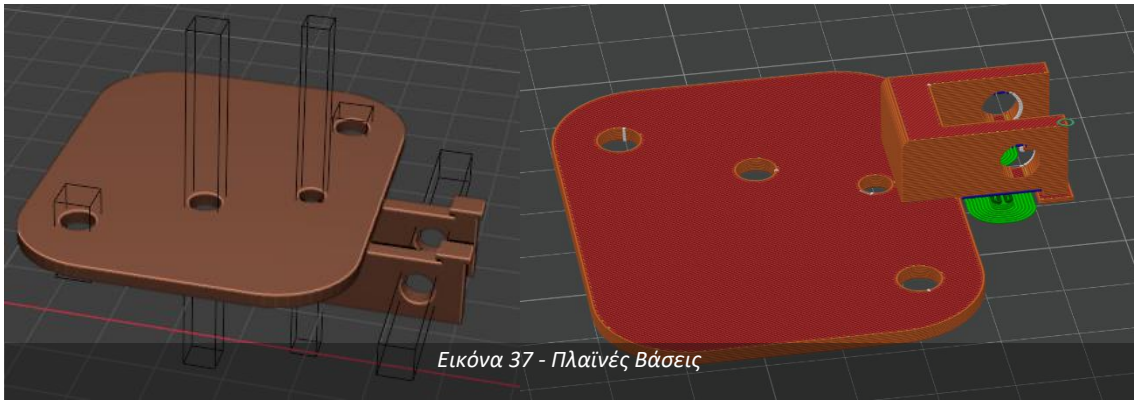


Εικόνα 36 - Slicer Settings



Εικόνα 35 - Εκτύπωση Δοκού

Τέλος οι πλαϊνές βάσεις στις οποίες εφαρμόζεται η κεντρική δοκός και μετακινούνται μαζί με τον άξονα X:



Εικόνα 37 - Πλαϊνές Βάσεις

Τα παραπάνω σχεδιάστηκαν με βάση τις διαστάσεις του επιλεγμένου V-wheel, το οποίο αποτελείται από τη ρόδα, δύο ρουλεμάν, δύο ροδέλες, ένα παξιμάδι, έναν αποστάτη και μία βίδα.

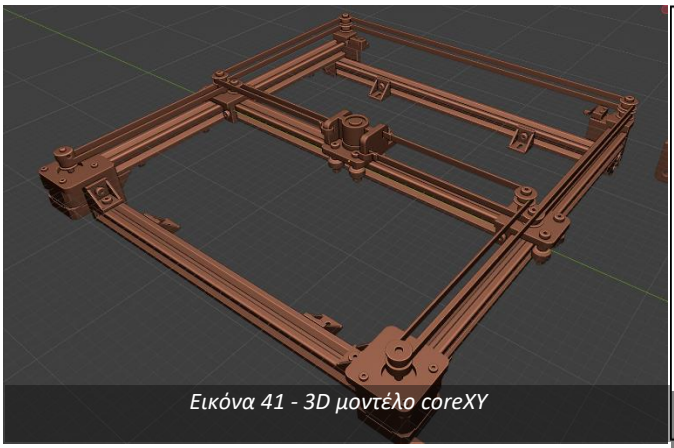


Εικόνα 39 - V-wheel kit

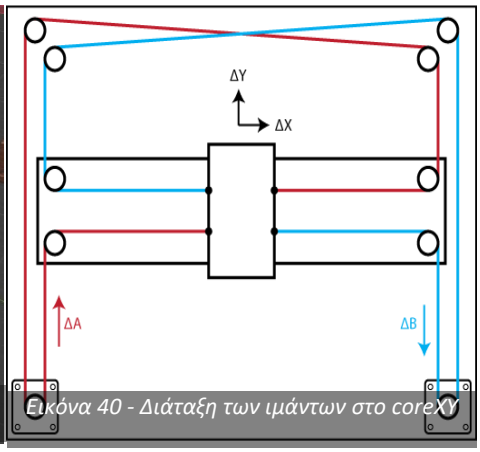


Εικόνα 38 - V-wheel assembly

Με τα κομμάτια έτοιμα από την έρευνα αγοράς και τις εκτυπώσεις έγινε η συναρμολόγηση ακολουθώντας τις παρακάτω ενδεικτικές αναφορές:

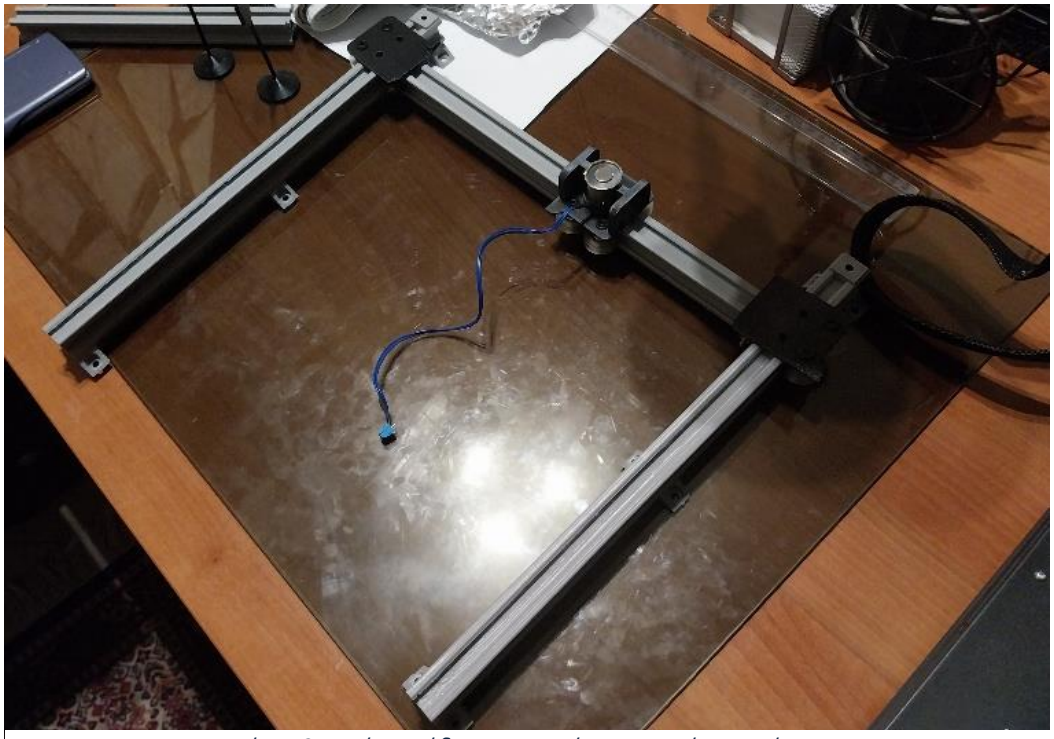


Εικόνα 41 - 3D μοντέλο coreXY



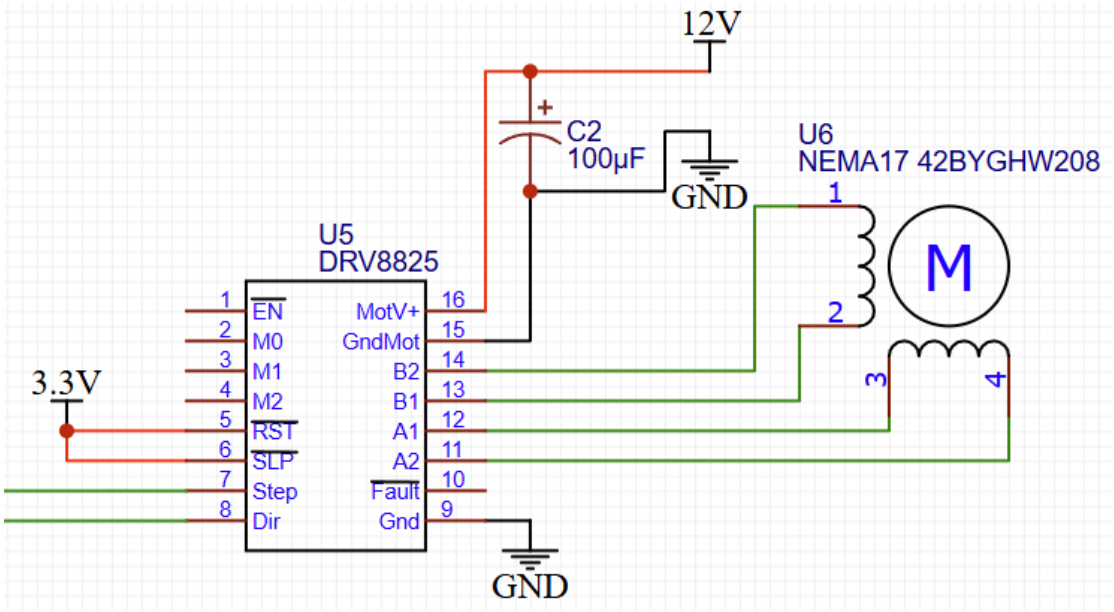
Εικόνα 40 - Διάταξη των ιμάντων στο coreXY

Οι εκτυπωμένες δοκοί περάστηκαν με γυαλόχαρτο για να ολισθαίνουν ευκολότερα οι ρόδες, διότι λόγω της κατακόρυφης τρισδιάστατης εκτύπωση οι γραμμές μεταξύ των layers είναι τραχείες. Αυτό δυσχεραίνει τη μεταφορά του άξονα X σε μεγάλες ταχύτητες κάτι που ήταν σημαντικό να αντιμετωπιστεί.



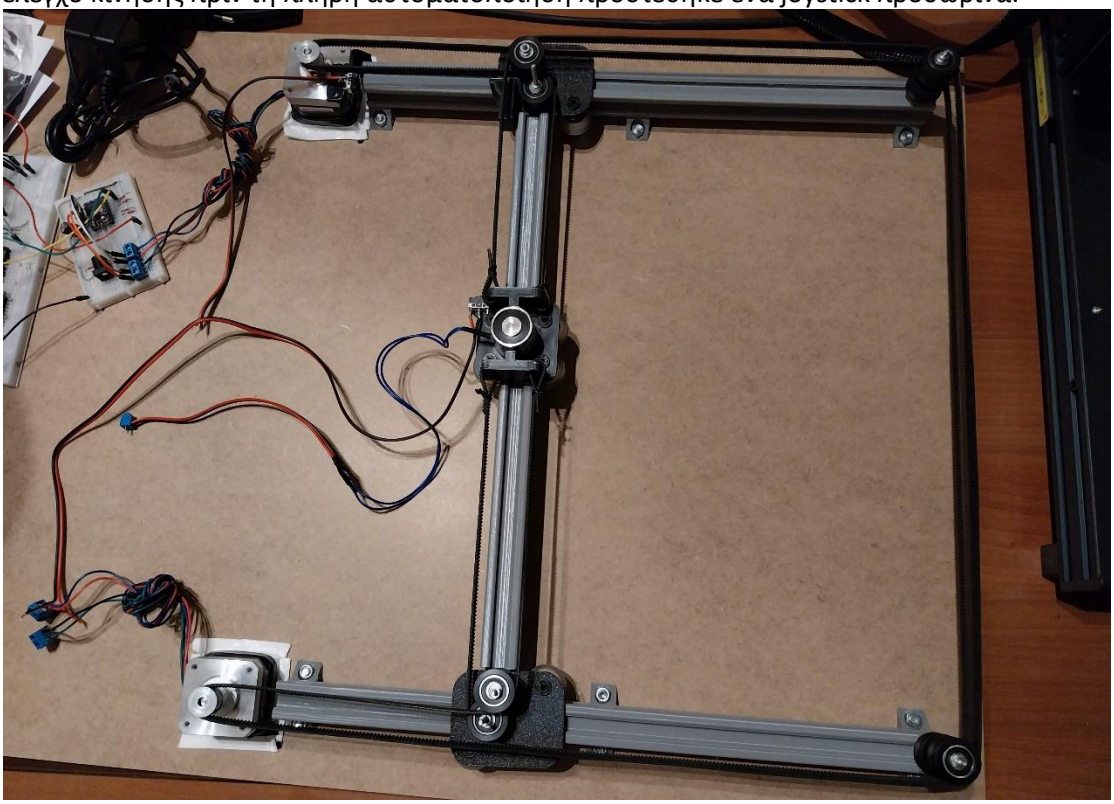
Εικόνα 42 - Πρώτο στάδιο κατασκευής του συστήματος κίνησης

Οι βηματικοί κινητήρες προστέθηκαν στο σύστημα μετά την αρχική συναρμολόγηση αρχικά με ταινία διπλής όψης, η συνδεσμολογία των μοτέρ και των driver τους φαίνεται στο schematic, με ιδιαίτερη προσοχή τον πυκνωτή 100μF που προστατεύει το DRV8825 από αιχμές υψηλής τάσης κατά τη λειτουργία του (DRV8825 Datasheet, 2014).



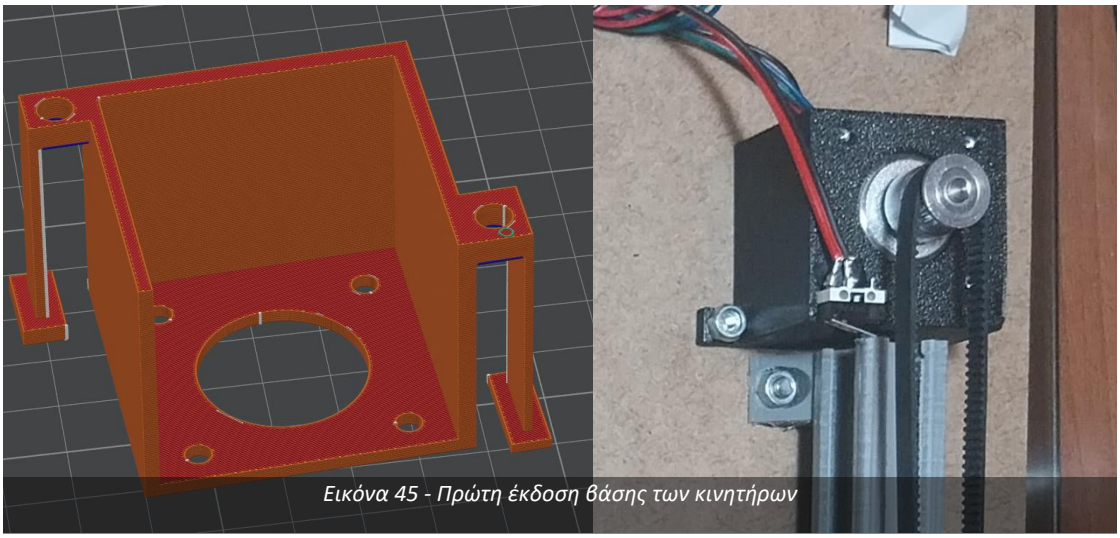
Εικόνα 43 - Συνδεσμολογία DRV8825

Προστέθηκαν οι δύο διακόπτες που καθορίζουν το σημείο μηδέν των αξόνων και η κίνηση σε αυτό το στάδιο γίνεται μέσω εντολών στο serial monitor. Στο επόμενο στάδιο για καλύτερο έλεγχο κίνησης πριν τη πλήρη αυτοματοποίηση προστέθηκε ένα joystick προσωρινά.



Εικόνα 44 - Δεύτερο στάδιο κατασκευής του συστήματος κίνησης

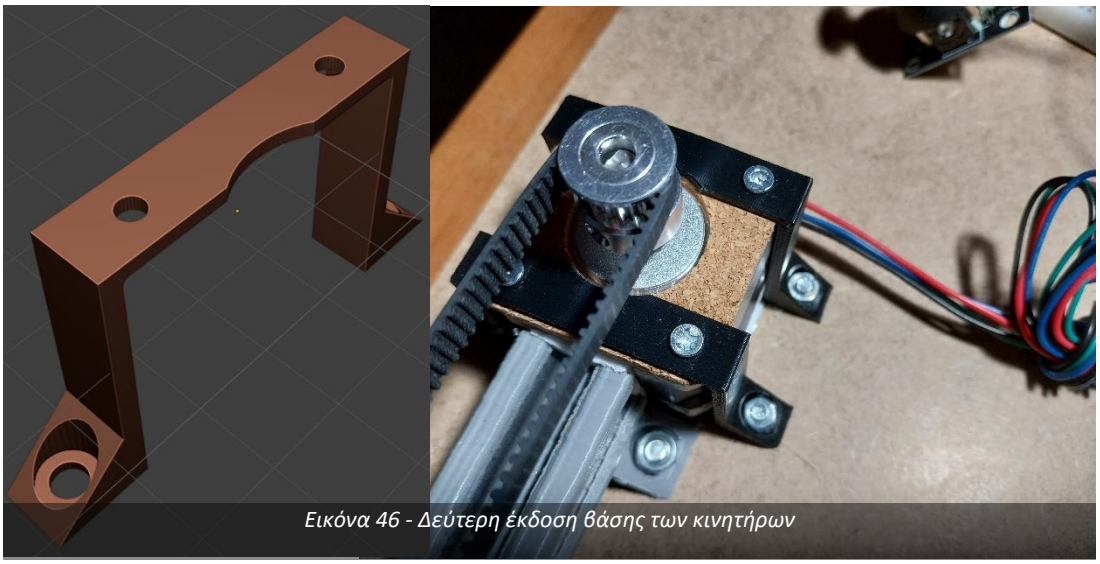
Με την ταινία διπλής όψης το τέντωμα των ιμάντων δεν ήταν σταθερό άρα χρειάστηκε σχεδίαση και εκτύπωση ειδικής βάσης που θα κράταγε τον άξονα περιστροφής σταθερό.



Εικόνα 45 - Πρώτη έκδοση βάσης των κινητήρων

Η πρώτη έκδοση αυτών των βάσεων ενθυλάκωναν τους κινητήρες κρατώντας τους τελείως ακίνητους. Αυτό είχε ως αποτέλεσμα, λόγω των δονήσεων κατά τη χρήση τους, η βάση ξύλου να δρα ως ενισχυτής ήχου με τόσο θόρυβο κατά τη λειτουργεία που θα εμπόδιζε το παιχνίδι.

Για την επίλυση αυτού του προβλήματος σχεδιάστηκαν διαφορετικές βάσεις ανοιχτού τύπου ώστε να μην απορροφούν τις δονήσεις αλλά να κρατάνε τον άξονα περιστροφής σταθερό.

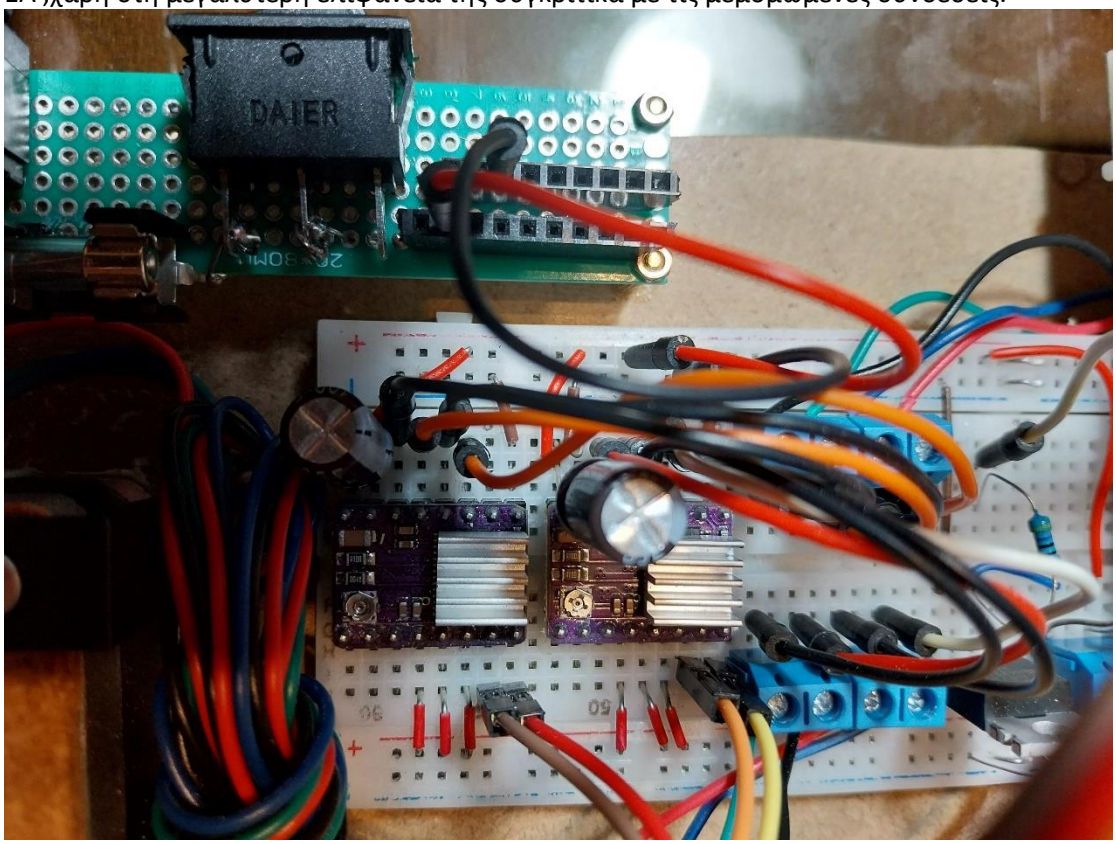


Εικόνα 46 - Δεύτερη έκδοση βάσης των κινητήρων

Αυτές οι βάσεις μείωσαν σημαντικά τον θόρυβο των κινητήρων και παρέμειναν μέχρι το τέλος της κατασκευής. Με αυτό το βήμα το σύστημα κίνησης είναι έτοιμο.

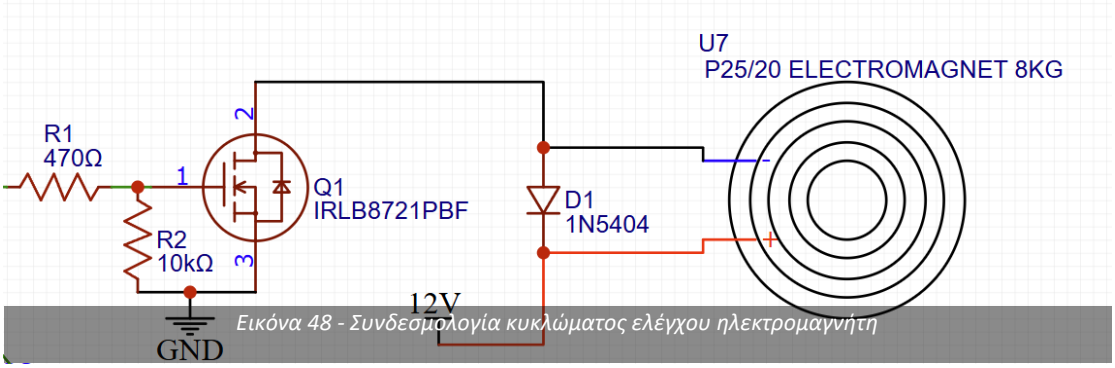
Οι συνδέσεις έγιναν σε breadboard με το κύκλωμα της πηγής σε perfboard για ασφάλεια λόγω μεγαλύτερου ρεύματος. Ο λόγος που δεν απαιτούνται παράλληλα καλώδια για τροφοδοσία είναι η χρήση της ράγας του breadboard που μπορεί να άγει περισσότερα από

1A ,χάρη στη μεγαλύτερη επιφάνεια της συγκριτικά με τις μεμομωμένες συνδέσεις.



Εικόνα 47 - Κάτοψη κυκλώματος των Drivers

Το λογικό επόμενο βήμα είναι το κύκλωμα του ηλεκτρομαγνήτη αφού το σύστημα κίνησης του είναι έτοιμο. Αυτό είναι απλό χάρη στην έρευνα αγοράς από την οποία καθορίστηκαν όλα τα υλικά και στη παρακάτω συνδεσμολογία τους στο schematic



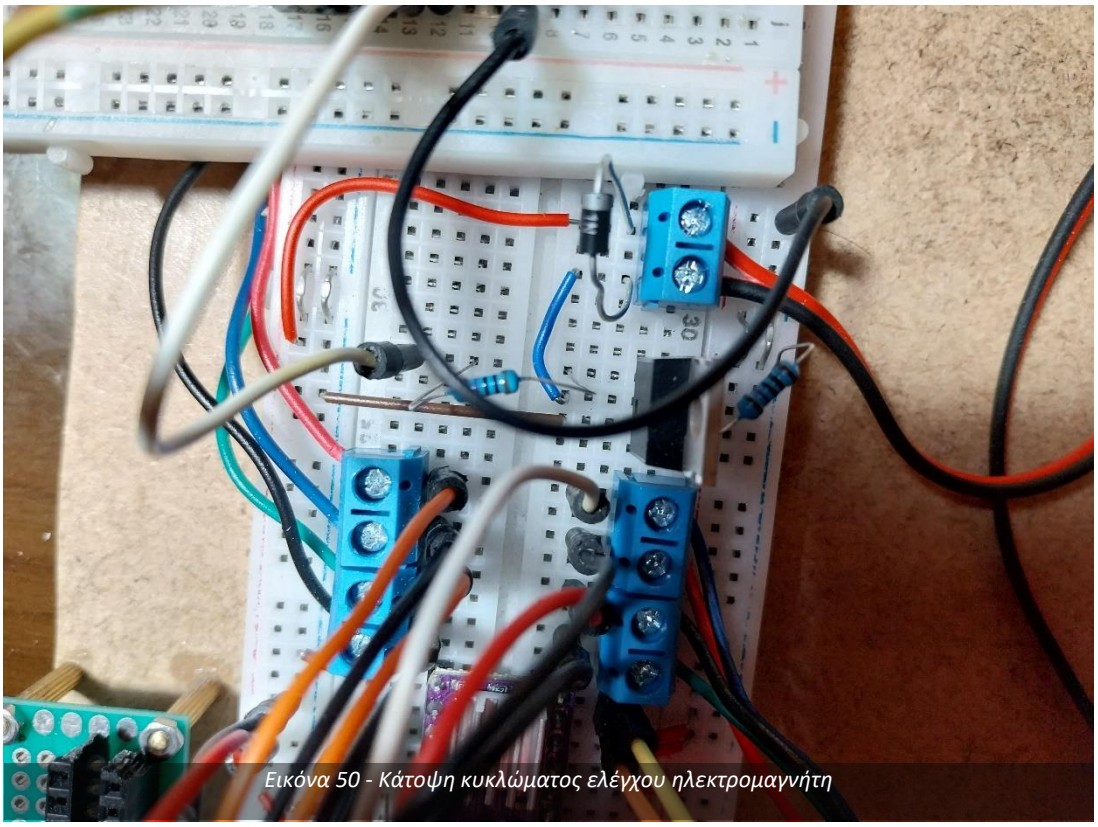
Εικόνα 48 - Συνδεσμολογία κυκλώματος ελέγχου ηλεκτρομαγνήτη

Σε αυτό το στάδιο επιλέχτηκαν οι μαγνήτες που θα χρησιμοποιηθούν στις βάσεις των πιονιών από το εξωτερικό με το βασικό χαρακτηριστικό που οδήγησε στην επιλογή τους τη δύναμη του μαγνητικού πεδίου που προσφέρουν. Οι μαγνήτες νεοδυμίου είναι η πιο δημοφιλής επιλογή για φυσιολογικές θερμοκρασίες δωματίου και με βάση τη καθαρότητα τους αγοράστηκαν 50 μαγνήτες τύπου N52 (Neodymium magnet, 2026), που είναι η μεγαλύτερη καθαρότητα που βρέθηκε με την μέση καθαρότητα να είναι N35.



Εικόνα 49 - Μαγνήτες N52

Το κύκλωμα ελέγχου του ηλεκτρομαγνήτη, όπως παρέμεινε για το μεγαλύτερο στάδιο της κατασκευής:



Εικόνα 50 - Κάτοψη κυκλώματος ελέγχου ηλεκτρομαγνήτη

Η μόνη αλλαγή που έγινε σε αυτό το κύκλωμα ήταν να μετακινηθεί το pull-down resistor από τη φυσική θέση του που βρισκόταν πίσω από το transistor, όπως φαίνεται στην εικόνα, σε μπροστινή θέση μαζί με το gate resistor. Αυτή η αλλαγή έγινε λόγω της διαπίστωσης πως το πίσω μέρος του transistor είναι μεταλλικό και είναι ασαφές πού συνδέεται εσωτερικά, άρα η αλλαγή ήταν καθαρά προληπτική για να μην αγγίξει το resistor στο πίσω μέρος κατά τη μεταφορά της κατασκευής.

Με το κάτω μέρος της σκακιέρας συναρμολογημένο ακολούθησε η κατασκευή του πάνω μέρους το οποίο ξεκίνησε από το σύστημα ανίχνευσης που τοποθετείται κάτω από τη σκακιέρα.

Η πρόκληση του συγκεκριμένου μέρους είναι το πλήθος των αισθητηρίων και οι ακριβής θέσεις που πρέπει να βρίσκονται για τη σωστή λειτουργία του πλέγματος. Τα καλώδια επίσης απαιτούν οργάνωση με 64 καλώδια και μία γείωση τα οποία θα πρέπει να συνδεθούν στο κύκλωμα με τους πολυπλέκτες στο breadboard. Το πλέγμα θα πρέπει να τοποθετηθεί με τέτοιο τρόπο ώστε να βρίσκεται πλήρως εντός της επιφάνειας που καλύπτει το σύστημα κίνησης και με το ελάχιστο δυνατό ύψος για την όσο δυνατόν μικρότερη μείωση της μαγνητικής έλξης. Η δυσκολία σύνδεσης ήταν ένας από τους κύριους λόγους που απορρίφθηκαν τα hall-effect sensors, αλλά εκ των υστέρων μετά την ολοκλήρωση της κατασκευής διαπιστώθηκε πως είχαν μεγαλύτερες προοπτικές παρά τη δυσκολία κατασκευής τους και χρειαζόντουσαν περισσότερη εξέταση. Ωστόσο επιλέχτηκαν reed switches για το πλέγμα που λειτουργούν σε ικανοποιητικό βαθμό για ένα πρωτότυπο (Reed switch, 2026).

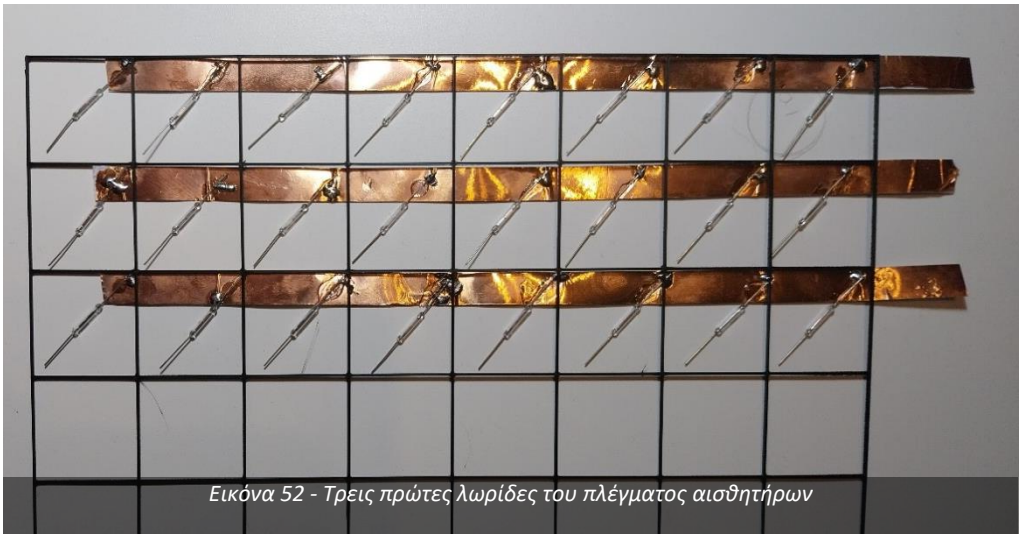
Για να διευκολυνθεί η κατασκευή του πλέγματος χρησιμοποιήθηκαν δύο μέθοδοι. Σχεδιάστηκε και τυπώθηκε ένας κάναβος με τις ακριβείς διαστάσεις τις σκακιέρας με διαχωριστικά οριζόντια και κάθετα ανά τετράγωνο για τη τοποθέτηση των αισθητηρίων. Η χρήση χαλκοταινίας (copper tape) ήταν ιδιαίτερα χρήσιμη καθώς παρείχε ταυτόχρονα μία επιφάνεια για να κολληθούν οι αισθητήρες, μία μονωμένη επιφάνεια για να οργανωθούν τα καλώδια και τέλος μία κοινή γείωση.

Κατά τη κατασκευή διαπιστώθηκε πως η επιλογή μίας επιφάνειας χαλκού θα δημιουργούσε eddy currents (Eddy current, 2026), ένα φαινόμενο που μία αγώγιμη επιφάνεια δημιουργεί αντίθετη δύναμη σε έναν κινούμενο μαγνήτη, σαν αυτούς που θα τοποθετηθούν στις βάσεις των κομματιών. Σε αυτό το φαινόμενο βασίζονται φρένα τρένων, καθώς δεν παρουσιάζει τριβές. Με παραπάνω μελέτη λόγω του ξύλου μεταξύ των κινούμενων μαγνητών και της χαλκοταινίας κρίθηκε πως αυτή η αντίθετη δύναμη θα ήταν τόσο μικρή που δεν θα σταματούσε τους μαγνήτες και άξιζαν τα πλεονεκτήματα της χαλκοταινίας το ρίσκο.



Εικόνα 51 - Χαλκοταινία

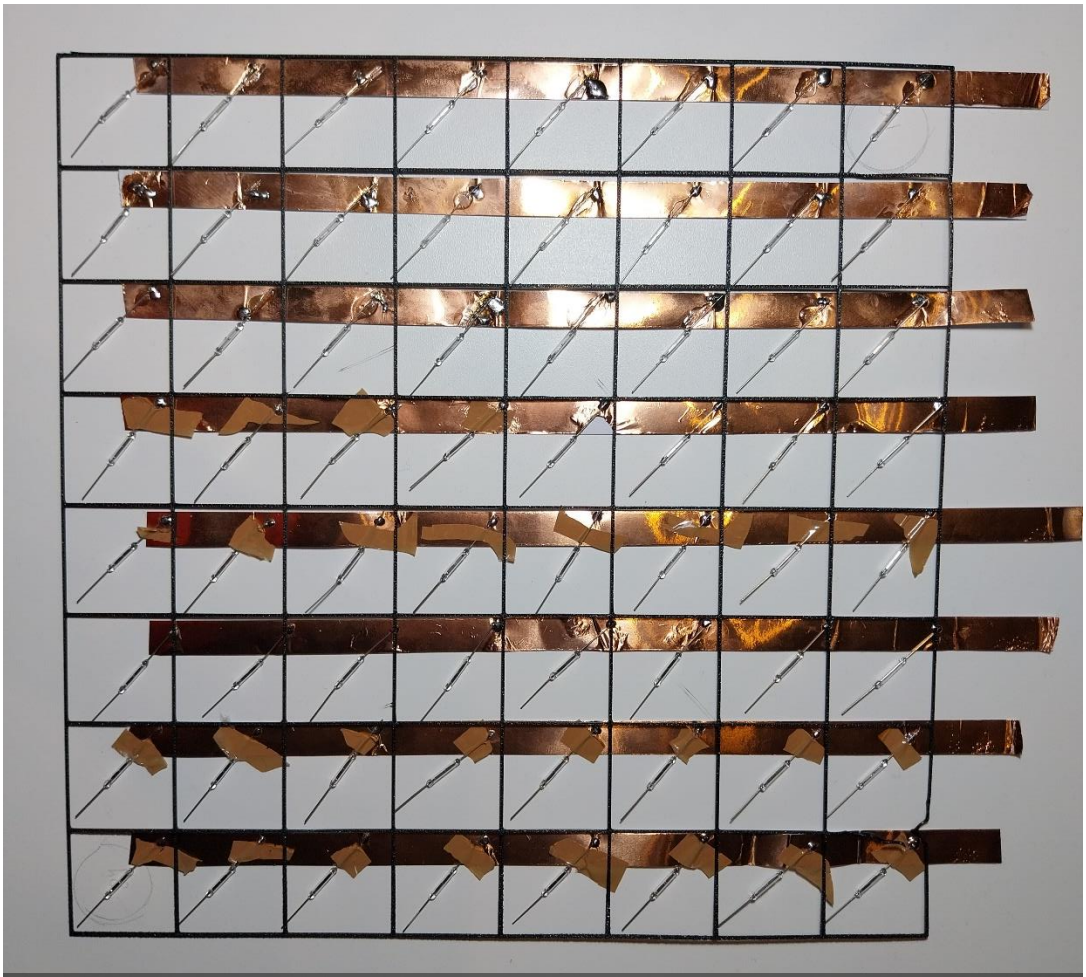
Η κατασκευή έγινε σε λωρίδες, κόβοντας τη ταινία στο σωστό μήκος, κολλώντας τους αισθητήρες πάνω της με κόλλα στιγμής και τέλος τη συγκόλληση. Στη παρακάτω φωτογραφία φαίνονται οι τρεις πρώτες λωρίδες που έγιναν με αυτή τη μέθοδο:



Εικόνα 52 - Τρεις πρώτες λωρίδες του πλέγματος αισθητήρων

Αυτή η διαδικασία ήταν αρκετά χρονοβόρα καθώς για την επιτυχία της τελικής κατασκευής έπρεπε να ελέγχονται με το πολύμετρο κατά μονάδα κρατώντας έναν μαγνήτη από πάνω τους για να διαπιστωθεί αν λειτουργεί σωστά ο αισθητήρας. Από τη μεταφορά, λόγω της παραγγελίας από το εξωτερικό, πολλά μεταλλικά pins είχαν στραβώσει και κατά το ίσιωμα λόγω της ευαίσθητης φύσης αυτών των αισθητήρων αποσυνδεόντουσαν απαιτώντας πλήρη αλλαγή. Για αυτά τα προβλεπόμενα σφάλματα έγινε παραγγελία 100 αισθητήρων αντί για μόνο των απαιτούμενων 64 και μέχρι τη τελική κατασκευή χάλασαν γύρω στους 10 από αυτούς. Η κατεύθυνση του αισθητήρα πρέπει να είναι σωστή για την ανίχνευση με τη μεριά με την μεγάλη επιφάνεια του μετάλλου μέσα στο γυαλί να δείνχει προς τα πάνω για να μεγιστοποιηθεί η περιοχή ανίχνευσης του αισθητήρα, κάτι που σύμβαλλε σε επόμενα προβλήματα επικάλυψης των αισθητήρων, περισσότερα για αυτό στην αποσφαλμάτωση του από το πρόγραμμα.

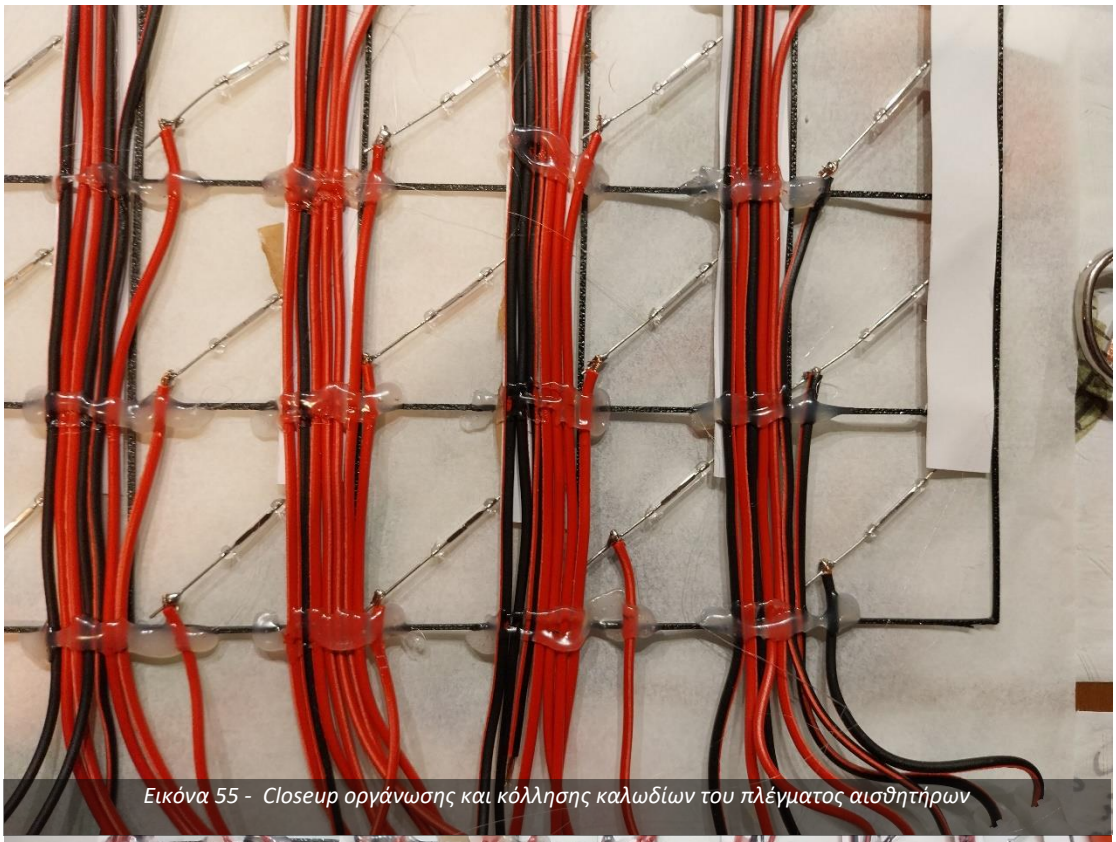
Συνεχίζοντας αυτή τη διαδικασία για τις υπόλοιπες λωρίδες κατασκευάστηκε η κοινή γείωση στην οποία θα ενωθούν μεταξύ τους οι χαλκοταινίες στο δεξί μέρος της φωτογραφίας.



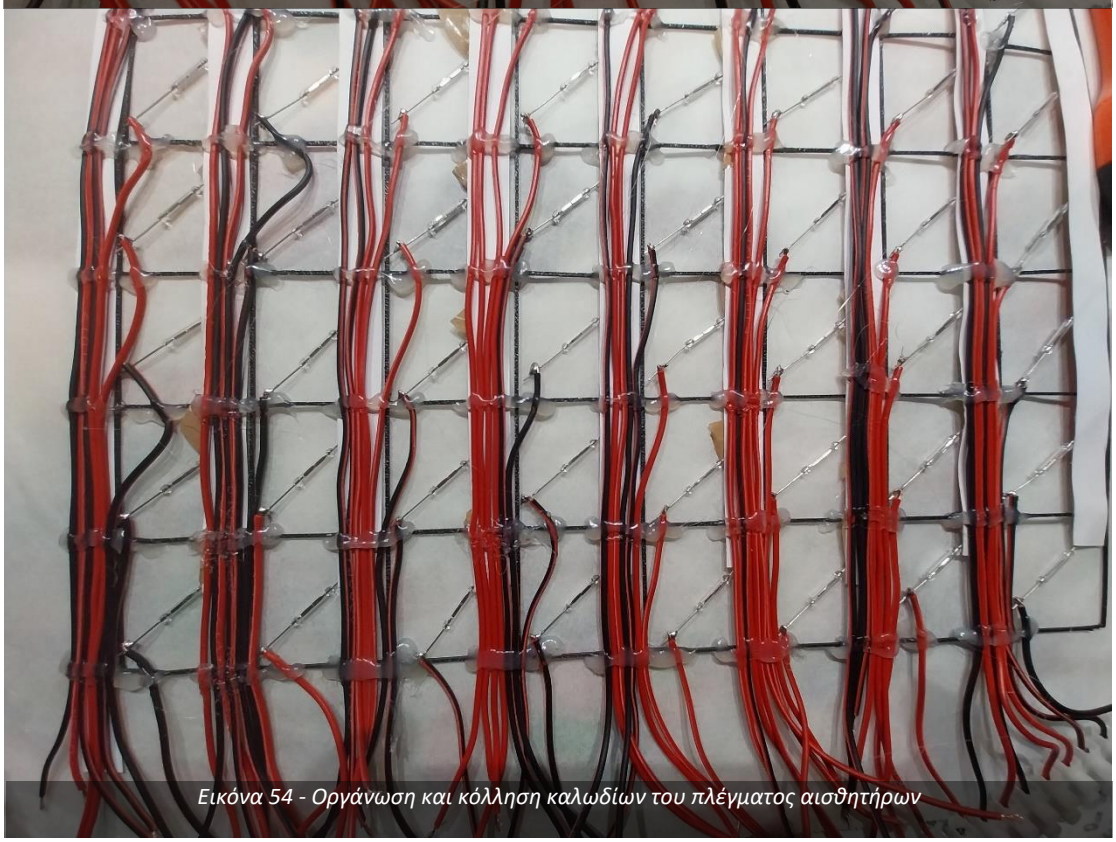
Εικόνα 53 - Οι λωρίδες του πλέγματος αισθητήρων

Τα καλώδια ήταν απλούστερα στο άλλο pin των αισθητήρων καθώς ο αισθητήρας έχει συγκολληθεί στη σωστή θέση πρόκειται για μία απλή συγκόλληση. Τα καλώδια κόπηκαν στα σωστά μήκη με ένα συντελεστή ασφαλείας με βάση τον κάναβο με σκοπό να χρησιμοποιηθούν κλέμες για την επέκταση τους στο breadboad. Οι κλέμες χρησιμεύουν για αποσφαλμάτωση και συντήρηση καθώς επιτρέπουν την αλλαγή μερικώς καλωδίου χωρίς να απαιτείται να κοπεί και να ανασυγκολληθεί. Με τα καλώδια συγκολλημένα άρχισε η διαδικασία κόλλησης με σιλικόνη διότι το πλέγμα θα τοποθετηθεί ανάποδα με τον ηλεκτρομαγνήτη να το έλκει προς τα κάτω απαιτείται αρκετή κόλλα για να μην πέσει. Τα καλώδια οργανώθηκαν από πάνω προς τα κάτω χρησιμοποιώντας τη μονωμένη μεριά της χαλκοταινίες ως επιφάνειας για να πατήσουν. Οι κολλήσεις σιλικόνης πρέπει επίσης να είναι όσο το δυνατόν πιο κοντές για να μην αυξήσουν το ύψος του πλέγματος και συγκρουστεί με τον ηλεκτρομαγνήτη στο σύστημα κίνησης.

Αρχίζοντας από το μακρύτερο καλώδιο και κολλώντας κάθε κοντότερο στο κοινό τους σημείο πάνω στην επιφάνεια της χαλκοταινίας τα καλώδια καταλήγουν οργανωμένα στο κάτω μέρος του πλέγματος σε οκτάδες.

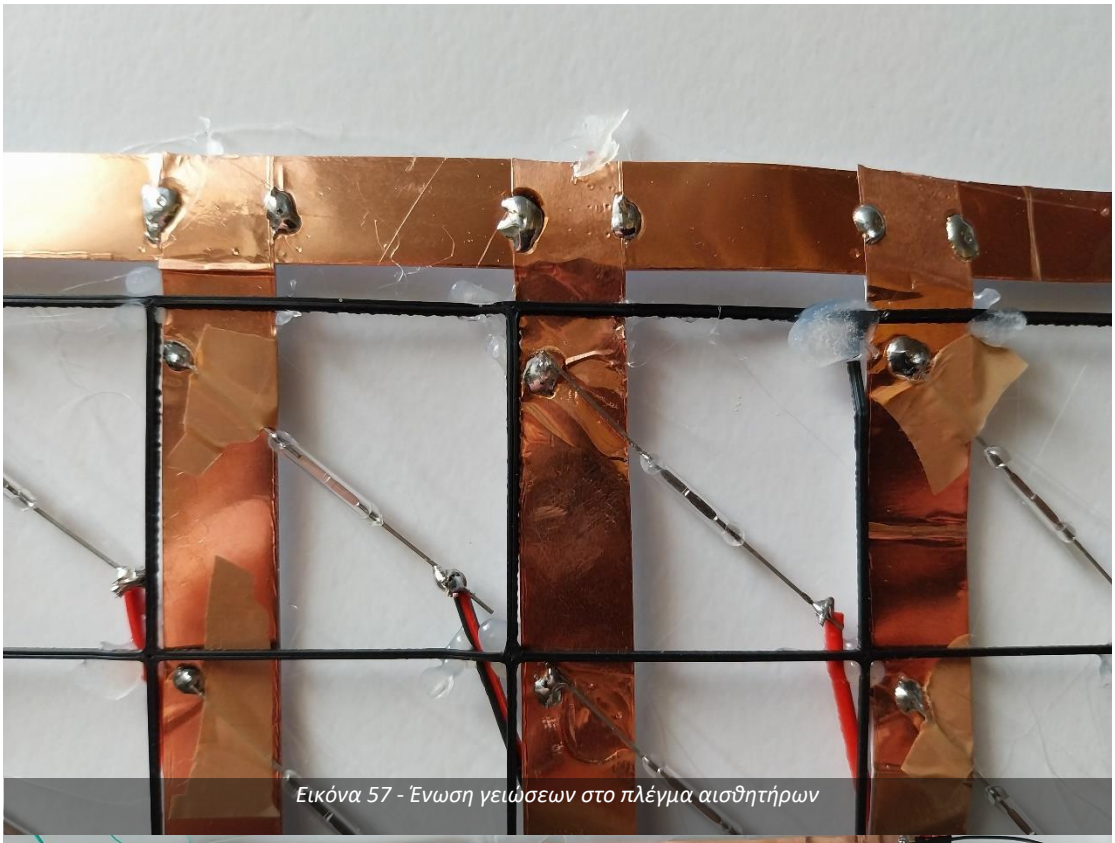


Εικόνα 55 - Closeup οργάνωσης και κόλλησης καλωδίων του πλέγματος αισθητήρων

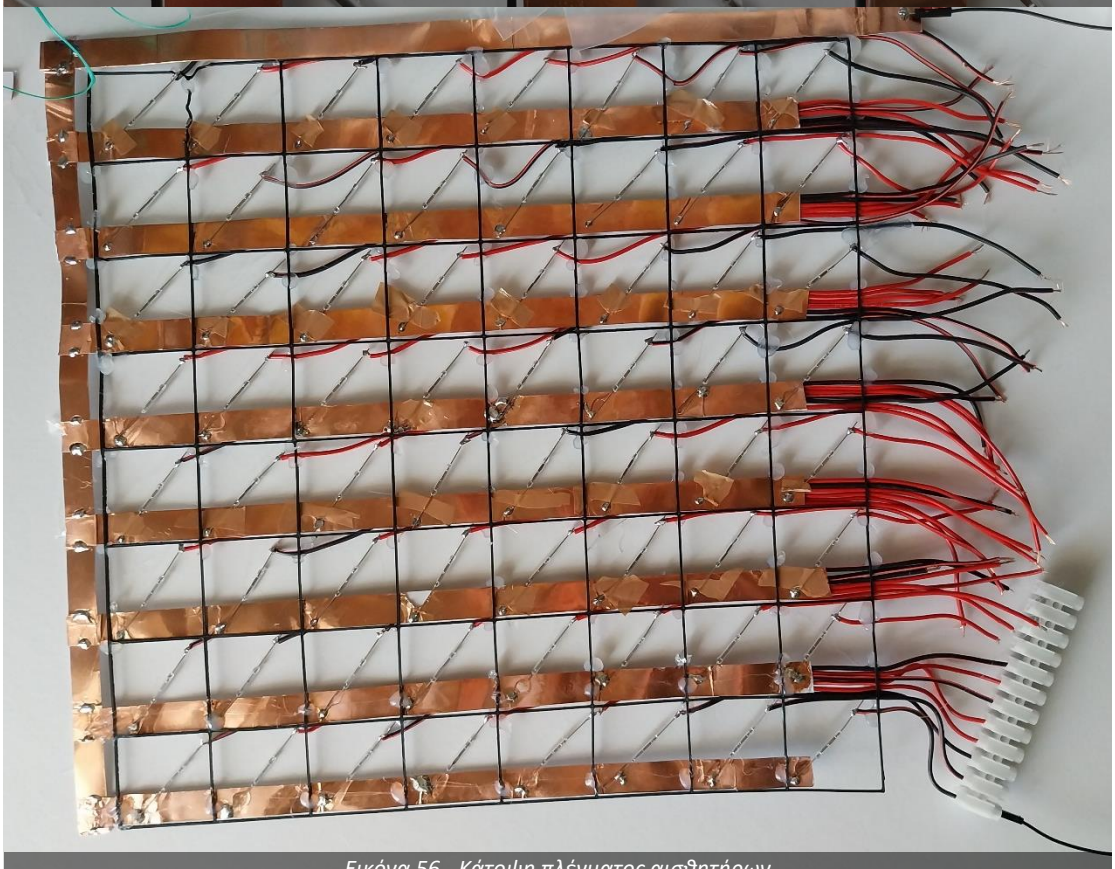


Εικόνα 54 - Οργάνωση και κόλληση καλωδίων του πλέγματος αισθητήρων

Από τη πάνω μεριά ενώθηκαν οι χαλκοταινίες για τη κοινή γείωση:

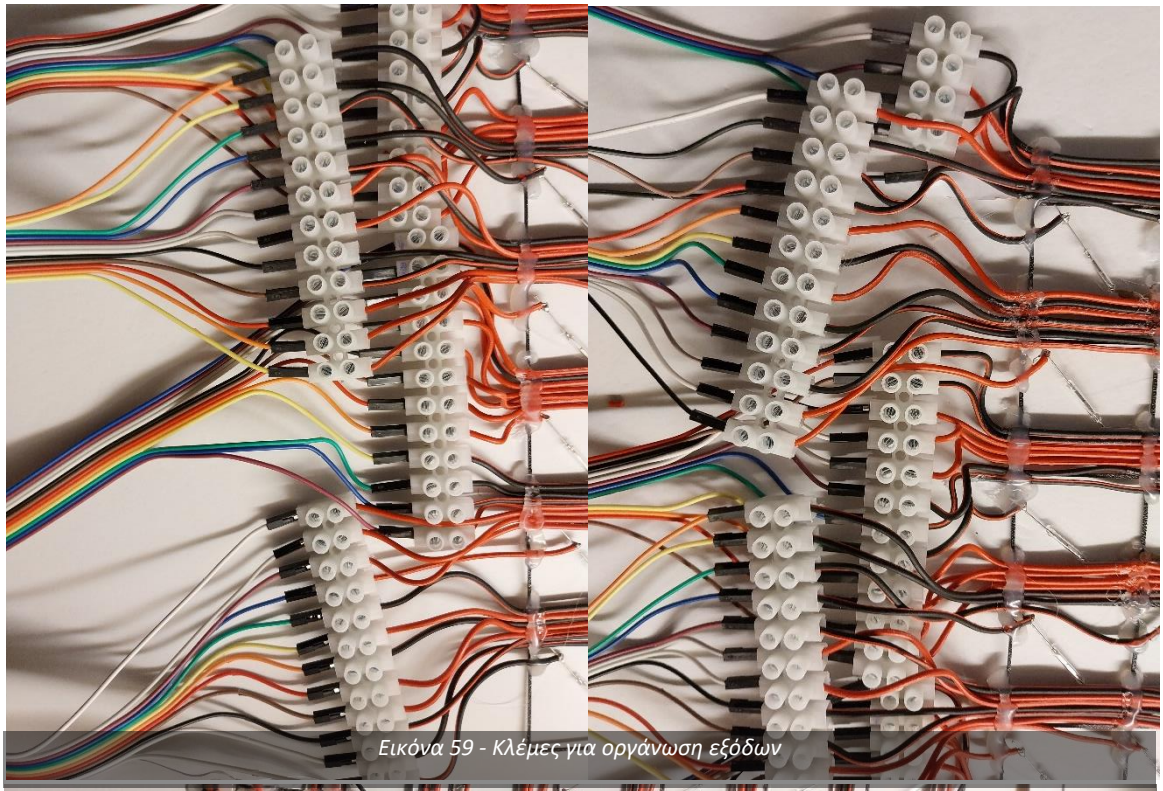


Εικόνα 57 - Ένωση γειώσεων στο πλέγμα αισθητήρων

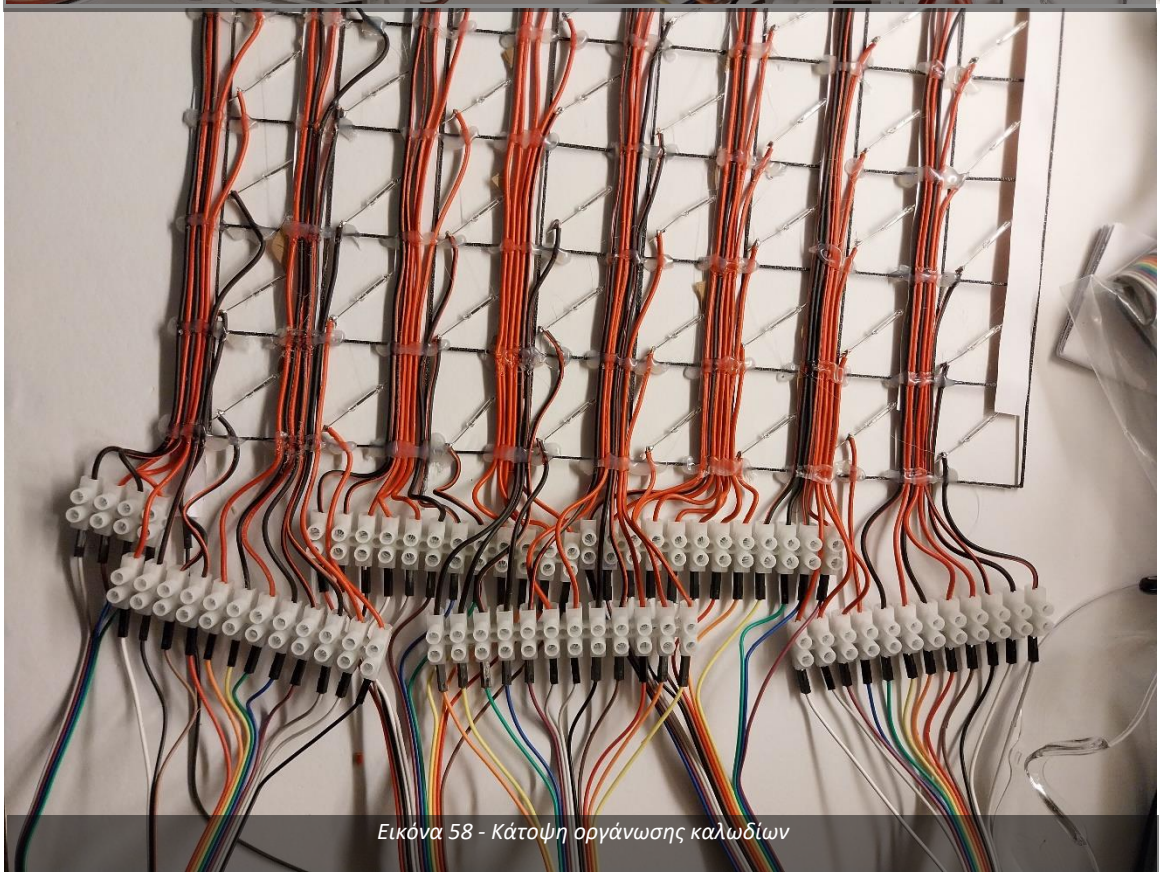


Εικόνα 56 - Κάτοψη πλέγματος αισθητήρων

Τέλος τοποθετήθηκαν όλες οι κλέμες για τα τελικά καλώδια για το breadboard.



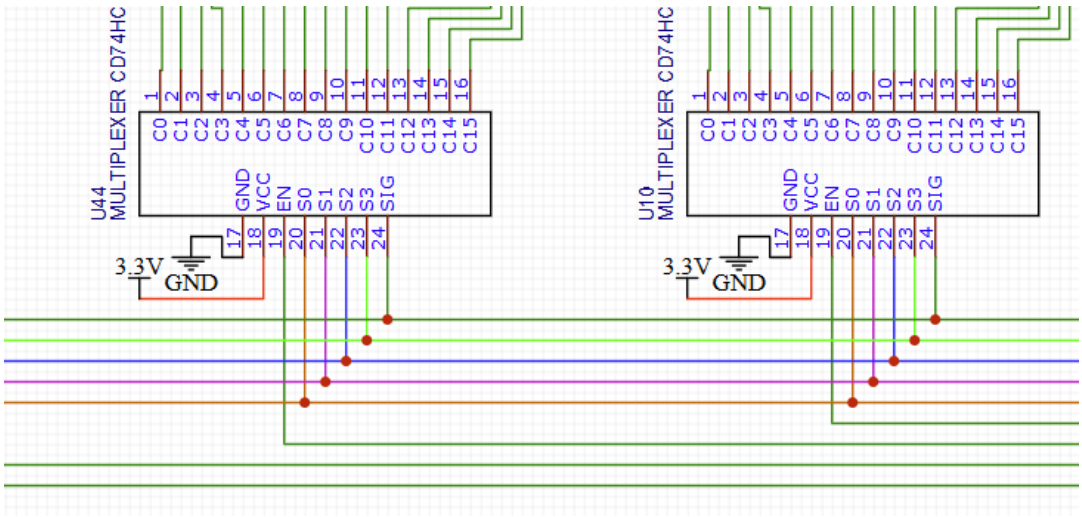
Εικόνα 59 - Κλέμες για οργάνωση εξόδων



Εικόνα 58 - Κάτοψη οργάνωσης καλωδίων

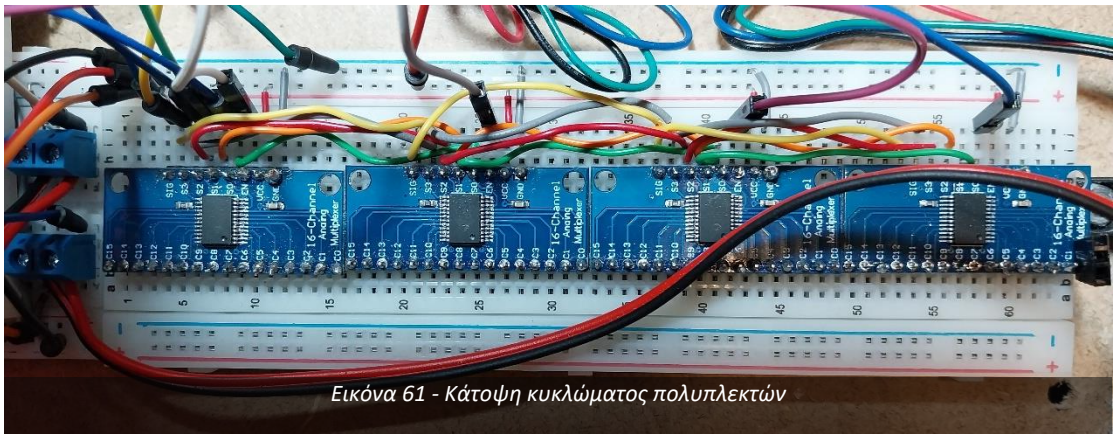
Με το πλέγμα αισθητήρων έτοιμο ακολουθεί το κύκλωμα των multiplexers τα οποία θα πάρουν τα 64 καλώδια και θα τα μειώσουν για να μπορούν να συνδεθούν στον μικροελεγκτή.

Από τέσσερις 16-bit multiplexers τα καλώδια θα μειωθούν από 64 σε 9 χάρη στην συνδεσμολογία πολυπλεξίας όπως φαίνεται στο schematic (Multiplexer, 2026).



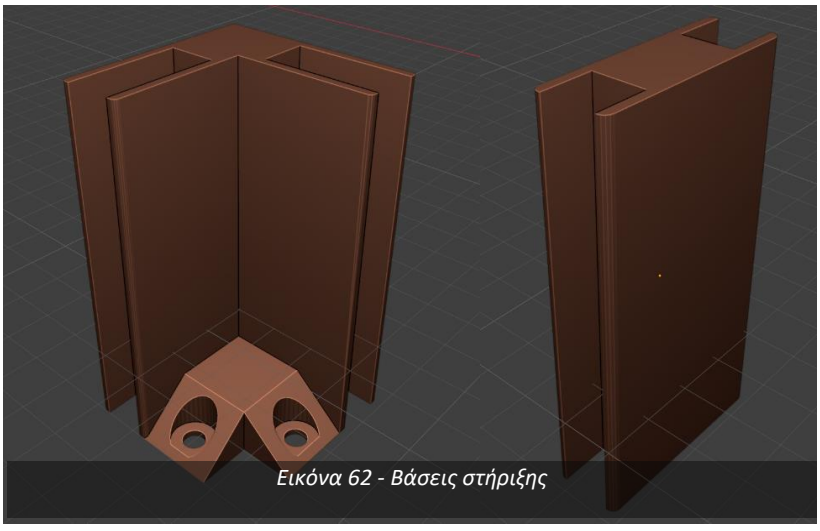
Εικόνα 60 - Συνδεσμολογία πολυπλεκτών

Για να γίνει αυτή η παράλληλη συνδεσμολογία των καλωδίων στο breadboard χρειάζονται πολλά μικρά καλώδια τα οποία καταλήγουν στις 9 εξόδους του κυκλώματος.



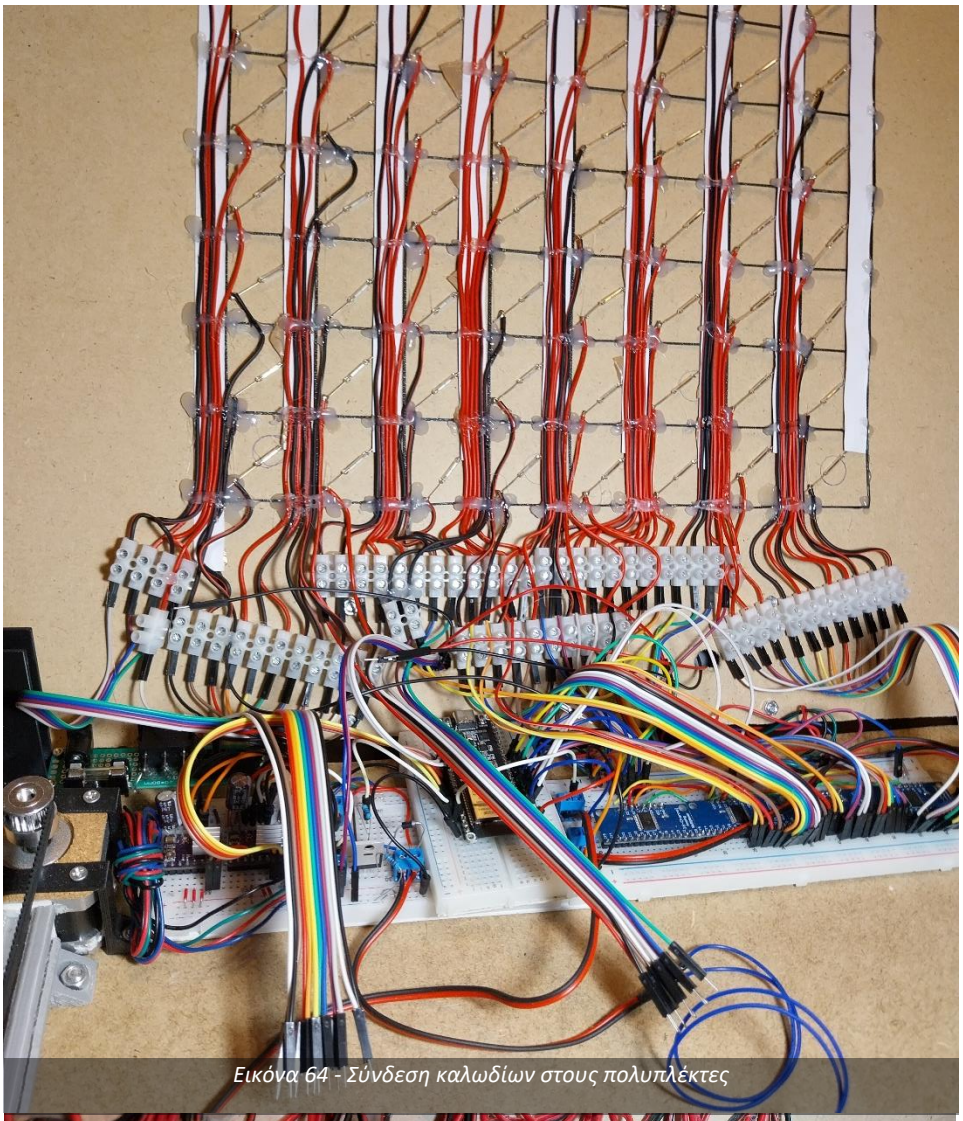
Εικόνα 61 - Κάτοψη κυκλώματος πολυπλεκτών

Σε αυτό το στάδιο σχεδιάστηκαν και τυπώθηκαν οι βάσεις στις οποίες στηρίζεται το πάνω μέρος της σκακιέρας με 8 συνολικά σημεία στήριξης λόγω αδυναμίας στήριξης στο κέντρο εξαιτίας του συστήματος κίνησης, κάτι απαραίτητο για να ελαχιστοποιηθεί το τόξο κάμψης.

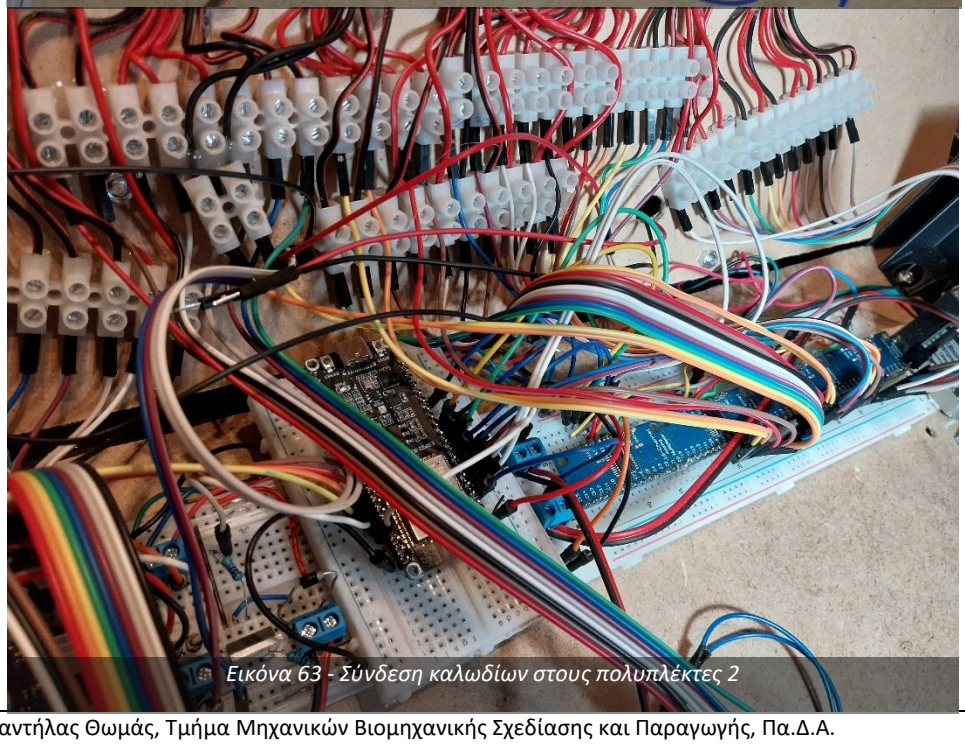


Εικόνα 62 - Βάσεις στήριξης

Με τα στηρίγματα έτοιμα ακολουθεί οι σύνδεση των καλωδίων στα multiplexers:

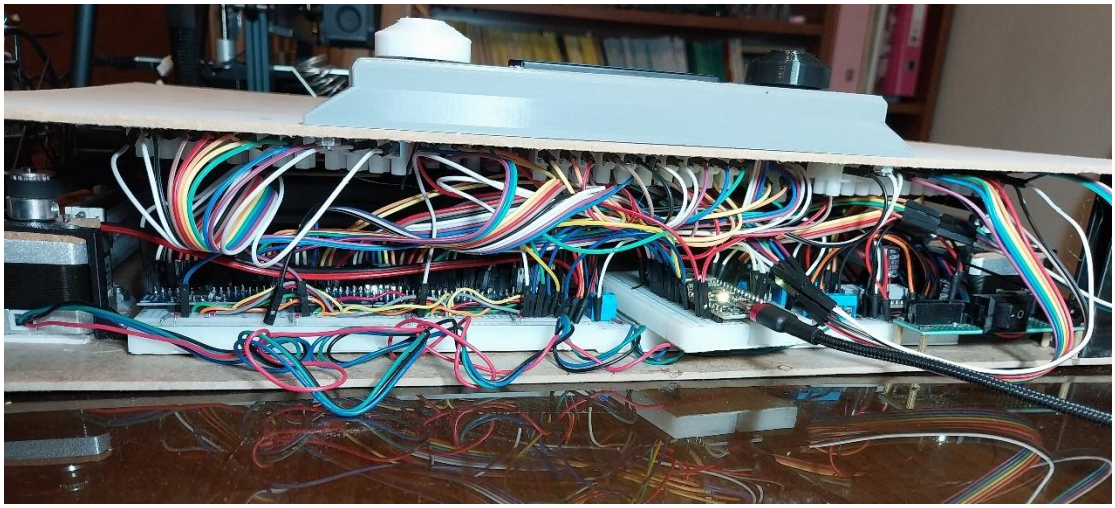


Εικόνα 64 - Σύνδεση καλωδίων στους πολυπλέκτες



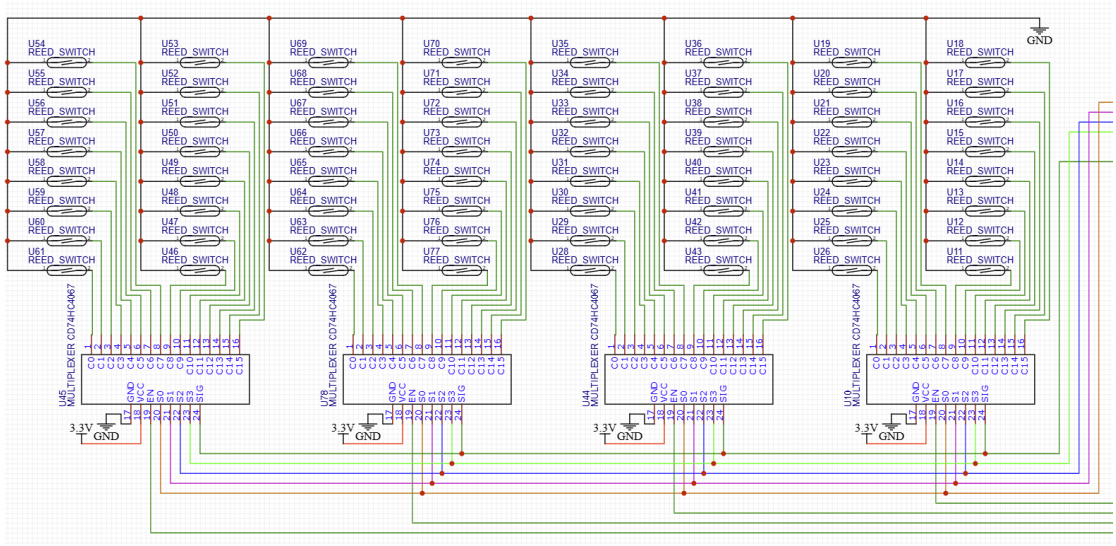
Εικόνα 63 - Σύνδεση καλωδίων στους πολυπλέκτες 2

Και η όψη των καλωδίων στη τελική κατασκευή:



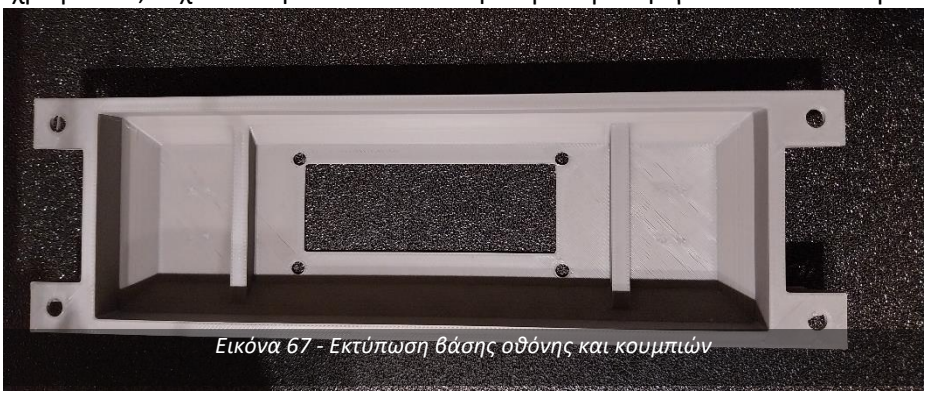
Εικόνα 65 - Πρόσοψη μετά τη σύνδεση του πλέγματος

Αυτό το μέρος της κατασκευής ήταν κατασκευαστικά το πιο δύσκολο για την απαιτούμενη ακρίβεια και το πλήθος των συνδέσεων, αλλά με το πλέγμα έτοιμο η κατασκευή πλησιάζει την ολοκληρωμένη της κατάσταση



Εικόνα 66 - Συνδεσμολογία πλέγματος αισθητήρων

Για την διεπαφή του χρήστη, που στο hardware αντιπροσωπεύεται με μία οθόνη και δύο κουμπιά των χρωμάτων, σχεδιάστηκε και τυπώθηκε μία βάση για τα απαιτούμενα εξαρτήματα:



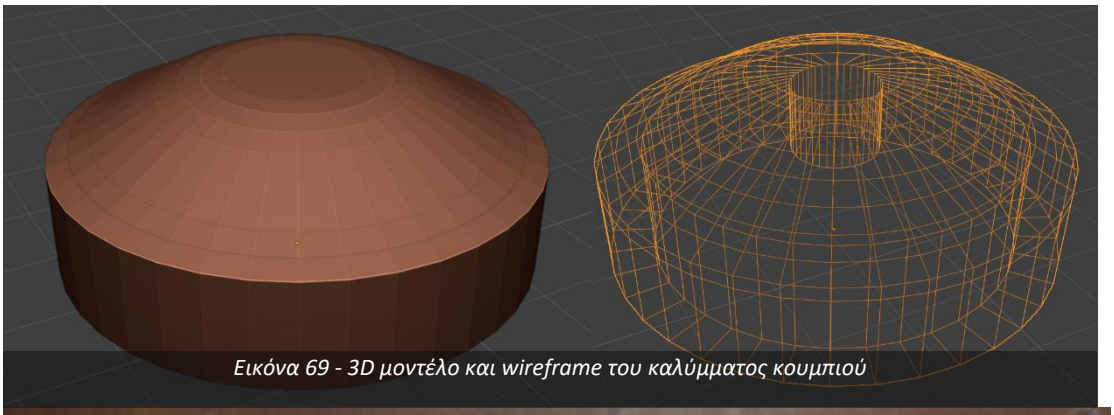
Εικόνα 67 - Εκτύπωση βάσης οθόνης και κουμπιών

Πάνω σε αυτήν τοποθετήθηκαν τα δύο pushbuttons και η LCD οθόνη η οποία καταλάθος βιδώθηκε ανάποδα τη πρώτη φορά:



Εικόνα 68 - Βάση διεπαφής χρήστη συνδεδεμένη ανάποδα

Για τα κουμπιά τα ίδια σχεδιάστηκαν καλύμματα:



Εικόνα 69 - 3D μοντέλο και wireframe του καλύμματος κουμπιού

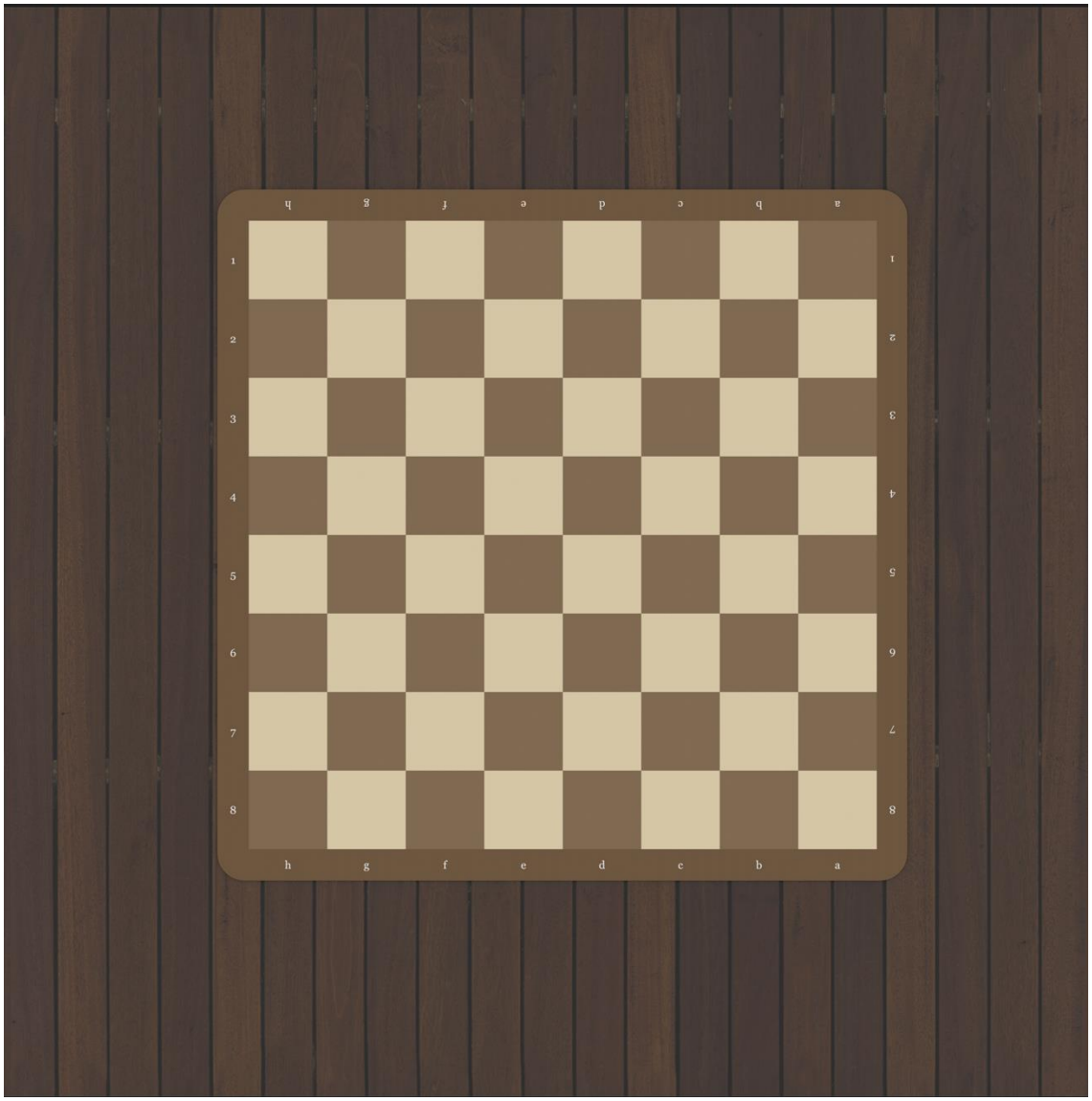


Εικόνα 70 - Κάτοψη διεπαφής χρήστη

Τα οποία κολλήθηκαν πάνω στα pushbuttons ολοκληρώνοντας τη διεπαφή του χρήστη με το σύστημα:

Αυτό ήταν το τελικό ηλεκτρονικό μέρος της κατασκευής και σε αυτό το στάδιο ξεκίνησε ο προγραμματισμός του τελικού συστήματος παράλληλα με την βελτίωση εμφάνισης της κατασκευής.

Το τελικό μέρος που μένει, εξαιρώντας τα πιόνια, είναι η εμφάνιση της σκακιέρας της ίδιας καθώς σε αυτό το μέρος της κατασκευής είναι ένα κενό ξύλο για το οποίο σχεδιάστηκε ένα render με τις απαραίτητες αποστάσεις για την ασσυμετρία του:



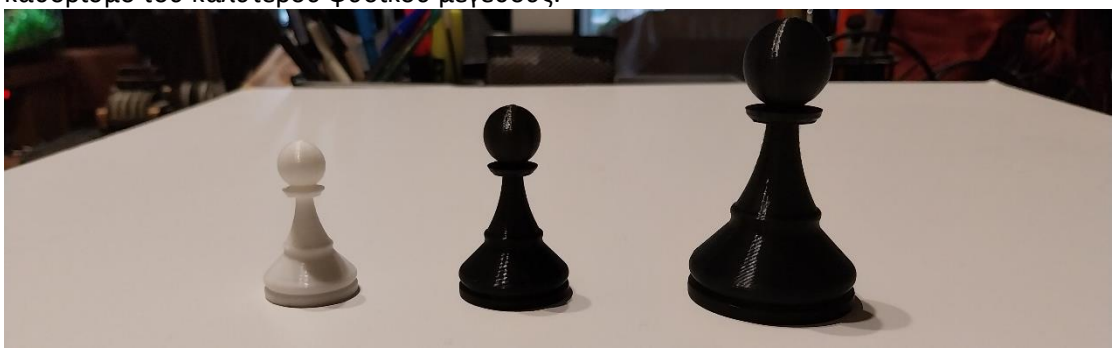
Εικόνα 71 - Σχέδιο εμφάνισης της σκακιέρας για εκτύπωση σε βινύλιο

Το σχέδιο υλοποιήθηκε από τοπικό τυπογραφείο σε αυτοκόλλητο βινύλιο καθώς η εκτύπωση σε απλό χαρτί απέδειξε πως η μεταφορά από ψηφιακό περιβάλλον σε φυσικό χαρτί έρχεται με απώλειες. Η τελική εκτύπωση είναι εξαιρετικά υψηλής ποιότητας και επιπρόσθετα μειώνει τη τριβή κάνοντας τις κινήσεις πιο ομαλές πέρα από την εμφάνιση.

Η εργονομία του παίκτη λήφθηκε υπόψη στο design τόσο της διεπαφής όσο και της σκακιέρας, με τα κουμπιά στη διεπαφή να κάνουν αντίθεση στο χρώμα του σώματος της, και οι γραμμές και στήλες να είναι ανεστραμμένες ανάλογα με το χρώμα του παίκτη ώστε να φαίνονται πάντα σωστά ανεξαρτήτως της μεριάς του. Τα τετράγωνα κάνουν έντονη αντίθεση μεταξύ τους και τα γράμματα και αριθμοί είναι ευδιάκριτα.

Η σταθερή κατασκευή είναι ολοκληρωμένη και μένει η σχεδίαση και εκτύπωση των πιονιών με τους μαγνήτες, που είναι το τελευταίο κομμάτι του hardware.

Για τα πιόνια τυπώθηκαν διαφορετικά μεγέθη και συγκρίθηκαν μεταξύ τους για το καθορισμό του καλύτερου φυσικού μεγέθους.



Εικόνα 72 - Πιόνια διαφορετικών μεγεθών

Με τις διαστάσεις από το πιόνι καθορισμένες, οι διαστάσεις των υπόλοιπων κομματιών ακολουθούν συγκεκριμένους κανονισμούς που για τουρνουά είναι το Staunton Design.

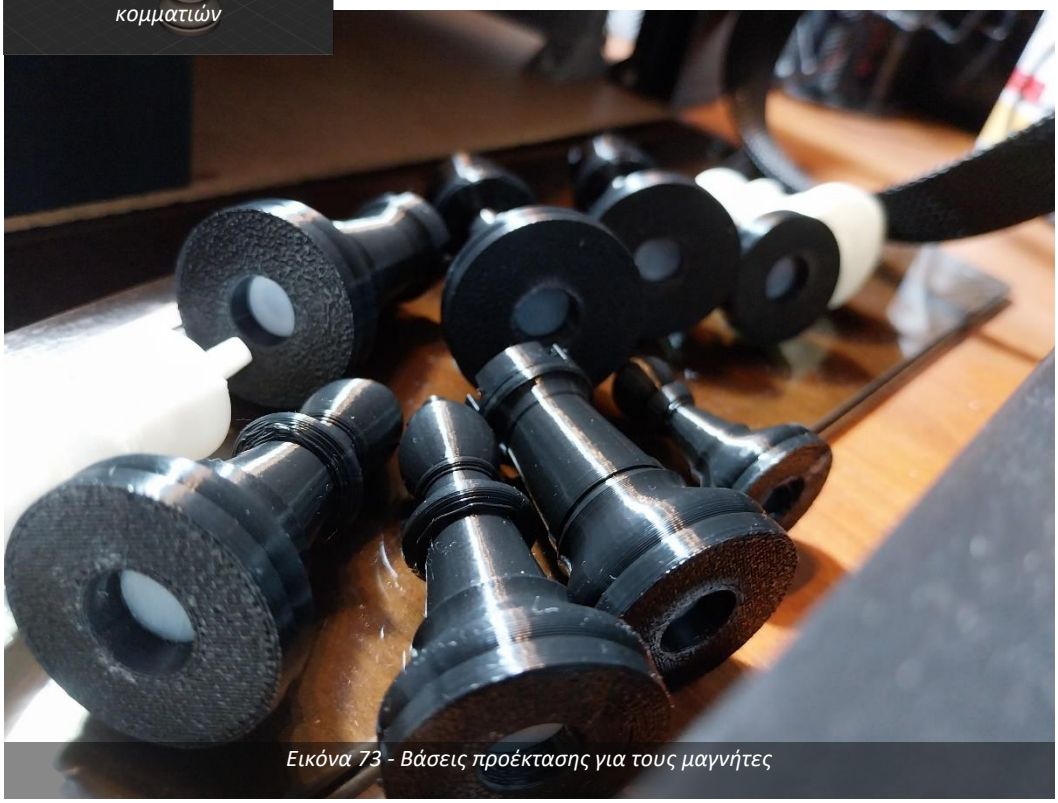


Εικόνα 74 - 3D μοντέλα των κομματιών

Χρησιμοποιήθηκε PLA για όλα τα κομμάτια καθώς είναι εξαιρετικά ελαφρύ, με το πιόνι στα 3 γραμμάρια και το βασιλιά στα 14, συγκριτικά με αγορασμένα ξύλινα ή πλαστικά σετ.

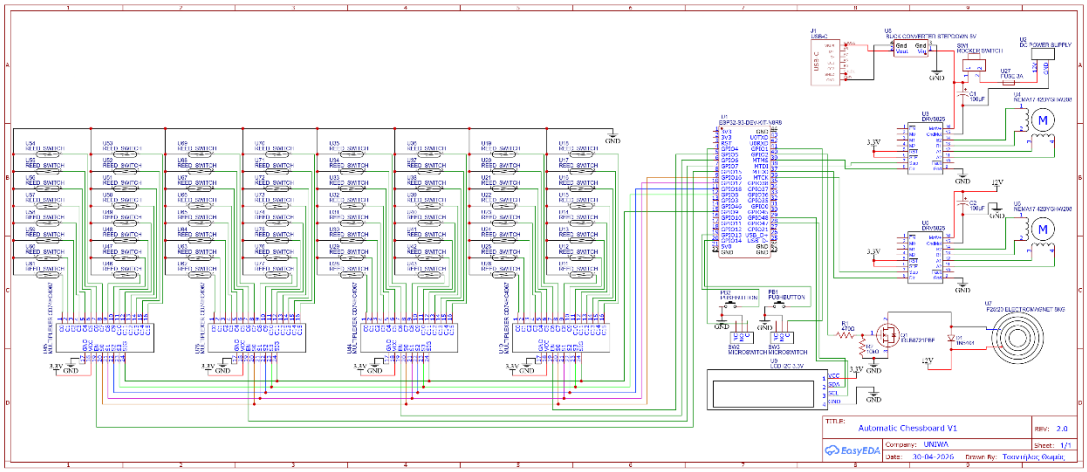
Τυπώθηκαν βάσεις για τον μαγνήτη να έρχεται σε επαφή με το ταμπλό λόγω της μείωσης της μαγνητικής έλξης από τη κόλλα.

Με τα κομμάτια έτοιμα ολοκληρώθηκε η κατασκευή και ακολούθησε το λογισμικό που αξιοποιεί τα λειτουργικά μέρη της σκακιέρας.



Εικόνα 73 - Βάσεις προέκτασης για τους μαγνήτες

Ηλεκτρονικό Κύκλωμα



Εικόνα 75 - Πλήρες schematic του συστήματος

Λίστα Υλικών

Ηλεκτρονικά

Reference	Όνομα	Model / Value	Qty	Σχόλια
U1	Microcontroller	ESP32-S3-DEV-KIT-N8R8	1	Ο μικροελεγκτής που τρέχει το software
U3, U5	Stepper Motor Driver	DRV8825	2	Δύο DRV8825 drivers βηματικών κινητήρων
U4, U6	Stepper Motor	NEMA17 42BGHW208	2	Βηματικοί κινητήρες για το σύστημα κίνησης
U7	Electromagnet (KK-P25/20)	8 kg	1	Μετακίνηση των μαγνητικών πιονιών
U8	Buck Converter (Step-Down)	5 V output	1	Μείωση της τάσης σε 5 V
U9	LCD Display (I2C)	LCD I2C 3.3 V	1	Η διεπαφή για τον παίκτη
U10, U44, U45, U78	16-Channel Analog Multiplexer	CD74HC4067	4	Πολυπλέκτες με 4×16 = 64 κανάλια για τα reed switches
U11–U26	Reed Switch	—	16	Μαγνητικοί διακόπτες για τις στήλες 1-2
U28–U43	Reed Switch	—	16	Μαγνητικοί διακόπτες για τις στήλες 3-4
U46–U61	Reed Switch	—	16	Μαγνητικοί διακόπτες για τις στήλες 5-6
U62–U77	Reed Switch	—	16	Μαγνητικοί διακόπτες για τις στήλες 7-8
U2	DC Power Supply	—	1	Η κύρια πηγή τροφοδοσίας
U27	Fuse	3 A	1	Προστασία από overcurrent

Q1	N-Channel MOSFET	IRLB8721PBF	1	Έλεγχος του ηλεκτρομαγνήτη
D1	Rectifier Diode	1N5404	1	Δίοδος flyback για προστασία από τον ηλεκτρομαγνήτη
R1	Resistor	470 Ω	1	Η αντίσταση της πύλης του transistor
R2	Resistor	10 kΩ	1	Pull-down αντίσταση για το transtistor
C1, C2	Electrolytic Capacitor	100 μF	2	Ομαλοποίηση τάσης για τα DRV8825 drivers
SW1	Rocker Switch	—	1	Κύριος διακόπτης ελέγχου τροφοδοσίας
SW2, SW3	Microswitch	—	2	Διακόπτες για την εύρεση του σημείου μηδέν (0,0) στο σύστημα κίνησης
PB1, PB2	Push Button	—	2	Τα δύο κουμπιά των χρωμάτων για τον παίκτη

Πίνακας 10 - Λίστα ηλεκτρονικών εξαρτημάτων

Μηχανικά:

Όνομα	**Qty/Value**	**Σχόλια**
Pulleys	2	Τροχαλίες για το σύστημα κίνησης.
Timing Belt	10m	Ιμάντες για το σύστημα κίνησης.
PLA White	1kg	Εκτύπωση λευκών κομματιών.
PLA Black	1kg	Εκτύπωση μαύρων κομματιών, στηριγμάτων και φορέων.
PLA Gray	1kg	Εκτύπωση δοκών.
Bearings	32	Ρουλεμάν για όλες τις ρόδες του συστήματος κίνησης.
V-slot wheels	8	Ρόδες για τους κινούμενους φορείς.
Idler wheels	8	Ρόδες για την δρομολόγηση των ιμάντων.
Wooden board	2	Υλικό βάσης και της σκακιέρας.
Vinyl	1	Βινύλιο για την εκτύπωση της εμφάνισης της σκακιέρας.
Wire	22AWG, 10m	Καλώδια για το πλέγμα.
Jumper Wires	100	Καλώδια για το breadboard.
Breadboard	2	Τα breadboards για τα εξαρτήματα.
Half-Breadboard	1	Μισό breadboard για το ESP32-S3.
Perfboard	5	Ένα perfboard για το κύκλωμα της πηγής.

Screws	100	Βίδες για σταθεροποίηση των δοκών και των κινητήρων.
Nuts	100	Παξιμάδια για σταθεροποίηση των δοκών και των κινητήρων.
Spacers	10	Αποστάτες για τις ρόδες των φορέων.

Πίνακας 11 – Λίστα μηχανικών εξαρτημάτων

Οικονομικός Προϋπολογισμός

Εξάρτημα	**Qty**	**Τιμή/τμχ (€)**	**Σύνολο (€)**
ESP32-S3-DevKitC-N8R8	1	13	13
DRV8825 Stepper Driver	2	2.50	5
NEMA17 Stepper Motor 42BYGHW208	2	12.60	25.20
Electromagnet KK-P25/20	1	10	10
Buck Converter 5 V	1	3	3
LCD I2C 3.3 V	1	3.50	3.50
CD74HC4067 MUX 16ch	4	1.5	6
Reed Switch	64	0.1875	12
DC Power Supply 12 V	1	10	10
Fuse 3 A	1	0.10	0.10
MOSFET IRLB8721PBF	1	1	1
Diode 1N5404	1	0.10	0.10
Resistor 470 Ω	1	0.01	0.01
Resistor 10 kΩ	1	0.01	0.01
Electrolytic Capacitor 100 μF	2	0.15	0.30
Rocker Switch	1	0.40	0.40

Microswitch (limit switch)	2	0.40	0.80
Push Button	2	0.30	0.60
Pulleys	2	1.6	3.2
Timing Belt GT2 (10m)	1	10	10
PLA White	1	10	10
PLA Black	1	10	10
PLA Gray	1	10	10
Bearings	32	0.25	8
V-slot Wheels	8	1	8
Idler Wheels	8	1	8
Wooden Board	2	6	12
Vinyl	1	15	15
Wire 22 AWG (10 m)	1	6	6
Jumper Wires (100τμχ)	1	4	4
Breadboard	2	3	6
Half Breadboard	1	4	4
Perfboard	5	0.60	3
Screws M3/M5 (100τμχ)	1	2	2
Nuts M3/M5 (100τμχ)	1	2	2
Spacers	10	0.1	1
Σύνολο Ηλεκτρονικών Εξαρτημάτων: **91.02 €**			
Σύνολο Μηχανικών Εξαρτημάτων: **122.2 €**			
Τελικό Σύνολο: **213.22 €**			

Πίνακας 12 - Οικονομικός προϋπολογισμός

Ανάλυση και Συγγραφή Λογισμικού (Software)

Διαγράμματα Ροής

Λόγω του πλάτους και ύψους τους, τα διαγράμματα ροής αναρτήθηκαν στο σχετικό github repository αντί τους παρόντος έντυπου.

Κώδικας Προγράμματος

Βιβλιοθήκες και παγκόσμιες μεταβλητές

```
#include "esp_log.h"
#include <FastAccelStepper.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

Η πρώτη βιβλιοθήκη που χρησιμοποιείται είναι η esp_log.h η οποία επιτρέπει την αποστολή μηνυμάτων στο serial monitor σε διαφορετικά επίπεδα προτεραιότητας ανάλογα με το ρυθμισμένο CoreDebugLevel.

Η δεύτερη βιβλιοθήκη, η FastAccelStepper.h επιλέχτηκε αντί της AccelStepper.h γιατί αξιοποιεί το ενσωματωμένο hardware Motor Control Pulse Width Modulation (MCPWM) του ESP32-S3, εξοικονομώντας επεξεργαστική ισχύ από τον επεξεργαστή για τη δημιουργία των παλμών των βηματικών κινητήρων (FastAccelStepper, 2026) (ESP32-S3 Technical Reference Manual, 2026).

Η βιβλιοθήκη Wire.h διευκολύνει τη χρήση της επικοινωνίας I2C, η οποία χρησιμοποιείται στην οθόνη LCD για μείωση των απαιτούμενων pins.
Η LiquidCrystal_I2C.h διευκολύνει την εκτύπωση μηνυμάτων στην LCD οθόνη αφού έχει αρχικοποιηθεί η επικοινωνία από τη Wire.h

```
// Neural Network (TFLite Micro)
#include "chess_model_data.h"
```

Στο αρχείο chess_model_data.h περιέχεται το νευρωνικό δίκτυο ως μια σειρά δεκαεξαδικών νούμερων. Το ESP δεν μπορεί να διαβάσει αρχεία .tflite απευθείας για αυτό γίνεται αυτή η μετατροπή. Ακολουθεί η επεξήγηση του αρχείου:

```
// Generated by tflite_to_header.py
// Original model: chess_model.tflite (6894616 bytes / 6.89 MB)
// This source file is larger but the compiled binary equals the original tflite size

#pragma once

const unsigned int chess_model_tflite_len = 6894616;
// Stored in flash memory as defined by the custom app partition
const uint8_t chess_model_tflite[] = {
0x20, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x00, 0x00, 0x00, 0x00,
 0x14, 0x00, 0x20, 0x00, 0x1c, 0x00, 0x18, 0x00, 0x14, 0x00, 0x10, 0x00, …
```

Η εντολή #pragma once εμποδίζει την βιβλιοθήκη να γίνει included παραπάνω από μια φορά σε ένα φάκελο αποφεύγοντας linking errors. Η μεταβλητής chess_model_tflite_len είναι το μέγεθος του μοντέλου σε bytes. Επειδή το μοντέλο είναι 6.9MB πρέπει να γραφτεί ένα custom partition για το κατάλληλο χωρισμό της μνήμης του μικροελεγκτή.

Ένα partition table ορίζει πώς χωρίζεται η μνήμη του συστήματος, η μνήμη που χρειάζεται για να φορτώσει το μοντέλο πρέπει να οριστεί η χωρητικότητά της πάνω από τα 6.9MB+το μέγεθος του υπόλοιπου προγράμματος.

Η μνήμη χωρίζεται σε nvs (non-volatile storage) στο οποίο αποθηκεύονται μερικές πληροφορίες όπως wifi passwords για να μην χαθούνε στο επόμενο upload κώδικα. Η περιοχή otadata έχει φορτωμένο το bootloader το οποίο διαχειρίζεται αυτόματα τα διαφορετικά app spaces που μπορεί να υπάρχουν για παράδειγμα με τη χρήση over-the-air (OTA) uploads. Ακόμη και με ένα app space προτείνεται η συμπερίληψη ενός bootloader για να μην χάσει το σύστημα αυτή τη λειτουργία του και να έπρεπε να ξαναγραφτεί manually. Στο app space (app0) κατοικεί το κύριο πρόγραμμα και καταλαμβάνει το μεγαλύτερο μέρος της μνήμης λόγω των απαιτούμενων 6.9MB του νευρωνικού δικτύου.

Τα offsets ορίζουν σε ποια θέση μνήμης αρχίζει κάθε κατηγορία. Στη περίπτωση του ESP32-S3 το σύστημα καταλαμβάνει 36863 bytes μνήμης, σε δεκαεξαδικό 0x8FFF, άρα η πρώτη διαθέσιμη θέση μνήμης είναι η 0x9000. Για το κάθε επόμενο offset προστίθεται το μέγεθος του προηγούμενου στην αρχική του θέση, δηλαδή για το otadata προστέθηκε 0x9000 + 0x5000 = 0xe000 για να βρεθεί το offset του.

Για τη κάθε κατηγορία του partition αφέθηκε αρκετή μνήμη και δεν ήταν ο σκοπός η βελτιστοποίηση αλλά η απλή φόρτωση του νευρωνικού δικτύου. Ακολουθεί το τελικό partition με όλες της πληροφορίες για το ESP32-S3 N8R8 (8MB Flash)

Name	*Type*	*Subtype*	*Offset*	*Size*
nvs	data	nvs	0x9000	0x5000
otadata	data	ota	0xe000	0x2000
app0	app	ota_0	0x10000	0x7E0000
coredump	data	coredump	0x7F0000	0x10000

Πίνακας 13 – Memory partition

Διαιρώντας με 1024 (2^{10}) για τη μετατροπή σε KiB από τον αριθμό σε bytes. Για το nvs ορίστηκαν 20KB, για το otadata 8KiB, για το app είναι 7.875MiB και το coredump για αποθήκευση σφαλμάτων είναι 64KiB.

Το coredump είναι σημαντικό γιατί ευθύνεται για τη μετάδοση του register dump κατά τη περίπτωση που ενεργοποιηθεί το watchdog timer ή «κρασάρει» ο επεξεργαστής στο serial monitor, που χωρίς αυτά τα δεδομένα θα ήταν δυσκολότερος ο καθορισμός του προβλήματος.

Για να φορτωθεί σωστά στο Arduino IDE πρέπει να γραφτεί σε μορφή .csv όπως παρακάτω:

```
# Name,     Type, SubType,  Offset,   Size
nvs,        data, nvs,      0x9000,   0x5000,
otadata,    data, ota,      0xe000,   0x2000,
app0,       app,  ota_0,    0x10000,  0x7E0000,
coredump,   data, coredump, 0x7F0000, 0x10000,
```

Πίνακας 14 - Memory partition σε CSV format

```
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"
```

Από τη βιβλιοθήκη TensorFlow Lite for Microcontrollers (TFLM) φορτώνονται τρεις συγκεκριμένες επικεφαλίδες.

Το interpreter παίρνει το input tensor, το περνάει μέσα από το νευρωνικό δίκτυο και επιστρέφει το output tensor άρα είναι αναγκαίο για τη χρήση του νευρωνικού δικύου όποτε πρέπει να καθοριστεί ο πίνακας με τις πιθανές απαντήσεις του υπολογιστή στις κινήσεις του παίκτη (tensorflow/tflite-micro, 2026).

Το operation resolver διαλέγει έξυπνα από όλες τις διαθέσιμες μαθηματικές πράξεις τις βιβλιοθήκης που χρησιμοποιούν διαφορετικά νευρωνικά δίκτυα, και απομονώνει μόνο τις σχετικές πράξεις που χρειάζεται το συγκεκριμένο μοντέλο εξοικονομώντας χώρο.

Στο schema περιέχεται η δομή του μοντέλου μέσα στο αρχικό αρχείο .tflite και είναι αναγκαία για να διαβαστεί σωστά από τα δυαδικά δεδομένα στα οποία μετατράπηκε στο header file. Χωρίς αυτό η βιβλιοθήκη δεν θα διάβαζε σωστά τις προδιαγραφές του μοντέλου.

```
// Motor Pins
#define DIR1_PIN 42
#define STEP1_PIN 41
#define DIR2_PIN 40
#define STEP2_PIN 39
```

Macros για τα pins στα DRV8825 για τους βηματικούς κινητήρες που ελέγχουν τη φορά και τους παλμούς, δηλαδή τη ταχύτητα τους.

```
// Path-clearance
// A square whose center is closer than this fraction of one square to the travel line is "in the path" of the electromagnet.
// 0.55 ≈ 20 mm at 37.5 mm/square — tune up if your magnet reaches further.
#define PATH_THRESHOLD_FRAC 0.55f
#define MAX_BLOCKERS 6
```

Το macro PATH_THRESHOLD_FRAC ορίζει το πόσο μακριά επιτρέπεται από το κέντρο ενός τετραγώνου, που έχει πιόνι, να κινηθεί ο ηλεκτρομαγνήτης κατά τη διαδρομή του. Ανάλογα με τη διάμετρο του κομματιού που κουνήθηκε και τη διάμετρο των κομματιών από τα οποία πρέπει να περάσει διαγωνίως, αυτό το ποσοστό καθορίζει αν είναι δυνατή η διαγώνια κίνηση ή αν πρέπει να αφαιρεθούν τα πιόνια που εμποδίζουν τη διαδρομή. Το macro MAX_BLOCKERS καθορίζει πόσα κομμάτια μπορούν να μετακινηθούν στη χειρότερη περίπτωση για να γίνει μία κίνηση.

Χρησιμοποιούνται μόνο κατά τη φυσική κίνηση κάποιου κομματιού και όχι στις διαβατικές κινήσεις του ηλεκτρομαγνήτη όταν είναι κλειστός. Επειδή η διάμετρος κάθε κομματιού διαφέρει το επιλεγμένο ποσοστό αποφεύγει τις συγκρούσεις στις περισσότερες περιπτώσεις, αλλά συναντώνται ακόμα σε κομμάτια με μεγάλη διάμετρο, όπως ο βασιλιάς και η βασίλισσα.

```
// Pins
const byte LIMIT_SWITCH_Y_PIN = 4;
const byte LIMIT_SWITCH_X_PIN = 1;
const byte WHITE_BTN_PIN = 13; // white confirm / e-stop half
const byte BLACK_BTN_PIN = 14; // black confirm / e-stop half
```

```
const byte ELECTROMAGNET_PIN = 2;
```

Οι ορισμοί από τα pins των διακόπτων οριοθέτησης των αξόνων του συστήματος κίνησης. Τα κουμπιά των δύο χρωμάτων που χρησιμοποιούνται για την καταχώρηση κίνησης, επανεκκίνηση παρτίδας και το emergency stop όταν πατιόνται ταυτόχρονα. Τελευταίο ορίζεται το pin που ελέγχει τον ηλεκτρομαγνήτη, το οποίο είναι συνδεδεμένο στη πύλη από το MOSFET transistor του σχετικού κυκλώματος.

```
// Physical Travel Limits
const int32_t MAX_X_STEPS = 21530;
const int32_t MAX_Y_STEPS = 22332;
```

Τα μέγιστα βήματα που επιτρέπονται σε κάθε άξονα πριν πέσει ο ηλεκτρομαγνήτης πάνω στη δοκό. Προσδιορίστηκαν χρησιμοποιώντας ένα joystick για τη χειροκίνητη μετακίνηση του ηλεκτρομαγνήτη στο πιο μακρινό σημείο κάθε άξονα και τη κλήση της homeAndMeasure() που μετράει τα βήματα μέχρι να πατηθεί ο περιοριστικός διακόπτης του άξονα.

```
// Speed Settings
const uint32_t CALIBRATION_SPEED = 1600;
const uint32_t ACCELERATION = 50000;
const int32_t SLOWDOWN_ZONE = 4000;
const int32_t STEPS_PER_SQUARE = 3000;
const uint32_t MOVE_SPEED = 8000;
const uint32_t MOVE_ACCEL = 40000;
```

Ορισμοί των ορίων ταχυτήτων των βηματικών κινητήρων σε βήματα ανά δευτερόλεπτο ή αλλιώς σε Hz, κατά την εύρεση των ορίων των αξόνων σε 1600 Hz που είναι αργή κίνηση για τη προστασία των φυσικών διακοπτών από κραδασμούς και κρούσεις. Κατά τη κανονική λειτουργία η μέγιστη ταχύτητα είναι 8000 Hz. Η επιτάχυνση είναι σε βήματα ανά δευτερόλεπτο στο τετράγωνο, δηλαδή για το όριο ταχύτητας των 8000 steps/second με επιτάχυνση 40000 steps/second^2 ο χρόνος που απαιτείται για να επιταχύνουν σε αυτή τη ταχύτητα είναι 8000/40000 = 0.2 δευτερόλεπτα. Ο χρόνος είναι για τα ηλεκτρονικά αρκετός ώστε να γίνεται πιο ομαλή η κίνηση και να μην χάνονται βήματα κατά την εκκίνηση. Αυτό σημαίνει πως μπορούν να επιβραδύνουν με αυτή τη ταχύτητα, πράγμα που χρησιμοποιείται στο SLOWDOWN_ZONE, 4000 βήματα από τη περίμετρο το όριο ταχύτητας αλλάζει σε 1600 Hz και η επιτάχυνση αυξάνεται για να επιβραδύνουν πιο αξιόπιστα.

Τα βήματα ανά τετράγωνο προσδιορίστηκαν διαιρώντας το περιοριστικό άξονα, τον άξονα X που έχει το μικρότερο μήκος, με το 7 και στρογγυλοποιώντας προς τα κάτω για συντελεστή ασφάλειας. Ο λόγος που γίνεται η διαίρεση με το 7 αντί για το 8 που είναι ο συνολικός αριθμός τετραγώνων, είναι διότι ο ηλεκτρομαγνήτης δεν χρειάζεται να κινείται στη περίμετρο και το πιο μακρινό πιόνι που θα χρειαστεί να μετακινηθεί θα βρίσκεται πάντα στο τετράγωνο που σχηματίζεται από τα κέντρα των τετραγώνων a1,h1,a8,h8.

```
// MUX Pins
#define PIN_S0 16
#define PIN_S1 17
#define PIN_S2 18
#define PIN_S3 8
#define PIN_SIG 9
#define PIN_EN0 5
#define PIN_EN1 6
```

```
#define PIN_EN2 7
#define PIN_EN3 15

// LCD
#define LCD_I2C_ADDRESS 0x27
#define LCD_SDA_PIN 47
#define LCD_SCL_PIN 48
#define LCD_COLS 16
#define LCD_ROWS 2
```

Ορισμός των pins για τα multiplexers και την οθόνη, η διεύθυνση I2C της οθόνης βρέθηκε τρέχοντας ένα πρόγραμμα I2C scanner που σαρώνει όλες τις δυνατές διευθύνσεις μέχρι να λάβει απάντηση.

```
#define FAST_SCAN_MS 50 // reed switch scan period
#define DEBOUNCE_TICKS 4 // 4 × 50ms = 200ms debounce
#define SETTLE_WINDOW_MS 350 // how long to collect placement candidates
#define LCD_SCROLL_INTERVAL_MS 300
```

Ορισμός της ταχύτητας σάρωσης του πλέγματος των αισθητήρων, πόσο να διαρκεί το debounce τους και πόσο πρέπει να σταθεροποιηθεί η μέτρηση για να καταχωρηθεί η κίνηση ως υποψήφια. Επίσης ορίζεται η ταχύτητα που μετακινείται το κείμενο στην οθόνη στη περίπτωση που είναι ενεργοποιημένοι περισσότεροι αισθητήρες από τον αριθμό που μπορεί ταυτοχρόνως να αναδειχθεί.

```
// Globals
static const char* TAG = "MAIN";
FastAccelStepperEngine Engine = FastAccelStepperEngine();
FastAccelStepper* Stepper1 = nullptr;
FastAccelStepper* Stepper2 = nullptr;
```

Οι συναρτήσεις της βιβλιοθήκης esp_log.h χρειάζονται ένα tag που επιτρέπει την καλύτερη οργάνωση τους με βάση αυτό συμπληρωματικά με τη ταξινόμηση με το CoreDebugLevel. Δεν αξιοποιούνται διαφορετικά tags στο πρόγραμμα αλλά χρειάζεται τουλάχιστον ένα για να κληθούν οι συναρτήσεις. Αρχικοποιείται το engine που διαχειρίζεται το MCPWM του ESP32 από τη βιβλιοθήκη FastAccelStepper.h καλώντας το constructor του. Τα αντικείμενα των βηματικών κινητήρων είναι pointers γιατί πρέπει να αρχικοποιηθούν από το engine άρα αρχικοποιούνται σε nullptr για να δειχθεί πως είναι άδεια (FastAccelStepper, 2026).

```
// MUX /
#define NUM_MUX 4
#define NUM_CHANNELS 16
const int EN_PINS[NUM_MUX] = { PIN_EN0, PIN_EN1, PIN_EN2, PIN_EN3 };
bool results[NUM_MUX][NUM_CHANNELS];
unsigned long lastFastScan = 0;
```

Αρχικοποιούνται τα enable pins των multiplexers σε έναν πίνακα και δεσμεύεται χώρος για τον πίνακα results όπου θα αποθηκεύονται τα αποτελέσματα των αισθητήρων.

```
bool isStopped = false;
bool isElectromagnetOn = false;
unsigned long lastPrintTime = 0;
```

Flags για εύκολη αντιστροφή κατάστασης για το eStop και τον ηλεκτρομαγνήτη. Αρχικοποιείται επίσης η τελευταία στιγμή που εκτυπώθηκε η θέση της σκακιέρας στο serial monitor για να χρησιμοποιηθεί σε non-blocking delay.

```
// LCD
LiquidCrystal_I2C Lcd(LCD_I2C_ADDRESS, LCD_COLS, LCD_ROWS);
String lcdContent = "";
int lcdScrollOffset = 0;
bool lcdNeedsScroll = false;
unsigned long lastLcdScroll = 0;
```

Αρχικοποιείται το αντικείμενο Lcd από τη βιβλιοθήκη LiquidCrystal_I2C.h με τις παραμέτρους που δηλώθηκαν προηγουμένως. Αρχικοποιείται ένα αντικείμενο String που αντιπροσωπεύει το περιεχόμενο που θα εκτυπωθεί στην οθόνη σε κενό “”. Οι υπόλοιπες μεταβλητές αποθηκεύουν το πόσο πρέπει να μετακινηθεί το κείμενο, αν χρειάζεται να μετακινηθεί και η τελευταία στιγμή που τυπώθηκε περιεχόμενο στην οθόνη για να χρησιμοποιηθεί σε ένα non-blocking delay.

```
// Board state tracking
bool boardState[NUM_MUX][NUM_CHANNELS];
bool rawPrev[NUM_MUX][NUM_CHANNELS];
uint8_t sameCount[NUM_MUX][NUM_CHANNELS];
bool boardInitialized = false;
bool suppressDetection = false;
```

Αρχικοποίηση πινάκων για επεξεργασία δεδομένων του πλέγματος αισθητήρων. Το boardState[] αντιπροσωπεύει τη θέση όλων των κομματιών στη σκακιέρα μετά την αποσφαλμάτωση των αισθητηρίων. Το rawPrev[] αποθηκεύει απευθείας της τιμές των αισθητήρων και το sameCount αποθηκεύει πόσοι αισθητήρες είναι ενεργοποιημένοι σε ένα τετράγωνο για το καθορισμό της επικάλυψης τους για αποσφαλμάτωση. Δύο flags καθορίζουν την σωστή αρχικοποίηση της κατάστασης της σκακιέρας και πότε χρειάζεται σάρωση των αισθητήρων.

```
// 3-state move detector
enum MoveDetectState { DETECT_IDLE, PIECE_IN_AIR, PIECE_SETTLING };
MoveDetectState detectState = DETECT_IDLE;
String liftedSquare = "";
```

Για το καθορισμό της πιθανότερης κίνησης του παίκτη χρησιμοποιείται ένα enum που καθορίζει τη κατάσταση του συστήματος. Όταν όλοι οι αισθητήρες είναι σταθεροί έχει τη τιμή DETECT_IDLE, όταν ένας αισθητήρας σβήσει καθορίζεται το τετράγωνο από το οποίο ο παίκτης σήκωσε ένα κομμάτι και η τιμή αλλάζει σε PIECE_IN_AIR. Τέλος όταν ενεργοποιηθεί κάποιος άλλος αισθητήρας σε ένα επιτρεπόμενο τετράγωνο η τιμή αλλάζει σε PIECE_SETTLING και προτείνεται η κίνηση στο παίκτη για έγκριση της. Επιτρέπεται από το παίκτη να αφήσει το κομμάτι που σήκωσε για να επιλέξει διαφορετική κίνηση και για αυτό αποθηκεύεται το τετράγωνο στο liftedSquare string object έτσι ώστε η κίνηση να είναι αναστρέψιμη.

```
// Settling window state
unsigned long settleStart = 0;
String settleBestSquare = ""; // candidate with highest sameCount so far
uint8_t settleBestCount = 0;
```

Όταν αφήνεται ένα κομμάτι στο τετράγωνο, λόγω επικάλυψης των μαγνητικών διακόπτων, απαιτείται αποσφαλμάτωση μετρώντας της τιμές και διαλέγοντας το τετράγωνο με βάση τους κανόνες του παιχνιδιού, τους περισσότερους ενεργοποιημένους αισθητήρες σε ένα τετράγωνο και το debouncing. Αυτές οι μεταβλητές διαχειρίζονται την έναρξη, την αρίθμηση και το τετράγωνο της καλύτερης πρόβλεψης της διαδικασίας.

```
// Legal move generation
// Filled when a piece is lifted; used to correct settling-window candidates.
#define MAX_LEGAL_MOVES 30
#define LEGAL_CORRECT_MAX_DIST 2
String legalDests[MAX_LEGAL_MOVES];
int numLegalDests = 0;
```

Αρχικοποιούνται οι μεταβλητές που χρησιμοποιούνται για την επαλήθευση επιτρεπτών κινήσεων με τους κανόνες του παιχνιδιού. Όλες οι νόμιμες κινήσεις αποθηκεύονται στο πίνακα legalDests[] και αν οι αισθητήρες δείξουν κάποια κίνηση που δεν επιτρέπεται σε μέγιστη απόσταση LEGAL_CORRECT_MAX_DIST διορθώνεται στη πλησιέστερη νόμιμη κίνηση. Για πιο γρήγορη απόκριση του συστήματος για feedback στο παίκτη, εφόσον αυτή είναι μία βοηθητική λειτουργία, οι νόμιμες κινήσεις περιορίζονται σε MAX_LEGAL_MOVES για κάθε κομμάτι και επιλέχτηκε ο αριθμός 30 γιατί υπερβαίνει τις διαθέσιμες κινήσεις που μπορεί να συναντήσει ένα κομμάτι κατά τη διάρκεια της παρτίδας. Αξίζει να σημειωθεί πως ο μέγιστος αριθμός νόμιμων κινήσεων σε μία θέση για όλα τα κομμάτια ταυτόχρονα μπορεί να φτάσει πάνω από 200 κοιτώντας το παιχνίδι με όλους τους κανόνες υπόψιν. Τέτοιοι αριθμοί συναντώνται στη μέση της παρτίδας αφού έχει «ανοίξει» το παιχνίδι και σε αυτές τις θέσεις λειτουργούν καλύτερα οι αισθητήρες λόγω της μικρότερης επικάλυψης από την αυξημένη απόσταση μεταξύ των κομματιών.

```
 // En passant & castling state (updated in passOrConfirm)
int epFile = -1; // file of pawn that can be captured en passant (-1 = none)
int epRank = -1; // rank of that pawn
bool castleWK = true; // white can castle kingside
bool castleWQ = true; // white can castle queenside
bool castleBK = true; // black can castle kingside
bool castleBQ = true; // black can castle queenside

String pendingMove = "";
String lastDisplayedMove = "";
```

Μεταβλητές για αποθήκευση διαθεσιμότητας τετραγώνου για en passant, ο κανόνας που σε περίπτωση που η πρώτη διπλή κίνηση ενός πιονιού είναι δίπλα σε εχθρικό πιόνι ο αντίπαλος μπορεί να αιχμαλωτίσει διαγώνια. Αρχικοποιούνται οι μεταβλητές που καθορίζουν αν επιτρέπεται από μία κατεύθυνση για ένα χρώμα ροκέ. Στα δύο string objects γράφονται οι κινήσεις που θα τυπωθούν στο παίκτη ως υποψήφιες από τη διαδικασία ανίχνευσης τους.

```
// Ambiguous move cycling
// cycleCandidates holds EITHER destination squares (cyclingSource=false)
// OR source squares (cyclingSource=true), never both at once.
#define MAX_CYCLE_CANDIDATES 4
String cycleCandidates[MAX_CYCLE_CANDIDATES];
int numCycleCandidates = 0;
int cycleIdx = 0;
bool cyclingSource = false;
unsigned long lastCycleTime = 0;
```

Το τελευταίο μέτρο για τη βοήθεια με την επαλήθευση κίνησης είναι να εναλλάσσονται στην οθόνη οι πιθανές νόμιμες κινήσεις, αν από τη διαδικασία αποσφαλμάτωσης διαπιστωθεί παραπάνω από μία κίνηση. Σε αυτή τη περίπτωση ασάφειας δείχνονται στην οθόνη εναλλάξ οι υποψήφιες κινήσεις μέχρι τέσσερις διαθέσιμες που αποθηκεύονται στον πίνακα cycleCandidates[]. Ανάλογα από τη προέλευση της ασάφειας, αν είναι από το τετράγωνο προορισμού ή αφετηρίας, αλλάζει το flag cyclingSource και επιλέγεται ποιο από τα δύο εναλλάσσεται στην οθόνη, μόνο ένα από τα δύο για να μην υπάρχει δυσκολία για το παίκτη να επιλέξει τη κίνηση. Σε περίπτωση αποτυχίας και αυτού του τελικού μέτρου αντιμετώπισης της ασάφειας μετρήσεων ο παίκτης θα πρέπει να καταχωρήσει χειροκίνητα τη κίνηση του στο serial monitor με την εντολή e2->e4 για παράδειγμα. Η μεταβλητής cycleIdx αποθηκεύει το τρέχον index του πίνακα cycleCandidates και η lastCycleTime αποθηκεύει τη τελευταία χρονική στιγμή που ενημερώθηκε η οθόνη ώστε να ενημερώνεται σε τακτά χρονικά διαστήματα χωρίς τη χρήση της delay() που θα εμπόδιζε τη λειτουργία του υπόλοιπου προγράμματος κατά την εκτέλεση της.

```
// Game state
enum GameColor { COLOR_NONE, COLOR_WHITE, COLOR_BLACK };
enum TurnState { TURN_WHITE, TURN_BLACK };
GameColor playerColor = COLOR_NONE;
TurnState currentTurn = TURN_WHITE; // white always moves first
```

Δύο enums δείχνουν το επιλεγμένο χρώμα του παίκτη και τη σειρά των χρωμάτων. Δύο μεταβλητές αυτών των enums χρησιμοποιούνται για αυτή τη χρήση και η μεταβλητής playerColor αρχικοποιείται σε COLOR_NONE για να δοθεί η δυνατότητα στο παίκτη να επιλέξει οποιοδήποτε χρώμα επιθυμείται πριν την έναρξη της παρτίδας.

```
// pieceBoard[file 0='a'..7='h'][rank 0='1'..7='8']
// Uppercase = white, lowercase = black, '.' = empty
char pieceBoard[8][8];
```

Ο πίνακας pieceBoard[] αποθηκεύει τη τρέχουσα κατάσταση της παρτίδας και είναι ο τελικός πίνακας στον οποίο γράφονται μόνο οι καταχωρημένες κινήσεις, αντίθετα με το πίνακα boardState[] στον οποίο γράφονται οι απασφαλματωμένες τιμές των αισθητήρων πριν γίνει η έγκριση από το παίκτη. Καθώς αυτός είναι ο τελικός και πιο έμπιστος πίνακας για τη θέση των κομματιών, από αυτόν τον πίνακα δημιουργείται το input tensor του νευρωνικού δικτύου. Λάθος καταχώρηση σε αυτόν τον πίνακα θα είχε ως συνέπεια τη λανθασμένη απόκριση του νευρωνικού δικτύου και του ελέγχου νόμιμων κινήσεων άρα είναι σημαντικό ο παίκτης να καταχωρεί της κινήσεις που επέλεξε σωστά.

```
// Neural Network
// Model: INT8 quantised, input [1,8,8,18] NHWC, output [1,4096] (from*64+to).
// Model is copied flash->PSRAM once at boot for fast inference
// PSRAM layout: model_buf ≈ 6.89 MB, nn_arena 128 KB (internal SRAM)
#define NN_ARENA_SIZE (128 * 1024)
#define NN_N_MOVES 4096
```

Σε αυτό το κομμάτι δηλώνονται οι προδιαγραφές του νευρωνικού δικτύου.
Το μοντέλο είναι κβαντοποιημένο σε signed int8_t, τιμές από -128 μέχρι 127.
Το input tensor έχει διαστάσεις [1,8,8,18] και έχει τη μορφή NHWC.
N ορίζει το batch size πόσες φορές θα τρέξει το μοντέλο, γίνεται πάντα ένα inference ανά θέση άρα είναι 1. Η ορίζει το height οπού είναι οι 8 γραμμές τις σκακιέρας, παρομοίως το W ορίζει το width που αναφέρεται στις 8 στήλες της σκακιέρας. Το C ορίζει τα channels, τα «κανάλια» πληροφορίας, δηλαδή για κάθε τετράγωνο το μοντέλο εξετάζει 18 διαφορετικά layers που είναι τα πιόνια, οι πύργοι, οι βασιλιάδες, το ροκέ, το en passant κτλ, ξεχωριστά για κάθε χρώμα.
Το output tensor είναι της μορφής [1,4096], με το 1 να είναι ξανά το batch size και το 4096 όλες οι δυνατές κινήσεις ανεξαρτήτως νομιμότητας σε μορφή τετραγώνου αφετηρίας και προορισμού (fromSquare->toSquare) (Silver, et al., 2017) για την εύκολη διερμηνεία τους στο σύστημα κίνησης. Το index μίας κίνησης δίνεται από τον τύπο from*64+to αλλά δεν χρησιμοποιείται διότι η κίνηση είναι ήδη στη σωστή μορφή για να τη δεχτεί το σύστημα κίνησης από το output tensor.

Το NN_ARENA_SIZE προσδιορίζει τον χώρο όπου γίνονται οι τρέχοντες γρήγοροι υπολογισμοί στην εσωτερική μνήμη SRAM αντί της PSRAM που θα φορτωθεί το μοντέλο. Αυτός ο χώρος, μόνο όταν τρέχει το νευρωνικό δίκτυο, φορτώνει το input και output tensor που καταναλώνουν 1.125KB + 4KB = 5.125 KB μαζί με τα ενδιάμεσα intermediate tensors, που αποθηκεύουν τους μεταβατικούς υπολογισμούς από layer σε layer, και τις συναρτήσεις τις ίδιες που ελέγχουν το πώς θα γίνουν οι πράξεις από το resolver.

Λόγω της αβεβαιότητας του απαιτούμενου χώρου μνήμης των δύο τελευταίων παραμέτρων αφέθηκε μνήμη 128 KB πολύ μεγαλύτερη της υπολογισμένης καθώς ήταν το τελευταίο μέρος του προγράμματος που γράφτηκε και περίσσευε αρκετή μνήμη χωρίς την ανάγκη βελτιστοποίησης της.

```
const tflite::Model* TflModel = nullptr;
tflite::MicroInterpreter* Interpreter = nullptr;
TfLiteTensor* InputTensor = nullptr; // [1,8,8,18] INT8
TfLiteTensor* OutputTensor = nullptr; // [1,4096] INT8
uint8_t* nnArena = nullptr; // arena (internal SRAM)
uint8_t* modelBuffer = nullptr; // model copy in PSRAM
```

Αρχικοποιούνται τα pointers του μοντέλου, του interpreter, του input tensor, του output tensor, του arena και του buffer αντιγραφής του μοντέλου για την μετακίνηση του στη PSRAM για γρηγορότερο inference έναντι της Flash.

```
bool computerMoveNeeded = false; // set when it becomes the computer's turn
bool computerThinking = false; // true during the 500 ms inference window
```

Τέλος από τα global variables αρχικοποιούνται δύο flags τα οποία καθορίζουν πότε πρέπει να επιλεχτεί μία κίνηση από το νευρωνικό δίκτυο.

Ακολουθεί η επεξήγηση των βασικών συναρτήσεων setup() και loop() πριν των επιμέρους συναρτήσεων που καλούνται για κατανόηση της ροής του προγράμματος πριν τις τεχνικές λεπτομέρειες.

setup()

```
void setup() {
 Serial.begin(115200);
 delay(2000);
 Serial.println("=== SETUP START ===");
```

Αρχικά εξασφαλίζεται η σειριακή επικοινωνία για αποσφαλμάτωση στο σωστό baud rate των 115200 για το ESP32, το delay 2 δευτερολέπτων χρειάζεται γιατί είναι ενεργοποιημένο το USB-CDC On Boot το οποίο δεν είναι όσο γρήγορο όσο το hardware UART.

```
 pinMode(WHITE_BTN_PIN, INPUT_PULLUP);
 pinMode(BLACK_BTN_PIN, INPUT_PULLUP);
 pinMode(LIMIT_SWITCH_X_PIN, INPUT_PULLUP);
 pinMode(LIMIT_SWITCH_Y_PIN, INPUT_PULLUP);
 pinMode(ELECTROMAGNET_PIN, OUTPUT);
```

Αρχικοποιούνται όλα τα pins των κουμπιών που αξιοποιούν τα pull-up resistor του μικροελεγκτή για ευκολότερη σύνδεση με μόνο καλώδια. Το pin του ηλεκτρομαγνήτη, στη πύλη του transistor δηλαδή, πρέπει να δηλωθεί ως output.

```
 Engine.init();
 Stepper1 = Engine.stepperConnectToPin(STEP1_PIN);
 Stepper2 = Engine.stepperConnectToPin(STEP2_PIN);
 if (!stepper1 || !stepper2) {
  Serial.println("Steppers failed to initialize!");
  delay(1000);
 }
 Stepper1->setDirectionPin(DIR1_PIN);
 Stepper2->setDirectionPin(DIR2_PIN);
 Stepper1->setAcceleration(ACCELERATION);
 Stepper2->setAcceleration(ACCELERATION);
 Stepper1->setCurrentPosition(0);
 Stepper2->setCurrentPosition(0);
```

Η βιβλιοθήκη που χρησιμοποιείται (FastAccel.h) χρησιμοποιεί ένα engine object για τον συγχρονισμό πολλών κινητήρων και λειτουργιών το οποίο πρέπει να κληθεί η init() του πριν να συνδέσει τα pins στα stepper objects. Αν κάποιο stepper motor object είναι null τυπώνεται προειδοποίηση στο serial monitor για αποσφαλμάτωση. Ύστερα για το κάθε βηματικό κινητήρα συσχετίζονται τα direction pins, αρχικοποιείται η μέγιστη επιτάχυνση και μηδενίζεται η θέση για αναφορά της ως το σημείο μηδέν.

Η χρήση του arrow operator (->) αντί για το assignment operator (=) είναι αναγκαία γιατί τα Stepper1 και Stepper2 είναι pointers to objects και πρέπει να γίνουν dereferenced για να αλλάξει κάποιο εσωτερικό member variable ή function. Το assignment θα άλλαζε τη θέση μνήμης που δείχνει το pointer αντί να αλλάξει τη τιμή του. Το arrow operator είναι ανάλογο του (*Stepper1).setDirectionPin(DIR1_PIN); άρα απλοποιεί το dereference operator (*) και το member access operator (.) σε ένα σύμβολο.

```
 pinMode(PIN_S0,OUTPUT);
 pinMode(PIN_S1,OUTPUT);
 pinMode(PIN_S2,OUTPUT);
```

```
pinMode(PIN_S3,OUTPUT);
for (int i = 0; i < NUM_MUX; i++) {
  pinMode(EN_PINS[i],OUTPUT);
  digitalWrite(EN_PINS[i],HIGH);
}
pinMode(PIN_SIG, INPUT_PULLUP);
muxSelectChannel(0);
```

Για τα multiplexers αρχικοποιούνται τα pins επιλογής σήματος (S0-S3) σε outputs. Τα enable pins γίνονται HIGH γιατί είναι active-low, δηλαδή ακολουθούν ανεστραμμένη λογική με 0 να τα ενεργοποιεί και 1 να τα απενεργοποιεί παρόμοια με τη χρήση του INPUT_PULLUP σε κουμπιά που αντιστρέφει τη λογική. Το signal pin είναι επίσης active-low δηλαδή ο αισθητήρας που διαβάζεται είναι ενεργοποιημένος μόνο όταν είναι LOW. Για αυτό όταν δεν διαβάζεται ενεργά από τον μικροελεγκτή γυρνάει στη κατάσταση ηρεμίας που είναι HIGH με το pullup resistor. Αν ήταν active-high στο ESP μπορεί να χρησιμοποιηθεί το INPUT_PULLDOWN.

```
Wire.begin(LCD_SDA_PIN, LCD_SCL_PIN);
Lcd.init();
Lcd.backlight();
Lcd.clear();
Lcd.setCursor(0,0); Lcd.print(" AutoChessboard");
Lcd.setCursor(0,1); Lcd.print(" Starting... ");
```

Για την οθόνη αρχικοποιείται η επικοινωνία I2C με τη βιβλιοθήκη Wire καλώντας τη Wire.begin και η ευελιξία του GPIO matrix που κατέχει το ESP σημαίνει πως οποιαδήποτε gpio pins μπορούν να χρησιμοποιηθούν για αυτή τη χρήση. Το αντικείμενο Lcd από τη βιβλιοθήκη LiquidCrystal_I2C.h αρχικοποιείται με την init(), το backlight ενεργοποιείται, καθαρίζεται πλήρως η οθόνη και γράφεται στην οθόνη το μήνυμα εκκίνησης (LiquidCrystal_I2C, 2016).

```
rgbLedWrite(38, 128, 128, 0);
homeAndMeasure(false);
rgbLedWrite(38, 0, 0, 255);
```

Η συνάρτηση rgbLedWrite είναι από την ESP-IDF (ESP32-S3 Technical Reference Manual, 2026), το framework που χρησιμοποιεί έμφυτα το ESP στο οποίο μεταφράζονται συναρτήσεις της Arduino.h. Ανάβει το onboard LED στο gpio 38 με το RGB_BUILTIN_LED_COLOR_ORDER που έχει δηλωθεί του οποίου η default τιμή είναι GRB (Green-Red-Blue). Άρα η πρώτη συνάρτηση ανάβει το πράσινο και το κόκκινο για ένα κίτρινο χρώμα μέσης φωτεινότητας ενώ η δεύτερη ανάβει το μπλε σε πλήρη φωτεινότητα.

Η συνάρτηση homeAndMeasure() βρίσκει τα σημεία μηδέν των δύο αξόνων κίνησης μετακινώντας με αργή ταχύτητα το σύστημα προς τα αριστερά μέχρι να πατηθεί ο διακόπτης του X άξονα και κάτω μέχρι το διακόπτη του Y άξονα. Με άλλα λόγια κάνει reset το σύστημα κίνησης CoreXY για το προσδιορισμό του σημείου (0,0).

```
// Reed switch snapshot (carriage at rest at h1)
muxScanAll();
memcpy(boardState, results, sizeof(boardState));
memcpy(rawPrev, results, sizeof(rawPrev));
memset(sameCount, 0, sizeof(sameCount));
```

```
  boardInitialized = true;
  lcdContent = muxBuildLcdString();
  lcdNeedsScroll = (lcdContent.length() > (size_t)LCD_COLS);
```

Αρχικοποιούνται οι πίνακες που περιγράφουν τη κατάσταση του πλέγματος των μαγνητικών διακόπτων. Η muxScanAll() διαβάζει τα 64 reed switches των οποίων οι τιμές γράφονται στο πίνακα results, τα results αντιγράφονται στο πίνακα boardState που θα χρησιμοποιηθεί αργότερα για τα αποτελέσματα των τιμών μετά από debouncing και legal move filtering. Αντιγράφονται επίσης στο πίνακα rawPrev που χρησιμοποιείται για επεξεργασία δεδομένων κατά το φιλτράρισμα των κινήσεων. Ο πίνακας sameCount αρχικοποιείται στο 0, ο οποίος χρησιμοποιείται για αποσφαλμάτωση αποθηκεύοντας πόσοι αισθητήρες ενεργοποιούνται λόγω της επικάλυψης τους σε κάθε τετράγωνο. Τα flags boardInitialized και lcdNeedsScroll θέτονται για τη χρήση τους από παρακάτω συναρτήσεις. Η muxBuildLcdString() συνθέτει ένα String object με όλους τους ενεργοποιημένους αισθητήρες.

```
  // Standard starting position regardless of reed switches
  initPieceBoard();
  Serial.println("Piece board set to standard starting position.");
  printBoardState();
```

Η συνάρτηση initPieceBoard() κατασκευάζει το πίνακα pieceBoard ο οποίος αποθηκεύει τη τελευταία τρέχουσα θέση της παρτίδας. Η printBoardState() τυπώνει τη τρέχουσα θέση της παρτίδας στο serial monitor, τα πιόνια είναι γραμμένα με κεφαλαία (P,K,N,R) για τα λευκά και σε πεζά (p,k,n,r) για τα μαύρα. Τυπώνονται κατά αυτόν το τρόπο λόγω του περιορισμού πως το serial monitor δεν υποστηρίζει Unicode χαρακτήρες άνω των 8-bit όπως ♜ ♞ ♝ ♛ ή ♖ ♘ ♗ ♕ για καλύτερη αναπαράσταση.

```
  // Neural network: copy model flash→PSRAM, build interpreter
  initNeuralNetwork();
```

Αρχικοποιείται το νευρωνικό δίκτυο αντιγράφοντας το στη PSRAM για ταχύτερο inference, προετοιμασία του «εγκεφάλου» του μοντέλου ώστε να μπορεί να κληθεί η invoke() στο loop().

```
  selectPlayerColor();
  // If the computer plays White it moves first
  if (playerColor == COLOR_BLACK) computerMoveNeeded = true;
  lcdUpdate();
```

Η selectPlayerColor() είναι η αρχική οθόνη κατά την οποία καλείται ο παίκτης να διαλέξει το επιθυμητό χρώμα πατώντας το κατάλληλο κουμπί. Αν ο παίκτης διαλέξει μαύρα το flag computerMoveNeeded γίνεται true για να διαβαστεί στη loop() και να τρέξει το νευρωνικό δίκτυο. Η lcdUpdate() διαχειρίζεται την αντιγραφή του String lcdContent στην οθόνη και το automatic scrolling αν οι ενεργοποιημένοι αισθητήρες καλύπτουν παραπάνω της γραμμή.

```
  ESP_LOGI(TAG, "Ready. MAX_X=%d MAX_Y=%d ZONE=%d",
           MAX_X_STEPS, MAX_Y_STEPS, (int)SLOWDOWN_ZONE);
  Serial.println("Commands: a1-h8 | e4-f3 | e4->f3 | board | home");
}
```

Στις τελευταίες γραμμές της setup τυπώνεται μόνο σε CoreDebugLevel = “info” ο συνολικός αριθμός βημάτων για κάθε άξονα και η ζώνη επιβράδυνσης. Τυπώνεται πάντα η ένδειξη των διαθέσιμων εντολών ως τελευταία εντολή στη setup() πριν κληθεί η loop().

loop()

Η συνάρτηση loop() αποτελεί τον κεντρικό βρόχο ελέγχου του συστήματος και εκτελείται επαναληπτικά από το λειτουργικό περιβάλλον του ESP32-S3. Κάθε επανάληψη εκτελεί, με σταθερή σειρά, έξι ανεξάρτητα υποσυστήματα: έλεγχο επείγουσας διακοπής, εκτέλεση κίνησης υπολογιστή, εκτύπωση θέσης για αποσφαλμάτωση, επεξεργασία εντολών σειριακής θύρας, ταχεία σάρωση αισθητήρων και διαχείριση κουμπιών επιβεβαίωσης.

```
void loop() {
 if (checkEStop()) return;
```

Η πρώτη εντολή κάθε επανάληψης είναι ο έλεγχος της κατάστασης έκτακτης ανάγκης μέσω της checkEStop(). Εάν το flag isStopped είναι ενεργό, η συνάρτηση επιστρέφει true και η loop() τερματίζει αμέσως με return, παγώνοντας όλη την επεξεργασία χωρίς να διακόπτει τη φυσική θέση των κινητήρων. Ενεργοποιείται πατώντας και τα δύο κουμπιά ταυτόχρονα και χρησιμεύει στη περίπτωση που έχει αποσυνδεθεί κάποιο κρίσιμο καλώδιο, όπως αυτά των δύο διακόπτων για τους άξονες του συστήματος κίνησης από τη μεταφορά της σκακιέρας.

```
 // Computer move
 // Triggered after the player confirms a move (passOrConfirm sets  the flag)
 // or at game start when the computer plays White.
 // Only runs when the motors are idle so executeMove() starts      cleanly.
 if (computerMoveNeeded && !isStopped &&
  !Stepper1->isRunning() && !Stepper2->isRunning()) {
  computerMoveNeeded = false;
  doComputerMove();
 }
```

Ελέγχεται το flag computerMoveNeeded, το οποίο τίθεται true από την passOrConfirm() μόλις ο παίκτης επιβεβαιώσει την κίνησή του, ή κατά την εκκίνηση του παιχνιδιού όταν ο υπολογιστής παίζει με τα λευκά. Η εκτέλεση επιτρέπεται μόνο εφόσον και οι δύο κινητήρες είναι σταματημένοι, ώστε η executeMove() να ξεκινά από καθορισμένη κατάσταση για να μη χαθεί η θέση του συστήματος κίνησης. Το flag isStopped καθορίζεται από τη συνάρτηση checkEStop() και χρησιμοποιείται για την ομαλή επαναφορά του συστήματος όταν ξαναπατηθούν και τα δύο κουμπιά ταυτοχρόνως.

```
 // — Position debug print
 if (millis() - lastPrintTime > 250) {
  int32_t logX, logY; getLogicalPos(logX, logY);
  int32_t absLogX = abs(logX);
  ESP_LOGI(TAG, "L:%ld R:%ld B:%ld T:%ld",
         (long)absLogX, (long)(MAX_X_STEPS-absLogX),
         (long)logY, (long)(MAX_Y_STEPS-logY));
  lastPrintTime = millis();
 }
```

Κάθε 250 ms εκτυπώνεται στο serial monitor η λογική θέση του άξονα X (αριστερά/δεξιά) και Y (κάτω/πάνω) σε βήματα κινητήρα, μαζί με την απόσταση που απομένει ως τα αντίστοιχα μηχανικά όρια. Η τιμή absLogX χρησιμοποιείται αντί της logX διότι το σύστημα CoreXY επιστρέφει αρνητικές τιμές κατά τη φορά της αρχικής κίνησης homing. Αυτή η εκτύπωση γίνεται μόνο όταν έχει τεθεί CoreDebugLevel = “info” στα flash settings για αποσφαλμάτωση του συστήματος διότι υπό κανονική λειτουργία δεν είναι σημαντική.

```
// Serial commands
if (Serial.available() > 0) {
  String cmd = Serial.readStringUntil('\n');
  cmd.trim();
  cmd.toLowerCase();
  if (cmd == "pass") {
    currentTurn = (currentTurn==TURN_WHITE)?TURN_BLACK:TURN_WHITE;
    pendingMove="";
    detectState=DETECT_IDLE;
    liftedSquare="";
    Serial.printf("Turn forced to:    %s\n",(currentTurn==TURN_WHITE)?"WHITE":"BLACK");
    lcdUpdateRow1();
```

Στο serial monitor δίνεται η δυνατότητα συγκεκριμένων εντολών για αποσφαλμάτωση και χειροκίνητο έλεγχο. Εφόσον είναι διαθέσιμο (Serial.available()>0) τότε το καταχωρημένο κείμενο αποθηκεύεται στο String cmd μέχρι το new line character που καταχωρείται με το enter. Η συνάρτηση trim αποσπά όλους τους special χαρακτήρες που μπορεί να βρίσκονται στο String όπως τα /r, /0, /n. Μετά καλείται η toLowerCase() στο String object με σκοπό να μην είναι οι εντολές case sensitive.

Η πρώτη εντολή που ελέγχεται είναι η «pass» η οποία περνάει τη κίνηση στο αντίθετο χρώμα χωρίς να γίνει έγκριση κάποιας κίνησης, κάτι που δεν επιτρέπεται στους κανόνες του σκακιού, χρησιμεύει για αποσφαλμάτωση σε περίπτωση που γίνονται δοκιμές μόνο για ένα χρώμα. Το currentTurn με τη χρήση ενός ternary operator γίνεται TURN_BLACK ή TURN_WHITE αντιστρέφοντας τη σειρά, καθαρίζεται η θέση που εκκρεμεί pendingMove και το τετράγωνο από το οποίο άρχισε η κίνηση σε ένα άδειο String “”, η κατάσταση ανίχνευσης γίνεται IDLE και τυπώνεται η νέα σειρά στο serial monitor και την οθόνη.

```
  } else if (cmd == "home") {
    rgbLedWrite(38, 128, 128, 0);
    homeAndMeasure(true);
    rgbLedWrite(38, 0, 0, 255);
```

Η εντολή «home» κάνει επανεκκίνηση στο σύστημα κίνησης για την εύρεση του σημείου (0,0) σε περίπτωση σφάλματος λόγω χαμένων βημάτων και η homeAndMeasure() καλείται με input true για την εκτύπωση διαγνωστικών στο serial monitor για αποσφαλμάτωση.

```
  } else if (cmd == "magnet_on") {
    digitalWrite(ELECTROMAGNET_PIN, HIGH);
    Serial.println("magnet on");
  } else if (cmd == "magnet_off") {
    digitalWrite(ELECTROMAGNET_PIN, LOW);
    Serial.println("magnet off");
```

Οι εντολές «magnet_on» και «magnet_off» δίνουν τη δυνατότητα χειροκίνητης χρήση του ηλεκτρομαγνήτη μαζί με τις εντολές κίνησης παρακάτω.

```
  } else if (cmd == "board") {
    printBoardState();
```

Η εντολή «board» τυπώνει στο serial monitor τη πλήρη εικόνα της τρέχουσας θέσης της παρτίδας για αποσφαλμάτωση ή επικύρωση σωστής καταχώρησης κίνησης.

```
  } else if (cmd.length() == 5 && cmd[2] == '-') {
    char f1=cmd[0],r1=cmd[1],f2=cmd[3],r2=cmd[4];
    if (f1>='a' && f1<='h' && r1>='1' && r1<='8' && f2>='a' && f2<='h'&& r2>='1' && r2<='8')
      executeMove(f1,r1,f2,r2);
    else Serial.printf("Invalid move '%s'. Format: e4-f3\n",    cmd.c_str());
```

Αυτή η εντολή παίρνει δύο κατάλληλα τετράγωνα, για παράδειγμα e2-e4 με μία παύλα μεταξύ τους και εκτελεί μία φυσική κίνηση του ηλεκτρομαγνήτη επιτρέποντας τη χειροκίνητη εισαγωγή θέσεων έναντι του νευρωνικού δικτύου για γρήγορη αποσφαλμάτωση.

```
} else if (cmd.length() == 2) {
    char file=cmd[0],rank=cmd[1];
    if (file>='a'&&file<='h'&&rank>='1'&&rank<='8') moveToSquare(file,rank);
    else Serial.printf("Unknown '%s'.\n", cmd.c_str());
```

Η εντολή δέχεται οποιοδήποτε έγκυρο τετράγωνο και κινεί τον ηλεκτρομαγνήτη εκεί επιλέγοντας τη συντομότερη διαδρομή, αν η καταχώρηση δεν είναι έγκυρη τυπώνεται το μήνυμα “Unkown” στο serial monitor με τη λανθασμένη καταχώρηση. Για να τυπωθεί σωστά το String object σε ένα formatted string στη θέση %s πρέπει να μετατραπεί σε char[] με την συνάρτηση c_str().

```
  } else if (cmd.length() == 6 && cmd[2] == '-' && cmd[3] == '>'){                char f1=cmd[0],
r1=cmd[1], f2=cmd[4], r2=cmd[5];
   if (f1>='a' && f1<='h' && r1>='1' && r1<='8' && f2>='a' && f2<='h'&& r2>='1' && r2<='8') {
     // Register as a confirmed player move — updates board model and flips turn
     int fx1 = f1-'a', ry1 = r1-'1', fx2 = f2-'a', ry2 = r2-'1';
     char moved = pieceBoard[fx1][ry1];
     char captured = pieceBoard[fx2][ry2];
     pieceBoard[fx2][ry2] = moved;
     pieceBoard[fx1][ry1] = '.';
```

Η εντολή δέχεται δύο έγκυρα τετράγωνα με το σύμβολο «->» ενδιάμεσα τους για να καταχωρήσει χειροκίνητα μία κίνηση του παίκτη σε περίπτωση αποτυχίας ανίχνευσης της. Οι μεταβλητές fx1,ry1,fx2,ry2 μετατρέπουν τη θέση που καταχωρήθηκε σε νούμερο, αφαιρώντας το χαρακτήρα “a” από το “a” γίνεται 0, αφαιρώντας από το “b” γίνεται 1 και ούτω καθεξής. Οι μεταβλητές moved και captured αναφέρονται στην απεικόνιση της θέσης στο serial monitor από τη printBoardState(), αποθηκεύουν το χαρακτήρα στο τετράγωνο αναχώρησης και άφιξης. Στο τετράγωνο άφιξης στη κατάλληλη θέση του πίνακα pieceBoard καταχωρείται η μεταβλητής moved ενώ στο τετράγωνο αναχώρησης η τελεία που συμβολίζει πως είναι άδειο.

```
     if (toupper(moved) == 'P'&& abs(ry2-ry1) == 2) {
       epFile = fx1;
       epRank = ry2 + (ImgIsWhite(moved) ? -1:1);
     }
     else {
       epFile = -1;
       epRank = -1;
     }
```

Αυτό το μέρος ελέγχει, στη περίπτωση που κουνήθηκε πιόνι, αν επιτρέπεται το en passant. Μετατρέπεται σε κεφαλαία για να μην υπάρχει διαχωρισμός μεταξύ χρωμάτων και ελέγχεται η απόλυτη τιμή γιατί για τα μαύρα η σκακιέρα είναι ανάποδα, έτσι ελέγχονται ταυτόχρονα αν το πιόνι κουνήθηκε δύο θέσεις μπροστά. Καταχωρείται η διαθεσιμότητα της στήλης (file) και γραμμής (rank), που στη περίπτωση της γραμμής προστίθεται η αφαιρείται 1 ανάλογα με το χρώμα για να δειχθεί πως το πίσω τετράγωνο από το πιόνι είναι διαθέσιμο για en passant. Σε κάθε άλλη περίπτωση δεν είναι έγκυρος αυτός ο κανόνας και οι μεταβλητές θέτονται σε -1.

```
      if(moved=='K') {
       castleWK=false;
       castleWQ=false;}
     if(moved=='k') {
       castleBK=false;
       castleBQ=false;}
```

Στη περίπτωση που κουνήθηκε βασιλιάς τότε το ροκέ για αυτό το χρώμα δεν επιτρέπεται.

```
   if(moved=='R'||captured=='R') {
     if(fx1==7||fx2==7) castleWK=false;
     if(fx1==0||fx2==0) castleWQ=false;
   }
   if(moved=='r'||captured=='r') {
     if(fx1==7||fx2==7) castleBK=false;
     if(fx1==0||fx2==0) castleBQ=false;
   }
```

Αν μετακινήθηκε πύργος, ανάλογα με τη θέση και το χρώμα το ροκέ από εκείνη τη κατεύθυνση και χρώμα δεν επιτρέπεται πλέον.

```
lastDisplayedMove=String(f1)+String(r1)+"->"+String(f2)+String(r2);
      pendingMove="";
      detectState=DETECT_IDLE;
      liftedSquare="";
      settleBestSquare="";
      settleBestCount=0;
      numCycleCandidates=0;
      cyclingSource=false;
currentTurn = (currentTurn==TURN_WHITE) ? TURN_BLACK:TURN_WHITE;
bool isNowComputerTurn = (playerColor == COLOR_WHITE && currentTurn == TURN_BLACK)
|| (playerColor == COLOR_BLACK && currentTurn == TURN_WHITE);
      if(isNowComputerTurn) computerMoveNeeded=true;
```

Θέτονται όλα τα flags παρόμοια με την εντολή «pass» έτσι ώστε να βρίσκεται το σύστημα στη σωστή κατάσταση και να αποφασιστεί αν είναι η σειρά του υπολογιστή ή του παίκτη.

```
      Serial.printf("Manual move: %c%c->%c%c piece=%c\n", f1,r1,f2,r2, moved);
      printBoardState();
      lcdUpdateRow1();
    } else {
```

```
      Serial.printf("Invalid move '%s'. Format: e2->e4\n", cmd.c_str());
    }
```

Τυπώνεται η καταχωρημένη κίνηση, η θέση της παρτίδας, γίνεται refreshed η οθόνη με τη σειρά του χρώματος που παίζει. Αν δεν είναι έγκυρη η εντολή τυπώνεται υπόδειξη χρήσης της μαζί με τη λανθασμένη καταχώρηση.

```
  } else {
    Serial.printf("Unknown '%s'. Commands: a1-h8 | e4-f3 | e4->f3 | board | home \n",
cmd.c_str()); }
```

Αν η καταχώριση δεν είναι καμία από τις παραπάνω εντολές τυπώνεται η υπόδειξη εντολών.

```
 // Fast scan detect + LCD
 unsigned long now = millis();
 if (now - lastFastScan >= FAST_SCAN_MS) {
  lastFastScan = now;
  muxScanAll();
  detectMove();
  updatePendingCandidates();
  String newContent = muxBuildLcdString();
  if (newContent!= lcdContent) {
   lcdContent = newContent;
   lcdScrollOffset = 0;
   lcdNeedsScroll = (lcdContent.length() > (size_t)LCD_COLS);
  }
  lcdUpdate();
 }
```

Σε αυτό το κομμάτι γίνεται η σάρωση του πλέγματος αισθητήρων και η αποσφαλμάτωση τους για τον καθορισμό της πιο πιθανής κίνησης που επέλεξε ο παίκτης. Η οθόνη ενημερώνεται με το αποτέλεσμα του φιλτραρίσματος. Όλη η αποσφαλμάτωση με το debouncing των αισθητήρων, τη δειγματοληψία τους και αν η κίνηση είναι νόμιμη με τους κανόνες του παιχνιδιού γίνονται στην detectMove() και στην updatePendingCandidates().

```
// — Cycle ambiguous candidates
 advanceCycling();
```

Αυτή η συνάρτηση εναλλάσσει μεταξύ υποψηφίων κινήσεων σε περίπτωση που αποτύχει ο παραπάνω έλεγχος, δρα ως δικλείδα ασφαλείας ώστε να μην απαιτείται ο παίκτης να χρησιμοποιεί το serial monitor για τη καταχώρηση κινήσεων σε περίπτωση ασάφειας.

```
 // Only responds when motors are stopped.
 // Single press = confirm pending move (or pass if no pending & not player turn).
 // Both pressed = e-stop (handled in checkEStop at top of loop).
 if (!stepper1->isRunning() && !stepper2->isRunning()) {
  bool bW = (digitalRead(WHITE_BTN_PIN) == LOW);
  bool bB = (digitalRead(BLACK_BTN_PIN) == LOW);
  if (!(bW && bB)) {
   if (bW) { passOrConfirm(true); delay(300); }
   if (bB) { passOrConfirm(false); delay(300); }
```

```
  }
 }
```

Ο έλεγχος των δύο κουμπιών χρώματος για καταχώρηση κινήσεων, επανεκκίνηση παρτίδας ή καταχώρηση ασαφής κίνησης (pass) που τα διαχειρίζεται η passOrConfirm(). Για τη περίπτωση του emergency stop (eStop) ο έλεγχος γίνεται στην αρχή της loop() στη σχετική συνάρτηση.

```
 // LCD scroll
 if (lcdNeedsScroll && (now - lastLcdScroll >= LCD_SCROLL_INTERVAL_MS)) {
  lastLcdScroll = now;
  lcdScrollOffset++;
  lcdUpdate();
 }
}
```

Τέλος στη loop() ανανεώνεται η οθόνη LCD με όλα τα flags που τέθηκαν από τις εντολές στο serial monitor, το scan του πλέγματος των αισθητήρων, τη τρέχουσα σειρά και την υποψήφια κίνηση του παίκτη ή του υπολογιστή.

getLogicalPos()

```
void getLogicalPos(int32_t &logX, int32_t &logY) {
 int32_t p1 = Stepper1->getCurrentPosition();
 int32_t p2 = Stepper2->getCurrentPosition();
 logX = (p1 + p2) / 2;
 logY = (p1 - p2) / 2;
}
```

Αυτή η συνάρτηση μετατρέπει τις φυσικές θέσεις των βηματικών κινητήρων στις λογικές συντεταγμένες (X,Y) για το σύστημα κίνησης. Η συνάρτηση getCurrentPosition() επιστρέφει τον αριθμό βημάτων του Stepper object σε όλη τη διάρκεια που λειτουργεί το σύστημα, όταν περιστρέφεται δεξιόστροφα τα βήματα προστίθενται και αριστερόστροφα αφαιρούνται. Σε ένα μηχανικό σύστημα κίνησης CoreXY, το σημείο X καθορίζεται όταν τα μοτέρ περιστρέφονται προς την ίδια κατεύθυνση και το σημείο Y σε αντίθετες κατευθύνσεις περιστροφής. Η διαίρεση με το δύο προκύπτει από της κινηματικές εξισώσεις του συστήματος κίνησης, συγκεκριμένα η κίνηση των μοτέρ σχετίζεται με τη μετατόπιση στους άξονες από:

$$\Delta_{p1} = \Delta X + \Delta Y \; και \; \Delta_{p1} = \Delta X - \Delta Y$$

Προσθέτοντας τις παραπάνω εξισώσεις προκύπτει για τον άξονα X:

$$p_1 + p_2 = (X + Y) + (X - Y) \Leftrightarrow p_1 + p_2 = 2X \Leftrightarrow X = \frac{p_1 + p_2}{2}$$

Αφαιρώντας τις εξισώσεις προκύπτει για τον άξονα Y:

$$p_1 - p_2 = (X + Y) - (X - Y) \Leftrightarrow p_1 - p_2 = 2Y \Leftrightarrow Y = \frac{p_1 - p_2}{2}$$

muxEnable() και muxDisableAll()

```
void muxEnable(int index) {
  for (int i = 0; i < NUM_MUX; i++)
    digitalWrite(EN_PINS[i], (i == index) ? LOW : HIGH);
}
```

Ενεργοποιεί ένα συγκεκριμένο multiplexer και απενεργοποιεί όλα τα υπόλοιπα.

```
void muxDisableAll() {
  for (int i = 0; i < NUM_MUX; i++) digitalWrite(EN_PINS[i], HIGH);
}
```

Απενεργοποιεί όλα τα multiplexers γράφοντας HIGH στο enable pin τους το οποίο είναι active-low.

muxSelectChannel()

```
void muxSelectChannel(uint8_t ch) {
  digitalWrite(PIN_S0, (ch >> 0) & 1);
  digitalWrite(PIN_S1, (ch >> 1) & 1);
  digitalWrite(PIN_S2, (ch >> 2) & 1);
  digitalWrite(PIN_S3, (ch >> 3) & 1);
}
```

Επιλέγει ένα κανάλι από τα multiplexers κάνοντας shift δεξιά το διάδικό αριθμό και απομονώνοντας το δεξιότερο ψηφίο. Για παράδειγμα για να επιλεχτεί το 6^{ο} κανάλι δέχεται input ch=6, η μεταβλητή στο δυαδικό είναι 0000110.
Τα shifts (>>) μεταφέρουν τα bits δεξιά κατά τον αριθμό δηλαδή το ch>>2 θα είναι 00000001, το λογικό bitwise “and” με το 00000001 είναι true άρα το pin γίνεται HIGH.
Για κάθε pin στο παράδειγμα η τιμή που θα γραφτεί είναι:

Pin	*Operation*	*Result*
PIN_S0	(6>>0) & 1	LOW
PIN_S1	(6>>1) & 1	HIGH
PIN_S2	(6>>2) & 1	HIGH
PIN_S3	(6>>3) & 1	LOW

Πίνακας 15 – Παράδειγμα bit shifting

muxScanAll()

```
void muxScanAll() {
 for (int m = 0; m < NUM_MUX; m++) {
  muxEnable(m);
  for (int ch = 0; ch < NUM_CHANNELS; ch++) {
   muxSelectChannel(ch);
   delayMicroseconds(10);
   results[m][ch] = (digitalRead(PIN_SIG) == LOW);
  }
 }
 muxDisableAll();
}
```

Εκτελεί μία πλήρη σάρωση των 64 αισθητήρων ενεργοποιώντας με τη σειρά κάθε multiplexer και ανοίγοντας διαδοχικά όλα τα κανάλια. Γράφονται τα αποτελέσματα στο δισδιάστατο πίνακα results, όπου οι αισθητήρες είναι active-low δηλαδή είναι true όταν επιστρέφουν 0.

muxToSquare(), muxToSquareLower() και muxBuildLcdString()

```
String muxToSquare(int m, int ch) {
  char file = 'H' - (ch % 8);
  int rank = (3 - m) * 2 + 1 + (ch / 8);
  String s = ""; s += file; s += rank; return s;
}
```

Η συνάρτηση μετατρέπει τον αριθμό του multiplexer και το κανάλι του από τη δυσανάγνωστη μορφή ένδειξης του, όπως M0:14, στο τετράγωνο της σκακιέρας (A1-H8) του αναλογεί.

Η πρώτη γραμμή υπολογίζει τη στήλη (file) αφαιρώντας από το χαρακτήρα 'H' το αποτέλεσμα του υπολοίπου ch%8 που επιστρέφει τιμές 0 εώς 7. Για παράδειγμα, για το κανάλι 13 η διαίρεση 13/8 έχει υπόλοιπο 5 και το αποτέλεσμα της πράξης 'H'-5 είναι 'C' εφόσον οι χαρακτήρες 'A' έως 'H' στο ASCII αναπαρίστανται από διαδοχικά νούμερα.

Η δεύτερη γραμμή υπολογίζει τη γραμμή (rank), κάθε multiplexer καλύπτει δύο γραμμές τις σκακιέρας για αυτό χρησιμοποιείται η πράξη (3-m)*2 για να καθορίσει ποιες γραμμές καλύπτει το επιλεγμένο multiplexer. Αναλυτικά:

Multiplexer	*Operation*	*Result*	
m = 3	(3-3) * 2	0	Καλύπτει τις γραμμές 1 - 2
m = 2	(3-2) * 2	2	Καλύπτει τις γραμμές 3 - 4
m = 1	(3-1) * 2	4	Καλύπτει τις γραμμές 5 - 6
m = 0	(3-0) * 2	6	Καλύπτει τις γραμμές 7 - 8

Πίνακας 16 - Κατανομή τετραγώνων στους πολυπλέκτες

Επειδή οι γραμμές σε μία σκακιέρα ξεκινούν από το 1 αντί του 0 προστίθεται μία μονάδα για να γίνουν τα αποτελέσματα 1,3,5 και 7. Ο τελευταίος όρος ch/8 επιλέγει ποια από τις δύο γραμμές είναι η σωστή, αξιοποιώντας την ιδιότητα της διαίρεσης ακέραιων αριθμών (integer division). Εφόσον η μεταβλητής ch είναι τύπου integer η πράξη ch/8 επιστρέφει 0 όταν το ch ισούται με 0 εώς 7 και επιστρέφει 1 όταν ισούται από 8 έως 15 προσθέτοντας ένα ακόμη στο αποτέλεσμα στο δεύτερο μισό τους έυρους.

Τέλος τα αποτελέσματα συνδέονται (concatenated) σε ένα string στο οποίο το operator "+" ενώνει τις λέξεις αντί να τις αθροίζει αριθμητικά για το τελικό string να έχει τη μορφή "A1/0" για παράδειγμα, με το /0 να ορίζει το τέλος του char[], το οποίο το String object χρησιμοποιεί, που προστίθεται αυτόματα σε κάθε String.

lcdWriteRow()

```
void lcdWriteRow(int row, const String& text, int offset) {
  Lcd.setCursor(0, row);
  for (int i = 0; i < LCD_COLS; i++) {
    int idx = offset + i;
    Lcd.print((idx < (int)text.length()) ? text[idx] : ' ');
  }
}
```

Η συνάρτηση τυπώνει στην επιλεγμένη γραμμή της οθόνης LCD ένα text τύπου String το οποίο λαμβάνει ως reference, έχοντας τη δυνατότητα για ορισμό ενός offset το οποίο εισάγει κενούς χαρακτήρες ‘’ πριν το κείμενο.

lcdUpdateRow1()

```
void lcdUpdateRow1() {
 // Computer is thinking: show "W>?->?" or "b>?->?"
 if (computerThinking) {
  String prefix = (currentTurn == TURN_WHITE) ? "W>" : "b>";
  String content = prefix + "?->?";
  Lcd.setCursor(0, 1);
  if (currentTurn == TURN_WHITE) {
   while ((int)content.length() < LCD_COLS) content = ' ' + content;
  } else {
   while ((int)content.length() < LCD_COLS) content += ' ';
  }
  return;
 }
```

Η συνάρτηση διαχειρίζεται τη δεύτερη γραμμή της οθόνης η οποία τυπώνει τις κινήσεις του υπολογιστή και του παίκτη. Στο πρώτο κομμάτι της, όταν είναι η σειρά του υπολογιστή προτού υπολογιστεί κίνηση, δείχνει W>?->? ή b>?->? ανάλογα με το χρώμα στο σωστό offset. Αν είναι η σειρά των λευκών το κείμενο στοιχίζεται δεξιά εισάγοντας επαναληπτικά κενά μέχρι το μήκος του String να ισούται με της οθόνης, αντίθετα αν είναι σειρά για τα μαύρα προστίθενται κενά μετά του content για να διαγράψουν τυχόν υπάρχων κείμενο.

```
  // Build action string
 String action;
 if (detectState == PIECE_IN_AIR && liftedSquare.length()) {
  action = liftedSquare + "->?";
 } else if (detectState == PIECE_SETTLING && liftedSquare.length()) {
  action = liftedSquare + "->?"; // still show as in-air during settling
 } else if (pendingMove.length()) {
  action = pendingMove;
 } else if (lastDisplayedMove.length()) {
  action = lastDisplayedMove;
 } else {
  action = "------";
 }
```

Αυτό το κομμάτι της συνάρτησης δείχνει το αποτέλεσμα της αποσφαλμάτωσης των αισθητήρων με σκοπό τη σωστή πρόβλεψη της κίνησης του. Στον αρχικό έλεγχο όταν ο παίκτης σηκώσει ένα κομμάτι, δείχνεται το τετράγωνο από το οποίο σηκώθηκε το πιόνι ως e2->? για παράδειγμα με το τετράγωνο προορισμού αβέβαιο. Ο δεύτερος έλεγχος χρησιμοποιούνταν για αποσφαλμάτωση και στο τελικό πρόγραμμα κατασκευάζει το ίδιο string. Αν το τετράγωνο προορισμού έχει αποφασιστεί τυπώνεται η πλήρης αναμενόμενη κίνηση ως e2->e4 ενδεικτικά. Στην αρχική κατάσταση πριν σηκώσει ο παίκτης κάποιο κομμάτι δείχνεται η προηγούμενη κίνηση του υπολογιστή σε περίπτωση που δεν εκτελέστηκε σωστά.

```
 // Prefix current turn's colour label
 String prefix = (currentTurn == TURN_WHITE) ? "W>" : "b>";
```

```
 String content = prefix + action;

 Lcd.setCursor(0, 1);
 if (currentTurn == TURN_WHITE) {
  // Right-align left padding
  while ((int)content.length() < LCD_COLS) content = ' ' + content;
 } else {
  // Left-align right padding
  while ((int)content.length() < LCD_COLS) content += ' ';
 }
}
```

Για την ένδειξη χρώματος του παίκτη κατασκευάζεται από το prefix W> ή b> ,παρομοίως με του υπολογιστή, μαζί με το προηγούμενο string και ακολουθείται η ίδια λογική στοίχισης, δεξιά εισάγοντας κενά πριν το κείμενο και αριστερά εισάγοντας μετά για καθαρισμό προηγούμενου κειμένου στην ίδια γραμμή.

lcdUpdate()

```
void lcdUpdate() {
  if (!lcdNeedsScroll) {
    lcdWriteRow(0, lcdContent, 0);
  } else {
    String padded = lcdContent + " ";
    int len = padded.length();
    Lcd.setCursor(0, 0);
    for (int i = 0; i < LCD_COLS; i++)
      Lcd.print(padded[(lcdScrollOffset + i) % len]);
  }
  lcdUpdateRow1();
}
```

Η συνάρτηση διαχειρίζεται την ανανέωση και των δύο γραμμών της οθόνης LCD, ελέγχοντας την ανάγκη μετακίνησης κειμένου σε τακτά χρονικά διαστήματα (scrolling) της πρώτης γραμμής. Σε περίπτωση που χρειάζεται μετατόπιση κειμένου προστίθεται ένα κενό στο τέλος του κειμένου ως διαχωριστικό για να μην κολλάνε οι λέξεις μεταξύ τους, δηλαδή να μην τυπώνεται A1B1 χωρίς κενό ανάμεσα. Η παγκόσμια μεταβλητής lcdScrollOffset διαλέγει το σημείο από το οποίο αρχίζει να διαβάζεται το κείμενο, και η επανάληψη τρέχει 16 φορές για να γεμίσει όλες τις στήλες. Το lcdScrollOffset αυξάνεται κατά 1 από τη loop() και γίνεται 0 όταν ανανεωθεί ο πίνακας με τους ενεργοποιημένους αισθητήρες που τυπώνονται σε αυτή τη γραμμή της οθόνης. Ο όρος %len χρησιμοποιείται για να επαναλαμβάνεται το κείμενο, επιστρέφει τιμές 0 έως το μήκος του κειμένου, μεγαλύτερες τιμές από το len το επιστρέφουν στο 0 επιτρέποντας αυτή τη λειτουργία. Τοποθετούνται μέσα στο index του String object για να απομονωθεί το γράμμα στην τρέχουσα στήλη της επανάληψης.

printBoardState()

```
void printBoardState() {
  Serial.println("  ┌─────────────────┐ ");
  for (int r = 7; r >= 0; r--) {
    Serial.printf(" | %d ", r + 1);
    for (int f = 0; f < 8; f++) {
      Serial.printf(" %c", pieceBoard[f][r]);
    }
    Serial.println(" |");
  }
  Serial.println("  └─────────────────┘ ");
  Serial.println("  a b c d e f g h");
}
```

Τυπώνει στο serial monitor για αποσφαλμάτωση τη θέση όλων των κομματιών από το πίνακα pieceBoard σε μορφή p,k,r,n για τα μαύρα κομμάτια και P,K,R,N για τα λευκά. Στις κενές θέσεις τυπώνεται τελεία '.' που περιέχονται μέσα στο πίνακα. Τυπώνει τις ενδεικτικές γραμμές και στήλες για να είναι πιο ευανάγνωστος ο πίνακας.

initPieceBoard()

```
void initPieceBoard() {
 for (int f = 0; f < 8; f++)
  for (int r = 0; r < 8; r++) pieceBoard[f][r] = '.';
 const char wBack[] = "RNBQKBNR";
 const char bBack[] = "rnbqkbnr";
 for (int f = 0; f < 8; f++) {
  pieceBoard[f][0] = wBack[f];
  pieceBoard[f][1] = 'P';
  pieceBoard[f][6] = 'p';
  pieceBoard[f][7] = bBack[f];
 }
}
```

Αρχικοποιεί τον πίνακα pieceBoard με τη θέση όλων των κομματιών της σκακιέρας στην αρχική θέση του παιχνιδιού σε μορφή p,k,r,n για τα μαύρα κομμάτια και P,K,R,N για τα λευκά. Στις κενές θέσεις τυπώνεται τελεία '.'.

selectPlayerColor()

```
void selectPlayerColor() {
  Lcd.clear();
  Lcd.setCursor(0, 0); Lcd.print("Choose your side");
  Lcd.setCursor(0, 1); Lcd.print("BLACK WHITE");
  Serial.println("Press BLACK (GPIO14) or WHITE (GPIO13) to choose color");
  while (true) {
    bool bB = (digitalRead(BLACK_BTN_PIN) == LOW);
    bool bW = (digitalRead(WHITE_BTN_PIN) == LOW);
    if (bB && bW) { delay(50); continue; }
    if (bB) {
      playerColor = COLOR_BLACK;
      Lcd.clear();
      Lcd.setCursor(0,0); Lcd.print("You play BLACK ");
      Lcd.setCursor(0,1); Lcd.print("White to move ");
      Serial.println("Player: BLACK. White moves first.");
      break;
    }
    if (bW) {
      playerColor = COLOR_WHITE;
      Lcd.clear();
      Lcd.setCursor(0,0); Lcd.print("You play WHITE ");
      Lcd.setCursor(0,1); Lcd.print("White to move ");
      Serial.println("Player: WHITE. White moves first.");
      break;
    }
    delay(50);
  }
  delay(400);
}
```

Καλείται κατά τη setup() μετά την αρχικοποίηση του συστήματος κίνησης ή στην επανεκκίνηση της παρτίδας και επιτρέπει στον παίκτη να επιλέξει το επιθυμητό χρώμα πατώντας το σχετικό κουμπί με μηνύματα για feedback στην οθόνη.

startNewGame()

```
void startNewGame() {
 Serial.println("\n=== NEW GAME ===");

 // Reset all state before homing
 detectState = DETECT_IDLE;
 pendingMove = "";
 lastDisplayedMove = "";
 liftedSquare = "";
 settleBestSquare = "";
 settleBestCount = 0;
 numCycleCandidates = 0;
 cycleIdx = 0;
 cyclingSource = false;
 numLegalDests = 0;
 suppressDetection = false;
 epFile = -1; epRank = -1;
 castleWK = true; castleWQ = true; castleBK = true; castleBQ =true;
 currentTurn = TURN_WHITE; playerColor = COLOR_NONE;
 computerMoveNeeded = false; computerThinking = false;
 initPieceBoard();
```

Ρυθμίζει τη κατάσταση του συστήματος για σωστή επανεκκίνηση παρτίδας χωρίς να απαιτείται πλήρης restart του μικροελεγκτή.

Αυτό το κομμάτι θέτει όλες τις παγκόσμιες μεταβλητές πίσω στην αρχική τους κατάσταση και γράφει στο πίνακα pieceBoard την αρχική θέση του παιχνιδιού με τη συνάρτηση initPieceBoard().

```
  // Re-init LCD
  Wire.begin(LCD_SDA_PIN, LCD_SCL_PIN);
  Lcd.init(); Lcd.backlight();
  Lcd.clear();
  Lcd.setCursor(0,0); Lcd.print(" NEW GAME ");
  Lcd.setCursor(0,1); Lcd.print(" Homing... ");
  // Re-home
  rgbLedWrite(38, 128, 128, 0);
  homeAndMeasure(false);
  isStopped = false;
  rgbLedWrite(38, 0, 0, 255);
```

Γίνεται επανεκκίνηση της οθόνης και ξανατρέχει η συνάρτηση homeAndMeasure(), η οποία μετακινεί τον ηλεκτρομαγνήτη αργά μέχρι τους διακόπτες για να επαναπροσδιοριστεί το σημείο (0,0) του συστήματος. Έτσι μειώνεται η πιθανότητα σφάλματος λόγω χαμένων βημάτων μεταξύ παρτίδων και εκτεταμένης χρήσης.

```
  muxScanAll();
  memcpy(boardState,results,sizeof(boardState));
  memcpy(rawPrev,results,sizeof(rawPrev));
  memset(sameCount,0,sizeof(sameCount));
  boardInitialized = true;
  lcdContent = muxBuildLcdString();
  lcdNeedsScroll = (lcdContent.length()>(size_t)LCD_COLS);
  printBoardState();
  selectPlayerColor();
  // If the computer plays White it moves first
  if (playerColor == COLOR_BLACK) computerMoveNeeded = true;
  lcdUpdate();
}
```

Αρχικοποιούνται οι πίνακες με τη κατάσταση των αισθητήρων, τυπώνονται οι ενεργοποιημένοι αισθητήρες στην οθόνη και γίνεται ξανά η επιλογή χρώματος από τον παίκτη. Τέλος κρίνεται αν απαιτείται κίνηση από τον υπολογιστή ή αν είναι σειρά του παίκτη.

passOrConfirm()

```
void passOrConfirm(bool isWhiteBtn) {
 bool isCurrentPlayer = (isWhiteBtn && currentTurn == TURN_WHITE) || (!isWhiteBtn && currentTurn == TURN_BLACK);
```

Αυτή η συνάρτηση διαχειρίζεται το πάτημα των κουμπιών κατά τη διάρκεια της παρτίδας ενημερώνοντας τη κατάσταση όλων των σχετικών μεταβλητών όταν ο παίκτης εγκρίνει μία κίνηση. Στην πρώτη γραμμή καθορίζεται αν πρόκειται για τη σειρά του παίκτη ή του υπολογιστή και ποιο κουμπί αντιστοιχεί στον καθένα.

```
 if (!isCurrentPlayer) {
  Lcd.clear();
  Lcd.setCursor(0,0); Lcd.print("Hold 1.5s: ");
  Lcd.setCursor(0,1); Lcd.print(" new game ");
```

Αν ο παίκτης πατήσει το κουμπί του αντιπάλου τυπώνεται στην οθόνη η δυνατότητα επανεκκίνησης της παρτίδας, που ενεργοποιείται με συνεχόμενο πάτημα άνω των 1.5 δευτερολέπτων.

```
  unsigned long t0 = millis();
  while (millis() - t0 < 1500) {
   bool held = isWhiteBtn ? (digitalRead(WHITE_BTN_PIN)==LOW)
                          : (digitalRead(BLACK_BTN_PIN)==LOW);
   if (!held) {
    // Short press cancel any pending/cycling move so the player can retry
    if (pendingMove.length() > 0 || detectState != DETECT_IDLE) {
     Serial.println("Opponent btn: clearing pending move.");
     pendingMove = "";
     detectState = DETECT_IDLE;
     liftedSquare = "";
     settleBestSquare = "";
     settleBestCount = 0;
     numCycleCandidates = 0;
     cycleIdx = 0;
     cyclingSource = false;
     muxScanAll();
     memcpy(boardState,results,sizeof(boardState));
     memcpy(rawPrev,results,sizeof(rawPrev));
     memset(sameCount,0,sizeof(sameCount));
    }
    lcdUpdate();
    return;
   }
   delay(20);
  }
  startNewGame();
```

```
  return;
 }
```

Σε αυτό το κομμάτι είναι η λογική για τη διαφορά επανεκκίνησης παρτίδας και χειροκίνητης επανεκκίνησης ανίχνευσης θέσης. Αν το πάτημα του κουμπιού του αντιπάλου από τον παίκτη είναι συντομότερο από 1.5 δευτερόλεπτα τότε καθαρίζεται η υποψήφια κίνηση από την οθόνη για να ξεκινήσει από την αρχή η ανίχνευση. Αυτό γίνεται από το παίκτη σε περίπτωση σφάλματος αν «κολλήσει» κάποιος αισθητήρας ως χειροκίνητο restart της ανίχνευσης αντί να καταχωρηθεί μέσω του serial monitor σε περίπτωση που δεν είναι συνδεδεμένη η κατασκευή σε υπολογιστή ή λάπτοπ. Καθαρίζονται όλες οι σχετικές μεταβλητές και πίνακες, σε περίπτωση που πατηθεί το κουμπί για παραπάνω από 1.5 δευτερόλεπτα εκτελείται πλήρης επανεκκίνηση της παρτίδας.

```
if (pendingMove.length() >= 6) {
  char f1 = pendingMove[0], r1 = pendingMove[1];
  char f2 = pendingMove[4], r2 = pendingMove[5];
  int fx1 = f1-'a', ry1 = r1-'1', fx2 = f2-'a', ry2 = r2-'1';
  char moved=pieceBoard[fx1][ry1], captured=pieceBoard[fx2][ry2];
```

Αν πατηθεί το κουμπί του επιλεγμένου χρώματος του παίκτη και υπάρχει μία έγκυρη υποψήφια κίνηση, αυτή η κίνηση καταχωρείται στο πίνακα pieceBoard για να αναπαρασταθεί η νέα κατάσταση της σκακιέρας. Αγνοώνται οι θέσεις 2 και 3 του πίνακα pendingMove διότι περιέχουν το βέλος -> μεταξύ των τετραγώνων που δηλώνουν τη κίνηση.

```
  // En passant, pawn moves diagonally one rank to empty square
  bool wasEP = (toupper(moved) == 'P'&& fx1!=fx2 && abs(ry2-ry1)==1 && captured=='.');
  if (wasEP) pieceBoard[fx2][ry1]='.';
```

Διαπιστώνεται αν η κίνηση ήταν en passant, στη περίπτωση που το πιόνι μετακινήθηκε διαγώνια (fx1 != fx2) και ακριβώς μία γραμμή μπροστά σε κενό τετράγωνο (‘.’) Χρησιμοποιείται η απόλυτη τιμή γιατί το αποτέλεσμα της κίνησης μπροστά για τα μαύρα είναι -1. Αν ισχύουν τα παραπάνω γράφεται η τιμή ‘.’ στο πίνακα μία θέση πίσω από το πιόνι που κουνήθηκε, δηλαδή πως αιχμαλωτίστηκε πιόνι με en passant.

```
  bool white = lmgIsWhite(moved);
  bool wasCastle = (toupper(moved) == 'K' && abs(fx2-fx1) == 2);
  if (wasCastle) {
   int bki = white ? 0 : 7;
   if (fx2 == 6){
     pieceBoard[5][bki] = pieceBoard[7][bki];
     pieceBoard[7][bki] = '.';}
   else if (fx2 == 2){
     pieceBoard[3][bki] = pieceBoard[0][bki];
     pieceBoard[0][bki] = '.';}
  }
  pieceBoard[fx2][ry2] = moved;
  pieceBoard[fx1][ry1] = '.';
```

Προσδιορίζεται αν ο παίκτης έκανε ροκέ και η κατεύθυνση του με βάση το κομμάτι που μετακινήθηκε αν ήταν βασιλιάς και μετακινήθηκε δύο τετράγωνα οριζόντια. Ανάλογα με το χρώμα προσδιορίζεται το bki (base king index) δηλαδή η γραμμή και με τη στήλη προορισμού

η κατεύθυνση του ροκέ. Ανανεώνονται στο πίνακα κατάλληλα οι νέες θέσεις του βασιλιά και του πύργου.

```
  if (toupper(moved) == 'P' && abs(ry2-ry1) == 2) {
    epFile = fx1;
    epRank = ry2 + (white ? -1 : 1);
  }
  else {
    epFile = -1;
    epRank = -1;
  }
  if (moved == 'K') {
    castleWK=  false;
    castleWQ = false;
  }
  if (moved == 'k') {
    castleBK = false;
    castleBQ = false;
  }
  if (moved == 'R'|| captured == 'R') {
    if (fx1==7 || fx2==7) castleWK = false;
    if (fx1==0 || fx2==0) castleWQ = false;
  }
  if (moved == 'r'|| captured == 'r') {
    if (fx1==7 || fx2==7) castleBK = false;
    if (fx1==0 || fx2==0) castleBQ = false;
  }
```

Ανανεώνονται οι διαθεσιμότητες για το en passant αποθηκεύοντας τη στήλη και γραμμή που παρουσιάζεται το πλεονέκτημα. Επίσης τη διαθεσιμότητα για τα ροκέ, τα οποία δεν επιτρέπονται με κίνηση του βασιλιά, ή κίνηση/αιχμαλωσία του πύργου για τη κατεύθυνση του.

```
Serial.printf("Confirmed: %s piece=%c",pendingMove.c_str(),moved);
  if (captured!='.') Serial.printf(" captures=%c",captured);
  if (wasEP) Serial.print(" [en passant]");
  if (wasCastle) Serial.print(" [castling]");
  Serial.println();
  Serial.printf(" EP: %s Castle: WK=%d WQ=%d BK=%d BQ=%d\n",
(epFile>=0) ? String((char)('a' + epFile) + String(epRank+1)).c_str() : "-",
castleWK,castleWQ,castleBK,castleBQ);
```

Τυπώνονται στο serial monitor διαγνωστικά για την εγκεκριμένη κίνηση όπως ποιο κομμάτι αιχμαλωτίστηκε, αν η κίνηση ήταν en passant και σε ποιο τετράγωνο το οποίο μεταφράζεται από τις μεταβλητές epFile και epRank σε στήλη και γραμμή, αλλιώς τυπώνεται παύλα. Τυπώνονται επίσης και τα δικαιώματα για ροκέ των χρωμάτων ανά χρώμα και κατεύθυνση.

```
  printBoardState();
```

```
  lastDisplayedMove=pendingMove;
  pendingMove="";
  detectState=DETECT_IDLE;
  liftedSquare="";
  settleBestSquare="";
  settleBestCount=0;
  numCycleCandidates=0;
  cycleIdx=0;
  cyclingSource=false;

  muxScanAll();
  memcpy(boardState,results,sizeof(boardState));
  memcpy(rawPrev,results,sizeof(rawPrev));
  memset(sameCount,0,sizeof(sameCount));
 }
```

Τυπώνεται η κατάσταση της σκακιέρας στο serial monitor και καθαρίζονται όλες οι μεταβλητές και πίνακες που σχετίζονται με την ανίχνευση κινήσεων του παίκτη.

```
} else {
   bool playerTurn =
(playerColor==COLOR_WHITE && currentTurn==TURN_WHITE) ||
(playerColor==COLOR_BLACK && currentTurn==TURN_BLACK);
   if (playerTurn) {
    Serial.println("No pending move yet.");
    return;
   }
   Serial.println("Turn passed.");
 }
 currentTurn = (currentTurn==TURN_WHITE) ? TURN_BLACK : TURN_WHITE;
 Serial.printf("Turn: %s\n",
(currentTurn==TURN_WHITE) ? "WHITE" : "BLACK");
```

Καθορίζεται αν είναι η σειρά του παίκτη και σε περίπτωση έγκρισης χωρίς υποψήφια κίνηση απορρίπτεται το πάτημα του κουμπιού. Αν είναι η σειρά του υπολογιστή ο παίκτης μπορεί να την ακυρώσει πατώντας το κουμπί του αν επιθυμεί. Τυπώνεται στο serial monitor ποιο χρώμα παίζει για αποσφαλμάτωση.

```
  // Check if it's now the computer's turn
  bool isNowComputerTurn =
(playerColor==COLOR_WHITE && currentTurn==TURN_BLACK) ||
(playerColor==COLOR_BLACK && currentTurn==TURN_WHITE);
  if (isNowComputerTurn) computerMoveNeeded = true;
  lcdUpdateRow1();
}
```

Προσδιορίζεται αν χρειάζεται κίνηση του υπολογιστή και ανανεώνεται η δεύτερη γραμμή της οθόνης προτού επιστρέψει η συνάρτηση.

Legal Move Generation

Η συντομογραφία lmg στις συναρτήσεις είναι για legal move generator (Move Generation, 2022).

```
bool lmgIsWhite(char p) {
  return p>='A' && p<='Z';
}
```

Επιστρέφει αν ένα κομμάτι είναι λευκό με βάση αν αναπαρίσταται με κεφαλαίο γράμμα.

```
bool lmgIsBlack(char p) {
  return p>='a' && p<='z';
}
```

Επιστρέφει αν ένα κομμάτι είναι μαύρο με βάση αν αναπαρίσταται με πεζό γράμμα.

```
bool lmgIsFriendly(char piece, int f, int r) {
 if (f<0 || f>=8 || r<0 || r>=8 ) return false;
 char t = pieceBoard[f][r];
 if (t=='.') return false;
 return lmgIsWhite(piece) ? lmgIsWhite(t) : lmgIsBlack(t);
}
```

Επιστρέφει αν ένα κομμάτι σε κάποιο τετράγωνο είναι το ίδιο χρώμα με το δοσμένο, δηλαδή αν είναι φιλικό προς το χρώμα.

```
bool lmgIsEnemy(char piece, int f, int r) {
 if (f<0||f>=8||r<0||r>=8) return false;
 char t = pieceBoard[f][r]; if (t=='.') return false;
 return lmgIsWhite(piece) ? lmgIsBlack(t) : lmgIsWhite(t);
}
```

Επιστρέφει αν ένα κομμάτι σε κάποιο τετράγωνο είναι διαφορετικό χρώμα με το δοσμένο, δηλαδή αν είναι εχθρικό προς το χρώμα.

```
void lmgAddDest(int f, int r) {
 if (f<0||f>=8||r<0||r>=8) return;
 if (numLegalDests >= MAX_LEGAL_MOVES) return;
 char s[3];
 s[0] = 'a'+ f;
 s[1] = '1'+ r;
 s[2] = 0;
 legalDests[numLegalDests++] = String(s);
}
```

Προσθέτει ένα τετράγωνο στο πίνακα επιτρεπόμενων κινήσεων legalDests αφού μετατραπεί πίσω σε χαρακτήρες αντί για αριθμούς.

```
void lmgSlide(char piece, int f, int r, int df, int dr) {
 for (int i=1; i<8; i++) {
  int nf = f + df*i, nr = r + dr*i;
  if (nf<0||nf>=8||nr<0||nr>=8) break;
  if (lmgIsFriendly(piece,nf,nr)) break;
```

```
    ImgAddDest(nf,nr);
    if (ImgIsEnemy(piece,nf,nr)) break;
  }
}
```

Υπολογίζει τις νόμιμες κινήσεις για κομμάτια όπως η βασίλισσα, οι πύργοι και οι αξιωματικοί που κινούνται πολλαπλά τετράγωνα σε μία κατεύθυνση.

Με βάση τις μεταβλητές df,dr επιλέγεται η κατεύθυνση ως εξής:

(df, dr)	*Direction*
(0, 1)	Πάνω
(0, -1)	Κάτω
(1, 0)	Δεξιά
(-1, 0)	Αριστερά
(1, 1)	Πάνω-Δεξιά
(1, -1)	Πάνω-Αριστερά
(-1, 1)	Κάτω-Δεξιά
(-1, -1)	Κάτω-Αριστερά

Πίνακας 17 - Κατευθύνσεις κομματιών

Η συνάρτηση διακόπτεται νωρίς αν συναντηθεί φιλικό κομμάτι που εμποδίζει τη κίνηση ή εχθρικό κομμάτι αφού καταχωρίσει την αιχμαλωσία του εχθρικού κομματιού ως έγκυρη κίνηση.

computeLegalDests()

```
void computeLegalDests(int fileIdx, int rankIdx) {
 numLegalDests = 0;
 if (fileIdx<0||fileIdx>=8||rankIdx<0||rankIdx>=8) return;
 char piece = pieceBoard[fileIdx][rankIdx];
 if (piece=='.') return;

 char pt = toupper(piece);
 bool white = lmgIsWhite(piece);
 int dir = white ? 1 : -1; // pawn advance direction
```

Υπολογίζει όλες τις νόμιμες κινήσεις για ένα κομμάτι, οι πρώτες γραμμές αποθηκεύουν σε μεταβλητές χρήσιμες πληροφορίες όπως το είδος του κομματιού, το χρώμα και τη κατεύθυνση που μετακινείται αν είναι πιόνι.

```
switch (pt) {
  case 'P': {
   // Advance one square
   int nr = rankIdx + dir;
   if (nr >= 0 && nr < 8 && pieceBoard[fileIdx][nr] == '.') {
    lmgAddDest(fileIdx, nr);
    // Advance two from starting rank
    int startRank = white ? 1 : 6;
    if (rankIdx == startRank &&
pieceBoard[fileIdx][rankIdx + 2*dir] == '.') {
      lmgAddDest(fileIdx, rankIdx + 2*dir);
    }
   }
   // Diagonal captures
   for (int df = -1; df <= 1; df += 2) {
    if (lmgIsEnemy(piece, fileIdx + df, rankIdx + dir))
     lmgAddDest(fileIdx + df, rankIdx + dir);
    }
   break;
  }
```

Στη πρώτη περίπτωση που εξετάζεται πιόνι ελέγχεται αν η θέση μπροστά είναι κενή, αν είναι η πρώτη κίνηση του που θα επέτρεπε να μετακινηθεί δύο τετράγωνα μπροστά και αν μπορεί να αιχμαλωτίσει εχθρικό κομμάτι διαγωνίως.

```
case 'N': {
    const int mv[8][2] = { {1,2}, {2,1}, {2,-1}, {1,-2},
{-1,-2}, {-2,-1}, {-2,1}, {-1,2} };
    for (int i = 0; i < 8; i++)
      if (!ImgIsFriendly(piece, fileIdx + mv[i][0],
rankIdx + mv[i][1]))
        ImgAddDest(fileIdx + mv[i][0], rankIdx + mv[i][1]);
    break;
  }
```

Για να ελεγχθεί η νομιμότητα κινήσεων του ίππου που κινείται σε σχήμα L εξετάζονται και οι 8 διαθέσιμες κινήσεις του. Οργανώνονται σε έναν πίνακα {df, dr} όπου για παράδειγμα το {2,-1} συμβολίζει 2 στήλες δεξιά και μία γραμμή κάτω, ή το {1,2} είναι 1 στήλη δεξιά και 2 γραμμές πάνω. Η μόνη περίπτωση να μην επιτρέπεται μία κίνηση ίππου, εξαιρώντας το ρουά στο βασιλιά ή να είναι καρφωμένο, είναι να βρίσκεται στη θέση προορισμού ένα φιλικό κομμάτι. Άρα αν το τετράγωνο προορισμού έχει εχθρικό κομμάτι ή είναι κενό η κίνηση προστίθεται στις νόμιμες για τώρα.

```
  case 'B': {
    ImgSlide(piece,fileIdx,rankIdx, 1, 1);
    ImgSlide(piece,fileIdx,rankIdx, 1,-1);
    ImgSlide(piece,fileIdx,rankIdx,-1, 1);
    ImgSlide(piece,fileIdx,rankIdx,-1,-1);
    break;
  }
```

Για τη περίπτωση του αξιωματικού ελέγχονται οι 4 διαγώνιες με τη συνάρτηση ImgSlide() που ελέγχει κάθε τετράγωνο στη προσδιορισμένη κατεύθυνση μέχρι να συναντήσει φιλικό ή εχθρικό κομμάτι.

```
  case 'R': {
    ImgSlide(piece,fileIdx,rankIdx, 1, 0);
    ImgSlide(piece,fileIdx,rankIdx,-1, 0);
    ImgSlide(piece,fileIdx,rankIdx, 0, 1);
    ImgSlide(piece,fileIdx,rankIdx, 0,-1);
    break;
  }
```

Για τη περίπτωση του πύργου ελέγχονται οι 4 ευθείες με τη συνάρτηση ImgSlide() που ελέγχει κάθε τετράγωνο στη προσδιορισμένη κατεύθυνση μέχρι να συναντήσει φιλικό ή εχθρικό κομμάτι.

```
  case 'Q': {
    ImgSlide(piece,fileIdx,rankIdx, 1, 1);
    ImgSlide(piece,fileIdx,rankIdx, 1,-1);
    ImgSlide(piece,fileIdx,rankIdx,-1, 1);
    ImgSlide(piece,fileIdx,rankIdx,-1,-1);
    ImgSlide(piece,fileIdx,rankIdx, 1, 0);
    ImgSlide(piece,fileIdx,rankIdx,-1, 0);
```

```
    lmgSlide(piece,fileIdx,rankIdx, 0, 1);
    lmgSlide(piece,fileIdx,rankIdx, 0,-1);
    break;
   }
```

Για τη περίπτωση της βασίλισσας ελέγχονται όλες οι κατευθύνσεις, 4 ευθείες και 4 διαγώνιες, με τη συνάρτηση lmgSlide() που ελέγχει κάθε τετράγωνο στη προσδιορισμένη κατεύθυνση μέχρι να συναντήσει φιλικό ή εχθρικό κομμάτι.

```
   case 'K': {
    for (int df = -1; df <= 1; df++)
     for (int dr = -1; dr <= 1; dr++) {
      if (df == 0 && dr == 0) continue;
      if (!lmgIsFriendly(piece, fileIdx+df, rankIdx+dr))
        lmgAddDest(fileIdx+df, rankIdx+dr);
    }
    break;
   }
  }
```

Για τον βασιλιά σε αυτό το σημείο υπολογίζονται τα 8 γειτονικά τετράγωνα, εξαιρώντας το τετράγωνο του βασιλιά του ίδιου, για αν δεν υπάρχουν φιλικά κομμάτια. Αν το τετράγωνο δηλαδή είναι κενό ή κατοικείται από εχθρικό κομμάτι η κίνηση επιτρέπεται για τώρα.

```
  // En passant
  if (pt == 'P' && epFile >= 0) {
   // epFile/epRank is where the capturable pawn sits, en passant means it can be taken diagonally to the square behind it (one rank in the moving side's direction)
   int epCaptureRank = epRank + dir; // where the capturing pawn lands
   if (abs(fileIdx - epFile) == 1 && rankIdx == epRank) {
    lmgAddDest(epFile, epCaptureRank);
   }
  }
```

Εδώ εξετάζεται επιπρόσθετα για το πιόνι με βάση τη διαθεσιμότητα για en passant αν επιτρέπεται η διαγώνια αιχμαλώτιση σε αυτή τη σειρά. Για να επιτρέπεται πρέπει από προηγούμενη κίνηση η μεταβλητής epFile να είναι έγκυρη, όχι -1, επειδή κάποιο πιόνι έκανε διπλή κίνηση μπροστά. Ύστερα υπολογίζεται το τετράγωνο πίσω από το εχθρικό πιόνι εκεί που θα καταλήξει το εξεταζόμενο σύμφωνα με τον κανόνα. Ελέγχεται επίσης αν το πιόνι βρίσκεται σε διπλανή στήλη και στην ίδια γραμμή που απαιτούνται για την αξιοποίηση του en passant.

```
// Castling
if (pt=='K') {
  int backRank = white ? 0 : 7;
  if (rankIdx == backRank && fileIdx == 4) { // king on e1/e8
  bool inCheck = lmgIsAttackedBy(4, backRank, !white, pieceBoard);
    if (!inCheck) {
      // Kingside (e->g, rook on h)
      bool ksr = white ? castleWK : castleBK;
      if (ksr && pieceBoard[5][backRank] == '.' &&
          pieceBoard[6][backRank] == '.' &&
          pieceBoard[7][backRank] == (white?'R':'r') &&
          !lmgIsAttackedBy(5,backRank,!white,pieceBoard) &&
          !lmgIsAttackedBy(6,backRank,!white,pieceBoard)) {
        lmgAddDest(6, backRank); // king lands on g file (index 6)
      }
      // Queenside (e->c, rook on a)
      bool qsr = white ? castleWQ : castleBQ;
      if (qsr && pieceBoard[3][backRank]=='.' &&
          pieceBoard[2][backRank] == '.' &&
          pieceBoard[1][backRank] == '.' &&
          pieceBoard[0][backRank] == (white?'R':'r') &&
          !lmgIsAttackedBy(3,backRank,!white,pieceBoard) &&
          !lmgIsAttackedBy(2,backRank,!white,pieceBoard)) {
        lmgAddDest(2, backRank); // king lands on c file (index 2)
      }
    }
  }
}
```

Ελέγχεται το δικαίωμα νομιμότητας για ροκέ για κάθε χρώμα σε όλες τις κατευθύνσεις. Υπολογίζεται η γραμμή του βασιλιά ανάλογα με το χρώμα και αν ο βασιλιάς βρίσκεται στο αρχικό του τετράγωνο και δεν είναι υπό απειλή εξετάζονται οι υπόλοιπες συνθήκες. Για το μικρό ροκέ στη μεριά του βασιλιά εξετάζεται αν έχει απορριφθεί προηγουμένως το ροκέ, τις τρεις δεξιές στήλες να είναι άδειες και την ύπαρξη του πύργου στη γωνία, όπως και να μην είναι υπό απειλή τα τετράγωνα από τα οποία πρέπει να περάσει ο βασιλιάς. Αν όλα τα παραπάνω είναι αληθή επιτρέπεται το μικρό ροκέ. Το μεγάλο ροκέ από τη μεριά της βασίλισσας εξετάζεται με τις ίδιες προϋποθέσεις με την αλλαγή πως τα κενά τετράγωνα πρέπει να βρίσκονται αριστερά για τρεις στήλες αντί για δύο.

```
// Filter moves that leave own king in check (handles pins too)
 filterChecks(fileIdx, rankIdx);
```

Η συνάρτηση που απορρίπτει τις ως στιγμής νόμιμες κινήσεις με βάση τα ρουά και τα καρφωμένα κομμάτια είναι η filterChecks() και αναλύεται παρακάτω στο σχετικό κεφάλαιο.

```
// Log legal destinations for debugging
 String dbg = "Legal dests for ";
```

```
  dbg += (char)('a'+fileIdx);
  dbg += (char)('1'+rankIdx);
  dbg += " (";
  dbg += piece;
  dbg += "):";
  for (int i = 0; i < numLegalDests; i++) {
    dbg += ' ';
    dbg += legalDests[i];
  }
  ESP_LOGI(TAG, "%s", dbg.c_str());
}
```

Εκτυπώνονται όλες οι νόμιμες κινήσεις για αποσφαλμάτωση μόνο στη περίπτωση που έχει τεθεί το CoreDebugLevel σε τουλάχιστον ‘info’ σε μορφή f3(P): f4 f5 … .

nearestLegal()

```
String nearestLegal(const String& candidate) {
  for (int i = 0; i < numLegalDests; i++)
    if (legalDests[i] == candidate) return candidate;
  ESP_LOGI(TAG, "Rule filter: %s not legal, discarding",   candidate.c_str());
  return "";
}
```

Η συνάρτηση εξετάζει αν μία υποψήφια κίνηση από την αποσφαλμάτωση των αισθητήρων είναι καταχωρημένη ως νόμιμη στο πίνακα legalDests και την επιστρέφει, αλλιώς τυπώνει σφάλμα και γυρνάει ένα άδειο string. Χρησιμοποιείται ως τελικό μέτρο ασφαλείας απόρριψης υποψήφιων κινήσεων των αισθητήρων με βάση τη νομιμότητα της κίνησης.

collectSourceCandidates()

```
int collectSourceCandidates(const String& dest, bool forWhite) {
 numCycleCandidates = 0;
 if (dest.length() < 2) return 0;
 int dfi = dest[0] - 'a', dri = dest[1] - '1';
 if (dfi < 0 || dfi >= 8 || dri < 0 || dri >= 8) return 0;
 String buf[MAX_CYCLE_CANDIDATES];
 int nBuf=0;
```

Σε αυτή τη συνάρτηση επαληθεύεται μία φυσική κίνηση του παίκτη συνδυάζοντας τη τρέχουσα θέση, τις μετρήσεις των αισθητήρων και το legal move generation. Στις πρώτες γραμμές αρχικοποιούνται οι μεταβλητές που χρησιμοποιούνται απορρίπτοντας τυχόν λάθος inputs και μετατρέποντας το τετράγωνο προορισμού σε αριθμητικό index. Προετοιμάζεται ένα buffer για την αποθήκευση όλων των δυνατών κινήσεων σε περίπτωση ασάφειας του πλέγματος αισθητήρων.

```
for (int fi = 0;  fi < 8 && nBuf < MAX_CYCLE_CANDIDATES; fi++) {
   for (int ri = 0; ri < 8 && nBuf < MAX_CYCLE_CANDIDATES; ri++) {
    char p = pieceBoard[fi][ri];
    if (p=='.') continue;
    if (forWhite != ImgIsWhite(p)) continue;
    if (fi == dfi && ri == dri) continue;
```

Μία διπλή επανάληψη σαρώνει όλα τα τετράγωνα χρησιμοποιώντας το πίνακα pieceBoard[] για το καθορισμό της θέσης όλων των κομματιών. Επιστρέφει νωρίς η επανάληψη αν το τετράγωνο προορισμού είναι άδειο, το κομμάτι ανήκει στον αντίπαλο ή ο προορισμός είναι ίδιος με το αρχικό τετράγωνο.

```
computeLegalDests(fi, ri);
    for (int i = 0; i < numLegalDests; i++) {
      if (legalDests[i] == dest) {
        String sq;
        sq += (char)('a'+fi);
        sq += (char)('1'+ri);
        buf[nBuf++] = sq;
        break;
      }
    }
   }
 }
  if (nBuf == 0) return 0;
```

Αν η επανάληψη δεν γύρισε νωρίς υπολογίζονται όλες οι νόμιμες κινήσεις για το κομμάτι ανάποδα, δηλαδή ποια κομμάτια θα μπορούσαν να πάνε στο dest. Η συνάρτηση computeLegalDests() αποθηκεύει τα αποτελέσματα στο πίνακα legalDests[], όπου συγκρίνεται με το input και αν υπάρχει ταύτιση μετατρέπεται πίσω σε τετράγωνο και αποθηκεύεται στο buffer. Αν το buffer μετά την επανάληψη είναι άδειο η συνάρτηση γυρνάει 0 για αποσφαλμάτωση.

```
for (int i = 0; i < nBuf; i++) {
  if (buf[i] == liftedSquare) {
    numCycleCandidates = 1;
    cycleCandidates[0] = liftedSquare;
    computeLegalDests(liftedSquare[0] - 'a', liftedSquare[1] - '1');
    return 1;
  }
 }
```

Εδώ συνδυάζεται με τη μέτρηση των αισθητήρων, αν το κομμάτι που σήκωσε ο παίκτης μπορεί να κινηθεί στα υπολογισμένα τετράγωνα του buffer ανανεώνεται ο πίνακας legalDests[] και γυρνάει 1 για επιτυχή εύρεση της κίνησης.

```
 numCycleCandidates = nBuf;
 for (int i = 0; i < nBuf; i++) cycleCandidates[i] = buf[i];
 computeLegalDests(cycleCandidates[0][0] - 'a', cycleCandidates[0][1] - '1');
 return nBuf;
}
```

Τέλος, αν κανένα υπολογισμένο τετράγωνο δεν είναι το liftedSquare τότε γράφονται οι κινήσεις στο πίνακα cycleCandidates[] για ανάδειξη τους με τη σειρά στο παίκτη καθώς δεν μπορεί να καθοριστεί μία μεμονωμένη κίνηση.

ImgIsAttackedBy()

```
bool ImgIsAttackedBy(int f, int r, bool byWhite, char board[8][8]) {
 char wp = byWhite ? 'P' : 'p';
 char wn = byWhite ? 'N' : 'n';
 char wb = byWhite ? 'B' : 'b';
 char wr = byWhite ? 'R' : 'r';
 char wq = byWhite ? 'Q' : 'q';
 char wk = byWhite ? 'K' : 'k';
 int pd = byWhite ? -1 : 1; // pawn attacks come from behind
```

Η συνάρτηση ελέγχει ένα τετράγωνο αν βρίσκεται υπό απειλή, αρχικά διαπιστώνεται από τη byWhite το χρώμα και αρχικοποιούνται τα λευκά ή τα μαύρα κομμάτια μαζί με τη κατεύθυνση των πιονιών ανάλογα του pieceBoard[].

```
// Pawn
 for (int df = -1; df <= 1; df += 2) {
  int sf = f + df, sr = r + pd;
  if (sf>=0 && sf<8 && sr>=0 && sr<8 && board[sf][sr] == wp) return true;
 }
```

Ελέγχονται αρχικά οι αριστερή και δεξιά στήλη, η γραμμή ανάλογα με τη κατεύθυνση των πιονιών, και αν υπάρχει εχθρικό πιόνι εκεί η συνάρτηση επιστρέφει true πως το τετράγωνο απειλείται.

```
// Knight
 const int kn[8][2] = {{1,2}, {2,1}, {2,-1}, {1,-2}, {-1,-2}, {-2,-1}, {-2,1}, {-1,2}};
 for (int i = 0; i < 8; i++) {
  int nf = f + kn[i][0], nr = r + kn[i][1];
  if (nf >= 0 && nf < 8 && nr >= 0 && nr < 8 && board[nf][nr] == wn) return true;
 }
```

Ελέγχονται όλα τα πιθανά τετράγωνα, σχετικά με το εξεταζόμενο τετράγωνο, για την παρουσία εχθρικού ίππου, αν βρεθεί επιστρέφεται πάλι true πως το τετράγωνο απειλείται.

```
// King
 for (int df = -1; df <= 1; df++) for (int dr = -1; dr <= 1; dr++) {
  if (!df && !dr) continue;
  int nf = f + df, nr = r + dr;
  if (nf >= 0 && nf < 8 && nr >=0 && nr < 8 && board[nf][nr] == wk) return true;
 }
```

Εξετάζονται τα 8 γειτονικά τετράγωνα, αγνοώντας το τετράγωνο που εξετάζεται, στα οποία αν βρεθεί εχθρικός βασιλιάς τότε το εξεταζόμενο τετράγωνο απειλείται.

```
 // Bishop/Queen diagonals
 const int diag[4][2] = {{1,1}, {1,-1}, {-1,1}, {-1,-1}};
 for (int d = 0; d < 4; d++) {
  for (int i = 1; i < 8; i++) {
   int nf = f + diag[d][0] * i, nr = r + diag[d][1] * i;
   if (nf < 0 || nf >= 8 || nr < 0 || nr >= 8) break;
```

```
    char t = board[nf][nr];
    if (t != '.') {
      if (t == wb || t == wq) return true;
      break; }
   }
  }
```

Εξετάζονται όλες οι διαγώνιοι για παρουσία εχθρικού αξιωματικού ή βασίλισσας χρησιμοποιώντας έναν διπλό βρόγχο και απορρίπτοντας τετράγωνα εκτός της σκακιέρας για απλούστευση του κώδικα.

```
// Rook/Queen straights
 const int straight[4][2]= {{1,0}, {-1,0}, {0,1}, {0,-1}};
 for (int d = 0; d < 4; d++) {
  for (int i = 1; i < 8; i++) {
    int nf = f + straight[d][0] * i, nr = r + straight[d][1] * i;
    if (nf<0|| nf >= 8 ||nr < 0 || nr >= 8) break;
    char t=board[nf][nr];
    if (t!='.') {
      if (t==wr||t==wq) return true;
      break;
    }
   }
  }
  return false;
}
```

Τέλος ελέγχονται οι ευθείες με παρόμοιο τρόπο με τις διαγώνιες για παρουσία εχθρικού πύργου ή βασίλισσας. Αν δεν βρεθεί καμία απειλή από όλους τους ελέγχους συνολικά η συνάρτηση επιστρέφει false.

ImgFindKing()

```
bool ImgFindKing(bool white, char board[8][8], int &kf, int &kr) {
 char k = white ? 'K' : 'k';
 for (int f = 0; f < 8; f++) for (int r = 0; r < 8; r++)
  if (board[f][r] == k) {
   kf = f;
   kr = r;
   return true;
  }
 return false;
}
```

Βοηθητική συνάρτηση που επιστρέφει αν υπάρχει ο βασιλιάς ενός χρώματος σε ένα τετράγωνο σαρώνοντας όλη τη σκακιέρα.

ImgInCheckAfterMove()

```
bool ImgInCheckAfterMove(int f1, int r1, int f2, int r2,
                         bool epCapture, bool white) {
 char tmp[8][8];
 memcpy(tmp, pieceBoard, 64);
 tmp[f2][r2] = tmp[f1][r1];
 tmp[f1][r1] = '.';
 if (epCapture) {
  // Removes the captured pawn (it is on epFile, r1 same rank as moving pawn)
  tmp[epFile][r1] = '.';
 }
 int kf, kr;
 if (!ImgFindKing(white, tmp, kf, kr)) return false; // no king found (shouldn't happen)
 return ImgIsAttackedBy(kf, kr, !white, tmp);
}
```

Επιστρέφει αν μετά από μία κίνηση ο βασιλιάς του ίδιου του παίκτη που παίζει όχι του αντιπάλου βρίσκεται σε ρουά, χρησιμοποιείται για να αποτρέψει παράνομες κινήσεις από καρφωμένα κομμάτια ή του ίδιου του βασιλιά.

filterChecks()

```
void filterChecks(int srcF, int srcR) {
  bool white = ImgIsWhite(pieceBoard[srcF][srcR]);
  int kept = 0;
  for (int i = 0; i < numLegalDests; i++) {
    int df = legalDests[i][0] - 'a';
    int dr = legalDests[i][1] - '1';
```

Το τελικό φίλτρο των παράνομων κινήσεων, ευθύνεται για την απόρριψη κινήσεων που θα ήταν νόμιμες αλλά αφήνουν το βασιλιά σε ρουά λόγω καρφώματος του κομματιού. Μετατρέπονται αρχικά οι αποθηκευμένες κινήσεις του πίνακα legalDests[] σε indexes.

```
// Detect en passant capture, pawn moves diagonally to empty square
    bool ep = false;
    if (toupper(pieceBoard[srcF][srcR]) == 'P' && df != srcF &&
        pieceBoard[df][dr] == '.' && epFile == df) {
      ep = true;
    }
    if (!ImgInCheckAfterMove(srcF, srcR, df, dr, ep, white)) {
      legalDests[kept++] = legalDests[i];
    }
  }
```

Ύστερα διαπιστώνεται αν επιτρέπεται το en passant. Αν η κίνηση δεν αφήνει τον βασιλιά εκτεθειμένο γράφεται ξανά στον πίνακα legalDests πάνω από τη προηγούμενη τιμή του.

```
  int removed = numLegalDests - kept;
  numLegalDests = kept;
  if (removed > 0) {
    ESP_LOGI(TAG, "filterChecks removed %d illegal moves (pins/checks)", removed);
  }
}
```

Οι απορριφθέντες κινήσεις μένουν με αυτό το τρόπο στο τέλος του πίνακα, αλλά αυτό αντιμετωπίζεται σώζοντας τον αριθμό των επιτρεπτών κινήσεων στο numLegalDests για να γνωρίζει το σύστημα μέχρι πού να διαβάσει τον πίνακα. Τυπώνονται οι κινήσεις που απορρίφθηκαν αν το CoreDebugLevel είναι info και πάνω για αποσφαλμάτωση.

resetBoardStateForLegalDests()

```
void resetBoardStateForLegalDests() {
  for (int i = 0; i < numLegalDests; i++) {
    int fi = legalDests[i][0] - 'a'; // 0-7
    int ri = legalDests[i][1] - '1'; // 0-7
    int m = 3 - ri / 2; // mux index
    int ch = (ri % 2) * 8 + (7 - fi); // channel index
    if (m >= 0 && m < NUM_MUX && ch >= 0 && ch < NUM_CHANNELS) {
      boardState[m][ch] = false;
      rawPrev[m][ch] = false;
      sameCount[m][ch] = 0;
    }
  }
}
```

Μετά την εύρεση των πιθανών νόμιμων τετραγώνων της κίνησης η οποία εκτελείται γίνεται reset των αισθητήρων για αυτά τα τετράγωνα, όπως και του συστήματος αποσφαλμάτωσης τους. Αυτό ανανεώνει τις μετρήσεις τους, συγκεκριμένα σε αυτά τα τετράγωνα στα οποία αναμένεται η άφιξη του κομματιού, για να μεγιστοποιηθεί η πιθανότητα επιτυχούς μέτρησης.

advanceCycling()

```
void advanceCycling() {
  if (numCycleCandidates < 2) return;
  if (millis() - lastCycleTime < 3000) return;
  lastCycleTime = millis();
  cycleIdx = (cycleIdx + 1) % numCycleCandidates;
  if (cyclingSource) {
    String dest = pendingMove.substring(4);
    pendingMove = cycleCandidates[cycleIdx] + "->" + dest;
  } else {
    String src = pendingMove.substring(0, 2);
    pendingMove = src + "->" + cycleCandidates[cycleIdx];
  }
  Serial.printf(" Cycling %s: %s\n", cyclingSource?"src":"dst", pendingMove.c_str());
  lcdUpdateRow1();
}
```

Η συνάρτηση εμφανίζει και εναλλάσει με τη σειρά στην οθόνη όλους δους διαθέσιμους προορισμούς ή sources σε περίπτωση αποτυχίας του συστήματος να εξακριβώσει μία κίνηση για να επιβεβαιώσει ο παίκτης τη σωστή. Χρησιμοποιείται η συνάρτηση substring του Arduino object String για να απομονωθεί γρήγορα το μέρος του string που εξετάζεται. Ανανεώνεται η κίνηση ανά 3 δευτερόλεπτα και το modulo (%) κάνει τις κινήσεις να εμφανίζονται κυκλικά. Δεν υποστηρίζεται να ανανεώνεται ταυτόχρονα το source και το destination, καλύπτεται η ασάφεια μόνο για ένα λόγω δυσκολίας εξακρίβωσης της κίνησης από τον παίκτη αν εναλλάσσονταν και τα δύο.

detectMove()

Η detectMove ευθύνεται να συνδυάσει όλες τις βοηθητικές συναρτήσεις για την ανίχνευση κινήσεων. Χρησιμοποιεί 3 καταστάσεις που ορίζονται από ένα enum, DETECT_IDLE κατά το οποίο ένα τετράγωνο αναμένεται να αδειάσει, PIECE_IN_AIR όπου κάποιο τετράγωνο αναμένεται να «κατοικηθεί» και PIECE_SETTLING όπου γίνεται η αποσφαλμάτωση λόγω της ασάφειας των αισθητήρων για να διαπιστωθεί με βεβαιότητα ότι το κομμάτι βρίσκεται στο τετράγωνο αντί για λάθος ενεργοποίηση αισθητήρα.

```
void detectMove() {
 if (!boardInitialized || suppressDetection) return;
 if (Stepper1->isRunning() || Stepper2->isRunning()) return;
 if (pendingMove.length() > 0) return;

 // Check if settling window has expired
 if (detectState == PIECE_SETTLING &&
    millis() - settleStart >= SETTLE_WINDOW_MS) {

  // Find active legal destinations
  String found[MAX_LEGAL_MOVES];
  int nFound=0;
  for (int i = 0; i < numLegalDests; i++) {
   int fi = legalDests[i][0] - 'a', ri = legalDests[i][1] - '1';
   int mm= 3 - ri / 2, cc = (ri % 2) * 8 + (7 - fi);
   if (mm >= 0 && mm<NUM_MUX && cc>=0 && cc<NUM_CHANNELS && results[mm][cc])
    if (legalDests[i] != liftedSquare) found[nFound++] = legalDests[i];
  }
```

Η ανίχνευση εκτελείται με τους βηματικούς κινητήρες σε ηρεμία και αν δεν υπάρχει ήδη κάποια υποψήφια κίνηση. Όταν ο χρόνος αναμονής έχει περάσει κατά το PIECE_SETTLING διαβάζονται οι αισθητήρες στα τετράγωνα που είναι νόμιμα για τη κίνηση και αντιγράφει τα αποτελέσματα στο πίνακα found[] αν είναι σύμφωνα με τον πίνακα results[], που αποθηκεύει τις τιμές των αισθητήρων.

```
  String chosen="";
  cyclingSource=false;
  numCycleCandidates=0;

  if (nFound == 1) {
   chosen = found[0];
  } else if (nFound > 1) {
   numCycleCandidates = min(nFound, MAX_CYCLE_CANDIDATES);
   for (int i = 0; i < numCycleCandidates; i++) cycleCandidates[i] = found[i];
   cycleIdx = 0;
   lastCycleTime = millis();
   chosen = found[0];
   Serial.printf(" >> Ambiguous dest: %d:", nFound);
```

```
    for (int i = 0; i < nFound; i++) Serial.printf(" %s", found[i].c_str());
    Serial.println(" -- cycling dests every 3s");
```

Αν βρέθηκε μία κίνηση γράφεται απευθείας ως η επιλεγμένη, ενώ στη περίπτωση που βρέθηκαν παραπάνω γίνεται η διαδικασία της αποσφαλμάτωσης από τον παίκτη στην οθόνη. Οι κινήσεις ανακυκλώνονται ανά 3 δευτερόλεπτα μέχρι ο παίκτης να επιβεβαιώσει ποια από τις υποψήφιες κινήσεις είναι η σωστή. Αυτό γίνεται μόνο σε περίπτωση αποτυχίας του συστήματος να αναγνωρίσει μία μεμονωμένη κίνηση.

```
} else {
    String activeFound = "";
    for (int i = 0; i < numLegalDests; i++) {
      int fi = legalDests[i][0] - 'a', ri = legalDests[i][1] - '1';
      int mm = 3 - ri / 2, cc = (ri % 2) * 8 + (7 - fi);
      if (mm>=0 && mm<NUM_MUX && cc>=0 && cc<NUM_CHANNELS &&
          results[mm][cc] && legalDests[i] != liftedSquare) {
        activeFound = legalDests[i];
        break;
      }
    }
```

Αν ο πίνακας found είναι άδειος άρα δεν υπάρχει κανένα εξακριβωμένο τετράγωνο προορισμού γίνεται ξανά σάρωση των αισθητήρων στα νόμιμα τετράγωνα και επιλέγεται το πρώτο που ο αισθητήρας του είναι HIGH ως το σωστό, αυτό είναι δικλείδα ασφαλείας και χρησιμοποιείται μόνο αν ο προηγούμενος έλεγχος δεν επέστρεψε αποτελέσματα.

```
if (activeFound.length() > 0) {
      chosen = activeFound;
      ESP_LOGI(TAG, "Fallback: using active sensor %s over settleBest %s",
            activeFound.c_str(), settleBestSquare.c_str());
    } else {
      chosen = nearestLegal(settleBestSquare); // if not legal
    }
  }
```

Στη περίπτωση που χρησιμοποιήθηκε η δικλείδα ασφαλείας, αν βρέθηκε ενεργός αισθητήρας γράφεται ως η επιλεγμένη κίνηση αλλιώς επιλέγεται η κοντινότερη νόμιμη κίνηση ανεξαρτήτως των μετρήσεων.

```
  // Find source
  if (chosen.length() > 0) {
    bool forWhite = (currentTurn==TURN_WHITE);
    int nSrc = collectSourceCandidates(chosen, forWhite);
    if (nSrc == 0) {
      chosen = "";
    } else if (nSrc == 1) {
      liftedSquare = cycleCandidates[0];
    } else {
```

```
      cyclingSource = true;
      cycleIdx = 0;
      lastCycleTime = millis();
      liftedSquare = cycleCandidates[0];
      Serial.printf(" >> Ambiguous src: %d:", nSrc);
      for (int i = 0; i < nSrc; i++) Serial.printf(" %s", cycleCandidates[i].c_str());
      Serial.println(" -- cycling sources every 3s");
    }
  }
```

Αφού καθοριστεί το τετράγωνο προορισμού εξακριβώνεται το τετράγωνο από το οποίο προήλθε το κομμάτι. Αν δεν βρεθεί κανένα τετράγωνο ακυρώνεται η κίνηση, αν βρεθεί ένα το σύστημα είναι σίγουρο πλέον για τη κίνηση. Στη περίπτωση που βρεθεί παραπάνω από ένα πιθανό τετράγωνο ακολουθείται η ίδια λογική επαλήθευσης από το παίκτη. Σε περίπτωση που υπάρχει αβεβαιότητα και στο τετράγωνο προορισμού και στο source τότε η κίνηση ακυρώνεται.

```
if (chosen.length() == 0 || chosen == liftedSquare) {
    ESP_LOGI(TAG,"Settled: no valid move, discarding");
    detectState=DETECT_IDLE;
    liftedSquare="";
    settleBestSquare="";
    settleBestCount=0;
    numCycleCandidates=0;
    cyclingSource=false;
    return;
  }
```

Αν δεν βρέθηκε προορισμός ή ο προορισμός είναι ίδιος με την αφετηρία τότε η συνάρτηση γυρνάει νωρίς για να επαναληφθεί η ανίχνευση κίνησης.

```
  pendingMove = liftedSquare + "->" + chosen;
  detectState = DETECT_IDLE;
  liftedSquare = "";
  ESP_LOGI(TAG,"Pending: %s%s", pendingMove.c_str(),
        numCycleCandidates > 1 ? (cyclingSource ? " [src-cyc]" : " [dst-cyc]") : "");
  Serial.printf(" >> Pending: %s (press colour button to confirm)\n",
           pendingMove.c_str());
  printBoardState();
  lcdUpdateRow1();
  return;
 }
```

Αυτό είναι το στάδιο έγκρισης της κίνησης από τον παίκτη όταν εκκρεμεί μία ή περισσότερες κινήσεις, η συνάρτηση επιστρέφει γιατί δεν απαιτείται η παρακάτω αποσφαλμάτωση με τη κίνηση εξακριβωμένη.

```
 // Per-square debounce
 for (int m = 0; m < NUM_MUX; m++) {
```

```
for (int ch = 0; ch < NUM_CHANNELS; ch++) {
  bool raw = results[m][ch];

  if (raw == rawPrev[m][ch]) {
    if (sameCount[m][ch] < 255) sameCount[m][ch]++;
  } else {
    sameCount[m][ch] = 1;
    rawPrev[m][ch] = raw;
  }

  if (sameCount[m][ch] < DEBOUNCE_TICKS) continue;
  if (raw == boardState[m][ch]) continue;

  boardState[m][ch] = raw;
  String sq = muxToSquareLower(m, ch);
```

Γίνεται debouncing των αισθητήρων για να διαβεβαιωθεί πως η μέτρηση είναι σταθερή πριν καταγραφεί στο πίνακα boardState[]. Για κάθε «ωμή» τιμή από τον πίνακα results[] εξετάζεται αν άλλαξε προς την προηγούμενη και αν έχει περάσει αρκετός χρόνος για να καταγραφτεί ως αληθής μέτρηση.

```
  if (!raw) {
    // Square went empty means candidate lift
    if (detectState == DETECT_IDLE) {
      int lfi = sq[0]-'a', lri = sq[1]-'1';

      if (lfi < 0 || lfi >= 8 || lri < 0 || lri >= 8) continue;
      if (pieceBoard[lfi][lri] == '.') {
        ESP_LOGI(TAG,"Ghost lift ignored at %s (empty in model)", sq.c_str());
        continue;
      }

      liftedSquare = sq;
      detectState = PIECE_IN_AIR;
      settleBestSquare = "";
      settleBestCount = 0;
      computeLegalDests(sq[0] - 'a', sq[1] - '1');
      resetBoardStateForLegalDests();
      ESP_LOGI(TAG,"Lifted from %s (%d legal dests)", sq.c_str(), numLegalDests);
      lastDisplayedMove = "";
      lcdUpdateRow1();
    }
```

Αν ένα τετράγωνο άδειασε ανανεώνεται η κατάσταση σε PIECE_IN_AIR απορρίπτοντας τη περίπτωση να διαβάστηκε λάθος μέτρηση σε κενό τετράγωνο του πίνακα pieceBoard[].

Υπολογίζονται οι νόμιμες κινήσεις για το κομμάτι που σηκώθηκε, ανανεώνονται οι αισθητήρες στα νόμιμα τετράγωνα για καινούργια μέτρηση και αναδεικνύεται η κατάσταση στην δεύτερη γραμμή της οθόνης.

```
    } else if (detectState == PIECE_IN_AIR) {
      settleStart = millis();
      detectState = PIECE_SETTLING;
      settleBestSquare = sq;
      settleBestCount = sameCount[m][ch];
      ESP_LOGI(TAG,"Settling started, candidate: %s (count=%d)",
            sq.c_str(), sameCount[m][ch]);
      lcdUpdateRow1();
```

Αν η συνάρτηση βρισκόταν ήδη σε κατάσταση PIECE_IN_AIR όταν κλήθηκε και ο αισθητήρας έγινε HIGH (καθώς βρίσκεται σε else if τρέχει μόνο αν raw == true) τότε μεταβαίνει σε κατάσταση PIECE_SETTLING για επαλήθευση της θέσης του κομματιού από το debouncing για ένα ορισμένο χρονικό διάστημα.

```
} else if (detectState == PIECE_SETTLING) {
      if (sameCount[m][ch] > settleBestCount) {
        settleBestSquare = sq;
        settleBestCount = sameCount[m][ch];
        ESP_LOGI(TAG,"Better candidate: %s (count=%d)",
              sq.c_str(), sameCount[m][ch]);
      }
    }
   }
  }
}
```

Τέλος σε κατάσταση PIECE_SETTLING διαβάζονται οι αισθητήρες και κρίνεται ο πιο έμπιστος ανάλογα με τη σταθερότητα μέτρησης του που καθορίζεται από το debouncing. Ολόκληρη η σκακιέρα συνεχίζει να σαρώνεται σε περίπτωση που κάποιος άλλος αισθητήρας που άλλαξε αποδειχθεί πιο σταθερός η κίνηση επαναπροσδιορίζεται στο πιο σταθερό αισθητήρα.

initNeuralNetwork()

```
void initNeuralNetwork() {
  Serial.println("\n=== Neural Network Init ===");

  if (!psramFound()) {
    Serial.println("ERROR: PSRAM not found — set Tools→PSRAM→OPI PSRAM");
  }
  Serial.printf("PSRAM free : %.2f MB\n", ESP.getFreePsram() / 1e6f);
  Serial.printf("Model size : %.2f MB (%u bytes)\n",
                chess_model_tflite_len / 1e6f, chess_model_tflite_len);
```

Η συνάρτηση αρχικοποιεί την απαραίτητη μνήμη για την αντιγραφή του νευρωνικού δικτύου στη PSRAM που επιτρέπει τη ταχύτερη λειτουργία του. Στις αρχικές γραμμές τυπώνεται για αποσφαλμάτωση στο serial monitor η κατάσταση της μνήμης του μικροελεγκτή χρησιμοποιώντας την συνάρτηση της ESP-IDF ESP.getFreePsram(). Το Arduino IDE έχει ήδη ορισμένο το αντικείμενο ESP καθώς κάνει #include πολλές βιβλιοθήκες της ESP-IDF αυτόματα για να τρέξει.

```
// 1. Copy model flash -> PSRAM
  Serial.print("Copying model flash->PSRAM... ");
  model_buf = (uint8_t*) ps_malloc(chess_model_tflite_len);
  if (!model_buf) {
      Serial.println("FAILED (ps_malloc)");
      return;
  }
  memcpy(model_buf, chess_model_tflite, chess_model_tflite_len);
  Serial.printf("OK (PSRAM free: %.2f MB)\n", ESP.getFreePsram() / 1e6f);
```

Ετοιμάζεται το buffer στο οποίο θα αντιγραφτεί το οποίο είναι στη PSRAM λόγω της χρήσης της συνάρτησης ps_malloc αντί για την απλή malloc, με τη memcpy γίνεται η αντιγραφή.

```
// 2. Parse model
  tfl_model = tflite::GetModel(model_buf);
  if (tfl_model->version() != TFLITE_SCHEMA_VERSION) {
    Serial.printf("Schema mismatch: model=%d expected=%d\n",
                  tfl_model->version(), TFLITE_SCHEMA_VERSION);
    return;
  }
```

Η GetModel() του namespace tflite μετατρέπει τα «ωμά» δεδομένα του πίνακα σε ένα object που είναι συμβατό με τη βιβλιοθήκη. Ελέγχεται πως το μοντέλο είναι συμβατό με την έκδοση του TensorFlow που χρησιμοποιεί η βιβλιοθήκη (schema version). Αν δεν είναι το μοντέλο δεν μπορεί να διαβαστεί από τη βιβλιοθήκη και η συνάρτηση επιστρέφει νωρίς.

```
  // 3. Op resolver (static,  must outlive interpreter)
  static tflite::MicroMutableOpResolver<7> resolver;
```

```
resolver.AddConv2D();
resolver.AddAdd();
resolver.AddRelu();
resolver.AddReshape();
resolver.AddFullyConnected();
resolver.AddPad();
resolver.AddTranspose();
```

Το resolver για να μειωθεί ο χώρος στη RAM που καταναλώνει το μοντέλο επιλέγει ποιες μαθηματικές πράξεις της βιβλιοθήκης θα συμπεριληφθούν, καθώς η βιβλιοθήκη περιέχει πολλές περισσότερες των χρησιμοποιουμένων. Για να διαπιστωθεί ποιες πράξεις χρειάζονται χρησιμοποιήθηκε το LLM Claude Sonnet. Οι πράξεις που αντιγράφονται στο πρόγραμμα είναι:

Συνάρτηση	**Πράξη**	**Επεξήγηση**
AddConv2D	Άθροισμα γινομένων	Το μοντέλο «βλέπει» τη σκακιέρα σαν μία εικόνα 8x8, κάθε «pixel» πολ/ται με το αντίστοιχο βάρος και προστίθονται τα γινόμενα μαζί.
AddAdd	Πρόσθεση πινάκων	Απαιτείται για το convolution2D
AddRelu	max(0,x)	Αν το x είναι θετικό μένει όπως είναι, αλλιώς μηδενίζεται. Αυτό επιτρέπει στο μοντέλο να απενεργοποιεί συγκεκριμένους νευρώνες.
AddReshape	Αναδιάταξη πίνακα χωρίς απώλεια δεδομένων (πχ μετατροπή από 8x8 σε 1x64)	Μετατροπή από το input tensor στα intermediate processing layers και τέλος στο tensor της εξόδου που έχουν όλα διαφορετικές διαστάσεις.
AddFullyConnected	Πολλαπλασιασμός κάθε τιμής με το βάρος της	Βλέπει καθολικά τη σκακιέρα πολ/ζοντας κάθε τετράγωνο με ένα βάρος συνδυάζοντας τις προηγούμενες πράξεις.

AddPad	Προσθέτει επιπλέον δεδομένα προσωρινά για την επεξεργασία σε περίπτωση μη ακριβής μετατροπής δεδομένων	Επιτρέπει σε όλα τα layers να λαμβάνουν τη σκακιέρα ως 8x8 ανεξαρτήτως αν έχει μετατραπεί σε μικρότερο πίνακα για επεξεργασία εισάγοντας 0 στα τετράγωνα που εισάγονται.
AddTranspose	Μετατρέπει τις γραμμές σε στήλες και τις στήλες σε γραμμές	Χρησιμοποιείται για να γυρνάει ανάποδα τη σκακιέρα έτσι ώστε το μοντέλο να μην βλέπει λευκά και μαύρα αλλά την ίδια θέση από τη μεριά του διαιρώντας την επεξεργασία στα μισά.

Πίνακας 18 - Resolver additions

```
// 4. Scratch arena in PSRAM
 nn_arena = (uint8_t*) ps_malloc(NN_ARENA_SIZE);
 if (!nn_arena) { Serial.println("ERROR: ps_malloc for arena failed"); return; }
```

Αρχικοποιείται στη PSRAM το arena για την φόρτωση των tensors που βρίσκονται υπό επεξεργασία. Εδώ αποθηκεύονται τα ενδιάμεσα αποτελέσματα, τα δεδομένα εισόδου και εξόδου.

```
// 5. Interpreter (static, must outlive setup())
 static tflite::MicroInterpreter static_interp(
    tfl_model, resolver, nn_arena, NN_ARENA_SIZE);
 interpreter = &static_interp;
```

Το interpreter αρχικοποιείται με το μοντέλο, το resolver και την arena. Μέσου αυτού γίνεται το inference, δηλαδή η χρήση του μοντέλου. Μεταφέρεται η διεύθυνση μνήμης του σε ένα global pointer για να αποφευχθεί η αντιγραφή του καθώς πρόκειται για ένα μεγάλο αντικείμενο.

```
  // 6. Allocate tensors
 Serial.print("Allocating tensors... ");
 if (interpreter->AllocateTensors() != kTfLiteOk) {
  Serial.println("FAILED -- increase NN_ARENA_SIZE"); return;
 }
 Serial.printf("OK (used %u / %u bytes)\n",
            interpreter->arena_used_bytes(), NN_ARENA_SIZE);

 nn_inp = interpreter->input(0);
 nn_out = interpreter->output(0);
 Serial.printf("Input : [%d,%d,%d,%d] %s scale=%.6f zp=%d\n",
    nn_inp->dims->data[0], nn_inp->dims->data[1],
    nn_inp->dims->data[2], nn_inp->dims->data[3],
    TfLiteTypeGetName(nn_inp->type),
    nn_inp->params.scale, nn_inp->params.zero_point);
 Serial.printf("Output: [%d,%d] %s scale=%.6f zp=%d\n",
    nn_out->dims->data[0], nn_out->dims->data[1],
    TfLiteTypeGetName(nn_out->type),
    nn_out->params.scale, nn_out->params.zero_point);
 Serial.println("=== Neural Network Ready ===\n");
}
```

Με την AllocateTensors το interpreter κρίνει αν το μέγεθος της arena είναι αρκετό, αυτό το μέγεθος βρέθηκε εμπειρικά με τη μέθοδο της διχοτόμησης και δεν έχει βελτιστοποιηθεί καθώς περισσεύει PSRAM στο σύστημα. Η αστάθεια του arena στη μνήμη σημαίνει πως πρέπει να προστεθεί ένας συντελεστής ασφαλείας στο arena_used_bytes γιατί αυτό είναι η πρόβλεψη του interpreter με τα δεδομένα που έχει ως στιγμής και όχι οι πραγματικές συνθήκες που είναι δύσκολο να υπολογιστούν. Σώζονται το input και output tensor στα παγκόσμια pointers nn_inp και nn_out. Τέλος τυπώνονται οι διαστάσεις και οι παράμετροι κβαντοποίησης του μοντέλου για την επαλήθευση σωστής αρχικοποίησης.

encodeBoardForModel()

```
void encodeBoardForModel(int8_t* buf) {
 memset(buf, (int8_t)(-128), 8 * 8 * 18);

 bool flip = (currentTurn == TURN_BLACK);
```

Αυτή η συνάρτηση μετατρέπει το πίνακα pieceBoard[] στη μορφή [1,8,8,18] tensor που περιμένει ως είσοδο το νευρωνικό δίκτυο. Αρχικά δεσμεύεται η μνήμη και γράφεται η τιμή -128 σε όλες τις θέσεις για να αρχικοποιηθούν. Το flip είναι σηματνικό για να βλέπει πάντα το νευρωνικό δίκτυο τη θέση από τη μεριά του, όπως κατά την εκπαίδευση, που μειώνει την απαιτούμενη εκπαίδευση στα μισά, γιατί δεν χρειάζεται να επαναληφθεί ανά χρώμα.

```
for (int col = 0; col < 8; col++) {
  for (int row = 0; row < 8; row++) {
    char p = pieceBoard[col][row];
    if (p == '.') continue;
    auto charToPlane = [](char c) -> int {
      switch (c) {
        case 'P': return 0;
        case 'N': return 1;
        case 'B': return 2;
        case 'R': return 3;
        case 'Q': return 4;
        case 'K': return 5;
        case 'p': return 6;
        case 'n': return 7;
        case 'b': return 8;
        case 'r': return 9;
        case 'q': return 10;
        case 'k': return 11;
        default: return -1;
      }
    };
    int plane = charToPlane(p);
    if (plane < 0) continue;
    if (flip) plane = (plane + 6) % 12;
    int drow = flip ? (7 - row) : row;
    buf[drow * 8 * 18 + col * 18 + plane] = 127;
  }
 }
```

Η διπλή επανάληψη περνάει κάθε τετράγωνο της σκακιέρας και το lambda charToPlane μετατρέπει το κομμάτι στο ανάλογο index που καταλαβαίνει το νευρωνικό δίκτυο. Τα κενά τετράγωνα προσπερνιόνται. Στο input tensor από τα 18 επίπεδα τα επίπεδα 0-6 είναι τα λευκά και τα 6-11 τα μαύρα. Η γραμμή (plane+6) % 12 μεταφράζει τα επίπεδα μεταξύ των χρωμάτων σε περίπτωση που ο υπολογιστής παίζει τα μαύρα. Η μεταβλητή drow (destination

row) αλλάζει γεωμετρικά την ίδια τη σκακιέρα για να τη δει από τη μεριά του το μοντέλο. Στη τελευταία γραμμή γράφεται στη σωστή θέση του input tensor το κομμάτι, το drow*8*18 δείχνει τη γραμμή, καθώς η γραμμή αποτελείται από 8 τετράγωνα και 18 επίπεδα στο καθένα. Το col*18 δείχνει τη στήλη, πολ/ται με το 18 για τα επίπεδα του tensor. Προστίθεται το plane για να δείχνει στο σωστό επίπεδο ώστε όταν συνολικά ενεργοποιηθούν με το 127 το μοντέλο να γνωρίζει ποιο κομμάτι βρίσκεται πού.

```
for (int r = 0; r < 8; r++) {
  for (int c = 0; c < 8; c++) {
    buf[r * 8 * 18 + c * 18 + 12] = 127;
  }
}

bool ourKS, ourQS, theirKS, theirQS;
if (!flip) {
  ourKS = castleWK;
  ourQS = castleWQ;
  theirKS = castleBK;
  theirQS = castleBQ;
} else {
  ourKS = castleBK;
  ourQS = castleBQ;
  theirKS = castleWK;
  theirQS = castleWQ;
}
```

Η διπλή επανάληψη γεμίζει το 12^{ο} επίπεδο του tensor με 127 το οποίο χρησιμοποιείται για να αναγνωρίζει το μοντέλο τη διαφορά τιμών μεταξύ κενών τετραγώνων και του χώρου που προστίθεται με το padding όταν το convolution kernel φιλτράρει τη περίμετρο της σκακιέρας. Ύστερα μεταφέρονται τα ροκέ των χρωμάτων ως «δικά μας» και «δικά τους» επειδή το μοντέλο δεν γνωρίζει τα χρώματα μόνο το ποιος παίζει.

```
auto fillPlane = [&](int plane) {
  for (int r = 0; r < 8; r++) {
    for (int c = 0; c < 8; c++) {
      buf[r * 8 * 18 + c * 18 + plane] = 127;
    }
  }
};
if (ourKS) fillPlane(13);
if (ourQS) fillPlane(14);
if (theirKS) fillPlane(15);
if (theirQS) fillPlane(16);

if (epFile >= 0 && epRank >= 0) {
  int drow = flip ? (7 - epRank) : epRank;
```

```
    buf[drow * 8 * 18 + epFile * 18 + 17] = 127;
  }
}
```

Το lambda fillPlane χρησιμοποιείται για να γεμίσει ένα επίπεδο με 127 για να μεταφερθούν σωστά τα δικαιώματα των ροκέ στα συγκεκριμένα επίπεδα. Τέλος στο τελευταίο επίπεδο 17 (επειδή η αρίθμηση ξεκινάει από το 0) γράφεται το δικαίωμα για en passant στη συγκεκριμένη γραμμή και στήλη.

doComputerMove()

```
void doComputerMove() {
 Serial.println("\n=== COMPUTER THINKING ===");

 computerThinking = true;
 lcdUpdateRow1();

 encodeBoardForModel(nn_inp->data.int8);

 uint32_t t0 = millis();
 if (interpreter->Invoke() != kTfLiteOk) {
  Serial.println("ERROR: inference failed!");
  computerThinking = false;
  lcdUpdateRow1();
  return;
 }
 Serial.printf("Inference: %lu ms\n", millis() - t0);
```

Η συνάρτηση υπολογίζει την απάντηση του νευρωνικού δικτύου σε μία θέση κάνοντας inference και εκτελεί την φυσική κίνηση που αναλογεί αυτής με τους κινητήρες.

Μετατρέπεται η θέση στη μορφή του input tensor και χρονομετρείται το inference για αποσφαλμάτωση, το οποίο καταλαμβάνει περίπου 500ms στο τελικό σύστημα, μια εξαιρετική ταχύτητα επεξεργασίας πολύ μικρότερη από τη διάρκεια που χρειάζεται το σύστημα κίνησης για να την εκτελέσει. Σε περίπτωση σφάλματος τυπώνεται στο serial monitor για αποσφαλμάτωση.

```
 int8_t* raw = nn_out->data.int8;
 bool flip = (currentTurn == TURN_BLACK);

 static int8_t scores_copy[NN_N_MOVES];
 memcpy(scores_copy, raw, NN_N_MOVES);

 char bestF1 = 0;
 char bestR1 = 0;
 char bestF2 = 0;
 char bestR2 = 0;
 bool found = false;
```

Γράφονται τα αποτελέσματα από το output tensor στο pointer raw και αντιγράφονται στο πίνακα scores_copy, ο οποίος είναι statically allocated για να μην αρχικοποιείται κάθε φορά που τρέχει η συνάρτηση. Αρχικοποιούνται οι μεταβλητές που θα αποθηκεύσουν τη κίνηση.

```
 for (int attempt = 0; attempt < NN_N_MOVES && !found; attempt++) {
  int best = 0;
  for (int i = 1; i < NN_N_MOVES; i++) {
   if (scores_copy[i] > scores_copy[best]) best = i;
  }
```

```
  if (scores_copy[best] == (int8_t)(-128)) break;

  int from_sq = best / 64;
  int to_sq = best % 64;
```

Η επανάληψη περνάει όλον το πίνακα scores_copy[] μέχρι να βρεθεί το index με τη υψηλότερη τιμή, η τρέχουσα μεγαλύτερη αποθηκεύεται στη μεταβλητή best.

Αν δεν βρέθηκε καλύτερη κίνηση η επανάληψη γυρίζει νωρίς. Επειδή το output tensor είναι όλες οι πιθανές κινήσεις (Silver, et al., 2017) σε format τετράγωνο προέλευσης προς τετράγωνο προορισμού (πχ e2e4) (Universal Chess Interface, 2026), είναι ένας μονοδιάστατος πίνακας με 4096 θέσεις οι οποίες αποδικοποιούνται διαιρώντας με το 64 για το τετράγωνο προέλευσης και το υπόλοιπο είναι το τετράγωνο προορισμού από το συνάρτηση που ακολουθεί το μοντέλο στη κωδικοποίηση κινήσεων:

$$unique\ index = (from\ square * 64) + to\ square$$

```
  if (flip) {
    from_sq = (7 - from_sq / 8) * 8 + (from_sq % 8);
    to_sq = (7 - to_sq / 8) * 8 + (to_sq % 8);
  }
```

Αν η σκακιέρα είναι αναστραμμένη η σκακιέρα αναστρέφεται ξανά μετά την encodeBoardForModel() για να γυρίσει στην πραγματική κατάσταση της.

```
  if (from_sq == 4 && to_sq == 7) to_sq = 6;
  else if (from_sq == 4 && to_sq == 0) to_sq = 2;
  else if (from_sq == 60 && to_sq == 63) to_sq = 62;
  else if (from_sq == 60 && to_sq == 56) to_sq = 58;
```

Διορθώνεται ο τρόπος που το μοντέλο αναπαριστά τα ροκέ σε διπλή κίνηση του βασιλιά αντί για το τετράγωνο του βασιλιά και του πύργου.

```
  char f1 = 'a' + (char)(from_sq % 8);
  char r1 = '1' + (char)(from_sq / 8);
  char f2 = 'a' + (char)(to_sq % 8);
  char r2 = '1' + (char)(to_sq / 8);

  if (f1 < 'a' || f1 > 'h' || r1 < '1' || r1 > '8' ||
      f2 < 'a' || f2 > 'h' || r2 < '1' || r2 > '8') {
    scores_copy[best] = (int8_t)(-128);
    continue;
  }
```

Μετατρέπονται τα indexes σε χαρακτήρες ASCII για καλύτερη ανάγνωση και συμβατότητα με το υπόλοιπο πρόγραμμα. Αν είναι εκτός των ορίων γράφεται η τιμή -128 στον πίνακα.

```
char piece = pieceBoard[f1 - 'a'][r1 - '1'];
  bool rightColor = (currentTurn == TURN_WHITE)
                    ? ImgIsWhite(piece)
                    : ImgIsBlack(piece);
  if (piece == '.' || !rightColor) {
    scores_copy[best] = (int8_t)(-128);
```

```
    continue;
  }
```

Αυτός ο έλεγχος επικυρώνει πως το κομμάτι που προσπαθεί να κινηθεί ανήκει στο χρώμα που παίζει (και το τετράγωνο δεν είναι άδειο), καθώς το νευρωνικό δίκτυο δίνει μία λίστα με όλες τις κινήσεις ανεξαρτήτως νομιμότητας, αυτή η λίστα πρέπει να φιλτραριστεί.

```
  computeLegalDests(f1 - 'a', r1 - '1');
  String dest = String(f2) + String(r2);
  if (nearestLegal(dest).length() > 0) {
    bestF1 = f1;
    bestR1 = r1;
    bestF2 = f2;
    bestR2 = r2;
    found = true;
  } else {
    scores_copy[best] = (int8_t)(-128);
  }
 }

 computerThinking = false;
```

Συνεχίζει το φιλτράρισμα καλώντας την συνάρτηση για όλες τις νόμιμες κινήσεις που γράφονται στον ανάλογο πίνακα και ελέγχονται αν συμπίπτουν με το τετράγωνο προορισμού.

```
 if (!found) {
   startNewGame
(currentTurn == TURN_WHITE);
   return;
 }

 Serial.printf("Computer plays: %c%c->%c%c\n", bestF1, bestR1, bestF2, bestR2);

 lastDisplayedMove = String(bestF1) + String(bestR1) + "->" +
                     String(bestF2) + String(bestR2);
 lcdUpdateRow1();
 delay(600);
```

Αν δεν βρέθηκε καμία κίνηση μετά από όλη τη λίστα κινήσεων η παρτίδα τελείωσε και καλείται η showGameOver() για να καθοριστεί ισοπαλία ή ήττα. Αν βρεθεί κίνηση, τυπώνεται στην οθόνη η κίνηση του υπολογιστή για επικοινωνία με τον παίκτη.

```
 char movingPiece = pieceBoard[bestF1 - 'a'][bestR1 - '1'];

 executeMove(bestF1, bestR1, bestF2, bestR2);

 if (!isStopped) {
```

```
  if ((movingPiece == 'P' && bestR2 == '8') ||
      (movingPiece == 'p' && bestR2 == '1')) {
    char queen = lmgIsWhite(movingPiece) ? 'Q' : 'q';
    pieceBoard[bestF2 - 'a'][bestR2 - '1'] = queen;
    Serial.printf(" [auto-promoted to %c]\n", queen);
    printBoardState();
  }
 }
}
```

Αποθηκεύεται το κομμάτι που κινείται για να διαπιστωθεί αν πρόκειται για προαγωγή, η οποία περιοριστικά είναι πάντα σε βασίλισσα. Γίνεται η φυσική εκτέλεση της κίνησης με την executeMove() η οποία μετακινεί τον ηλεκτρομαγνήτη στα σωστά τετράγωνα, και τυπώνεται στο serial monitor η θέση για επαλήθευση σωστής καταχώρησης.

squareToMotorPos()

```
bool squareToMotorPos(char file, char rank, int32_t &m1, int32_t &m2) {
 if (file < 'a' || file > 'h' || rank < '1' || rank > '8') return false;
 int32_t tx = (int32_t)(rank - '1') * STEPS_PER_SQUARE;
 int32_t ty = (int32_t)(7 - (file - 'a')) * STEPS_PER_SQUARE;
 m1 = ty - tx;
 m2 = -(tx + ty);
 return true;
}
```

Η συνάρτηση μετατρέπει τα τετράγωνα της σκακιέρας σε βήματα για τους δύο κινητήρες χρησιμοποιώντας της κινηματικές εξισώσεις ενός συστήματος CoreXY (CoreXY, 2020) και τα υπολογισμένα βήματα ανά τετράγωνο.

checkEStop()

```
bool checkEStop() {
  static uint8_t eStopCount = 0;

  bool bW = (digitalRead(WHITE_BTN_PIN) == LOW);
  bool bB = (digitalRead(BLACK_BTN_PIN) == LOW);

  if (bW && bB) {
    if (++eStopCount >= ESTOP_DEBOUNCE_COUNT) {
      eStopCount = 0;
      isStopped = !isStopped;
      delay(400);
      if (isStopped) {
        rgbLedWrite(38, 255, 0, 0);
        Serial.println("\n!!! EMERGENCY STOP — press both buttons again to resume !!!");
        stepper1->forceStop();
        stepper2->forceStop();
      } else {
        rgbLedWrite(38, 0, 0, 255);
        Serial.println("\n>>> SYSTEM READY");
      }
    }
  } else {
    eStopCount = 0;
  }

  if (isStopped) {
    stepper1->forceStop();
    stepper2->forceStop();
    return true;
  }
  return false;
}
```

Η συνάρτηση ελέγχει αν πατήθηκαν ταυτόχρονα και τα δύο κουμπιά των χρωμάτων, το οποίο λειτουργεί ως «συναγερμός» έκτακτης ανάγκης και σταματά την κίνηση των κινητήρων ακαριαία. Χρησιμοποιείται αν κατά τη μεταφορά αποσυνδεθεί κάποιος σημαντικός διακόπτης και το σύστημα κίνησης πέσει πάνω στη δοκό για να ελαχιστοποιηθεί η καταπόνηση του συστήματος και να μην χρειάζεται να διακοπεί η παροχή ρεύματος. Γίνεται ένα debouncing για να μπορεί να επαναφερθεί το σύστημα στην κανονική λειτουργία πατώντας ξανά τα δύο κουμπιά χωρίς να εναλλάσει συνεχώς μεταξύ των καταστάσεων.

moveToSquare()

```
void moveToSquare(char file, char rank) {
 if (file < 'a' || file > 'h' || rank < '1' || rank > '8') {
  Serial.printf("Invalid square: %c%c\n", file, rank);
  return;
 }
 int32_t logX, logY;
 getLogicalPos(logX, logY);
 float x0 = (float)abs(logX);
 float y0 = (float)logY;
 float x1 = (float)(rank - '1') * STEPS_PER_SQUARE;
 float y1 = (float)(7 - (file - 'a')) * STEPS_PER_SQUARE;
 float dx = x1 - x0;
 float dy = y1 - y0;
```

Εκτελεί τη φυσική κίνηση του ηλεκτρομαγνήτη στο κέντρο ενός καθορισμένο τετραγώνου. Αρχικά αρχικοποιούνται όλες οι μεταβλητές για τη λογική θέση στις καρτεσιανές συντεταγμένες, κατά τις οποίες το σύστημα αρχίζει από το σημείο (0,0) μετά το calibration.

```
 if (fabsf(dx) < 0.5f && fabsf(dy) < 0.5f) {
  ESP_LOGI(TAG, "Already at %c%c", (int)file, (int)rank);
  return;
 }

 int32_t m1T = (int32_t)roundf(y1) - (int32_t)roundf(x1);
 int32_t m2T = -((int32_t)roundf(x1) + (int32_t)roundf(y1));
 int32_t d1 = abs(m1T - stepper1->getCurrentPosition());
 int32_t d2 = abs(m2T - stepper2->getCurrentPosition());
 int32_t dom = max(d1, d2);
 if (dom < 10) {
  ESP_LOGI(TAG, "Already at %c%c", (int)file, (int)rank);
  return;
 }
```

Υπολογίζονται οι στόχοι για το κάθε μοτέρ (σε ακέραια βήματα στρογγυλοποιώντας τα floats) με βάση τη τρέχουσα θέση τους. Η μεταβλητή dom αποθηκεύει τη μεγαλύτερη απόσταση για το μοτέρ που πρέπει να κάνει τα περισσότερα βήματα, για τον υπολογισμό των ταχυτήτων και επιταχύνσεων τους αργότερα στη συνάρτηση. Για διαφορές 10 βημάτων που θα ήταν φαινομενικά στο ίδιο σημείο η κίνηση απορρίπτεται.

```
 float r1 = (float)d1 / (float)dom;
 float r2 = (float)d2 / (float)dom;
 uint32_t distCap = (uint32_t)sqrtf((float)MOVE_ACCEL * (float)dom);
 uint32_t safeSpd = min(MOVE_SPEED, distCap);
```

Υπολογίζεται ο συγχρονισμός των κινητήρων, για να κινηθεί ο ηλεκτρομαγνήτης σε μία ομαλή ευθεία ανάλογα με το προορισμό το κάθε μοτέρ χρειάζεται να κάνει διαφορετικά βήματα, αλλά πρέπει να αρχίσουν και να σταματήσουν ακριβώς την ίδια χρονική στιγμή. Για αυτό

υπολογίζονται οι ταχύτητες αναλογικά, οι αναλογίες r1 και r2 δείχνουν πόσο γρήγορα πρέπει να περιστρέφεται ο κινητήρας συγκριτικά με την μεγαλύτερη απόσταση που πρέπει να διανυθεί. Για παράδειγμα αν το ένα μοτέρ πρέπει να κάνει 10,000 βήματα και το δεύτερο μοτέρ 5,000 τότε η αναλογία δείχνει πως το δεύτερο πρέπει να περιστρέφεται με τη μισή ταχύτητα για να ολοκληρωθούν ταυτόχρονα οι κινήσεις τους.

Η distCap υπολογίζει την απόσταση που χρειάζεται το μοτέρ για να επιβραδύνει πλήρως και η safeSpd περιορίζει τη ταχύτητα στη καθορισμένη μέγιστη σε Hz.

```
auto inSlowZone = [&](float wx, float wy) -> bool {
  if (wx < SLOWDOWN_ZONE && dx < 0) return true;
  if (wx > (MAX_X_STEPS - SLOWDOWN_ZONE) && dx > 0) return true;
  if (wy < SLOWDOWN_ZONE && dy < 0) return true;
  if (wy > (MAX_Y_STEPS - SLOWDOWN_ZONE) && dy > 0) return true;
  return false;
 };
```

Αυτό το lambda καθορίζει αν ο ηλεκτρομαγνήτης κινείται προς τη περίμετρο της κατασκευής συγκρίνοντας τις προκαθορισμένες τιμές που ορίζουν το SLOWDOWN_ZONE με την κατεύθυνση κίνησης. Χωρίς να συμπεριλαμβάνεται η κατεύθυνση κίνησης το σύστημα θα κινούταν αργά ακόμη και όταν φεύγει από τη περίμετρο. Αυτό ήταν απαραίτητο καθώς οι φυσικοί διακόπτες για τα σημεία μηδέν των αξόνων είναι πολύ ευαίσθητοι σε κρούσεις και δονήσεις, κατά τη κατασκευή χάλασαν αρκετοί διακόπτες πριν διαπιστωθεί η σημαντικότητα αργής κίνησης στη περίμετρο. Σε μελλοντικοί κατασκευή μπορούν να χρησιμοποιηθούν διαφορετικοί αισθητήρες όπως hall effect sensors για τα σημεία μηδέν, έτσι ώστε να μπορεί η κίνηση να γίνεται με ταχύτητα και στη περίμετρο.

```
 auto applySpd = [&](uint32_t spd, uint32_t acc) {
  uint32_t s1 = (d1 > 0) ? max((uint32_t)(spd * r1), (uint32_t)200) : 200;
  uint32_t s2 = (d2 > 0) ? max((uint32_t)(spd * r2), (uint32_t)200) : 200;
  uint32_t a1 = (d1 > 0) ? max((uint32_t)(acc * r1), (uint32_t)1000) : 1000;
  uint32_t a2 = (d2 > 0) ? max((uint32_t)(acc * r2), (uint32_t)1000) : 1000;
  stepper1->setAcceleration(a1);
  stepper2->setAcceleration(a2);
  stepper1->setSpeedInHz(s1);
  stepper2->setSpeedInHz(s2);
  stepper1->applySpeedAcceleration();
  stepper2->applySpeedAcceleration();
 };
```

Αυτό το lambda εφαρμόζει την ταχύτητα και την επιτάχυνση των κινητήρων με τα κατώτατα όρια των 200Hz για ταχύτητα και 1000 Hz/s επιτάχυνση λαμβάνοντας υπόψη τις προηγουμένως υπολογισμένες αναλογίες (FastAccelStepper, 2026).

```
bool curSlow = inSlowZone(x0, y0);
 applySpd(curSlow ? CALIBRATION_SPEED : safeSpd,
       curSlow ? ACCELERATION : MOVE_ACCEL);
 stepper1->moveTo(m1T);
 stepper2->moveTo(m2T);
 ESP_LOGI(TAG, ">> %c%c safeSpd=%lu slow=%d",
```

```
        (int)file, (int)rank, (unsigned long)safeSpd, (int)curSlow);

 while (stepper1->isRunning() || stepper2->isRunning()) {
  if (checkEStop()) {
   stepper1->forceStop();
   stepper2->forceStop();
   stepper1->setAcceleration(ACCELERATION);
   stepper2->setAcceleration(ACCELERATION);
   return;
  }
```

Η κίνηση αρχίζει με την συνάρτηση moveTo() των κινητήρων, αυτή η συνάρτηση είναι ασύγχρονη και δεν επιβαρύνει τον επεξεργαστεί γιατί οι παλμοί δημιουργούνται στο MCPWM του N8R8 που είναι περιφερειακό hardware που έχει το dev board. Ωστόσο για να μην γυρίσει η συνάρτηση νωρίς και το πρόγραμμα προχωρήσει στη κίνηση του παίκτη προτού ολοκληρωθεί η κίνηση του ηλεκτρομαγνήτη, η κίνηση συνεχώς ελέγχεται μέσα στη while για τυχών επείγουσα διακοπής της. Αυτό είναι προαιρετικό αλλά προσδίδει ασφάλεια και σαφή σειρά κινήσεων.

```
 getLogicalPos(logX, logY);
  bool nowSlow = inSlowZone((float)abs(logX), (float)logY);
  if (nowSlow != curSlow) {
   curSlow = nowSlow;
   applySpd(curSlow ? CALIBRATION_SPEED : safeSpd,
         curSlow ? ACCELERATION : MOVE_ACCEL);
  }
  delay(5);
 }
 stepper1->setAcceleration(ACCELERATION);
 stepper2->setAcceleration(ACCELERATION);
 ESP_LOGI(TAG, "Arrived at %c%c", (int)file, (int)rank);
}
```

Ελέγχεται αν κατά τη διάρκεια της κίνησης εισήλθε στην περίμετρο για να επιβραδύνει, το delay 5ms αποτρέπει τον επεξεργαστή να τρέχει πολύ γρήγορα loops καθώς δεν ευθύνεται ο ίδιος για την δημιουργία των παλμών. Η επιτάχυνση των μοτέρ επαναφέρεται στην αρχική ρύθμιση με την ολοκλήρωση της κίνησης και τυπώνεται μήνυμα στο serial monitor αν είναι το CoreDebugLevel από «info» και πάνω.

squareToStepCoords()

```
static void squareToStepCoords(char file, char rank, float &tx, float &ty) {
  tx = (float)(rank - '1') * STEPS_PER_SQUARE;
  ty = (float)(7 - (file - 'a')) * STEPS_PER_SQUARE;
}
```

Μετατρέπει ένα τετράγωνο σε συντεταγμένες βημάτων για να βρεθεί η τρέχουσα θέση, δεν υπολογίζει τα βήματα που απαιτούνται για να φτάσει εκεί το σύστημα κίνησης από τη παρούσα θέση του, αντίθετα με την squareToMotorPos(). Είναι πιο ελαφριά και χρησιμοποιείται στις επαναλήψεις που ελέγχουν διαρκώς τη θέση αφού έχει γίνει ο υπολογισμός για τον προορισμό.

distToSegSq()

```
static float distToSegSq(float px, float py,
                         float ax, float ay,
                         float bx, float by) {
  float dx = bx - ax;
  float dy = by - ay;
  float len2 = dx * dx + dy * dy;
  if (len2 < 1.f) {
    float ex = px - ax;
    float ey = py - ay;
    return ex * ex + ey * ey;
  }
```

Η συνάρτηση υπολογίζει το τετράγωνο της ελάχιστης απόστασης μεταξύ ενός σημείου (px,py) και ενός ευθύγραμμου τμήματος (ax,ay)->(bx,by), δηλαδή τη θέση της ορθής προβολής διανύσματος του σημείου στο ευθύγραμμο τμήμα. Η συνάρτηση γράφτηκε από το LLM Claude Sonnet. Χρησιμοποιείται για να καθοριστεί αν ο ηλεκτρομαγνήτης θα κινηθεί πολύ κοντά σε υπάρχοντα κομμάτια και τα παρασύρει. Οι αρχικές γραμμές υπολογίζουν τις συνιστώσες του διανύσματος $\overrightarrow{AB}$ το οποίο αντιπροσωπεύει τη διαδρομή που θα ακολουθηθεί. Τα dx, dy εκφράζουν πόσο μετακινείται το σύστημα στον άξονα X και Y. Αποθηκεύεται το τετράγωνο του μήκους και απορρίπτονται αποστάσεις όπου το σημείο αφετηρίας και προορισμού βρίσκονται πολύ κοντά.

```
  float t = ((px - ax) * dx + (py - ay) * dy) / len2;
  if (t < 0.f) t = 0.f;
  else if (t > 1.f) t = 1.f;
```

Το t είναι το εσωτερικό γινόμενο των διανυσμάτων $\overrightarrow{AP}$ και $\overrightarrow{AB}$. Η διαίρεση με το len2 βρίσκει τη θέση του σημείου πάνω στη διαδρομή ως ποσοστό μεταξύ 0 και 1. Σε περίπτωση που το σημείο βρίσκεται πριν ή μετά της διαδρομής μετατρέπεται στην κοντινότερη τιμή.

```
  float cx = ax + t * dx - px;
  float cy = ay + t * dy - py;
  return cx * cx + cy * cy;
}
```

Το σημείο (ax+t*dx , ay+t*dy) αντιπροσωπεύει το σημείο που βρίσκεται πιο κοντά στο (px,py) το οποίο είναι το «εμπόδιο» που εξετάζεται. Επιστρέφει την απόσταση από το εμπόδιο σε μονάδες μήκους στο τετράγωνο, στην περίπτωση που χρησιμοποιείται βήματα στο τετράγωνο.

squareInPath()

```
static bool squareInPath(char pf, char pr, char f1, char r1, char f2, char r2) {
  if (pf == f1 && pr == r1) return false;
  if (pf == f2 && pr == r2) return false;
  float ax, ay, bx, by, px, py;
  squareToStepCoords(f1, r1, ax, ay);
  squareToStepCoords(f2, r2, bx, by);
  squareToStepCoords(pf, pr, px, py);
  float thr = STEPS_PER_SQUARE * PATH_THRESHOLD_FRAC;
  return distToSegSq(px, py, ax, ay, bx, by) < thr * thr;
}
```

Η συνάρτηση γυρίζει αν ένα τετράγωνο ενδιάμεσα στη κίνηση του ηλεκτρομαγνήτη είναι κάτω από ένα επιτρεπτό όριο απόστασης, με άλλα λόγια αν ο μαγνήτης θα περάσει από αυτό το τετράγωνο. Αρχικά απορρίπτονται η αφετηρία και ο προορισμός της κίνησης, μετατρέπονται σε συντεταγμένες (σε βήματα) και προσδιορίζεται ένα threshold κάτω του οποίου το τετράγωνο θεωρείται πως είναι στη διαδρομή. Χρησιμοποιείται η προηγούμενη συνάρτηση distToSegSq() για τον υπολογισμό της απόστασης στο τετράγωνο και συγκρίνεται με το threshold στο τετράγωνο για τη τελική έξοδο της συνάρτησης.

countNearSegment()

```
static int countNearSegment(char f1, char r1, char f2, char r2) {
  int n = 0;
  for (int fi = 0; fi < 8; fi++) {
    for (int ri = 0; ri < 8; ri++) {
      if (pieceBoard[fi][ri] == '.') continue;
      if (squareInPath((char)('a' + fi), (char)('1' + ri), f1, r1, f2, r2)) n++;
    }
  }
  return n;
}
```

Η συνάρτηση μετράει τον αριθμό εμποδίων στη διαδρομή που πρέπει να εκτελέσει ο ηλεκτρομαγνήτης κατά τη κίνηση ενός κομματιού. Η διπλή επανάληψη περνάει κάθε τετράγωνο της σκακιέρας αγνοώντας τα άδεια τετράγωνα και καλείται η squareInPath() που καθορίζει αν το κομμάτι σε ένα τετράγωνο βρίσκεται πολύ κοντά. Ο μετρητής n αυξάνεται κατά 1 για κάθε εμπόδιο και χρησιμοποιείται για την εύρεση της διαδρομής με τα λιγότερα εμπόδια.

computeMagnetRoute()

```
static int computeMagnetRoute(char f1, char r1, char f2, char r2,
                              char wpF[], char wpR[]) {
 if (f1 == f2 || r1 == r2) {
  wpF[0] = f2;
  wpR[0] = r2;
  return 1;
 }
```

Αυτή η συνάρτηση υπολογίζει τη διαδρομή που πρέπει να ακολουθήσει ο ηλεκτρομαγνήτης ώστε να μην παρασύρει τα υπόλοιπα κομμάτια κατά την εκτέλεση της κίνησης. Αρχικά ελέγχεται αν η κίνηση είναι στην ίδια γραμμή ή στήλη, και αν είναι, η διαδρομή είναι ευθεία και στέλνεται απευθείας εκεί.

```
 if (countNearSegment(f1, r1, f2, r2) == 0) {
  wpF[0] = f2;
  wpR[0] = r2;
  return 1;
 }
```

Αν δεν υπάρχουν κοντινά κομμάτια σε μία διαγώνια κίνηση το σύστημα κινεί πάλι ευθεία τον ηλεκτρομαγνήτη μεταξύ των δύο σημείων.

```
 char afF = f2;
 char afR = r1;
 char bfF = f1;
 char bfR = r2;
```

Αν δεν είναι τίποτα από τα παραπάνω η διαδρομή που πρέπει να ακολουθηθεί είναι πιο περίπλοκη, σε σχήμα L όπως κινείται ο ίππος. Αποθηκεύονται οι γωνίες (waypoints) δύο πιθανών διαδρομών για να αποφασιστεί η καλύτερη μεταξύ τους.

```
 bool aFree = (pieceBoard[afF - 'a'][afR - '1'] == '.');
 bool bFree = (pieceBoard[bfF - 'a'][bfR - '1'] == '.');
```

Αποθηκεύεται για τις γωνίες των δύο πιθανών διαδρομών αν είναι άδειες ή εμποδίζονται από κομμάτι.

```
int sA = countNearSegment(f1, r1, afF, afR) +
         countNearSegment(afF, afR, f2, r2) +
         (aFree ? 0 : 1);
 int sB = countNearSegment(f1, r1, bfF, bfR) +
         countNearSegment(bfF, bfR, f2, r2) +
         (bFree ? 0 : 1);
 bool useA;
```

Εδώ μετριούνται τα «scores» των διαδρομών για να γίνει η σύγκριση, στο μέτρο προστίθεται ο αριθμός εμποδίων στο πρώτο και δεύτερο τμήμα της διαδρομής, και αν η γωνία είναι ελεύθερη. Τέλος προετοιμάζεται η μεταβλητής που κρατάει την προτιμότερη διαδρομή.

```
 if (sA < sB) useA = true;
 else if (sB < sA) useA = false;
 else if (aFree && !bFree) useA = true;
```

```
  else if (bFree && !aFree) useA = false;
  else useA = true;
  wpF[0] = useA ? afF : bfF;
  wpR[0] = useA ? afR : bfR;
  wpF[1] = f2;
  wpR[1] = r2;
  return 2;
}
```

Τα σκορ συγκρίνονται και γράφεται στη useA η διαδρομή με τα λιγότερα εμπόδια. Αν τα εμπόδια είναι ίσα καθορίζεται με τη κενή γωνία. Αποθηκεύονται στους πίνακες η ενδιάμεση στήλη και γραμμή από την οποία πρέπει να περάσει πριν τον προορισμό.

findPathBlockersRoute()

```
static int findPathBlockersRoute(char f1, char r1, char f2, char r2,
                                 char bF[], char bR[]) {
  char wpF[3], wpR[3];
  int nWP = computeMagnetRoute(f1, r1, f2, r2, wpF, wpR);
  int n = 0;
  char pF = f1;
  char pR = r1;
```

Η συνάρτηση ευθύνεται για τον εντοπισμό εμποδίων τα οποία θα πρέπει να μετακινηθούν προσωρινά για να ολοκληρωθεί η κίνηση. Αρχικοποιούνται οι πίνακες και υπολογίζονται τα waypoints που πρέπει να ακολουθηθούν κατά τη κίνηση. Αποθηκεύεται η αρχική στήλη και γραμμή.

```
for (int seg = 0; seg < nWP && n < MAX_BLOCKERS; seg++) {
    char nxF = wpF[seg];
    char nxR = wpR[seg];
    for (int fi = 0; fi < 8 && n < MAX_BLOCKERS; fi++) {
      for (int ri = 0; ri < 8 && n < MAX_BLOCKERS; ri++) {
        if (pieceBoard[fi][ri] == '.') continue;
        char f = (char)('a' + fi);
        char r = (char)('1' + ri);
        bool dup = false;
        for (int k = 0; k < n; k++) {
          if (bF[k] == f && bR[k] == r) {
            dup = true;
            break;
          }
        }
        if (!dup && squareInPath(f, r, pF, pR, nxF, nxR)) {
          bF[n] = f;
          bR[n] = r;
          n++;
        }
      }
    }
```

Η επανάληψη περνάει όλα τα μέρη της διαδρομής (1 για ευθεία, 2 για γωνία L). Ύστερα ο διπλός βρόγχος διαβάζει όλα τα τετράγωνα της σκακιέρας αγνοώντας τα κενά και τα διπλά. Γράφεται στους πίνακες bF[] και bR[] τα κομμάτια που εμποδίζουν που κρίνει η squareInPath() αν βρίσκονται εντός της υπολογισμένης διαδρομής.

```
if (seg < nWP - 1 && pieceBoard[nxF - 'a'][nxR - '1'] != '.') {
      bool dup = false;
      for (int k = 0; k < n; k++) {
        if (bF[k] == nxF && bR[k] == nxR) {
```

```
      dup = true;
      break;
     }
    }
    if (!dup && n < MAX_BLOCKERS) {
     bF[n] = nxF;
     bR[n] = nxR;
     n++;
    }
   }
   pF = nxF;
   pR = nxR;
 }
 return n;
}
```

Σε αυτό το τμήμα τση συνάρτησης με τη συνθήκη seg < nWP – 1, όπου μόνο για κινήσεις σχήματος L το nWP είναι δύο, εξετάζεται η γωνία αν δεν είναι κενή. Στο πρώτο τμήμα της διαδρομής απορρίπτονται τα διπλά αλλιώς καταγράφονται τα εμπόδια στους πίνακες και ενημερώνεται η θέση με τα pF, pR για να περάσει η επανάληψη από όλα τα τετράγωνα της διαδρομής.

findParkingSquare()

```
static bool findParkingSquare(char origF, char origR,
                              char pathF1, char pathR1,
                              char pathF2, char pathR2,
                              char resF[], char resR[], int nRes,
                              char &outF, char &outR) {
int ofi = origF - 'a';
int ori = origR - '1';
```

Αυτή η συνάρτηση για μία διαδρομή με εμπόδια εντοπίζει κενά τετράγωνα στα οποία τα κομμάτια μπορούν να μετακινηθούν προσωρινά. Αρχικά αποθηκεύονται τα indexes της στήλης και γραμμής του εμποδίου.

```
for (int dist = 1; dist <= 7; dist++) {
  for (int df = -dist; df <= dist; df++) {
    for (int dr = -dist; dr <= dist; dr++) {
      if (abs(df) != dist && abs(dr) != dist) continue;
      int fi = ofi + df;
      int ri = ori + dr;
      if (fi < 0 || fi >= 8 || ri < 0 || ri >= 8) continue;
```

Οι τρεις εμφωλευμένες επαναλήψεις σαρώνουν τα τετράγωνα σε σπειροειδή μορφή (spiral search) ώστε να βρεθεί το πιο κοντινό κενό τετράγωνο για τη προσωρινή μετακίνηση. Ο έλεγχος abs(df) != dist && abs(dr) != dist εμποδίζει να σαρωθούν τα εσωτερικά τετράγωνα ξανά όταν αυξάνεται το dist. Στις μεταβλητές fi, ri είναι τα τρέχοντα indexes του τετραγώνου προς εξέταση και ελέγχεται αν είναι εκτός σκακιέρας.

```
      char f = (char)('a' + fi);
      char r = (char)('1' + ri);
      if (pieceBoard[fi][ri] != '.') continue;
      if (squareInPath(f, r, pathF1, pathR1, pathF2, pathR2)) continue;
      bool conflict = false;
      for (int e = 0; e < nRes; e++) {
        if (resF[e] == f && resR[e] == r) {
          conflict = true;
          break;
        }
      }
```

Μετατρέπονται σε μορφή χαρακτήρων και ελέγχεται αν υπάρχει ήδη άλλο κομμάτι στο τετράγωνο ή αν το εξεταζόμενο καινούργιο τετράγωνο εμποδίζει τη διαδρομή. Η μεταβλητής conflict ελέγχει αν το τετράγωνο έχει ήδη δεσμευτή για τη προσωρινή μετακίνηση διαφορετικού εμποδίου, καθώς μπορεί να πρέπει να μετακινηθεί πάνω από ένα κομμάτι για την ολοκλήρωση της κίνησης. Τα δεσμευμένα τετράγωνα «παρκαρίσματος» γράφονται στους πίνακες resF[] και resR[] για reserved file/rank.

```
      if (conflict) continue;
      if (f != origF && r != origR) {
        bool wpAFree = (pieceBoard[f - 'a'][origR - '1'] == '.');
```

```
      bool wpBFree = (pieceBoard[origF - 'a'][r - '1'] == '.');
      if (!wpAFree && !wpBFree) continue;
     }
     outF = f;
     outR = r;
     return true;
    }
   }
  }
 return false;
}
```

Αν η κίνηση προς το τετράγωνο «παρκαρίσματος» δεν είναι ευθεία (στην ίδια γραμμή ή στήλη) τότε πρόκειται για κίνηση σχήματος L, στην οποία εξετάζονται οι δύο πιθανές διαδρομές με βάση ποια γωνία είναι άδεια. Στην περίπτωση που καμία γωνία δεν είναι άδεια συνεχίζει η αναζήτηση για καλύτερο τετράγωνο. Τέλος αποθηκεύονται στις μεταβλητές outF και outR το τετράγωνο «παρκαρίσματος» και η συνάρτηση γυρίζει true πως βρέθηκε.

physicalPickAndPlace()

```
static void physicalPickAndPlace(char f1, char r1, char f2, char r2) {
  moveToSquare(f1, r1);
  if (isStopped) return;

  isElectromagnetOn = true;
  digitalWrite(ELECTROMAGNET_PIN, HIGH);
  rgbLedWrite(38, 255, 0, 0);
```

Η συνάρτηση συνδυάζει τις προηγούμενες για να εκτελέσει μία πλήρη κίνηση μεταφέροντας τον μαγνήτη στο τετράγωνο αφετηρίας και υπολογίζοντας τη διαδρομή για το τετράγωνο προορισμού. Αρχικά καλείται η moveToSquare() που μετακινεί τον ηλεκτρομαγνήτη σε μία ευθεία γραμμή (καθώς δεν μετατοπίζει κάποιο κομμάτι κλειστός). Ύστερα ενεργοποιείται ο ηλεκτρομαγνήτης και αλλάζει το ενδεικτικό φως στον μικροελεγκτή για αποσφαλμάτωση.

```
  float srcTx, srcTy;
  squareToStepCoords(f1, r1, srcTx, srcTy);
  bool nearWall = (srcTx > (float)(MAX_X_STEPS - SLOWDOWN_ZONE) ||
                   srcTy > (float)(MAX_Y_STEPS - SLOWDOWN_ZONE) ||
                   srcTx < (float)SLOWDOWN_ZONE ||
                   srcTy < (float)SLOWDOWN_ZONE);
  delay(nearWall ? 600 : 400);
```

Υπολογίζεται η τρέχουσα θέση του ηλεκτρομαγνήτη σε συντεταγμένες (σε βήματα) και κρίνεται αν βρίσκεται κοντά στη περίμετρο. Για να «ανοίξει» πλήρως ο ηλεκτρομαγνήτης η εκκίνηση της κίνησης καθυστερείται για ένα μικρό χρονικό διάστημα. Επειδή η κίνηση από τη περίμετρο είναι πιο ξαφνική η εκκίνηση καθυστερείται περισσότερο.

```
  char wpF[3], wpR[3];
  int nWP = computeMagnetRoute(f1, r1, f2, r2, wpF, wpR);
  for (int i = 0; i < nWP; i++) {
    moveToSquare(wpF[i], wpR[i]);
    if (isStopped) {
      isElectromagnetOn = false;
      digitalWrite(ELECTROMAGNET_PIN, LOW);
      return;
    }
  }
  isElectromagnetOn = false;
  digitalWrite(ELECTROMAGNET_PIN, LOW);
  rgbLedWrite(38, 255, 140, 0);
  Serial.printf(" [phys] %c%c->%c%c (%dseg%s)\n",
                f1, r1, f2, r2, nWP, nearWall ? " wall" : "");
}
```

Η διαδρομή υπολογίζεται με τη computeMagnetRoute() που καταγράφει όλα τα ενδιάμεσα τετράγωνα από τα οποία πρέπει να περάσει ο ηλεκτρομαγνήτης. Σε περίπτωση έκτακτης

ανάγκης σβήνει ο ηλεκτρομαγνήτης μαζί με τα μοτέρ. Με την ολοκλήρωση της κίνησης γράφεται στο serial monitor και ανάβει το onboard LED του ESP.

updatePendingCandidates()

```
void updatePendingCandidates() {
  if (pendingMove.length() < 6) return;
  if (numLegalDests == 0) return;
  if (cyclingSource) return;

  char srcF = pendingMove[0];
  char srcR = pendingMove[1];
  String srcSq = String(srcF) + String(srcR);
```

Αυτή η συνάρτηση συνδυάζει τον έλεγχο νομιμότητας (legal move generation) με τη διεπαφή του χρήστη για να διαχειριστεί την εναλλαγή πιθανών κινήσεων σε περίπτωση ασάφειας. Επιστρέφει νωρίς αν δεν υπάρχει υποψήφια κίνηση ή αν δεν υπάρχουν νόμιμες κινήσεις. Αποθηκεύεται το τετράγωνο αφετηρίας σε επιμέρους chars και ένα String object.

```
  String found[MAX_CYCLE_CANDIDATES];
  int nFound = 0;
  for (int i = 0; i < numLegalDests && nFound < MAX_CYCLE_CANDIDATES; i++) {
    if (legalDests[i] == srcSq) continue;
    int fi = legalDests[i][0] - 'a';
    int ri = legalDests[i][1] - '1';
    int mm = 3 - ri / 2;
    int cc = (ri % 2) * 8 + (7 - fi);
    if (mm < 0 || mm >= NUM_MUX || cc < 0 || cc >= NUM_CHANNELS) continue;
    if (!results[mm][cc]) continue;
    found[nFound++] = legalDests[i];
  }
```

Ελέγχονται τα νόμιμα τετράγωνα στα οποία μπορεί να μετακινηθεί το κομμάτι και αναμένεται να ενεργοποιηθεί ένας αισθητήρας σε αυτά τα τετράγωνα για να γραφεί στον πίνακα found[].

```
  if (nFound == 0) {
    char df = pendingMove[4];
    char dr = pendingMove[5];
    int fi = df - 'a';
    int ri = dr - '1';
    int mm = 3 - ri / 2;
    int cc = (ri % 2) * 8 + (7 - fi);
    bool destStillActive = (mm >= 0 && mm < NUM_MUX &&
                            cc >= 0 && cc < NUM_CHANNELS && results[mm][cc]);
    int sfi = srcF - 'a';
```

```
  int sri = srcR - '1';
  int smm = 3 - sri / 2;
  int scc = (sri % 2) * 8 + (7 - sfi);
  bool srcNowActive = (smm >= 0 && smm < NUM_MUX &&
                       scc >= 0 && scc < NUM_CHANNELS && results[smm][scc]);
  bool isCaptureDest = (fi >= 0 && fi < 8 && ri >= 0 && ri < 8 &&
                        pieceBoard[fi][ri] != '.');
  if (srcNowActive || (!destStillActive && !isCaptureDest)) {
    Serial.printf(" pendingMove %s cleared (dest low or piece returned)\n",
                  pendingMove.c_str());
    pendingMove = "";
    detectState = DETECT_IDLE;
    liftedSquare = "";
    settleBestSquare = "";
    settleBestCount = 0;
    numCycleCandidates = 0;
    cycleIdx = 0;
    cyclingSource = false;
    lcdUpdateRow1();
  }
  return;
 }
```

Αυτό το τμήμα ελέγχει αν η κίνηση είναι άκυρη επειδή ο παίκτης επέστρεψε το κομμάτι πίσω στην αρχική του θέση. Αν δεν ανιχνεύτηκε σε καμία νόμιμη θέση η μεταβλητής destStillActive ελέγχει αν ο αισθητήρας είναι ενεργός και η srcNowActive ελέγχει τον αισθητήρα του αρχικού τετραγώνου. Αν το srcNowActive είναι true ο παίκτης επέστρεψε το κομμάτι πίσω στην αρχική θέση για να αλλάξει τη κίνση του, για επαλήθευση ελέγχεται ότι το τετράγωνο προορισμού δεν είναι ενεργό και δεν έχει αιχμαλωτίσιμο κομμάτι. Αν ισχύουν τα παραπάνω όλες οι μεταβλητές γυρνούν στις αρχικές τους τιμές για να ξαναεκτελεστεί η αναγνώριση κίνησης και ενημερώνεται η οθόνη.

```
bool changed = (nFound != numCycleCandidates);
 if (!changed) {
  for (int i = 0; i < nFound && !changed; i++) {
    bool present = false;
    for (int j = 0; j < numCycleCandidates; j++) {
      if (cycleCandidates[j] == found[i]) {
        present = true;
        break;
      }
    }
    if (!present) changed = true;
  }
 }
```

```
if (!changed) return;
```

Ελέγχεται αν η λίστα πιθανών προορισμών είναι ίδια με τη προηγούμενη για να γυρίσει νωρίς η συνάρτηση καθώς δεν χρειάζεται ενημέρωση η οθόνη.

```
String prevDst = pendingMove.substring(4);
numCycleCandidates = nFound;
for (int i = 0; i < nFound; i++) cycleCandidates[i] = found[i];

cycleIdx = 0;
for (int i = 0; i < nFound; i++) {
  if (cycleCandidates[i] == prevDst) {
    cycleIdx = i;
    break;
  }
}

String newPending = srcSq + "->" + cycleCandidates[cycleIdx];
if (newPending != pendingMove) {
  pendingMove = newPending;
  lcdUpdateRow1();
}
Serial.printf(" Candidates updated: %s [%d option%s]\n",
          pendingMove.c_str(), nFound,
          nFound > 1 ? " -- cycling every 3s" : "");
}
```

Απομονώνεται με τη substring() το τετράγωνο προορισμού και ελέγχεται πως υπάρχει επόμενο για να τυπωθεί στην οθόνη, η συνάρτηση καλείται ανά 3 δευτερόλεπτα που καθορίζει το ρυθμό ανανέωσης για να δοθεί χρόνος στο παίκτη να επικυρώσει τη κίνηση.

physicalMoveViaWaypoint()

```
static bool physicalMoveViaWaypoint(char f1, char r1,
                                    char fm, char rm,
                                    char f2, char r2) {
  char piece = pieceBoard[f1 - 'a'][r1 - '1'];
  physicalPickAndPlace(f1, r1, fm, rm);
  if (isStopped) return false;
  pieceBoard[fm - 'a'][rm - '1'] = piece;
  pieceBoard[f1 - 'a'][r1 - '1'] = '.';

  physicalPickAndPlace(fm, rm, f2, r2);
  if (isStopped) return false;
  pieceBoard[f2 - 'a'][r2 - '1'] = piece;
  pieceBoard[fm - 'a'][rm - '1'] = '.';
  return true;
}
```

Η συνάρτηση εκτελεί μία κίνηση σε σχήμα L μεταξύ του σημείου αφετηρίας f1,r1 ενός ενδιάμεσου σημείου fm,rm και το τετράγωνο προορισμού f2,r2. Επίσης ενημερώνει κατάλληλα το πίνακα pieceBoard[] με τη νέα θέση του κομματιού.

executeMove()

```
void executeMove(char f1, char r1, char f2, char r2) {
 Serial.printf("\nMachine move: %c%c->%c%c\n", f1, r1, f2, r2);
 if (f1 < 'a' || f1 > 'h' || r1 < '1' || r1 > '8' ||
     f2 < 'a' || f2 > 'h' || r2 < '1' || r2 > '8') {
  Serial.println(" Invalid square, aborting.");
  return;
 }

 suppressDetection = true;
 rgbLedWrite(38, 255, 140, 0);

 int fx1 = f1 - 'a';
 int ry1 = r1 - '1';
 int fx2 = f2 - 'a';
 int ry2 = r2 - '1';
 char movingPiece = pieceBoard[fx1][ry1];
 char capturedPiece = pieceBoard[fx2][ry2];
 bool isCapture = (capturedPiece != '.');
 bool handled = false;
```

Η κύρια συνάρτηση που συνδυάζει όλες τις τεχνικές εύρεσης διαδρομής (pathfinding) για να εκτελεστεί μία πλήρης κίνηση επιλεγμένη από το νευρωνικό δίκτυο. Αρχικά προετοιμάζονται απαραίτητες μεταβλητές για τα αριθμητικά indexes των τετραγώνων, ποιο κομμάτι κινείται, αν κάποιο κομμάτι αιχμαλωτίζεται. Η handled διαχειρίζεται τη περίπτωση των ίππων των οποίων η κίνηση εκτελείται στο παρακάτω κομμάτι του κώδικα.

```
struct KnightBlock { char nf1, nr1, nf2, nr2, pf, pr, pmf, pmr, pkf, pkr; };
 static const KnightBlock KB[] = {
  {'g','1','f','3', 'g','2','g','3','h','3'},
  {'b','1','c','3', 'b','2','b','3','a','3'},
  {'g','8','f','6', 'g','7','g','6','h','6'},
  {'b','8','c','6', 'b','7','b','6','a','6'},
 };
```

Αποθηκεύεται ένα εσωτερικό struct για να καταγραφούν οι αρχικές κινήσεις των ίππων για διευκόλυνση τους. Γράφεται η αρχική και τελική θέση του ίππου, η αρχική, η ενδιάμεση και η τελική θέση του πιονιού που εμποδίζει.

```
 for (int k = 0; k < 4; k++) {
  if (f1 != KB[k].nf1 || r1 != KB[k].nr1 ||
      f2 != KB[k].nf2 || r2 != KB[k].nr2) continue;
  if (pieceBoard[KB[k].pf - 'a'][KB[k].pr - '1'] == '.') break;

  Serial.printf(" [knight-L] parking pawn %c%c via %c%c to %c%c\n",
                KB[k].pf, KB[k].pr, KB[k].pmf, KB[k].pmr,
```

```
            KB[k].pkf, KB[k].pkr);
  if (!physicalMoveViaWaypoint(KB[k].pf, KB[k].pr,
                               KB[k].pmf, KB[k].pmr,
                               KB[k].pkf, KB[k].pkr)) {
    suppressDetection = false;
    return;
  }

  Serial.printf(" [knight-L] moving knight %c%c->%c%c\n", f1, r1, f2, r2);
  physicalPickAndPlace(f1, r1, f2, r2);
  if (isStopped) {
    suppressDetection = false;
    return;
  }
  pieceBoard[fx2][ry2] = movingPiece;
  pieceBoard[fx1][ry1] = '.';

  Serial.printf(" [knight-L] restoring pawn %c%c via %c%c to %c%c\n",
            KB[k].pkf, KB[k].pkr, KB[k].pmf, KB[k].pmr,
            KB[k].pf, KB[k].pr);
  if (!physicalMoveViaWaypoint(KB[k].pkf, KB[k].pkr,
                               KB[k].pmf, KB[k].pmr,
                               KB[k].pf, KB[k].pr)) {
    suppressDetection = false;
    return;
  }
```

Αν η κίνηση ταιριάζει με τις τέσσερις καταγεγραμμένες το σύστημα ακολουθεί τα δεδομένα χωρίς να χρειάζεται υπολογισμός τους.

```
  if (movingPiece == 'K') {
    castleWK = false;
    castleWQ = false;
  }
  if (movingPiece == 'k') {
    castleBK = false;
    castleBQ = false;
  }
  epFile = -1;
  epRank = -1;
  currentTurn = (currentTurn == TURN_WHITE) ? TURN_BLACK : TURN_WHITE;
  lastDisplayedMove = String(f1) + String(r1) + "->" +
                      String(f2) + String(r2);
```

```
  Serial.printf("Move done. Turn: %s\n",
            (currentTurn == TURN_WHITE) ? "WHITE" : "BLACK");
  printBoardState();
  handled = true;
  break;
 }
```

Αν μετακινείται βασιλιάς ενημερώνονται τα δικαιώματα ροκέ ανάλογα με το χρώμα, τυχών προηγούμενα δικαιώματα για en passant ενημερώνονται καθώς διαρκούν μόνο μία κίνηση. Αλλάζεται η σειρά και ενημερώνεται στο serial monitor και στην οθόνη.

```
if (!handled) {
  char resF[MAX_BLOCKERS + 6];
  char resR[MAX_BLOCKERS + 6];
  int nRes = 0;
  resF[nRes] = f1;
  resR[nRes] = r1;
  nRes++;
  resF[nRes] = f2;
  resR[nRes] = r2;
  nRes++;
  char mainWpF[3], mainWpR[3];
  int nMainWp = computeMagnetRoute(f1, r1, f2, r2, mainWpF, mainWpR);
  for (int i = 0; i < nMainWp - 1; i++) {
   resF[nRes] = mainWpF[i];
   resR[nRes] = mainWpR[i];
   nRes++;
   Serial.printf(" Route wp reserved: %c%c\n", mainWpF[i], mainWpR[i]);
  }
```

Αν δεν πρόκειται για τις ειδικές περιπτώσεις των ίππων αρχικοποιούνται οι πίνακες των δεσμευμένων τετραγώνων (reserved) resF[] και resR[]. Υπολογίζεται η διαδρομή με την computeMagnetRoute() που διαλέγει αν πρόκειται για ευθεία ή κίνηση σχήματος L. Οι πίνακες γεμίζονται με τα τετράγωνα αφετηρίας και προορισμού, μαζί με το τετράγωνο γωνίας σε περίπτωση που απαιτείται κίνηση σχήματος L.

```
  char capParkF = 0;
  char capParkR = 0;
  if (isCapture) {
   if (findParkingSquare(f2, r2, f1, r1, f2, r2,
                resF, resR, nRes, capParkF, capParkR)) {
     Serial.printf(" Parking captured %c: %c%c->%c%c\n",
     capturedPiece, f2, r2, capParkF, capParkR);
     physicalPickAndPlace(f2, r2, capParkF, capParkR);
    if (isStopped) {
     suppressDetection = false;
```

```
      return;
    }
    pieceBoard[capParkF - 'a'][capParkR - '1'] = capturedPiece;
    pieceBoard[fx2][ry2] = '.';
    resF[nRes] = capParkF;
    resR[nRes] = capParkR;
    nRes++;
   } else {
    Serial.printf(" WARNING: no parking for captured %c at %c%c\n",
                  capturedPiece, f2, r2);
   }
  }
```

Αν η κίνηση πρόκειται για αιχμαλωσία αντίπαλου κομματιού πρέπει να μετατοπιστεί προσωρινά για να αφαιρεθεί από τον παίκτη. Το τετράγωνο μετακίνησης του αντίπαλου κομματιού προσδιορίζεται με τη findParkingSquare() και γίνεται η μετακίνηση του εκεί. Τυπώνονται μηνύματα στο serial monitor για παρακολούθηση σε πραγματικό χρόνο και σε περίπτωση αποτυχίας εύρεσης τετραγώνου «παρκαρίσματος».

```
char bF[MAX_BLOCKERS];
  char bR[MAX_BLOCKERS];
  char pkF[MAX_BLOCKERS];
  char pkR[MAX_BLOCKERS];
  bool parked[MAX_BLOCKERS] = {};
  int nBlockers = findPathBlockersRoute(f1, r1, f2, r2, bF, bR);
  for (int i = 0; i < nBlockers; i++) {
   if (isStopped) break;
   if (findParkingSquare(bF[i], bR[i], f1, r1, f2, r2,
                         resF, resR, nRes, pkF[i], pkR[i])) {
    Serial.printf(" Parking blocker %c: %c%c->%c%c\n",
                  pieceBoard[bF[i] - 'a'][bR[i] - '1'],
                  bF[i], bR[i], pkF[i], pkR[i]);
    physicalPickAndPlace(bF[i], bR[i], pkF[i], pkR[i]);
    if (isStopped) break;
    pieceBoard[pkF[i] - 'a'][pkR[i] - '1'] = pieceBoard[bF[i] - 'a'][bR[i] - '1'];
    pieceBoard[bF[i] - 'a'][bR[i] - '1'] = '.';
    resF[nRes] = pkF[i];
    resR[nRes] = pkR[i];
    nRes++;
    parked[i] = true;
   } else {
    Serial.printf(" WARNING: no parking for blocker at %c%c\n", bF[i], bR[i]);
   }
  }
```

Ανιχνεύονται εμπόδια που πρέπει να μετατοπιστούν προσωρινά με τη findPathBlockersRoute() και χρησιμοποιώντας ξανά τη findParkingSquare() μετακινούνται σε ελεύθερα τετράγωνα. Καθόλη τη διάρκεια η πίνακες resF[] και resR[] ενημερώνονται για να μην επιχειρήσει το σύστημα να χρησιμοποιήσει το ίδιο τετράγωνο για τη μετακίνηση περισσότερου από ενός κομματιού.

```
if (!isStopped) {
    Serial.printf(" Main: %c%c->%c%c\n", f1, r1, f2, r2);
    physicalPickAndPlace(f1, r1, f2, r2);
  }

  if (!isStopped && toupper(movingPiece) == 'K' && abs(fx2 - fx1) == 2) {
    bool white = lmgIsWhite(movingPiece);
    int bki = white ? 0 : 7;
    char bk = white ? '1' : '8';
    char rook = white ? 'R' : 'r';
    if (fx2 == 6) {
      physicalPickAndPlace('h', bk, 'f', bk);
      if (!isStopped) {
        pieceBoard[5][bki] = rook;
        pieceBoard[7][bki] = '.';
      }
    } else if (fx2 == 2) {
      physicalPickAndPlace('a', bk, 'd', bk);
      if (!isStopped) {
        pieceBoard[3][bki] = rook;
        pieceBoard[0][bki] = '.';
      }
    }
  }
```

Εκτελείται η κύρια κίνηση μετά τις μετακινήσεις των απαραίτητων κομματιών και ενημερώνεται κατάλληλα ο πίνακας pieceBoard[]. Ελέγχεται ξεχωριστά η περίπτωση του ροκέ διότι είναι η μόνη κατά την οποία μετακινούνται δύο κομμάτια ταυτόχρονα.

```
for (int i = nBlockers - 1; i >= 0 && !isStopped; i--) {
    if (!parked[i]) continue;
    Serial.printf(" Restoring %c: %c%c->%c%c\n",
                  pieceBoard[pkF[i] - 'a'][pkR[i] - '1'],
                  pkF[i], pkR[i], bF[i], bR[i]);
    physicalPickAndPlace(pkF[i], pkR[i], bF[i], bR[i]);
    if (isStopped) break;
    pieceBoard[bF[i] - 'a'][bR[i] - '1'] = pieceBoard[pkF[i] - 'a'][pkR[i] - '1'];
    pieceBoard[pkF[i] - 'a'][pkR[i] - '1'] = '.';
  }
```

```
  if (!isStopped) {
    pieceBoard[fx2][ry2] = movingPiece;
    pieceBoard[fx1][ry1] = '.';
    if (isCapture && capParkF != 0) {
      pieceBoard[capParkF - 'a'][capParkR - '1'] = '.';
      Serial.printf(" Captured %c cleared from model at %c%c\n",
                capturedPiece, capParkF, capParkR);
    }
```

Μετά τη κύρια κίνηση όλα τα εμποδίζοντα κομμάτια επαναφέρονται στις προηγούμενες θέσεις τους και ενημερώνεται ο πίνακας πως μετακινήθηκαν πίσω. Σε περίπτωση αιχμαλωσίας το κομμάτι διαγράφεται από τη θέση και είναι ευθύνη του παίκτη να το αφαιρέσει από τη σκακιέρα.

```
if (movingPiece == 'K') {
      castleWK = false;
      castleWQ = false;
    }
    if (movingPiece == 'k') {
      castleBK = false;
      castleBQ = false;
    }
    if (movingPiece == 'R' || capturedPiece == 'R') {
      if (fx1 == 7 || fx2 == 7) castleWK = false;
      if (fx1 == 0 || fx2 == 0) castleWQ = false;
    }
    if (movingPiece == 'r' || capturedPiece == 'r') {
      if (fx1 == 7 || fx2 == 7) castleBK = false;
      if (fx1 == 0 || fx2 == 0) castleBQ = false;
    }
```

Ενημερώνονται τα δικαιώματα ροκέ ανά χρώμα και κατεύθυνση ανάλογα αν μετακινήθηκε πύργος ή βασιλιάς και ποιος.

```
    if (toupper(movingPiece) == 'P' && abs(ry2 - ry1) == 2) {
      epFile = fx1;
      epRank = ry2 + (ImgIsWhite(movingPiece) ? -1 : 1);
    } else {
      epFile = -1;
      epRank = -1;
    }
    currentTurn = (currentTurn == TURN_WHITE) ? TURN_BLACK : TURN_WHITE;
    lastDisplayedMove = String(f1) + String(r1) + "->" +
                        String(f2) + String(r2);
    Serial.printf("Move done. Turn: %s\n",
```

```
            (currentTurn == TURN_WHITE) ? "WHITE" : "BLACK");
    printBoardState();
  }
}
```

Ενημερώνονται τα δικαιώματα en passant, αλλάζεται η σειρά και τυπώνεται η κατάσταση στο serial monitor και στην οθόνη.

```
 rgbLedWrite(38, 0, 0, 255);
 delay(500);
 muxScanAll();
 memcpy(boardState, results, sizeof(boardState));
 memcpy(rawPrev, results, sizeof(rawPrev));
 memset(sameCount, 0, sizeof(sameCount));
 detectState = DETECT_IDLE;
 pendingMove = "";
 liftedSquare = "";
 settleBestSquare = "";
 settleBestCount = 0;
 numCycleCandidates = 0;
 cyclingSource = false;
 suppressDetection = false;
 lcdUpdate();
}
```

Όλες οι σχετικές μεταβλητές παίρνουν τις αρχικές τιμές τους για να προετοιμαστεί η ανίχνευση κίνησης του παίκτη και ενημερώνεται η οθόνη.

homeAndMeasure()

```
void homeAndMeasure(bool printMeasurement) {
 if (printMeasurement) ESP_LOGI(TAG, "=== RE-HOME + MEASURE ===");
 else ESP_LOGI(TAG, "=== AUTO-HOMING ===");

 stepper1->setAcceleration(ACCELERATION);
 stepper2->setAcceleration(ACCELERATION);
 stepper1->setSpeedInHz(CALIBRATION_SPEED);
 stepper2->setSpeedInHz(CALIBRATION_SPEED);
 stepper1->applySpeedAcceleration();
 stepper2->applySpeedAcceleration();
```

Η συνάρτηση εκτελεί το αρχικό calibration (και στο τέλος κάθε παρτίδας) για να καθοριστούν τα σημεία μηδέν των δύο αξόνων. Αρχικά οι κινητήρες ρυθμίζονται σε αργή ταχύτητα για την προστασία των διακόπτων που είναι ευαίσθητοι σε κρούσεις και τη μείωση της καταπόνησης του συστήματος.

```
 long startX = stepper1->getCurrentPosition();
 stepper1->runForward();
 stepper2->runForward();
 while (digitalRead(LIMIT_SWITCH_X_PIN) == HIGH) {
  if (checkEStop()) return;
  delay(1);
 }
 stepper1->forceStop();
 stepper2->forceStop();
 long xSteps = abs(stepper1->getCurrentPosition() - startX);
 ESP_LOGI(TAG, "X limit. Steps: %ld", xSteps);
 delay(300);
```

Καταγράφεται η αρχική θέση για να γίνει το calibration στον άξονα X (δεξιά-αριστερά). Τα μοτέρ περιστρέφονται προς την ίδια μεριά μέχρι να πατηθεί ο διακόπτης LIMIT_SWITCH_X_PIN που σημαίνει το σημείο μηδέν του άξονα X. Τα μοτέρ σταματάνε ακαριαία καθώς πρόκειται για το φυσικό όριο του συστήματος κίνησης. Υπολογίζεται η απόσταση που διανύθηκε για αποσφαλμάτωση και αρχική χαρτογράφηση του χώρου κίνησης.

```
 stepper1->setSpeedInHz(CALIBRATION_SPEED);
 stepper2->setSpeedInHz(CALIBRATION_SPEED);
 stepper1->applySpeedAcceleration();
 stepper2->applySpeedAcceleration();
 long startY = stepper1->getCurrentPosition();
 stepper1->runBackward();
 stepper2->runForward();
 while (digitalRead(LIMIT_SWITCH_Y_PIN) == HIGH) {
  if (checkEStop()) return;
  delay(1);
```

```
 }
 stepper1->forceStop();
 stepper2->forceStop();
 long ySteps = abs(stepper1->getCurrentPosition() - startY);
 ESP_LOGI(TAG, "Y limit. Steps: %ld", ySteps);
 delay(300);
```

Μετά τη forceStop() επειδή η βιβλιοθήκη κάνει πιθανώς reset της ταχύτητες αρχικοποιούνται ξανά στο CALIBRATION_SPEED. Ο δεύτερος άξονα Y (πάνω-κάτω) ακολουθεί την ίδια διαδικασία με τη διαφορά πως οι κινητήρες πρέπει να περιστρέφονται σε αντίστροφη φορά. Κινείται προς τα κάτω μέχρι να πατηθεί ο διακόπτης LIMIT_SWITCH_Y_PIN και υπολογίζονται ξανά τα βήματα.

```
 stepper1->setCurrentPosition(0);
 stepper2->setCurrentPosition(0);

 if (!printMeasurement) {
  Serial.println("\n============================");
  Serial.println(" HOMING COMPLETE ");
  Serial.println(" h1 = origin (0,0) ");
  Serial.println("============================\n");
  return;
 }
 Serial.println("\n==========================================");
 Serial.println(" RE-HOME + MEASUREMENT COMPLETE ");
 Serial.printf(" X: %ld steps expected %d drift %+ld (%+.2f mm)\n",
          xSteps, MAX_X_STEPS,
          xSteps - (long)MAX_X_STEPS,
          (xSteps - (long)MAX_X_STEPS) * 0.0125f);
 Serial.printf(" Y: %ld steps expected %d drift %+ld (%+.2f mm)\n",
          ySteps, MAX_Y_STEPS,
          ySteps - (long)MAX_Y_STEPS,
          (ySteps - (long)MAX_Y_STEPS) * 0.0125f);
 Serial.println(" Origin reset to (0,0) = h1.");
 Serial.println("==========================================\n");
}
```

Μετά το calibration των δύο αξόνων ορίζεται με τη setCurrentPosition() η θέση περιστροφής των κινητήρων ως το σημείο (0,0) του συστήματος κίνησης. Για τη χαρτογράφηση του χώρου κίνησης τυπώνονται τα διαγνωστικά με τον αριθμό βημάτων της κάθε κίνησης συγκριτικά με τα αναμενόμενα υπολογισμένα. Με τη βοήθεια ενός joystick για χειροκίνητη κίνηση και αυτών των διαγνωστικών καθορίστηκαν τα MAX_X_STEPS και MAX_Y_STEPS του τελικού συστήματος.

Εκπαίδευση Νευρωνικού Δικτύου

Βάση δεδομένων

Για την μηχανική εκμάθηση απαιτείται μία μεγάλη βάση δεδομένων ανεξαρτήτως σκοπού άρα η εύρεση των κατάλληλων δεδομένων ήταν το πρώτο βήμα στην εκπαίδευση του νευρωνικού δικτύου. Η βάση δεδομένων που αξιοποιήθηκε ήταν η open-chess database από την ιστοσελίδα Lichess.org που αποτελείται από 369,477,725 θέσεις αξιολογημένες από το Stockfish, τον κορυφαίο σκακιστικό αλγόριθμο (Open Database, 2026) (Stockfish, 2022).

Η βάση δεδομένων αποτελείται από ένα αρχείο lichess_db_eval.jsonl.zst μεγέθους 19GB με τα δεδομένα σε μορφή JSON (JavaScript Object Notation) ως εξής (Evaluations, 2026):

```
{
  "fen": // the position FEN only contains pieces, active color, castling rights, and en passant square.
  "evals": [ // a list of evaluations, ordered by number of PVs.
    "knodes": // number of kilo-nodes searched by the engine
    "depth": // depth reached by the engine
    "pvs": [ // list of principal variations
      "cp": // centipawn evaluation. Omitted if mate is certain.
      "mate": // mate evaluation. Omitted if mate is not certain.
      "line": // principal variation, in UCI_Chess960 format.
}
```

Το FEN format (Forsyth–Edwards Notation) συμπιέζει τις θέσεις σε ένα string μαζί με το ποιος παίζει, αν επιτρέπεται ροκέ ή en passant μειώνοντας τον απαιτούμενο χώρο αποθήκευσης (Forsyth–Edwards Notation, 2026). Στα evals αποθηκεύονται όλες οι αναλύσεις της μηχανής Stockfish για τη θέση, συγκεκριμένα όλες οι γραμμές ή ακολουθίες κινήσεων που εξετάστηκαν.

Το knodes (kilo-nodes) δείχνει πόσες χιλιάδες θέσεις εξέτασε το Stockfish, δηλαδή αν το knodes είναι 1,000 εξετάστηκαν 1,000,000 θέσεις.

Depth είναι το «βάθος» των υπολογισμών δηλαδή πόσες κινήσεις μπροστά αναλύθηκαν, όσο μεγαλύτερο το βάθος τόσο πιο αξιόπιστη η ανάλυση.

Pvs (Principal Variations) είναι οι καλύτερες γραμμές, εννοώντας τις ακολουθίες κινήσεων που βρέθηκαν από το Stockfish, μαζί με την αξιολόγηση.

Cp (Centipawns) είναι η μέτρηση της αξιολόγησης που δείχνει πιο χρώμα έχει πλεονέκτημα και κατά πόσο. Για τα λευκά παίρνει θετικές τιμές με +1 ανά πιόνι ή +3 ανά αξιωματικό, ενώ για τα μαύρα αρνητικές με -5 για πύργο ή -9 για βασίλισσα.

Mate (ματ) δείχνει αν η μηχανή βρήκε αναγκαστικό ματ και ο αριθμός του δείχνει τον αριθμό απαιτούμενων κινήσεων για αυτό.

Line δίνει τη σειρά κινήσεων σε μορφή UCI, για παράδειγμα e2e4 e7e5, που διευκολύνει σημαντικά την μετάφραση των δεδομένων με έτοιμα τετράγωνα προέλευσης και προορισμού.

Ένα παράδειγμα θέσης στη βάση δεδομένων είναι το εξής:


```
{
 "fen": "2bq1rk1/pr3ppn/1p2p3/7P/2pP1B1P/2P5/PPQ2PB1/R3R1K1 w - -
",
 "evals": [
  {
   "pvs": [
    {
     "cp": 311,
     "line": "g2e4 f7f5 e4b7 c8b7 f2f3 b7f3 e1e6 d8h4 c2h2 h4g4"
    }
   ],
   "knodes": 206765,
   "depth": 36
  },
  {
   "pvs": [
    {
     "cp": 292,
     "line": "g2e4 f7f5 e4b7 c8b7 f2f3 b7f3 e1e6 d8h4 c2h2 h4g4"
    },
    {
     "cp": 277,
     "line": "f4g3 f7f5 e1e5 d8f6 a1e1 b7f7 g2c6 f8d8 d4d5 e6d5"
    }
   ],
   "knodes": 92958,
   "depth": 34
  },
  {
   "pvs": [
    {
     "cp": 190,
     "line": "h5h6 d8h4 h6g7 f8d8 f4g3 h4g4 c2e4 g4e4 g2e4 g8g7"
    },
    {
     "cp": 186,
     "line": "g2e4 f7f5 e4b7 c8b7 f2f3 b7f3 e1e6 d8h4 c2h2 h4g4"
    },
    {
     "cp": 176,
```

```
      "line": "f4g3 f7f5 e1e5 f5f4 g2e4 h7f6 e4b7 c8b7 g3f4 f6g4"
     }
    ],
    "knodes": 162122,
    "depth": 31
   }
  ]
}
```


Για την ακόλουθη θέση:



Εικόνα 76 - Παράδειγμα θέσης FEN

Το επόμενο βήμα με τη βάση δεδομένων κατεβασμένη είναι να καθοριστεί ο αριθμός θέσεων που θα εκπαιδευτεί το μοντέλο και οι ποιότητα αυτών των δεδομένων. Θέσεις οι οποίες έχουν μία καλύτερη κίνηση παρουσιάζουν μικρή ασάφεια ενώ αντίθετα θέσεις όπως το endgame κατά το οποίο υπάρχουν πολλές διαθέσιμες κινήσεις εισάγουν θόρυβο κατά την εκπαίδευση. Επιπρόσθετα το βάθος του υπολογισμού και η ισοπαλία μιας θέσης (cp κοντά στο μηδέν) παρέχουν πιο αξιόπιστες αναλύσεις και δυναμικές θέσεις. Με βάση τα παραπάνω

πρέπει να γίνει ένα φιλτράρισμα στη βάση δεδομένων ώστε να μείνουν μόνο οι πιο καρποφόρες μετρήσεις.

Για να συμβεί αυτό το φιλτράρισμα χρησιμοποιήθηκε προσωρινό python script, εξ ολοκλήρου με τη χρήση του LLM Claude Sonnet, το οποίο ξεχωρίζει τις πιο αξιόπιστες μετρήσεις και τις μετατρέπει στο input και output tensor με τα οποία μπορεί να εκπαιδευτεί το μοντέλο.

Επιλέχτηκε ο αριθμός δειγμάτων να έχει ύψος 5,000,000 θέσεων, για την ισορρόπηση της πρόκλησης και της δυσκολίας προς τους παίκτες, και τα δεδομένα να αποθηκευτούν σε tensors της ακόλουθης μορφής:

Input tensor επιλέχτηκε της μορφής [N,18,8,8] όπου N ο αριθμός δειγμάτων, 18 δεδομένα μίας θέσης σε bitmaps αναλυτικά

Bitmap	**Δεδομένα**
0	Λευκά Πιόνια
1	Λευκοί Ίπποι
2	Λευκοί Αξιωματικοί
3	Λευκοί Πύργοι
4	Λευκή Βασίλισσα
5	Λευκός Βασιλιάς
6	Μαύρα Πιόνια
7	Μαύροι Ίπποι
8	Μαύροι Αξιωματικοί
9	Μαύροι Πύργοι
10	ΜαύρηΒασίλισσα
11	Μαύρος Βασιλιάς
12	Φόντο, μία σειρά με όλα 1.0 για να οριοθετήσει τη σκακιέρα και χρησιμοποιηθεί ως δείκτης όλων το τετραγώνων για περιορισμού της ασάφειας από τις προηγούμενες διαστάσεις
13	1.0 αν ο παίκτης που παίζει δικαιούται μικρό ροκέ
14	1.0 αν ο παίκτης που παίζει δικαιούται μεγάλο ροκέ
15	1.0 αν ο αντίπαλος δικαιούται μικρό ροκέ
16	1.0 αν ο αντίπαλος δικαιούται μεγάλο ροκέ
17	1.0 αν υπάρχει δικαίωμα en passant

Πίνακας 19 - Ανάλυση επιπέδων του input tensor

Οι υπόλοιπες διαστάσεις αντιπροσωπεύουν τις στήλες (files) και γραμμές (ranks) της σκακιέρας τα οποία είναι 8*8 για 64 τετράγωνα. Ακολουθεί η επεξήγηση του parse_lichess_evals.py που έκανε τη μετατροπή:

```
"""
Lichess Eval DB Parser & Feature Extractor

Parses lichess_db_eval.jsonl.zst and produces:
  - X.npy : float32 array of shape (N, 18, 8, 8) — board bitmaps
  - y.npy : int64 array of shape (N,) — move labels (from*64 + to)
  - meta.npy : cp, depth, fullmove for each sample
Requirements:
    pip install python-chess zstandard numpy tqdm
"""

from __future__ import annotations

import argparse
import json
import os
import sys
import time
from pathlib import Path

import chess
import numpy as np
import zstandard as zstd
```

Εισάγονται οι απαιτούμενες βιβλιοθήκες, συγκεκριμένα:

Βιβλιοθήκη	**Χρήση**
annotations	Επιτρέπει τη χρήση καθορισμένων τύπων δεδομένων (type hinting) που κανονικά θα υποστηρίζονταν σε νεότερες εκδόσεις της Python μετατρέποντας το type checking σε strings
argparse	Χρησιμοποιείται για καταχώρηση εντολών στο πρόγραμμα μέσω του terminal
json	Διάβασμα αρχείων μορφής JSON
os	Πρόσβαση σε λειτουργίες του συστήματος όπως διαδρομές αρχείων.
sys	Πρόσβαση σε λειτουργίες του συστήματος όπως έξοδος από το πρόγραμμα.
time	Μέτρηση του χρόνου
Path	Διευκόλυνση διαχείρισης διαδρομών αρχείων (filepaths)

chess (python-chess)	Παρέχει όλους τους κανόνες του σκακιού και επικύρωση νομιμότητας κινήσεων (Fiekas, 2024)
numpy	Για μαθηματικούς υπολογισμούς και διαχείριση μεγάλων πινάκων δεδομένων
zstandard	Διαβάζει τεράστια αρχεία δεδομένων .zst όπως η επιλεγμένη βάση δεδομένων (Evaluations, 2026)

Πίνακας 20 - Βιβλιοθήκες του parser της βάσης δεδομένων

```
# Board encoding: 18 × 8 × 8 bitmaps

PIECE_TO_PLANE = {
   (chess.PAWN, chess.WHITE): 0,
   (chess.KNIGHT, chess.WHITE): 1,
   (chess.BISHOP, chess.WHITE): 2,
   (chess.ROOK, chess.WHITE): 3,
   (chess.QUEEN, chess.WHITE): 4,
   (chess.KING, chess.WHITE): 5,
   (chess.PAWN, chess.BLACK): 6,
   (chess.KNIGHT, chess.BLACK): 7,
   (chess.BISHOP, chess.BLACK): 8,
   (chess.ROOK, chess.BLACK): 9,
   (chess.QUEEN, chess.BLACK): 10,
   (chess.KING, chess.BLACK): 11,
}
```

Ο πίνακας PIECE_TO_PLANE αντιστοιχεί κάθε συνδυασμό (τύπος_κομματιού, χρώμα) στον αριθμό επιπέδου του tensor εισόδου. Τα επίπεδα 0–5 περιέχουν τα λευκά κομμάτια (πιόνι, ίππος, αξιωματικός, πύργος, βασίλισσα, βασιλιάς) και τα επίπεδα 6–11 τα αντίστοιχα μαύρα. Η αρίθμηση ακολουθεί την ίδια σύμβαση με τον encoder του μικροελεγκτή ώστε τα δύο συστήματα να παράγουν ίδια tensors για ίδιες θέσεις.

```
def encode_board(board: chess.Board) -> np.ndarray:
   planes = np.zeros((18, 8, 8), dtype=np.float32)
   flip = (board.turn == chess.BLACK)

   for square, piece in board.piece_map().items():
      row = square // 8
      col = square % 8
      if flip:
         row = 7 - row
         color = not piece.color
      else:
         color = piece.color
```

```
        plane = PIECE_TO_PLANE[(piece.piece_type, color)]
        planes[plane, row, col] = 1.0
```

Η συνάρτηση encode_board() επιστρέφει ένα tensor διαστάσεων [18, 8, 8]. Η επανάληψη περνάει όλα τα κομμάτια της σκακιέρας και τοποθετεί την τιμή 1.0 στο αντίστοιχο επίπεδο και τετράγωνο. Η αναστροφή (flip) εφαρμόζεται όταν κινούνται τα μαύρα, αναστρέφεται η γραμμή ώστε το μοντέλο να βλέπει πάντα τη θέση από την πλευρά της πλευράς που κινείται, και εναλλάσσεται το χρώμα (color = not piece.color) ώστε τα κομμάτια της κινούμενης πλευράς να πέφτουν πάντα στα επίπεδα 0–5. Αυτή η τεχνική μειώνει την απαιτούμενη εκπαίδευση στη μέση καθώς το μοντέλο δεν διαχωρίζει μεταξύ λευκών και μαύρων κατά την αξιολόγηση που εκτελεί και δεν πρέπει να εκπαιδευτεί ξεχωριστά ανά χρώμα.

```
    planes[12, :, :] = 1.0
    our_color = board.turn
    their_color = not board.turn
    planes[13, :, :] = float(board.has_kingside_castling_rights(our_color))
    planes[14, :, :] = float(board.has_queenside_castling_rights(our_color))
    planes[15, :, :] = float(board.has_kingside_castling_rights(their_color))
    planes[16, :, :] = float(board.has_queenside_castling_rights(their_color))

    if board.ep_square is not None:
        row = board.ep_square // 8
        col = board.ep_square % 8
        if flip:
            row = 7 - row
        planes[17, row, col] = 1.0
    return planes
```

Οι υπόλοιπες διαστάσεις περιέχουν τις πληροφορίες κατάστασης του παιχνιδιού. Το επίπεδο 12 τίθεται ολόκληρο σε 1.0 ως φόντος για να χρησιμοποιηθεί ως δείκτης όλων το τετραγώνων για περιορισμού της ασάφειας από τις προηγούμενες διαστάσεις. Τα bitmaps 13–16 κωδικοποιούν τα δικαιώματα ροκέ, γεμίζει ολόκληρο το επίπεδο με 1.0 εάν το αντίστοιχο δικαίωμα είναι ενεργό. Το bitmap 17 ενεργοποιεί μόνο το τετράγωνο του en passant εάν υπάρχει διαθέσιμη αιχμαλώτιση.

```
def move_to_label(move: chess.Move, flip: bool) -> int:
    from_sq = move.from_square
    to_sq = move.to_square
    if flip:
        from_sq = (7 - from_sq // 8) * 8 + (from_sq % 8)
        to_sq = (7 - to_sq // 8) * 8 + (to_sq % 8)
    return from_sq * 64 + to_sq
```

Κάθε κίνηση κωδικοποιείται ως ακέραιος, παράγοντας 4096 δυνατές τιμές (64*64). Τα τετράγωνα αριθμούνται from_sq=0 (a1) έως from_sq=63 (h8) κατά τη σύμβαση της python-chess. Η ίδια αναστροφή γραμμής εφαρμόζεται και στα τετράγωνα κίνησης ώστε το label να είναι συνεπής με την αναστραμμένη φορά που δίνεται ως είσοδος ώστε να είναι ίδια ανεξαρτήτως ποιος παίζει.

```
# Filtering

def should_keep(record: dict, board: chess.Board, best_eval: dict):
    depth = record.get("depth", 0)
    if depth < 20:
        return False, "low_depth"

    pvs = best_eval.get("pvs", [])
    if not pvs:
        return False, "no_pvs"

    top_pv = pvs[0]
    if "mate" in top_pv:
        if abs(top_pv["mate"]) <= 1:
            return False, "mate_in_1"
    elif "cp" in top_pv:
        if abs(top_pv["cp"]) > 1000:
            return False, "blowout_position"
    else:
        return False, "no_score"

    if sum(1 for _ in board.legal_moves) < 2:
        return False, "forced_move"

    return True, "ok"
```

Η συνάρτηση should_keep() εφαρμόζει τέσσερα φίλτρα ποιότητας σε κάθε εγγραφή. Πρώτον αποκλείονται θέσεις που αναλύθηκαν σε βάθος μικρότερο από 20 κινήσεις, καθώς το Stockfish δεν έχει εξερευνήσει επαρκώς τον χώρο αναζήτησης και η βέλτιστη κίνηση μπορεί να αλλάξει σε μεγαλύτερο βάθος. Δεύτερον, αποκλείονται θέσεις ματ σε 1 κίνηση, όπου ο αλγόριθμος δεν χρειάζεται να αξιολογήσει καρποφόρες κινήσεις. Τρίτον, αποκλείονται θέσεις με μεγάλη υπεροχή (|cp| > 1000 centipawns) διότι το πλεονέκτημα είναι τόσο μεγάλο άρα η καλύτερη κίνηση δεν είναι στρατηγική και το μοντέλο δεν ωφελείται από αυτές. Τέλος, αποκλείονται αναγκαστικές κινήσεις (μία μόνο νόμιμη κίνηση), οι οποίες δεν προσφέρουν καμία πληροφορία σχετικά με στρατηγική επιλογή.

```
# Streaming parser

def stream_jsonl_zst(filepath: str):
    """
    Generator yielding (record_dict, bytes_decompressed_so_far).
    Uses 16 MB read buffer for NVMe throughput.
    """
    READ_SIZE = 16 * 1024 * 1024 # 16 MB
    dctx = zstd.ZstdDecompressor(max_window_size=2**31)
```

```
    bytes_read = 0

    with open(filepath, "rb") as fh:
        with dctx.stream_reader(fh) as reader:
            buffer = b""
            while True:
                chunk = reader.read(READ_SIZE)
                if not chunk:
                    break
                bytes_read += len(chunk)
                buffer += chunk
                lines = buffer.split(b"\n")
                buffer = lines[-1]
                for line in lines[:-1]:
                    line = line.strip()
                    if line:
                        try:
                            yield json.loads(line), bytes_read
                        except json.JSONDecodeError:
                            continue
            if buffer.strip():
                try:
                    yield json.loads(buffer), bytes_read
                except json.JSONDecodeError:
                    pass
```

Το αρχείο lichess_db_eval.jsonl.zst είναι αποθηκευμένο σε μορφή Zstandard (zstd), ο αλγόριθμος αποσυμπιέζει σε streaming χωρίς να απαιτεί ολόκληρο το αρχείο στη μνήμη. Χρησιμοποιείται buffer 16 MB ανά ανάγνωση, προσαρμοσμένος στη βέλτιστη απόδοση NVMe δίσκων. Το buffer διαχωρίζεται σε γραμμές με split("\n") και αν κοπεί το τελευταίο κομμάτι αποθηκεύεται για σύνθεση στο επόμενο chunk. Κάθε ολοκληρωμένη γραμμή γράφεται ως JSON και επιστρέφεται μία μία με τη yield, αποφεύγοντας τη φόρτωση ολόκληρης της βάσης στη μνήμη.

```
def parse_dataset(
    input_path: str,
    output_dir: str,
    max_samples: int = 5_000_000,
    save_every: int = 500_000,
    log_every: int = 10_000,
):
```

Η συνάρτηση parse_dataset() αποτελεί τη κεντρική επανάληψη επεξεργασίας οριοθετόντας τις παραμέτρους του προγράμματος. Τρέχει διαδοχικά τα strings από τη stream_jsonl_zst() και για κάθε μία εφαρμόζει την should_keep().

```
def _flush_chunk(batch_X, batch_y, batch_meta, output_dir, chunk_idx):
```

```
    X_arr = np.array(batch_X, dtype=np.float32)
    y_arr = np.array(batch_y, dtype=np.int64)
    meta_arr = np.array(batch_meta, dtype=np.int32)
    np.save(f"{output_dir}/X_chunk_{chunk_idx:04d}.npy", X_arr)
    np.save(f"{output_dir}/y_chunk_{chunk_idx:04d}.npy", y_arr)
    np.save(f"{output_dir}/meta_chunk_{chunk_idx:04d}.npy", meta_arr)
    print(f" --> Chunk {chunk_idx:04d} saved: {len(batch_X):,} samples "
          f"({X_arr.nbytes/1e6:.0f} MB)")
```

Κάθε 500.000 δείγματα (save_every) το batch αποθηκεύεται σε αρχεία .npy με αριθμό chunk (π.χ. X_chunk_0000.npy). Αυτή η στρατηγική chunked αποθήκευσης αποτρέπει απώλεια δεδομένων σε περίπτωση διακοπής και επιτρέπει παράλληλη επεξεργασία chunks. Τα τελικά arrays αποθηκεύονται ως float32 (X), int64 (y) και int32 (meta) για συμβατότητα με τη βιβλιοθήκη NumPy και αποδοτική χρήση μνήμης.

```
def _merge_chunks(output_dir, num_chunks):
    X_list, y_list, meta_list = [], [], []
    for i in range(num_chunks):
        X_list.append(np.load(f"{output_dir}/X_chunk_{i:04d}.npy"))
        y_list.append(np.load(f"{output_dir}/y_chunk_{i:04d}.npy"))
        meta_list.append(np.load(f"{output_dir}/meta_chunk_{i:04d}.npy"))
        print(f" Loaded chunk {i+1}/{num_chunks}", end="\r", flush=True)

    print("\n Concatenating ...", flush=True)
    X = np.concatenate(X_list); del X_list
    y = np.concatenate(y_list); del y_list
    meta = np.concatenate(meta_list); del meta_list

    print(" Shuffling ...", flush=True)
    rng = np.random.default_rng(42)
    perm = rng.permutation(len(y))
    X, y, meta = X[perm], y[perm], meta[perm]

    print(" Saving ...", flush=True)
    np.save(f"{output_dir}/X.npy", X)
    np.save(f"{output_dir}/y.npy", y)
    np.save(f"{output_dir}/meta.npy", meta)

    for i in range(num_chunks):
        for prefix in ["X", "y", "meta"]:
            try:
                os.remove(f"{output_dir}/{prefix}_chunk_{i:04d}.npy")
            except FileNotFoundError:
                pass
```

```
    print(f" X : {X.shape} {X.dtype} {X.nbytes/1e9:.2f} GB")
    print(f" y : {y.shape} {y.dtype} {y.nbytes/1e6:.1f} MB")
    print(f" meta : {meta.shape} {meta.dtype}")
```

Η _merge_chunks() φορτώνει όλα τα chunk αρχεία, τα συνενώνει σε ενιαία arrays και εφαρμόζει τυχαία ανακατάταξη (shuffle) με σταθερό seed 42 για αναπαραγωγιμότητα. Η ανακατάταξη είναι απαραίτητη γιατί τα δεδομένα Lichess οργανώνονται χρονολογικά και η εκπαίδευση χωρίς shuffle θα εισήγαγε ένα bias προς πρόσφατες καταχωρήσεις στη βάση δεδομένων. Μετά την αποθήκευση των τελικών X.npy, y.npy και meta.npy διαγράφονται αυτόματα τα ενδιάμεσα chunk αρχεία για εξοικονόμηση χώρου δίσκου.

```
# Main parse function

    Path(output_dir).mkdir(parents=True, exist_ok=True)
    bytes_total = os.path.getsize(input_path)

    stats = {
        "processed": 0, "kept": 0,
        "low_depth": 0, "blowout_position": 0, "mate_in_1": 0,
        "forced_move": 0, "opening_start": 0, "no_pvs": 0,
        "no_score": 0, "bad_move": 0, "other_skip": 0,
    }

    batch_X, batch_y, batch_meta = [], [], []
    chunk_idx = 0
    t0 = time.time()

    print("=" * 62)
    print(" Lichess Eval DB Parser")
    print("=" * 62)
    print(f" Input : {input_path}")
    print(f" Size : {bytes_total/1e9:.1f} GB (compressed)")
    print(f" Output : {output_dir}")
    print(f" Target : {max_samples:,} samples")
    print(f" Filter : depth>=20, |cp|<=1000, >=2 legal moves")
    print("=" * 62)
    print(" Opening decompression stream...")
    print(" (First records may take 10-30s on a large file)")
    print()
```

Ο αρχή του κορμού του προγράμματος, εισάγονται τα file paths και οι αρχικές μεταβλητές. Αντί να ξεκινήσει αμέσως την ανάλυση, προετοιμάζει τις υποδομές και τυπώνει στον χρήστη το τι πρόκειται να συμβεί για αποσφαλμάτωση.

```
    sys.stdout.flush()
    first_seen = False
```

```
for record, bytes_read in stream_jsonl_zst(input_path):
    if stats["kept"] >= max_samples:
        break

    stats["processed"] += 1

    # Confirm stream is alive on very first record
    if not first_seen:
        first_seen = True
        elapsed = time.time() - t0
        print(f" Stream alive! First record after {elapsed:.1f}s")
        print(f" Record keys : {list(record.keys())}")
        # Show a sample eval so we can verify the data looks right
        evals = record.get("evals", [])
        if evals:
            print(f" Sample eval : depth={evals[0].get('depth','?')} "
                  f"pvs={evals[0].get('pvs',[{}])[:1]}")
        print()
        sys.stdout.flush()
```

Η αρχή της κύριας επεξεργασία, κατά την ανάγνωση της πρώτης καταχώρησης (first_seen), ο κώδικας εκτελεί μια διαγνωστική εκτύπωση, αναφέροντας τον χρόνο που απαιτήθηκε για την προθέρμανση της αποσυμπίεσης και τη δομή των κλειδιών του dictionary (keys) της παρτίδας. Αυτό το στάδιο είναι κρίσιμο για την επιβεβαίωση της ακεραιότητας των δεδομένων, καθώς εμφανίζει ένα δείγμα από το βάθος ανάλυσης και τις προτεινόμενες κινήσεις (pvs), επιτρέποντας την επαλήθευση της αναμενόμενης πληροφορίας του input του προγράμματος πριν γίνει η επεξεργασία εκατομμυρίων δεδομένων.

```
try:
        fen = record["fen"]
        evals = record.get("evals", [])
        if not evals:
            stats["no_pvs"] += 1
            continue

        best_eval = max(evals, key=lambda e: e.get("depth", 0))
        record["depth"] = best_eval.get("depth", 0)

        board = chess.Board(fen)
        keep, reason = should_keep(record, board, best_eval)
        if not keep:
            stats[reason] = stats.get(reason, 0) + 1
            continue
```

```
        pvs = best_eval["pvs"]
```

Η διαδικασία επεξεργασίας εισέρχεται σε ένα μπλοκ ελέγχου (try), όπου εξάγεται η θέση της σκακιέρας σε μορφή FEN και οι διαθέσιμες αναλύσεις της μηχανής. Αν μια παρτίδα δεν περιέχει αξιολογήσεις (evals), απορρίπτεται αμέσως και καταγράφεται στα στατιστικά ως no_pvs. Για κάθε έγκυρη εγγραφή, ο κώδικας απομονώνει την ποιοτικότερη ανάλυση (αυτή με το μεγαλύτερο βάθος/depth) και τη χρησιμοποιεί ως σημείο αναφοράς για την αξιολόγηση της θέσης.

```
def pv_score(pv):
            if "mate" in pv:
                return 100000 - abs(pv["mate"]) if pv["mate"] > 0 else -100000
            return pv.get("cp", 0)

        pvs_sorted = sorted(pvs, key=pv_score, reverse=True)
        best_pv_moves = pvs_sorted[0].get("line", "").split()
        if not best_pv_moves:
            stats["bad_move"] += 1
            continue

        move = chess.Move.from_uci(best_pv_moves[0])
        if move not in board.legal_moves:
            stats["bad_move"] += 1
            continue

        flip = (board.turn == chess.BLACK)
        enc = encode_board(board)
        label = move_to_label(move, flip)

        top_pv = pvs_sorted[0]
        if "mate" in top_pv:
            cp_val = 30000 if top_pv["mate"] > 0 else -30000
        else:
            cp_val = int(np.clip(top_pv.get("cp", 0), -32000, 32000))

        batch_X.append(enc)
        batch_y.append(label)
        batch_meta.append((cp_val, record["depth"], 0))
        stats["kept"] += 1

    except Exception as e:
        stats["other_skip"] += 1
        continue
```

Η εσωτερική συνάρτηση pv_score κανονικοποιεί τις αξιολογήσεις της σκακιστικής μηχανής, μετατρέποντας τα σενάρια ματ σε αριθμητικές τιμές ώστε να είναι συγκρίσιμα με τις μονάδες centipawns (cp). Με βάση αυτό το σκορ, οι διαθέσιμες κινήσεις ταξινομούνται και επιλέγεται η βέλτιστη κίνηση (best move). Αν η κίνηση αυτή είναι άκυρη ή δεν μπορεί να αναλυθεί, η παρτίδα απορρίπτεται ως bad_move. Για τις έγκυρες θέσεις, ο κώδικας ελέγχει ποιος παίκτης έχει σειρά και, αν είναι τα μαύρα, ενεργοποιεί την αναστροφή (flip). Η σκακιέρα μετατρέπεται σε αριθμητικό πίνακα μέσω της encode_board, ενώ η κίνηση μετασχηματίζεται σε έναν μοναδικό αριθμό-ετικέτα (label). Τέλος, η κωδικοποιημένη θέση, το label και τα μεταδεδομένα (αξιολόγηση, βάθος) προστίθενται στο τρέχον batch, αυξάνοντας τον μετρητή των «ποιοτικών» δειγμάτων (kept) που θα χρησιμοποιηθούν για την εκπαίδευση.

```
# Progress bar
    if stats["processed"] % log_every == 0:
        print_progress(stats, max_samples, t0, bytes_read, bytes_total)
        sys.stdout.flush()

    # Save chunk
    if len(batch_X) >= save_every:
        print() # newline after progress bar
        _flush_chunk(batch_X, batch_y, batch_meta, output_dir, chunk_idx)
        chunk_idx += 1
        batch_X, batch_y, batch_meta = [], [], []
        print_filter_breakdown(stats)
        sys.stdout.flush()

# Final flush
if batch_X:
    print()
    _flush_chunk(batch_X, batch_y, batch_meta, output_dir, chunk_idx)
    chunk_idx += 1

print(f"\n\nMerging {chunk_idx} chunk(s)...")
sys.stdout.flush()
_merge_chunks(output_dir, chunk_idx)

elapsed = time.time() - t0
print(f"\n{'=' * 62}")
print(f" PARSE COMPLETE")
print(f"{'=' * 62}")
print(f" Records scanned : {stats['processed']:>10,}")
print(f" Samples kept : {stats['kept']:>10,}")
print(f" Keep rate : {stats['kept']/max(stats['processed'],1)*100:.2f}%")
print(f" Time : {fmt_time(elapsed)}")
print(f" Throughput : {stats['processed']/elapsed:,.0f} rec/s")
```

```
    print()
    print_filter_breakdown(stats)
    print(f"\n Output: {output_dir}/")
    print(f" X.npy (N, 18, 8, 8) float32")
    print(f" y.npy (N,) int64")
    print(f" meta.npy (N, 3) int32 [cp, depth, fullmove]")
```

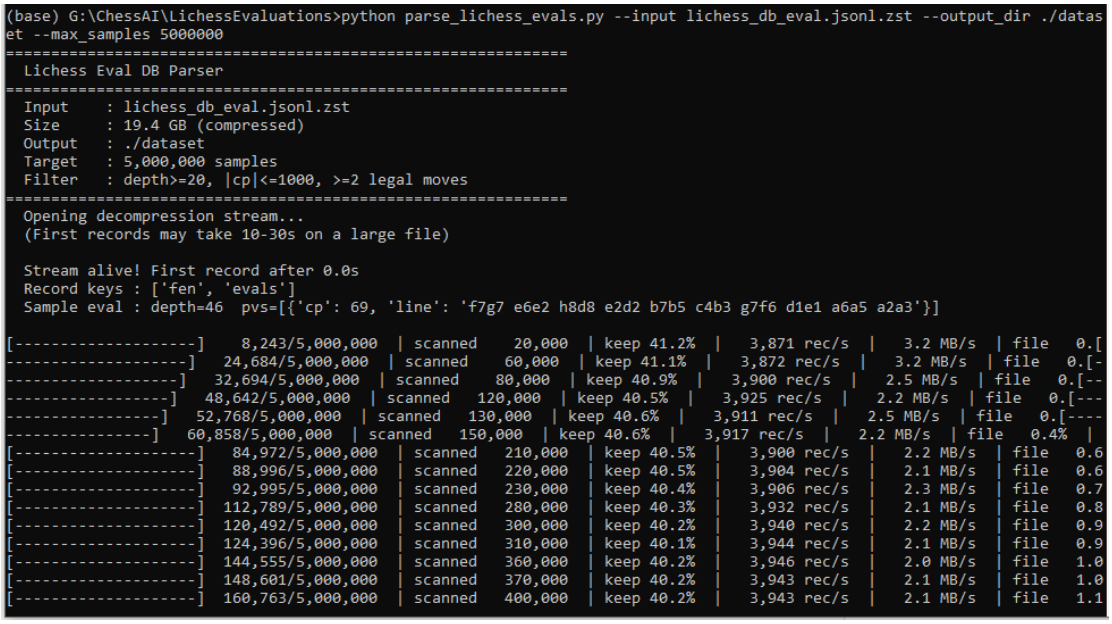
Τυπώνεται η διαγνωστική κατάσταση της προόδου και γίνεται η τελική οργάνωση ως το τελικό φιλτράρισμα πριν τη αποθήκευση του συνολικού dataset.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Parse Lichess eval DB")
    parser.add_argument("--input", default="lichess_db_eval.jsonl.zst")
    parser.add_argument("--output_dir", default="./dataset")
    parser.add_argument("--max_samples", type=int, default=5_000_000)
    parser.add_argument("--save_every", type=int, default=500_000)
    parser.add_argument("--stats_only", action="store_true")
    args = parser.parse_args()

    if args.stats_only:
        print_dataset_stats(args.output_dir)
    else:
        parse_dataset(
            input_path = args.input,
            output_dir = args.output_dir,
            max_samples = args.max_samples,
            save_every = args.save_every,
        )
```

Αυτό το τμήμα αποτελεί το σημείο εισόδου (entry point) του προγράμματος, επιτρέποντας την εκτέλεσή του απευθείας από τη γραμμή εντολών. Μέσω της βιβλιοθήκης argparse, ορίζονται οι παράμετροι λειτουργίας, όπως η διαδρομή του αρχείου εισόδου, ο φάκελος αποθήκευσης και το όριο των δειγμάτων, παρέχοντας ευελιξία χωρίς την ανάγκη τροποποίησης του κώδικα.

Τρέχοντας το παραπάνω πρόγραμμα που έγραψε το Claude Sonnet με βάση των επιλεγμένων παραμέτρων έγινε το φιλτράρισμα, η παρακάτω εικόνα δείχνει το script να τρέχει τυπώνοντας διαγνωστικά για τυχόν αλλαγές:



Εικόνα 77 - Parser running

Το φιλτράρισμα της βάσης δεδομένων πήρε συνολικά 26 λεπτά και αποθηκεύτηκαν σε 10 chunks πριν τη συγχώνευση στα τελικά αρχεία. Χρειάστηκαν να σαρωθούν 6 εκατομμύρια θέσεις προτού συγκεντρωθούν οι ζητούμενες 5 εκατομμύρια θέσεις που αποδεικνύει την υψηλή ποιότητα του open-chess database.

```
  --> Chunk 0009 saved: 500,000 samples  (2304 MB)
  Filter breakdown (1,098,896 skipped total):
    low_depth            :  838,002  (76.3%)
    blowout_position     :   96,625  ( 8.8%)
    mate_in_1            :   91,346  ( 8.3%)
    forced_move          :   72,644  ( 6.6%)
    bad_move             :      279  ( 0.0%)


Merging 10 chunk(s)...
  Loaded chunk 10/10
  Concatenating ...
  Shuffling ...
  Saving ...
  X    : (5000000, 18, 8, 8)  float32  23.04 GB
  y    : (5000000,)  int64  40.0 MB
  meta : (5000000, 3)  int32

============================================================
  PARSE COMPLETE
============================================================
  Records scanned :  6,098,896
  Samples kept    :  5,000,000
  Keep rate       : 81.98%
  Time            : 26.3min
  Throughput      : 3,859 rec/s

  Filter breakdown (1,098,896 skipped total):
    low_depth            :  838,002  (76.3%)
    blowout_position     :   96,625  ( 8.8%)
    mate_in_1            :   91,346  ( 8.3%)
    forced_move          :   72,644  ( 6.6%)
    bad_move             :      279  ( 0.0%)

  Output: ./dataset/
    X.npy    (N, 18, 8, 8) float32
    y.npy    (N,)          int64
    meta.npy (N, 3)        int32  [cp, depth, fullmove]

(base) G:\ChessAI\LichessEvaluations>
```

Εικόνα 78 - Τελικά αποτελέσματα parser

Τα τελικά αρχεία που παράχθηκαν είχαν τα εξής μεγέθη:

meta.npy	2/28/2026 1:06 AM	NPY File	58,594 KB
X.npy	2/28/2026 1:06 AM	NPY File	22,500,001 KB
y.npy	2/28/2026 1:06 AM	NPY File	39,063 KB

Εικόνα 79 - Αρχεία από το parser

Το αρχείο meta.npy περιέχει βοηθητικούς αριθμούς συγκεκριμένα το cp_score, depth και το fullmove για κάθε δείγμα με σκοπό τον έλεγχο της ακεραιότητας των δεδομένων. Το X.npy περιέχει τα input tensors μορφής (N, 18, 8, 8) για αυτό παρατηρείται η μεγάλη διαφορά μεγέθους του συγκριτικά με το y.npy που περιέχει τα ouput tensors μορφής (N,) από 0 έως 4096 που είναι όλες οι πιθανές θέσεις σε μορφή UCI (Universal Chess Interface) όπως e2e4 e7e5 (Universal Chess Interface, 2026).

Εκπαίδευση μοντέλου

Το επόμενο βήμα ήταν η εκπαίδευση του νευρωνικού δικτύου με το έτοιμο dataset. Έγινε με ένα δεύτερο Python script, train_chess_model.py αναρτημένο στο github, το οποίο εκπαιδεύει 20 γενιές (epochs) με σταθερό learning rate εκμεταλλευόμενο τη κάρτα γραφικών του συστήματος με το cuda (PyTorch documentation, 2026). Επίσης σκοπός είναι να μετατραπεί το μοντέλο σε μορφή ONNX (Open Neural Network Exchange) (Introduction to ONNX, 2024) αντί για το παρόν format της PyTorch. Επιλέχθηκε το συγκεκριμένο είδους αρχείου (.onnx) γιατί επιτρέπει την μεταγενέστερη μετατροπή του μοντέλου σε διαφορετικά formats όπως .tflite ή .esp-dl

```
(pytorch_env) G:\ChessAI\LichessEvaluations>python train_chess_model.py

============================================================
  Chess Move Prediction — Training
============================================================
  Device    : cuda
  GPU       : NVIDIA GeForce RTX 3060
  VRAM      : 12.9 GB
  Channels  : 64   Blocks: 6
  Batch     : 2048   Epochs: 20
  AMP       : True

[1/4] Loading dataset...
  Loading dataset (memory-mapped, no RAM spike)...
  Loading X into RAM (this takes ~60s but GPU will run at 100% after)...
  X : (5000000, 18, 8, 8)  float32
  y : (5000000,)  int64
  Unique moves in dataset: 1792
  Train : 4,750,000  |  Val : 250,000

[2/4] Building model...
  Parameters : 6,751,536  (~6.75M)
  INT8 size  : ~6.8 MB  (after quantisation)

[3/4] Setting up optimiser...
G:\ChessAI\LichessEvaluations\train_chess_model.py:263: FutureWarning: `torch.cuda.amp.GradSc
  scaler = torch.cuda.amp.GradScaler(enabled=cfg["amp"] and device.type=="cuda")

[4/4] Training...

   Ep     Loss    Top1    Top5  Val-Top1  Val-Top5        LR     Time
  ------------------------------------------------------------------
G:\ChessAI\LichessEvaluations\train_chess_model.py:289: UserWarning: To copy construct from a
equires_grad_(True), rather than torch.tensor(sourceTensor).
  y_batch = torch.tensor(y_batch, dtype=torch.long).to(device, non_blocking=True)
G:\ChessAI\LichessEvaluations\train_chess_model.py:293: FutureWarning: `torch.cuda.amp.autoca
  with torch.cuda.amp.autocast(enabled=cfg["amp"] and device.type=="cuda"):
G:\ChessAI\LichessEvaluations\train_chess_model.py:321: UserWarning: To copy construct from a
equires_grad_(True), rather than torch.tensor(sourceTensor).
  y_batch = torch.tensor(y_batch, dtype=torch.long).to(device, non_blocking=True)
G:\ChessAI\LichessEvaluations\train_chess_model.py:322: FutureWarning: `torch.cuda.amp.autoca
  with torch.cuda.amp.autocast(enabled=cfg["amp"] and device.type=="cuda"):
    1   3.7990   0.222   0.568     0.264     0.645  0.000994   2.9min  ETA 55.9min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep01_0.264.pth
    2   3.3853   0.289   0.683     0.294     0.694  0.000976   2.9min  ETA 52.4min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep02_0.294.pth
```

Εικόνα 80 - Αρχή εκπαίδευσης νευρωνικού δικτύου

Από τα διαγνωστικά φαίνεται πως το script δουλεύει με μετρήσιμη βελτίωση απόδοσης μεταξύ του πρώτου και του δεύτερου epoch. Η σωστή επιλογή του αριθμού των epochs είναι σημαντική γιατί ισορροπεί την δημιουργικότητα του μοντέλου, χαμηλές τιμές και συμπεριφέρεται φαινομενικά τυχαία ενώ πολύ υψηλές και ανταποκρίνεται μόνο στις τιμές των input tensors. Η τιμή των 20 epochs δόθηκε παρατηρώντας την βελτίωση του μοντέλου ανά epoch η οποία άρχισε να παρουσιάζει φθίνουσες αποδόσεις μετά την τιμή των 30 epochs οριοθετώντας τη μέγιστη ωφέλιμη εκπαίδευση με το παρόν dataset, με άλλα λόγια καθρίζουν τα όρια της υποπροσαρμογής και της υπερπροσαρμογής (underfitting και overfitting).

Το τελικό μοντέλο αποθηκεύτηκε σε 20 checkpoints (chunks) τα οποία συνδεθήκαν μαζί και πήρε 54 λεπτά χάρη στη κάρτα γραφικών:

```
G:\ChessAI\LichessEvaluations\train_chess_model.py:322: FutureWarning: `torch.cuda.amp.autocast(args..
  with torch.cuda.amp.autocast(enabled=cfg["amp"] and device.type=="cuda"):
    1   3.7990   0.222   0.568     0.264     0.645  0.000994   2.9min  ETA 55.9min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep01_0.264.pth
    2   3.3853   0.289   0.683     0.294     0.694  0.000976   2.9min  ETA 52.4min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep02_0.294.pth
    3   3.2676   0.313   0.719     0.307     0.713  0.000946   2.9min  ETA 49.2min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep03_0.307.pth
    4   3.1974   0.328   0.740     0.321     0.732  0.000905   2.9min  ETA 46.2min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep04_0.321.pth
    5   3.1497   0.338   0.753     0.323     0.741  0.000855   2.9min  ETA 43.2min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep05_0.323.pth
    6   3.1131   0.346   0.763     0.331     0.748  0.000796   2.9min  ETA 40.3min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep06_0.331.pth
    7   3.0818   0.353   0.772     0.337     0.757  0.000730   2.9min  ETA 37.4min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep07_0.337.pth
    8   3.0527   0.360   0.780     0.344     0.765  0.000658   2.6min  ETA 34.1min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep08_0.344.pth
    9   3.0251   0.366   0.787     0.344     0.767  0.000582   2.6min  ETA 31.0min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep09_0.344.pth
   10   2.9977   0.373   0.794     0.349     0.772  0.000505   2.6min  ETA 27.9min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep10_0.349.pth
   11   2.9706   0.381   0.801     0.354     0.776  0.000428   2.6min  ETA 25.0min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep11_0.354.pth
   12   2.9426   0.388   0.808     0.355     0.779  0.000352   2.6min  ETA 22.1min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep12_0.355.pth
   13   2.9132   0.397   0.815     0.361     0.783  0.000280   2.6min  ETA 19.2min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep13_0.361.pth
   14   2.8828   0.406   0.823     0.364     0.788  0.000214   2.6min  ETA 16.4min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep14_0.364.pth
   15   2.8514   0.416   0.830     0.366     0.790  0.000155   2.6min  ETA 13.6min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep15_0.366.pth
   16   2.8198   0.426   0.838     0.368     0.792  0.000105   2.6min  ETA 10.9min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep16_0.368.pth
   17   2.7895   0.437   0.845     0.370     0.793  0.000064   2.6min  ETA 8.1min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep17_0.370.pth
   18   2.7627   0.446   0.851     0.372     0.795  0.000034   2.6min  ETA 5.4min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep18_0.372.pth
   19   2.7413   0.454   0.855     0.373     0.795  0.000016   2.6min  ETA 2.7min
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep19_0.373.pth
   20   2.7272   0.460   0.859     0.373     0.795  0.000010   2.6min  ETA 0s
    Checkpoint saved: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep20_0.373.pth

============================================================
  TRAINING COMPLETE
============================================================
  Best val top-1 : 0.3730  (37.30%)
  Total time     : 53.8min
  Best checkpoint: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_ep20_0.373.pth
```

Εικόνα 81 - Αποτελέσματα εκπαίδευσης

Το αρχείο που εξάγεται είναι στα 26MB άρα χρειάζεται κβαντοποίηση για να φορτωθεί στα 8MB του ESP32-S3:



Εικόνα 82 - Αρχείο Pytorch

Η μετατροπή του σε onnx από το export_onnx.py:

```
(pytorch_env) G:\ChessAI\LichessEvaluations>python export_onnx.py
Checkpoint loaded successfully
ONNX exported: G:\ChessAI\LichessEvaluations\checkpoints\chess_model.onnx  (27.0 MB)
Clean weights: G:\ChessAI\LichessEvaluations\checkpoints\chess_model_final.pth
```

Εικόνα 83 - Εξαγωγή σε ONNX

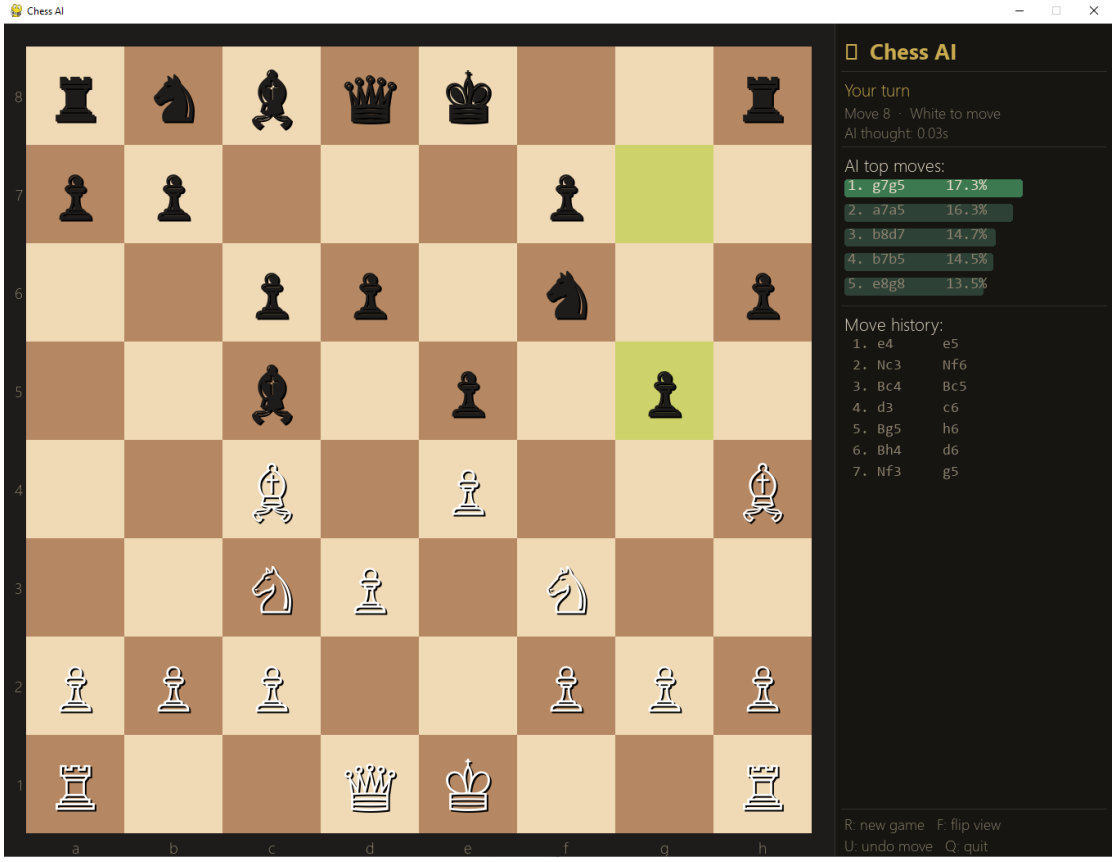


Εικόνα 84 - Αρχείο ONNX

Επαλήθευση αποτελεσμάτων

Πριν τη κβαντοποίηση χρησιμοποιήθηκε μία custom εφαρμογή για την δοκιμή της ακεραιότητας του μοντέλου γραμμένη από το LLM Claude Sonnet η οποία είχε ως σκοπό να δώσει την ικανότητα πειραματισμού πριν προχωρήσει στη κβαντοποίηση για μεταγενέστερη σύγκριση ποιότητας κβαντοποίησης λόγω της μείωσης ακρίβειας (Krishnamoorthi, 2018). Η εφαρμογή για είναι στο αρχείο play_chess.py του σχετικού github repository.

Το περιβάλλον χρήστη (UI) της εφαρμογής παρουσιάζει τις 5 κινήσεις από το μοντέλο σε σειρά αυτοπεποίθησης και η εφαρμογή ήταν κρίσιμη για τη διαπίστωση των δυνάμεων και αδυναμιών του μοντέλου. Όπως φαίνεται από τη παρακάτω εικόνα από μία δοκιμαστική παρτίδα το μοντέλο, στη αντίπαλη μαύρη πλευρά της παρτίδας, δείχνει λογικές κινήσεις με όλες τις πέντε να είναι καλές για το επιθυμητό επίπεδο δυσκολίας.



Εικόνα 85 - Διεπαφή χρήστη της ψηφιακής εφαρμογής για testing του μοντέλου

Κβαντοποίηση μοντέλου

Με το μοντέλο πλήρως λειτουργικό το επόμενο βήμα είναι η κβαντοποίηση του, επιλέχθηκε uniform quantization για την απλότητα του και μορφή .espdl, το κορυφαίο format της ESP Deep-Learning library που χρησιμοποιεί assembly instructions για την αξιοποίηση των special registers του ESP32-S3 (ESP-DL, 2026). Η προβλεπόμενη μείωση του απαιτούμενου χώρου είναι 19MB, με το μοντέλο να καταλαμβάνει 7MB. Αυτό είναι αρκετά μικρό για να χωράει στο ESP32-S3 με τα 8MB flash, αλλά όλο το υπόλοιπο πρόγραμμα θα πρέπει να χωρέσει στο εναπομένων 1MB και θα πρέπει να γραφτεί ένα custom partition για το διαχωρισμό μνήμης, καθώς τα δοσμένα από το Arduino IDE δεν δίνουν στο app space τέτοια χωρητικότητα.

Η κβαντοποίηση σε .espdl με uniform quantization γίνεται με το εργαλείο ESP-PPQ (ESP-PPQ, 2026) με το quantize_for_esp32.py:

```
(pytorch_env) G:\ChessAI\LichessEvaluations>python quantize_for_esp32.py

   ____  ____  ____  ____  ____
  / ___/ ___// __ \/ __ \/ __ \
 / /__/ /__ / /_/ / /_/ / /_/ /
/_____/____/ .___/ .___/ .___/
          /_/   /_/   /_/

Loading calibration data...
Quantizing G:\ChessAI\LichessEvaluations\checkpoints\chess_model.onnx
Target: esp32s3  Precision: INT8
Calibration samples: 500

[02:20:03] ConvTranspose Decomposition Pass Running ... Finished.
[02:20:03] PPQ Quantization Fusion Pass Running ...       Finished.
[02:20:04] PPQ Quantize Simplify Pass Running ...         Finished.
[02:20:04] PPQ Parameter Quantization Pass Running ...    Finished.
Calibration Progress(Phase 1): 100%|████████████████████████████████████████| 16/16 [00:00<00:00, 25.68it/s]
Calibration Progress(Phase 2): 100%|████████████████████████████████████████| 16/16 [00:00<00:00, 24.96it/s]
Finished.
[02:20:05] PPQ Quantization Alignment Pass Running ...    Finished.
[02:20:05] PPQ Passive Parameter Quantization Running ... Finished.
--------- Network Snapshot ---------
Num of Op:                  [36]
Num of Quantized Op:        [36]
Num of Variable:            [67]
Num of Quantized Var:       [67]
------- Quantization Snapshot ------
Num of Quant Config:        [108]
ACTIVATED:                  [31]
OVERLAPPED:                 [44]
PASSIVE:                    [33]
Network Quantization Finished.
Analysing Graphwise Quantization Error(Phrase 1):: 100%|████████████████████| 8/8 [00:00<00:00, 27.49it/s]
Analysing Graphwise Quantization Error(Phrase 2):: 100%|████████████████████| 8/8 [00:00<00:00, 12.56it/s]
Layer                              | NOISE:SIGNAL POWER RATIO
/res_tower/res_tower.5/conv2/Conv: | ████████████████████ | 3.531%
/res_tower/res_tower.4/conv2/Conv: | ███████████████████  | 3.450%
/policy_conv/Conv:                 | ██████████████████   | 3.353%
/res_tower/res_tower.3/conv2/Conv: | ████████████████     | 2.927%
/policy_fc/Gemm:                   | ███████████████      | 2.902%
/res_tower/res_tower.2/conv2/Conv: | ██████████████       | 2.656%
/res_tower/res_tower.5/conv1/Conv: | ██████████           | 1.902%
/res_tower/res_tower.1/conv2/Conv: | ████████             | 1.696%
/res_tower/res_tower.4/conv1/Conv: | ██████               | 1.268%
/res_tower/res_tower.3/conv1/Conv: | ████                 | 0.987%
/res_tower/res_tower.0/conv2/Conv: | ████                 | 0.911%
/res_tower/res_tower.2/conv1/Conv: | ███                  | 0.823%
/res_tower/res_tower.1/conv1/Conv: | ██                   | 0.612%
/input_conv/input_conv.0/Conv:     | ██                   | 0.495%
/res_tower/res_tower.0/conv1/Conv: | ██                   | 0.409%
Analysing Layerwise quantization error:: 100%|████████████████████| 15/15 [00:02<00:00,  5.34it/s]
Layer                              | NOISE:SIGNAL POWER RATIO
/policy_fc/Gemm:                   | ████████████         | 0.950%
/input_conv/input_conv.0/Conv:     | ██████               | 0.523%
/res_tower/res_tower.0/conv2/Conv: | ██                   | 0.197%
/res_tower/res_tower.1/conv2/Conv: | ██                   | 0.135%
/res_tower/res_tower.2/conv2/Conv: | ██                   | 0.118%
/res_tower/res_tower.1/conv1/Conv: | ██                   | 0.114%
/res_tower/res_tower.2/conv1/Conv: | ▌                    | 0.062%
/res_tower/res_tower.3/conv2/Conv: | ▌                    | 0.062%
/res_tower/res_tower.0/conv1/Conv: | ▌                    | 0.048%
/res_tower/res_tower.4/conv2/Conv: | ▌                    | 0.047%
/res_tower/res_tower.5/conv2/Conv: | ▌                    | 0.036%
/res_tower/res_tower.5/conv1/Conv: |                      | 0.025%
/res_tower/res_tower.4/conv1/Conv: |                      | 0.021%
/res_tower/res_tower.3/conv1/Conv: |                      | 0.017%
/policy_conv/Conv:                 |                      | 0.009%
```

Εικόνα 86 - Κβαντοποίηση με ESP-PPQ

Η κβαντοποίηση απέδειξε πως το μοντέλο μπορεί να χωρέσει στο ESP32-S3 και πρέπει να εξεταστεί ξανά στην εφαρμογή για να προσδιοριστεί η μείωση της ακρίβειας του.



Εικόνα 87 - Κβαντοποιημένο μοντέλο ESP-DL

Σε αυτό το σημείο διαπιστώθηκε πως δεν υπάρχουν πόροι και βιβλιοθήκες για τα αρχεία .espdl στο περιβάλλον Arduino IDE καθώς είναι φτιαγμένα να χρησιμοποιηθούν ως components σε περιβάλλον ESP-IDF όπως στο VS Code. Θεωρητικά μπορεί να μετατραπεί σε δεκαεξαδική μορφή και να αξιοποιηθεί η βιβλιοθήκη ESP-NN για να τρέξει το οποίο δεν θα απαιτούσε αλλαγή περιβάλλοντος (ESP-NN, 2026) αλλά εξετάστηκαν πιο απλές λύσεις από αυτό το custom integration στο Arduino IDE.

Η πιο διαδεδομένη μορφή με βιβλιοθήκη που την υποστηρίζει στο περιβάλλον ανάπτυξης είναι η .tflite για tensor flow lite, στην οποία μπορεί να μετατραπεί από την μορφή .onnx που επιλέχθηκε (Introduction to ONNX, 2024). Μετατράπηκε με το convert_to_tflite.py, το οποίο είναι αναρτημένο στο github.

Η τελική μορφή του μοντέλου είναι σε .tflite και είναι 6.7KB



Εικόνα 88 - Κβαντοποιημένο μοντέλο tensor flow lite

Αυτή η μορφή θα μπορούσε να διαβαστεί απευθείας με τη χρήση SPIFFS ή του πιο σύγχρονου LittleFS στο ESP32-S3, τα οποία είναι file browsers καθώς ο επεξεργαστής τρέχει το λογισμικό σύστημα FreeRTOS υπάρχει η δυνατότητα (FreeRTOS Reference Manual, 2026).

Ωστόσο η πιο συνηθισμένη προσέγγιση είναι η μετατροπή του μοντέλου σε ένα στατικό πίνακα δεδομένων μέσα σε ένα header file (.h) που να γίνει included στο sketch. Με ένα τελευταίο script, από το LLM Claude Sonnet, γίνεται η μετατροπή σε C array:

```
"""
Converts chess_model.tflite → chess_model_data.h
A C array included directly in the Arduino sketch.
No PROGMEM — data goes into .rodata in the app partition
"""
import os

TFLITE = r"G:\ChessAI\LichessEvaluations\esp32_model\chess_model.tflite"
HEADER = r"G:\ChessAI\LichessEvaluations\chess_inference_demo\chess_model_data.h"

with open(TFLITE, "rb") as f:
    data = f.read()

size = len(data)
print(f"Input : {TFLITE}")
print(f"Size : {size:,} bytes ({size/1e6:.2f} MB)")
print(f"Source will be ~{size*6/1e6:.0f} MB of text but compiles back to {size/1e6:.2f} MB binary")

lines = [
    "// Auto-generated by tflite_to_header.py do not edit manually",
    f"// Source: chess_model.tflite ({size} bytes / {size/1e6:.2f} MB)",
    "// Note: source file is large but compiled binary = original tflite size",
    "",
    "#pragma once",
    "",
    f"const unsigned int chess_model_tflite_len = {size};",
    "",
```

```
    "// No PROGMEM — stored in .rodata within the app partition",
    "const uint8_t chess_model_tflite[] = {",
]

row = []
for i, byte in enumerate(data):
    row.append(f"0x{byte:02x}")
    if len(row) == 12:
        lines.append("  " + ", ".join(row) + ",")
        row = []
if row:
    lines.append("  " + ", ".join(row))

lines += ["};", ""]

with open(HEADER, "w") as f:
    f.write("\n".join(lines))

print(f"Output: {HEADER}")
print("Done — add chess_model_data.h to sketch folder with #include")
```

Η επανάληψη For i, byte in enumerate(data) διαβάζει το αρχείο .tflite ένα byte τη φορά, το f"0x{byte:02x} μετατρέπει τον αριθμό κάθε byte σε δεκαεξαδική μορφή (Hex). Για παράδειγμα, αν ένα byte έχει την τιμή 255, το μετατρέπει στο κείμενο "0xff". Ο έλεγχος if len(row) == 12 γίνεται καθαρά για να είναι το αρχείο .h αναγνώσιμο, αν ήταν όλα σε μία μεγάλη σειρά το Arduino IDE θα προσπαθούσε να τα φορτώσει ταυτόχρονα για να τα δείξει στο χρήστη με αποτέλεσμα να κολλήσει. Το ", ".join(row): προσθέτει κόμματα ανάμεσα στους αριθμούς, ώστε να δημιουργηθεί ένας σωστός πίνακας της C {0x01, 0x02, ...}.

Το αρχείο προστίθεται στο ίδιο directory με το .ino του sketch και έχει τη μορφή ενός C array Το οποίο καταλαμβάνει 574550 γραμμές κώδικα με 12 δεκαξαδικές τιμές ανά γραμμή:

```
1  // Auto-generated by tflite_to_header.py do not edit manually
2  // Source: chess_model.tflite  (6894616 bytes / 6.89 MB)
3  // Note: source file is large but compiled binary = original tflite size
4
5  #pragma once
6  #include <stdint.h>
7
8  const unsigned int chess_model_tflite_len = 6894616;
9
10 // Stored in flash as defined by the app partition
11 const uint8_t chess_model_tflite[] = {
12   0x20, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, 0x00, 0x00, 0x00, 0x00,
13   0x14, 0x00, 0x20, 0x00, 0x1c, 0x00, 0x18, 0x00, 0x14, 0x00, 0x10, 0x00,
14   0x0c, 0x00, 0x00, 0x00, 0x08, 0x00, 0x04, 0x00, 0x14, 0x00, 0x00, 0x00,
```

Εικόνα 89 - Αρχή του header file του μοντέλου

Αυτό ολοκληρώνει το κομμάτι της εκπαίδευσης, πληροφορίες για τη χρήση του μοντέλου στο ESP32-S3 αναλύονται στο κεφάλαιο επεξήγηση κώδικα.

```
574558   0x08, 0x00, 0x04, 0x00, 0x0c, 0x00, 0x00, 0x00, 0x27, 0x00, 0x00, 0x00,
574559   0x02, 0x00, 0x00, 0x00, 0x00, 0x27, 0x0a, 0x00, 0x08, 0x00, 0x00, 0x00,
574560   0x00, 0x00, 0x04, 0x00, 0x0a, 0x00, 0x00, 0x00, 0x02, 0x00, 0x00, 0x00,
574561   0x0c, 0x00, 0x10, 0x00, 0x0f, 0x00, 0x00, 0x00, 0x08, 0x00, 0x04, 0x00,
574562   0x0c, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00,
574563   0x00, 0x00, 0x00, 0x03
574564 };
```

Εικόνα 90 - Τέλος του header file του μοντέλου

Αποτελέσματα

Με τη διεκπεραίωση της κατασκευής μπορούν να διαπιστωθούν οι επιδόσεις της συγκριτικά με τους αρχικούς στόχους και διαφορετικές αυτόματες σκακιέρες στην αγορά (Square Off by Miko, 2026) (Millennium Chess Shop, 2026) (Tournament e-Boards, 2026).

Ως πρωτότυπο και πρώτη έκδοση η κατασκευή πληρεί τις αρχικές προϋποθέσεις. Το σύστημα είναι πλήρως λειτουργικό ως προς τους τρεις πυλώνες, της αντίληψης, της επεξεργασίας και της κίνησης.

Το κομμάτι της επεξεργασίας είναι το πιο βελτιστοποιημένο με το νευρωνικό δίκτυο να επιλέγει δυνατές κινήσεις για το επίπεδο που σχεδιάστηκε (1500 ELO). Η φύση της βάσης δεδομένων πάνω στην οποία εκπαιδεύτηκε προσδίδει ένα εύρος στη δύναμη του ανάλογα με τις κινήσεις του παίκτη. Αν ο παίκτης παίξει καλές αρχικές κινήσεις (διαδεδομένα ανοίγματα), επειδή αυτές οι θέσεις ήταν στο εκπαιδευτικό υλικό το νευρωνικό δίκτυο γνωρίζει καλές απαντήσεις. Αντίθετα αν ο παίκτης είναι αρχάριος, τότε οι κινήσεις που θα παίξει καταλήγουν σε θέσεις που δεν βρίσκονταν στη βάση δεδομένων, κάτι που μειώνει την ακρίβεια του αντιπάλου και δημιουργεί ένα δυναμικό επίπεδο που «νιώθει» σαν ο αντίπαλος να βρίσκεται κοντά στο επίπεδο του παίκτη. Επίσης, ο υπολογισμός της απάντησης είναι εξαιρετικά γρήγορος χάρη στον επιλεγμένο μικροελεγκτή. Αυτό ήταν το κομμάτι που ξεπέρασε τις προσδοκίες και αποδεικνύει τι είναι δυνατό σε ένα φθηνό μικροελεγκτή χωρίς να έχει γίνει εξειδικευμένη βελτιστοποίηση χωρητικότητας ή ταχύτητας του μοντέλου πέρα της κβαντοποίησης.

Το σύστημα της κίνησης αποφέρει μία ευχάριστη εμπειρία με κινήσεις αρκετά γρήγορες για μία παρτίδα. Ωστόσο υπάρχουν μέρη στα οποία μπορεί να βελτιστοποιηθεί σε μία δεύτερη εκδοχή της σκακιέρας. Η εκτύπωση των δοκών αντί της αγοράς μεταλλικών αντίστοιχων περιορίζει τη μέγιστη ταχύτητα, και οι κινητήρες αν και ασφαλείς για το σύστημα είναι αρκετά θορυβώδεις κατά τη κίνηση εν μέρη εξαιτίας των drivers. Αυτό καθιστά το σύστημα ακατάλληλο για τουρνουά ή γρήγορα παιχνίδια υψηλού επιπέδου καθώς στο σκάκι η ταχύτητα των κινήσεων είναι σημαντική λόγω του περιορισμένου χρόνου. Για φιλικές παρτίδες χαμηλού ή μέτριου επιπέδου το σύστημα κίνησης είναι πλήρως κατάλληλο και προσομοιώνει επιτυχώς αληθοφανείς κινήσεις που μπορεί να συναντήσει ο παίκτης σε μία πραγματική παρτίδα.



Βίντεο 1 - Πρώτες Κινήσεις

Το σύστημα ανίχνευσης κινήσεων είναι ο κύριος περιοριστικός παράγοντας της κατασκευής, η χρήση μαγνητικών διακοπτών (reed switches) είχε ως αποτέλεσμα την υποβάθμιση της παρτίδας. Επειδή οι διακόπτες είναι δυαδικοί, χωρίς ένα εύρος τιμών για να διαπιστωθεί η δύναμη του μαγνητικού πεδίου, η αντίληψη του συστήματος δυσχεραίνεται εξαιτίας τις επικάλυψης των αισθητήρων που συμβάλλει σε λανθασμένες μετρήσεις ή μη ανταποκρίσιμους αισθητήρες. Παρ ’όλα αυτά, η απλούστερη κατασκευή του πλέγματος χάρη σε αυτούς τους αισθητήρες έναντι των hall-effect sensors, οι οποίοι έχουν τρία pins ο καθένας και πολύ μικρότερο μέγεθος, έκανε τη κατασκευή του πρωτότυπου εφικτή χωρίς την αγορά εξωτερικού PCB. Έδωσε επίσης αφορμή για εκτεταμένη αποσφαλμάτωση από τη μεριά του λογισμικού, με το τελικό σύστημα να έχει πάνω από τέσσερις δικλείδες ασφαλείας για τη σωστή επικύρωση κινήσεων από τον παίκτη. Αυτές οι γνώσεις δειγματοληψίας και αποσφαλμάτωσης αισθητήρων είναι χρήσιμες ανεξαρτήτως συστήματος και θα χρησιμοποιηθούν σε μελλοντικές κατασκευές. Η εμφάνιση των δεδομένων σε πραγματικό χρόνο στην οθόνη επιτρέπει την επικοινωνία του παίκτη με το σύστημα, αλλά ανάλογα με τη θέση μπορεί να χρειαστεί να επαναληφθεί μία κίνηση ή να εισαχθεί από το serial monitor αν οι αισθητήρες δεν ανταποκρίνονται σε αλλαγές επειδή έχουν ενεργοποιηθεί από κοντινούς μαγνήτες.

Η εμφάνιση της σκακιέρας με το αυτοκόλλητο εκτυπωμένο βινύλιο είναι άνω των προσδοκιών με πλούσια χρώματα πιστά στην ψηφιακή μορφή του στο πρόγραμμα σχεδίασης (Blender). Η χρήση τρισδιάστατων εκτυπωμένων στηρίξεων επέτρεψε την ομοιομορφία της κατασκευής ώστε τα εξωτερικά χαρακτηριστικά της να συμπληρώνουν το ένα το άλλο.

Η χρήση μαγνητικών κινήσεων επιτρέπει μία μεγάλη ευελιξία προς το σετ κομματιών που μπορεί να χρησιμοποιηθεί. Το μόνο που απαιτείται είναι μαγνήτες στις βάσεις τους, κάτι που πολλά σετ ήδη παρουσιάζουν. Αλλιώς μπορεί να χρησιμοποιηθεί κάποιο υλικό που μαγνητίζεται στις βάσεις, όπως μεταλλικές ροδέλες, και οι μαγνήτες να προστεθούν από κάτω έτσι ώστε να είναι αποσπώμενοι για την επαναχρησιμοποίηση τους μεταξύ διαφορετικών σετ κομματιών.

Το μέγεθος της κατασκευής παρουσιάζει μερικούς «νεκρούς» χώρους λόγω των δοκών του συστήματος κίνησης, εναλλακτικά συστήματα εκτός του CoreXY μπορούν να αναλυθούν για μελλοντικές εκδοχές.

Συνοψίζοντας, η κατασκευή είναι πλήρως λειτουργική για καθημερινή χρήση και φιλικές παρτίδες μικρού ή μεσαίου επιπέδου. Τα αποτελέσματα είναι πολύ ικανοποιητικά για πρώτη εκδοχή με τη δυνατότητα διεκπεραίωσης μιας πλήρους παρτίδας, η οποία είναι ευχάριστη χάρη στο νευρωνικό δίκτυο να υπολογίζει αρκετά καλές κινήσεις, έτσι ώστε να είναι διασκεδαστικό χωρίς να είναι ανίκητο από ανθρώπους όπως οι περισσότεροι σκακιστικοί αλγόριθμοι. Οι αυτόματες σκακιέρες στην αγορά που προορίζονται για τουρνουά και υψηλά επίπεδα δείχνουν καλύτερα χαρακτηριστικά σε όλους τους πυλώνες, κάτι που δείχνει πόσες εκδοχές πρέπει να φτιάχτηκαν μέχρι να βελτιστοποιηθούν. Εξάλλου η κατασκευή πολλών εκδοχών είναι απαραίτητη για την εμπορευματοποίηση ενός προϊόντος και οι προσδοκίες και τα αποτελέσματα πρέπει να μετριάζονται με βάση τη παρούσα εκδοχή.

Συμπεράσματα

Η κατασκευή ενός πολυδιάστατου συστήματος, όπως μία αυτόματη σκακιέρα, παρουσιάζει πολλές προκλήσεις οι οποίες είναι αδύνατο να βελτιστοποιηθούν από τη πρώτη εκδοχή. Με τη κατασκευή ενός λειτουργικού πρωτοτύπου διαπιστώνονται πλεονεκτήματα και μειονεκτήματα διαφορετικών λύσεων, με ιδιαίτερη βάση στα υποσυστήματα που πρέπει να δοθεί προτεραιότητα στην αποτελεσματικότητα και επίδοση τους.

Από αυτό το πρωτότυπο τα μέρη των οποίων συμπεραίνεται η προτεραιότητα βελτιστοποίησης τους σε μία δεύτερη μεταγενέστερη εκδοχή μεταξύ άλλων είναι:

- Η χρήση αισθητηρίων με μεγάλο εύρος τιμών σε πλέγματα για την ελαχιστοποίηση προβλημάτων επικάλυψης και μεγαλύτερη ανάλυση θέσεως.

- Η οριζόντια εκτύπωση ή αγορά μεταλλικών δοκών στο σύστημα κίνησης για ελαχιστοποίηση της τριβής κατά τη κίνηση θα επέτρεπε υψηλότερες ταχύτητες.

- Η προτίμηση σύγχρονων driver βηματικών κινητήρων που δίνουν προτεραιότητα στην αθόρυβη λειτουργία, εφόσον οι κινητήρες προσφέρουν περισσότερη ροπή από την απαιτούμενη.

- Η αγορά τυπωμένων κυκλωμάτων PCB αντί για breadboards θα βελτίωνε την αξιοπιστία των ηλεκτρονικών αλλά με μεγάλη αύξηση του κόστους.

- Η χρήση αποσπώμενων μαγνητών στις βάσεις των κομματιών θα επέτρεπε την επαναχρησιμοποίηση τους σε διαφορετικά σκακιστικά σετ.

Μέσω διαφορετικών εκδοχών μπορεί ένα πρωτότυπο να εξελιχθεί σε εμπορευματοποιήσιμο προϊόν, βελτιώνοντας τμηματικά τα επιμέρους λειτουργικά του μέρη. Συνδυαστικά με αυτή τη μεθοδολογία η ένταξη καινούργιων στόχων αποτελεί μέρος μεταγενέστερων εκδοχών, μελλοντικά πλαίσια έρευνας μεταξύ άλλων είναι:

- Η χρήση μπαταρίας για την φορητότητα της κατασκευής, συνοδευόμενη από το σύστημα διαχείρισης της και επαναφόρτισης. Προαιρετικά μπορεί να συμπεριληφθεί και ένα μικρό ηλιακό πάνελ στον «νεκρό» χώρο γύρω από το σύστημα κίνησης για παθητική επαναφόρτιση.

- Μία μεγαλύτερη οθόνη αφής που επιτρέπει την καλύτερη επικοινωνία με το χρήστη, υποστηρίζοντας πολλαπλές ρυθμίσεις προσβασιμότητας όπως διαφορετικές γλώσσες, φωτεινότητα, αντίθεση, κτλ. Οι οθόνες γραφικών χρησιμοποιούν περίπου 50pins και θα απαιτούταν ξεχωριστό driver για την κωδικοποίηση στο HDMI καλώδιο της.

- Η σύνδεση στο διαδίκτυο θα επέτρεπε απομακρυσμένες παρτίδες μεταξύ παικτών, είτε με τη χρήση πολλαπλών κατασκευών ή μέσω πλατφόρμων όπως το Chess.com και το Lichess λαμβάνοντας τα σχετικά API keys για τις παροχές τους.

- Η ικανότητα εκπαίδευσης σε διαφορετικές θέσεις από την αρχική, για εκμάθηση διαφορετικών ανοιγμάτων ή σταδίων του παιχνιδιού. Αυτή η δυνατότητα θα μετέτρεπε την κατασκευή σε διδακτική με σκοπό τη χρήση της από σκακιστικούς συλλόγους. Προαιρετικά μια εξωτερική κάρτα microSD για φόρτωση πολλαπλών νευρωνικών δικτύων για τη προσομοίωση αντιπάλων σε όλα τα επίπεδα.

Χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs)

Κατά τη διάρκεια της κατασκευής, της συγγραφής κώδικα και του παρόν έντυπου χρησιμοποιήθηκαν LLMs με την παρακάτω μεθοδολογία:

Για θέματα τα οποία δεν θέτουν σε κίνδυνο το σύστημα, με άλλα λόγια χαμηλού ρίσκου, προτιμήθηκε η απόκτηση γνώσεων από το Claude Sonnet της Anthropic λόγω της ταχύτητας, της αποδοτικότητας και της σχετικότητας με το παρόν εξεταζόμενο πρόβλημα. Θέματα υψηλού ρίσκου που η αποτυχία τους θα επέφερε καταπόνηση των εξαρτημάτων ή κρούση του συστήματος κίνησης μελετήθηκαν από άρθρα στις ιστοσελίδες που αναγράφονται στη σχετική βιβλιογραφία.

Η συγγραφή προσωρινών προγραμμάτων για την εξακρίβωση μεθοδολογιών και απόκτηση γνώσεων, όπως η εύρεση του μέτρου ορθής προβολής κατά τη δρομολόγηση του ηλεκτρομαγνήτη ή η εξαγωγή δεδομένων υψηλής ποιότητας από τη βάση δεδομένων, γράφτηκαν συνδυάζοντας προτεινόμενες μεθόδους και συναρτήσεις δοσμένες από το Claude Sonnet και το Gemini της Google. Λόγω του περιοριστικού διαθέσιμου context αυτών, στο δωρεάν πρόγραμμα δεν διατίθενται αρκετά tokens για τη συγγραφή ενός πλήρης προγράμματος, απαιτήθηκε ο χειροκίνητος συνδυασμός των παραγόμενων συναρτήσεων και η κατανόηση τους. Ο σχολιασμός του τελικού προγράμματος, εκτός του παρόν έντυπου, γράφτηκε από το Claude Sonnet, λόγω της ήδη διαθέσιμης εκτεταμένης επεξήγησης του που γράφτηκε χειροκίνητα σε αυτό το έγγραφο. Τα διαγράμματα ροής γράφτηκαν σε UML μετά την συγγραφή τους σε C++ μετατρέποντας μεταξύ των δύο γλωσσών με το Claude Sonnet.

Η αποσφαλμάτωση προβλημάτων κώδικα λόγω της απλότητας του διαθέσιμου debugging στο Arduino IDE έγινε με το Claude Sonnet όπως και το profiling του κώδικα. Η διαδικασία περιλάμβανε αντιγραφή των register dumps που έδινε το ESP32 λόγω του watchdog timer που περιέχει στο LLM, το οποίο εξακρίβωνε το είδος του προβλήματος. Χάρη αυτής της διαδικασίας γράφτηκε το partition table και ανακτήθηκαν πολλές γνώσεις για τη σωστή χρήση των pointers και γενικότερα για την ασφάλεια μνήμης ενός επεξεργαστή.

Στο παρόν έντυπο τα μεγάλα γλωσσικά μοντέλα χρησιμοποιήθηκαν για επαλήθευση, εύρεση λογικών αστοχιών και ασάφειας μεθοδικά μετά τη χειροκίνητη συγγραφή κάθε επιμέρους κεφαλαίου. Οι διορθώσεις και παρατηρήσεις που πρότειναν βελτίωσαν την ορθότητα και συνέχεια του εγγράφου και γράφτηκαν χειροκίνητα στο έγγραφο αναδιατυπώνοντας στο προσωπικό ύφος της γλώσσας αντί της αντιγραφής τους.

Η ταχύτητα κατασκευής και συγγραφής κώδικα ήταν ιδιαίτερα αποτελεσματική χάρη στη χρήση αυτών των γλωσσικών μοντέλων. Είναι αδιαφιλονίκητο πως καθιστούν το δυνατότερο εργαλείο απόκτησης γνώσεων, αλλά με σημαντικά μειονεκτήματα και δυσκολία χρήσης λόγω κατανόησης του prompting. Απαιτείται τεχνογνωσία και φιλτράρισμα των απαντήσεων τους για τη συγκράτηση μόνο εποικοδομητικών πληροφοριών κάτι που αποκτιέται μόνο με τη χρήση τους.

Παράδειγμα Prompt στο Gemini της Google

Prompt: «I'm trying to make a neural network that can play chess with the endgoal of integrating it onto an automatic chessboard using an esp32s3. I know a typical tiny chess AI uses a residual convolutional neural network, could you write a mock python script that demonstrates the workflow and outputs a tensor of 4096 channels for every possible move in UCI disregarding promotions with pytorch?»

Response: «To map every possible move in chess without promotions, we use a **64 x 64 flat output layer**, which equals exactly **4096 channels**. Each channel represents a unique combination of a starting square and an ending square 64 x 64 = 4096.

Here is a complete, mock Python script using **PyTorch**. It demonstrates the workflow of a Residual Convolutional Neural Network (ResNet) that takes a chess board state and outputs this 4096-move tensor.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ResidualBlock(nn.Module):
"""
Standard ResNet block: Conv -> BatchNorm -> ReLU -> Conv -> BatchNorm -> Add
"""
def __init__(self, channels):
super(ResidualBlock, self).__init__()
self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
self.bn1 = nn.BatchNorm2d(channels)
self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
self.bn2 = nn.BatchNorm2d(channels)

def forward(self, x):
residual = x
out = F.relu(self.bn1(self.conv1(x)))
out = self.bn2(self.conv2(out))
out += residual # Skip connection
return F.relu(out)

class ChessResNet(nn.Module):
def __init__(self, num_residual_blocks=4, hidden_channels=64):
super(ChessResNet, self).__init__()

# Input layer: 12 channels for pieces (6 white, 6 black) on an 8x8 board
```

```
# You can add more channels for castling rights, en passant, etc.
self.conv_input = nn.Conv2d(12, hidden_channels, kernel_size=3, padding=1)
self.bn_input = nn.BatchNorm2d(hidden_channels)

# Residual Tower
self.res_blocks = nn.ModuleList([
ResidualBlock(hidden_channels) for _ in range(num_residual_blocks)
])

# Policy Head (Outputs the move probabilities)
# Reduces channels before flattening to control parameter count for ESP32
self.policy_conv = nn.Conv2d(hidden_channels, 32, kernel_size=1)
self.policy_bn = nn.BatchNorm2d(32)

# Fully connected layer to map 32 channels * 8 * 8 board to 4096 moves
self.policy_fc = nn.Linear(32 * 8 * 8, 4096)

def forward(self, board_state):
# Initial Convolution
x = F.relu(self.bn_input(self.conv_input(board_state)))

# Pass through residual tower
for block in self.res_blocks:
x = block(x)

# Policy Head execution
policy = F.relu(self.policy_bn(self.policy_conv(x)))
policy = policy.view(policy.size(0), -1) # Flatten to (batch_size, 2048)

# Move logits output (4096 channels)
move_logits = self.policy_fc(policy)
return move_logits

# --- Workflow Demonstration ---
if __name__ == "__main__":
# 1. Instantiate the model
# Kept small (4 blocks, 64 channels) to fit inside ESP32-S3 memory constraints
model = ChessResNet(num_residual_blocks=4, hidden_channels=64)
model.eval() # Set to evaluation mode

# 2. Simulate a batch of 1 chess board state
```

```
# Size: (Batch_size=1, Channels=12, Height=8, Width=8)
mock_board_input = torch.randn(1, 12, 8, 8)

# 3. Forward pass
with torch.no_grad():
output_tensor = model(mock_board_input)

# 4. Verify output shape
print(f"Input shape: {mock_board_input.shape}")
print(f"Output shape: {output_tensor.shape} (1 Batch, 4096 Move Channels)")

# 5. Concept: How to map a UCI move (e.g., 'e2e4') to an index
# square_index = row * 8 + col (e.g., a1=0, h8=63)
from_square = 12 # e2
to_square = 28 # e4
move_index = from_square * 64 + to_square

print(f"The logic channel for 'e2e4' maps to index: {move_index}")
print(f"Logit value for this move: {output_tensor[0][move_index].item():.4f}")
```

Critical ESP32-S3 Design Considerations

- **Memory Limits**: The ESP32-S3 has 512KB of SRAM and up to 8MB of PSRAM. A standard AlphaZero network is too large. Keep hidden_channels at 64 or lower and residual_blocks under 6.
- **Quantization**: You must convert this PyTorch model to **TensorFlow Lite (TFLite)** or **ONNX**, and then use 8-bit quantization (INT8) to shrink the model file size and boost execution speed.
- **ESP-DL**: Espressif provides a deep learning library called **ESP-DL** optimized specifically for the vector instructions of the ESP32-S3 chip.

If you want to move forward with hardware deployment, let me know:

- Will you use **TensorFlow Lite Micro** or Espressif's native **ESP-DL** framework?
- Do you want to see the code for **masking illegal moves** out of that 4096 tensor?
- Do you need advice on how to structure the **12-channel board input**? »

Από αυτή την απάντηση καθορίστηκαν τα βήματα που χρειάζονται για την εκπαίδευση ενός νευρωνικού δικτύου. Η απάντηση δεν ήταν πλήρης λόγω των απλοποιημένων 12 καναλιών αντί για τα απαιτούμενα 18 που έχει το τελικό μοντέλο, για αυτό χρειάστηκαν πολλά παραπάνω prompts και μελέτη της διαδικασίας πριν το τελικό αποτέλεσμα. Εκτός από το prompting στο LLM, χρησιμοποιήθηκαν και σχετικά github repositories από χρήστες που έφτιαξαν παρόμοια νευρωνικά δίκτυα. Η διαδικασία θα ήταν πολύ δυσκολότερη χωρίς τη χρήση LLMs που επεξήγησαν με ικανοποιητική ακρίβεια αυτά τα repositories, τα οποία παρέχουν περιορισμένο documentation ως προς τον κώδικα.

Βιβλιογραφία

Algebraic Notation (chess). (2026, April 13). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Algebraic_notation_(chess)

Alpha-Beta. (2026). Ανάκτηση από chessprogramming.org: https://www.chessprogramming.org/Alpha-Beta

APDahlen. (2024, January). *Breadboard Current Limitations with Terminal Block Alternatives.* Ανάκτηση από forum.digikey.com: https://forum.digikey.com/t/breadboard-current-limitations-with-terminal-block-alternatives/38009

Arduino Documentation. (2024). Ανάκτηση από Arduino.cc: https://docs.arduino.cc/

Blind Mode. (2026). Ανάκτηση από Lichess.com: https://lichess.org/page/blind-mode-tutorial

C++ Reference. (2026). Ανάκτηση από cppreference.com: https://cppreference.com/

Chang, C., & Tran, T. (2025, March). *Mastering Precision: Understanding Microstepping in Motion Control.* Ανάκτηση από www.analog.com: https://www.analog.com/en/resources/analog-dialogue/articles/mastering-precision-understanding-microstepping.html

Chess Programming Wiki. (2024, July 18). Ανάκτηση από chessprogramming.org: https://www.chessprogramming.org/Main_Page

Chess Stats. (2026). Ανάκτηση από Chess.com: https://www.chess.com/terms/chess-stats

CoreXY. (2020, August 31). Ανάκτηση από reprap.org: https://reprap.org/wiki/CoreXY

DRV8825 Datasheet. (2014, July). Ανάκτηση από https://www.ti.com/lit/ds/symlink/drv8825.pdf

DRV8825 missing microsteps. (2015, February 11). Ανάκτηση από cabristor.blogspot.com: https://cabristor.blogspot.com/2015/02/drv8825-missing-steps.html

Eddy current. (2026, March 25). Ανάκτηση από wikipedia: https://en.wikipedia.org/wiki/Eddy_current

Electromagnet. (2026, May 18). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Electromagnet

Elo Rating System. (2026, May 19). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Elo_rating_system

ESP32-S3 Datasheet. (2026, March 5). Ανάκτηση από www.espressif.com: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

ESP32-S3 Technical Reference Manual. (2026, March 4). Ανάκτηση από documentation.espressif.com: https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf

ESP-DL. (2026). Ανάκτηση από github.com: https://github.com/espressif/esp-dl

ESP-NN. (2026). Ανάκτηση από github.com: https://github.com/espressif/esp-nn

ESP-PPQ. (2026). Ανάκτηση από github.com: https://github.com/espressif/esp-ppq

Evaluations. (2026). Ανάκτηση από Lichess.com: https://database.lichess.org/#evals

FastAccelStepper. (2026). Ανάκτηση από github.com: https://github.com/gin66/FastAccelStepper

FIDE LAWS of CHESS. (2026). Ανάκτηση από www.fide.com: https://www.fide.com/FIDE/handbook/LawsOfChess.pdf

Fiekas, N. (2024). *python-chess: a chess library for Python.* Ανάκτηση από python-chess.readthedocs.io: https://python-chess.readthedocs.io/en/latest/

Flyback diode. (2026, March 25). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Flyback_diode

Forsyth–Edwards Notation. (2026, February 12). Ανάκτηση από wikipedia: https://en.wikipedia.org/wiki/Forsyth%E2%80%93Edwards_Notation

FreeRTOS Reference Manual. (2026). Ανάκτηση από www.freertos.org: https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf

Greg06. (2026). *Automated Chessboard.* Ανάκτηση από instructables.com: https://www.instructables.com/Automated-Chessboard/

GT2 Timing Belt Datasheet. (2026). Ανάκτηση από www.handsontec.com: https://www.handsontec.com/dataspecs/gt2-belt-B.pdf

Introduction to ONNX. (2024). Ανάκτηση από onnx.ai: https://onnx.ai/onnx/intro/

IRLB8721PbF Datasheet. (2026). Ανάκτηση από www.infineon.com: https://www.infineon.com/assets/row/public/documents/24/49/infineon-irlb8721-datasheet-en.pdf?fileId=5546d462533600a40153566056732591

Kaiming, H., Xiangyu, Z., Shaoqing, R., & Jian, S. (2015, December 10). *Deep Residual Learning for Image Recognition.* Ανάκτηση από arxiv.org: https://arxiv.org/pdf/1512.03385

Krishnamoorthi, R. (2018, June 21). *Quantizing deep convolutional networks for.* Ανάκτηση από arxiv.org: https://arxiv.org/pdf/1806.08342

Language Reference. (2026). Ανάκτηση από docs.arduino.cc: https://docs.arduino.cc/language-reference/

LiquidCrystal_I2C. (2016, March 7). Ανάκτηση από github.com: https://github.com/johnrickman/LiquidCrystal_I2C

MadisonJames. (2025, June 7). *Why Are CoreXY 3D Printers Faster?* Ανάκτηση από kingroon.com: https://kingroon.com/blogs/3d-print-101/why-are-corexy-3d-printers-faster

Millennium Chess Shop. (2026). Ανάκτηση από computerchess.com: https://computerchess.com/en/

Move Generation. (2022, March 12). Ανάκτηση από chessprogramming.org: https://www.chessprogramming.org/Move_Generation

Multiplexer. (2026). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Multiplexer

Neodymium magnet. (2026). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Neodymium_magnet

Neural Networks. (2026). Ανάκτηση από chessprogramming.org: https://www.chessprogramming.org/Neural_Networks

Open Database. (2026). Ανάκτηση από Lichess.com: https://database.lichess.org/

Peiling. (2025, July 3). *PLA 3D Printing: Ultimate Guide with Tips and Considerations.* Ανάκτηση από www.raise3d.com: https://www.raise3d.com/blog/pla-3d-printing/

PyTorch documentation. (2026). Ανάκτηση από docs.pytorch.org: https://docs.pytorch.org/docs/2.12/index.html

Reed switch. (2026). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Reed_switch

Rules of Chess. (2026, May 16). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Rules_of_chess

Sejal, A., Delara, F., Brandon, L., Thomas, D., & Anthony, M.-R. (2026, April 30). *What is (H)CI: Why Does the "Human" Matter?* Ανάκτηση από arxiv.org: https://arxiv.org/pdf/2605.00109

Shannon, C. E. (1950, March). *Programming a Computer for Playing Chess.* Ανάκτηση από computerhistory.org: https://www.computerhistory.org/chess/doc-431614f453dde/

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., . . . Hassabis, D. (2017, December 5). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm.* Ανάκτηση από arxiv.org: https://arxiv.org/pdf/1712.01815

Square Off by Miko. (2026). Ανάκτηση από squareoffnow.com: https://squareoffnow.com/

Stepper Motor Drivers. (2026). Ανάκτηση από www.pololu.com: https://www.pololu.com/category/120/stepper-motor-drivers

Stepper Motor Selection Guide for 3D Printers: NEMA Size, Torque & Precision Explained. (2026, january 13). Ανάκτηση από LeanMotor.com : https://www.leanmotor.com/stepper-motor-selection-guide-for-3d-printers.html

Stockfish. (2022). Ανάκτηση από stockfishchess.org: https://stockfishchess.org/

tensorflow/tflite-micro. (2026). Ανάκτηση από github.com: https://github.com/tensorflow/tflite-micro

TMC2208/2 & TMC2224 family Datasheet. (2023, February 16). Ανάκτηση από www.analog.com: https://www.analog.com/media/en/technical-documentation/data-sheets/TMC2202_TMC2208_TMC2224_datasheet_rev1.14.pdf

Torque. (2026, March 17). Ανάκτηση από en.wikipedia.org: https://en.wikipedia.org/wiki/Torque

Tournament e-Boards. (2026). Ανάκτηση από digitalgametechnology.com: https://www.digitalgametechnology.com/products/tournament-e-boards

Universal Chess Interface. (2026, March 30). Ανάκτηση από wikipedia.org: https://en.wikipedia.org/wiki/Universal_Chess_Interface

Προσθήκες

Datasheets / Ιστοσελίδες Πωλητών

Βηματικοί κινητήρες

42BYGHW208
42BYGHW Series
NEMA 17
OK42STH22-0804AC Nema 17
Creality 42-34
42BYGHM809

Drivers

DRV8825
A4988
TMC2208 V3.0
L9110S

Ηλεκτρομαγνήτες

P25-20
WF-P30/22
WF-P25/20 (24 V)
ZYE1-P25/20

Μικροελεγκτές

ESP32-S3 (Technical Reference Manual)
Arduino UNO R3
Raspberry Pi Zero 2 W

Λοιπά

Reed Switch
Micro Switch
Transistor IRLB8721PBF
100 μF Electrolytic Capacitor
1N5404 Diode
440 Ω Resistor/ 10 kΩ Resistor
CD74HC4067 Multiplexer
16×2 3.3V LCD
PCF8574 backpack

Github Repositories

Repository της κατασκευής

Automated Chessboard

Βιβλιοθήκες C++/C

FastAccelStepper
LiquidCrystal_I2C
arduino-esp32
esp-tflite-micro
esp-idf
esp-dl
esp-nn
esp-dsp
tflite-micro

Βιβλιοθήκες Python

pytorch
numpy
python-chess
onnx
tensorflow-onnx
esp-ppq

Παρόμοιες ατομικές κατασκευές από διαφορετικούς χρήστες

ESP-Chess-Board
Wizards-Chess
ChessBoard
gambit
Imperium
esp32-chess-engine